DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY

Deep Learning for Computer Vision with TensorFlow 2

CNN architectures, transfer learning, object detection and YOLO

Volume 2 of 2 · Sections 46–56

11 SECTIONS

108 FIGURES

102 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 46 Classification Training 3
LECTURE 47 YOLO Model and Left Corner 16
LECTURE 48 Double Dash and Traffic Light 20
LECTURE 49 YOLO: Real-Time Object Detection 36
LECTURE 50 Don't Void the Original or YOLO 53
LECTURE 51 Darknet Folder and Google Drive 72
LECTURE 52 YOLO Version and File 78
LECTURE 53 MS COCO Object Detection 81
LECTURE 54 Attention Module 88
LECTURE 55 YOLO Version 98
LECTURE 56 Traffic Lights 102

LECTURE 46

Classification Training

Hello everyone, welcome to a new class. This class is for YOLO version 2, which represents an upgrade from the initial version, YOLO version 1. The idea with this new version is to improve the accuracy results as well as the speed. So, let's start checking the main improvements.

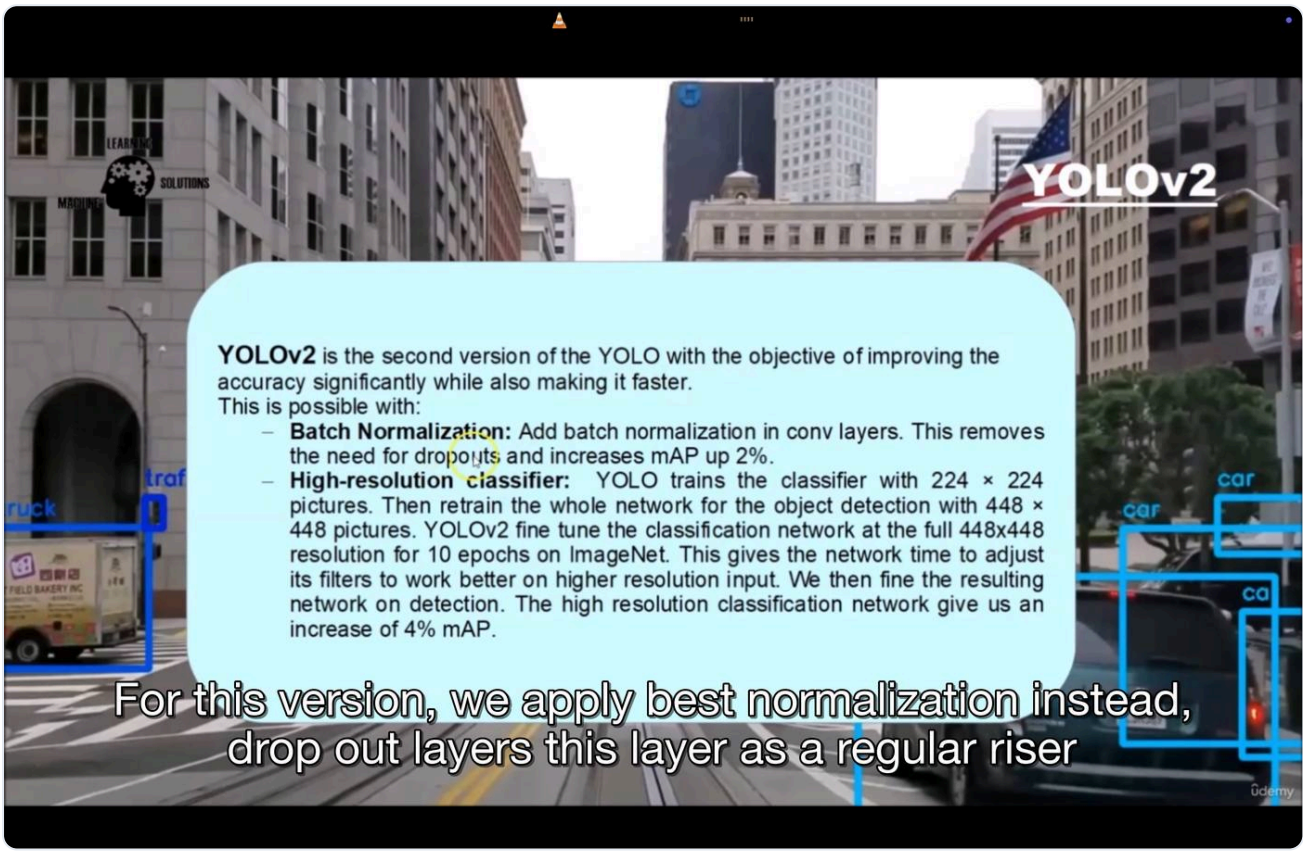


FIGURE 1.1 | This Is Possible with:

For this version we apply patch normalization instead dropout layers. This layer acts as a regularizer and increases the main average precision in 2%. The model also increases the input resolution for the classifier. The initial YOLO trains the classifier with this resolution 224 by 224, but in YOLO version 2 the resolution is doubled 448 pixels in height and width, trained for 10 epochs on ImageNet dataset. This high resolution in classification increases the main average precision in 4%. On the other hand, YOLO version 2 takes a different approach about the predictions of the Greek cells. Let's check it out. YOLO version 2 uses the approach of anchor boxes, which are predefined bounding boxes with specific ratios and measures.

Let's start illustrating an issue in the approach of YOLO version 1. Here I have two objects, an SUV in horizontal and a person in vertical.

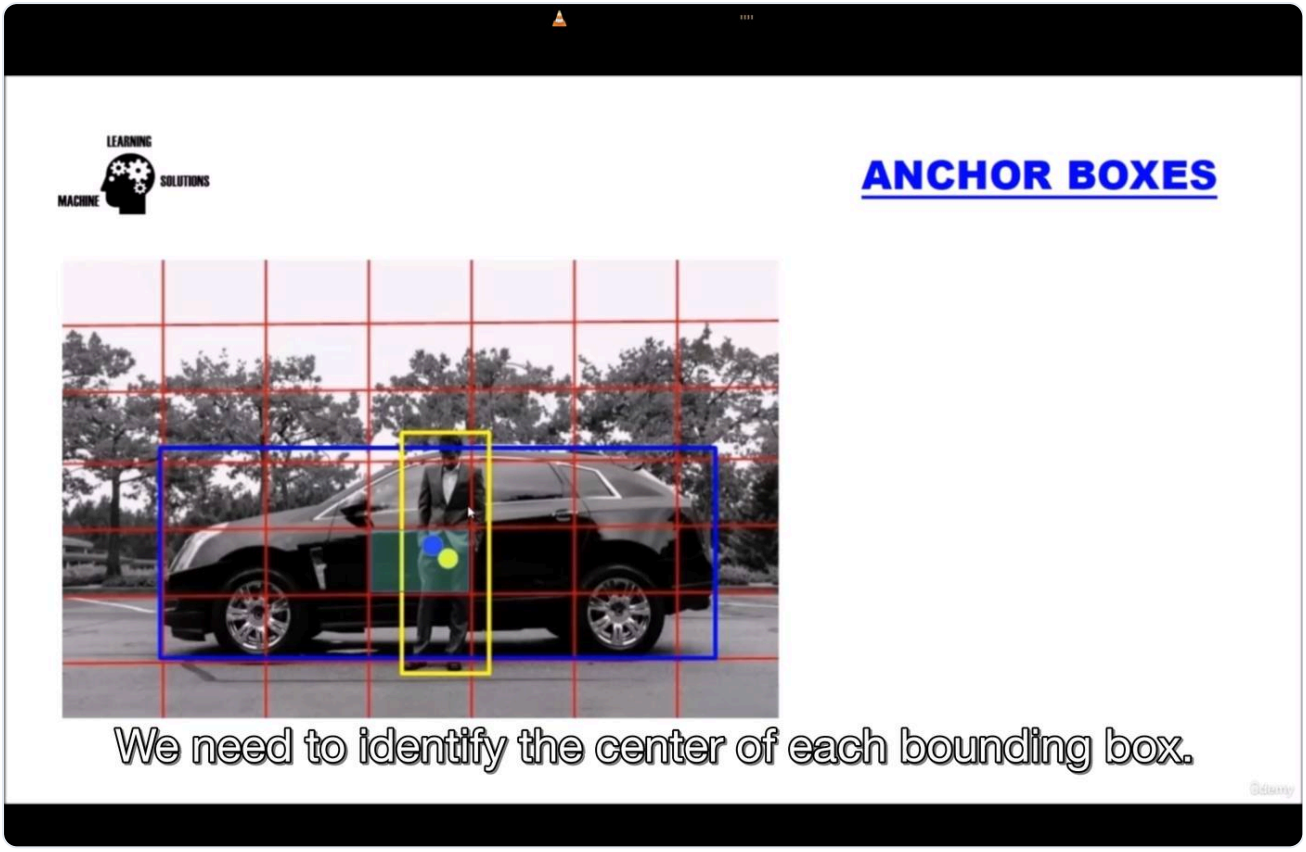


FIGURE 1.2

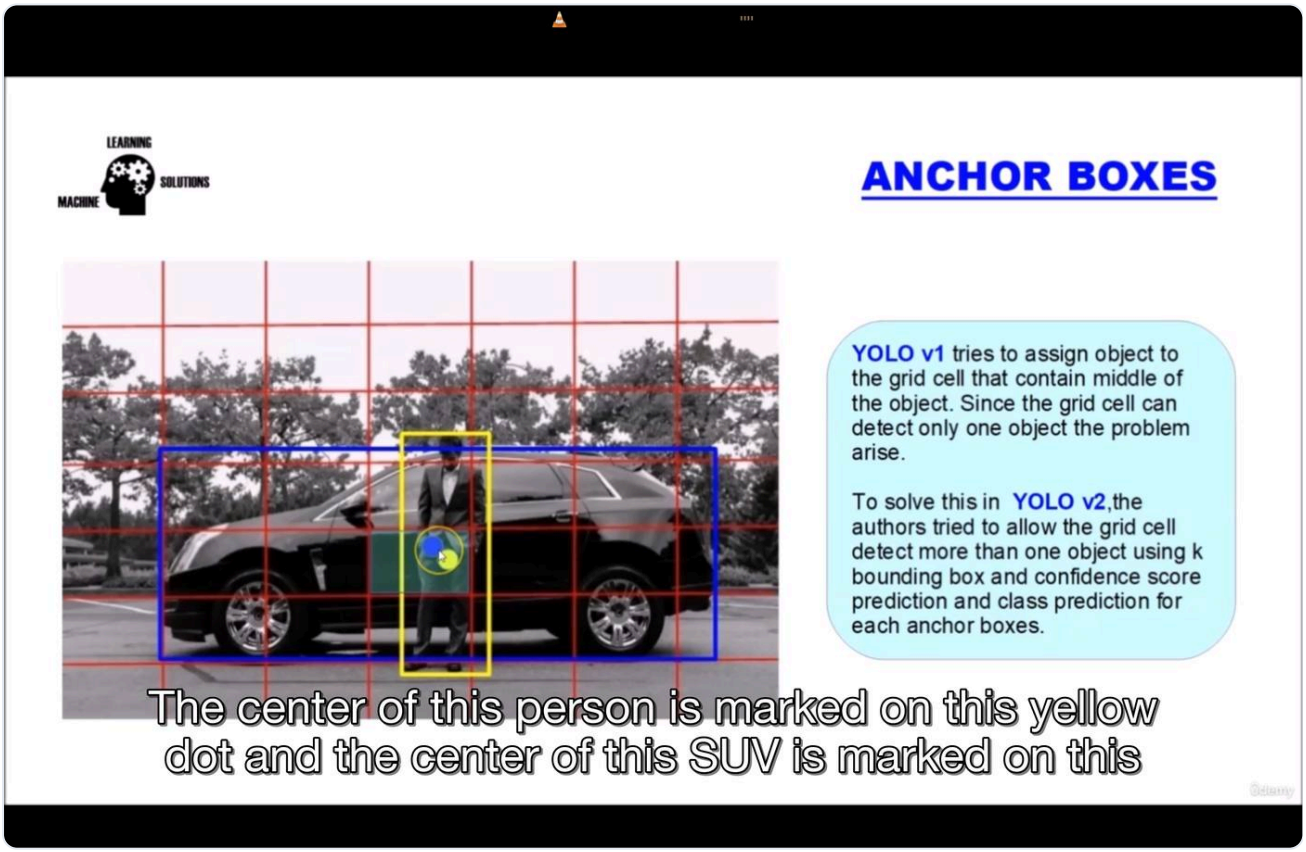


FIGURE 1.3 | YOLO v1 Tries to Assign Object to

As you can remember, we need to identify the center of each bounding box, the center of this person is marked on this yellow dot and the center of this SUV is marked on this blue dot. Then this Greek cell is responsible to detect these two objects. So, there is an issue in YOLO version 1, because each Greek cell in version 1 only can predict one class, then we can't predict more than one object. So, in order to solve this problem, YOLO version 2 uses anchor boxes per Greek cell and also takes a different approach in the predictions. We are going to predict based on these anchor boxes, not based on a Greek cell. Then with these anchor boxes, we can manage objects with different sizes and ratios for each Greek cell.

In order to optimize the predictions, there is a new spatial dimension, let's see it. Here we are going to change the size of the grid. The idea is to locate the center point of the output feature map in a specific Greek cell, because the initial resolution 448 in height and width produces even Greek cells. We will produce an output feature map with a size 14 by 14, then it will be very complicated to assign the center point to a specific Greek cell. Let's see the representation.

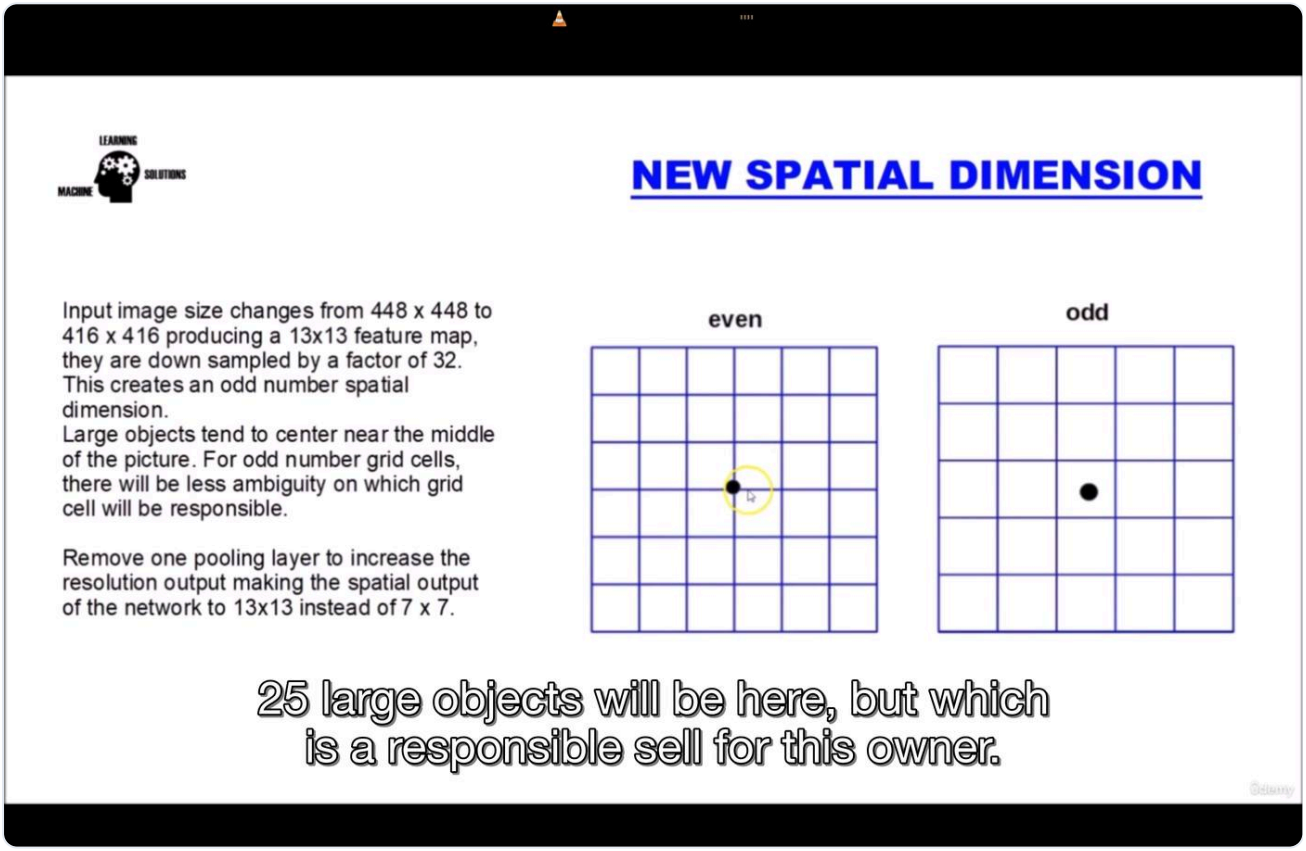


FIGURE 1.4 | Input Image Size Changes from 448 X 448 to

If the number of Greek cells is even, we have this situation, the center point, which is mainly responsible to identify large objects, will be here, but which is a responsible cell for this center? We have four candidates. It would be very complicated to assign a specific Greek cell. However, if we reduce this input resolution to this resolution 416 pixels, we are going to produce a 13 by 13 feature map. The Greek cell in size is an odd number and it's very easy to identify the cell which is responsible to the center point. These final sizes in the Greek cells occur because we have a downsample factor of 32, reducing the dimensionality of the input data to the output feature map.

In addition, the new architecture removes one pooling layer to increase the default 7 by 7 feature map, increasing the output to 13 by 13 feature map. Now, per each Greek cell we are going to apply anchor boxes. Let's check it out. We are going to have k number of anchor boxes per Greek cell, something like this. Some anchor boxes are in vertical, others in horizontal. Here I have a square anchor box with a specific size. This is another square anchor box with a different size.

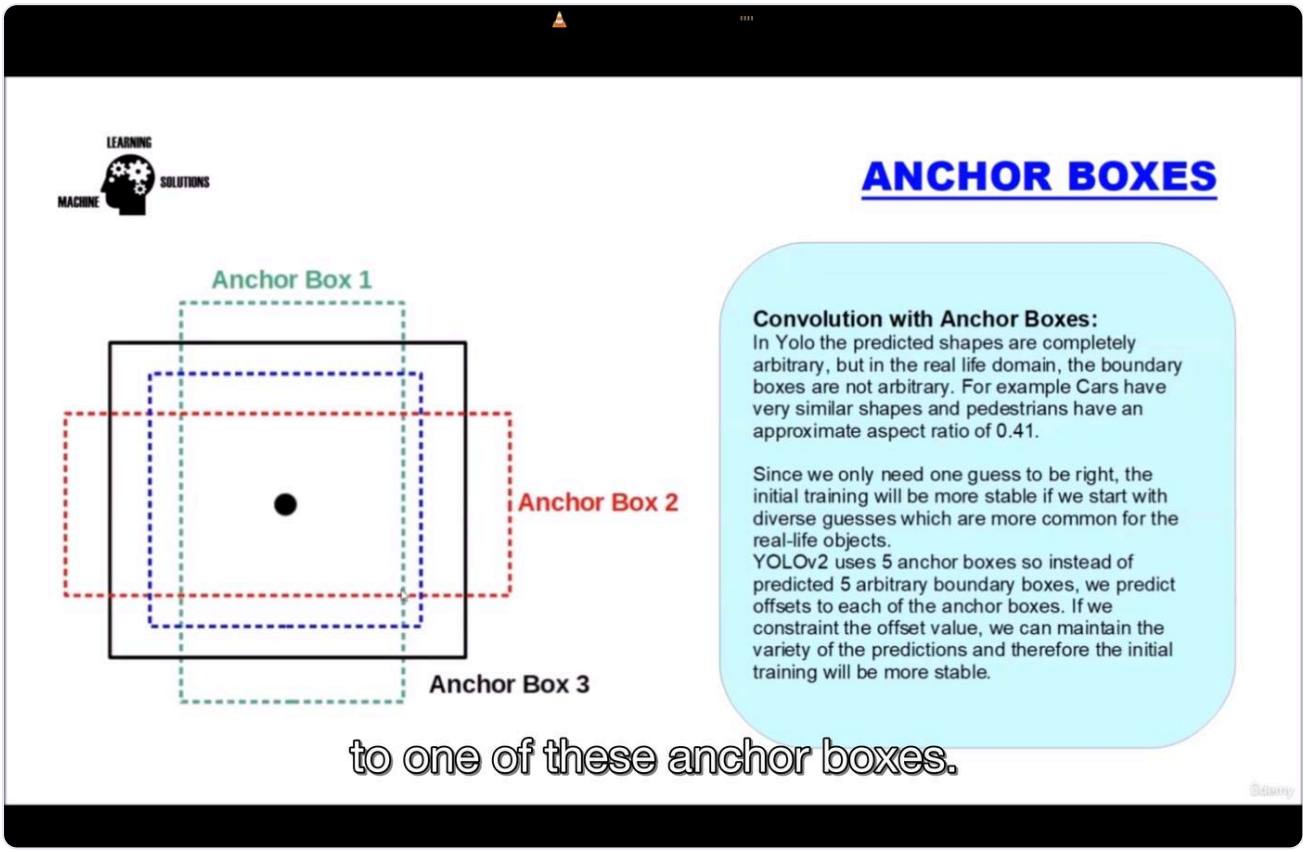


FIGURE 1.5 | Convolution with Anchor Boxes:

So we need different sizes and ratios because in real life the shape of the objects are very close to one of these anchor boxes. For instance, pedestrians can fit better in vertical boxes and cars may fit better in horizontal anchor boxes. Then this version of YOLO uses five different anchor boxes and now the predictions are made for each anchor box and not for each Greek cell. So we are going to predict k number of anchor boxes. For this prediction we need to remove the fully connected layers which are present in the version 1 and work with the information of the anchor boxes.

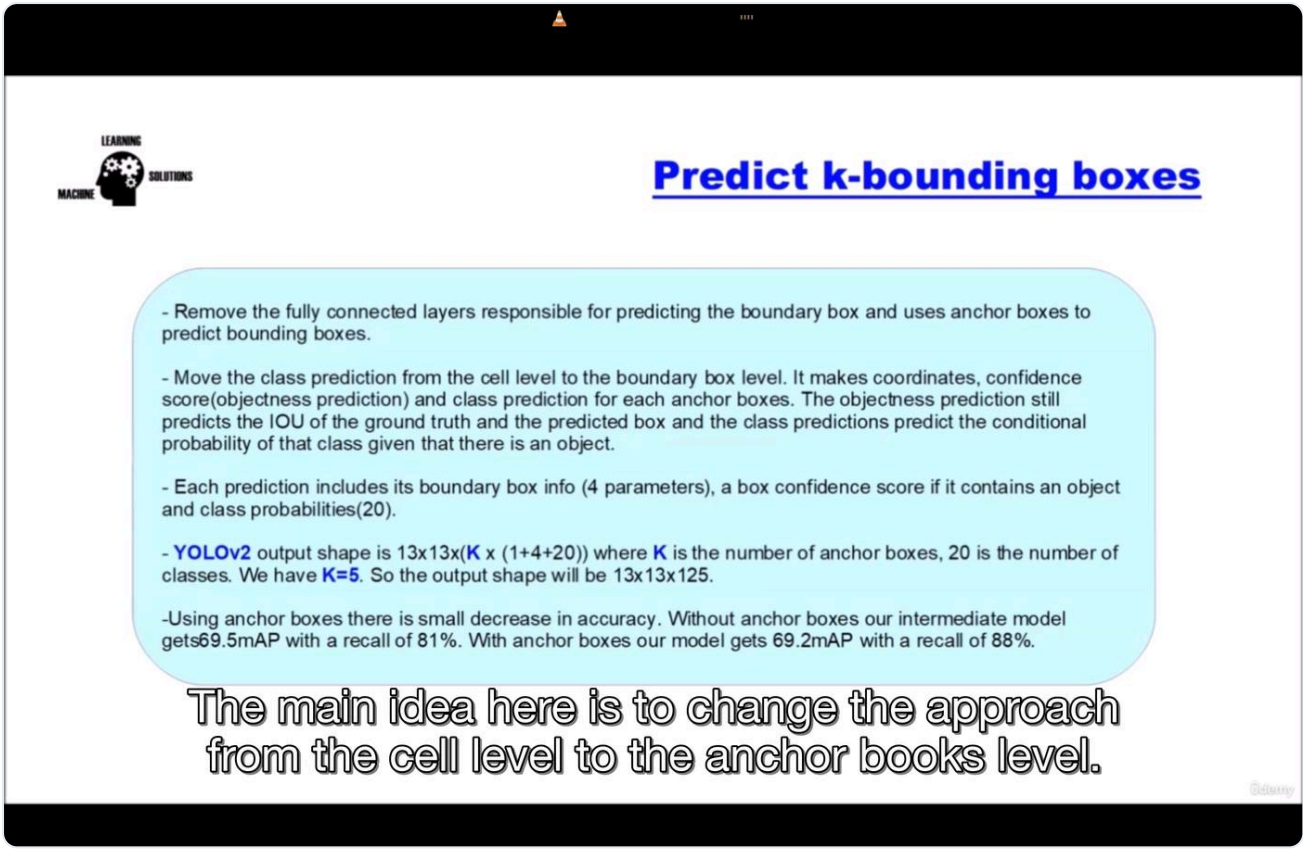


FIGURE 1.6

The main idea here is to change the approach from the cell level to the anchor box level. Then we are going to generate the confidence score which is the probability to find an object, the class predictions which is the probability that an object belongs to a specific class for each anchor box. In order to calculate the objectness score which is the probability to find an object this version of YOLO uses the same approach as the version 1 calculating the IOU metric. This metric is applied for the objectness score and also to calculate the class prediction which is a conditional probability. We are going to calculate a specific class only if we detect an object.

So each banding box prediction will contain four parameters to identify the center of the banding box and the offset value in height and width related to the anchor box. There is also a confidence score and finally the information of the classes. YOLO version 2 output has a shape 13 by 13 because we apply a reducing factor in 32 multiplied by curly brackets, k is the number of anchor boxes by curly brackets, 1 is the objectness score plus 4 which is the number of parameters required to define a banding box and here is a change, plus 20 in this case the number of classes for Pascal VOC. So take note in this case the classes and the objectness score are calculated for every banding box.

This is the main difference from the initial version of YOLO. In the initial version we stack the information of the banding boxes in only one tensor because the information of the classes is the same despite the banding boxes. For this example the final output is 13 by 13 by 125. These anchor boxes produce a small decrease in the accuracy but it increases the recall metric.

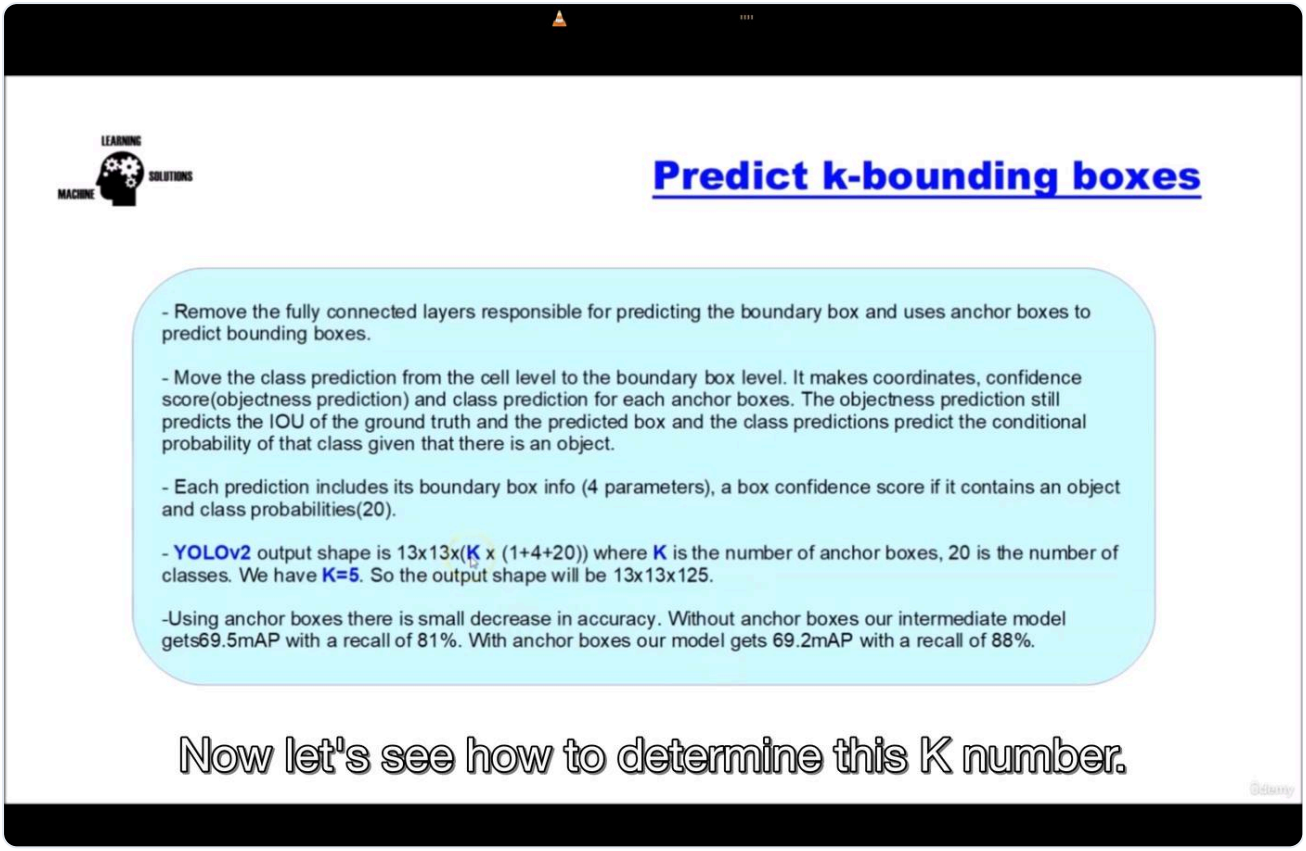


FIGURE 1.7

Now let's see how to determine this k number. How many banding boxes we are going to apply per grid cell. YOLO version 2 uses a clustering algorithm to identify the best number of banding boxes. So we are going to apply the k-minx algorithm to identify the number of groups or clusters. Here we have the plot for COCO dataset and here we have the plot for Pascal VOC 2007.

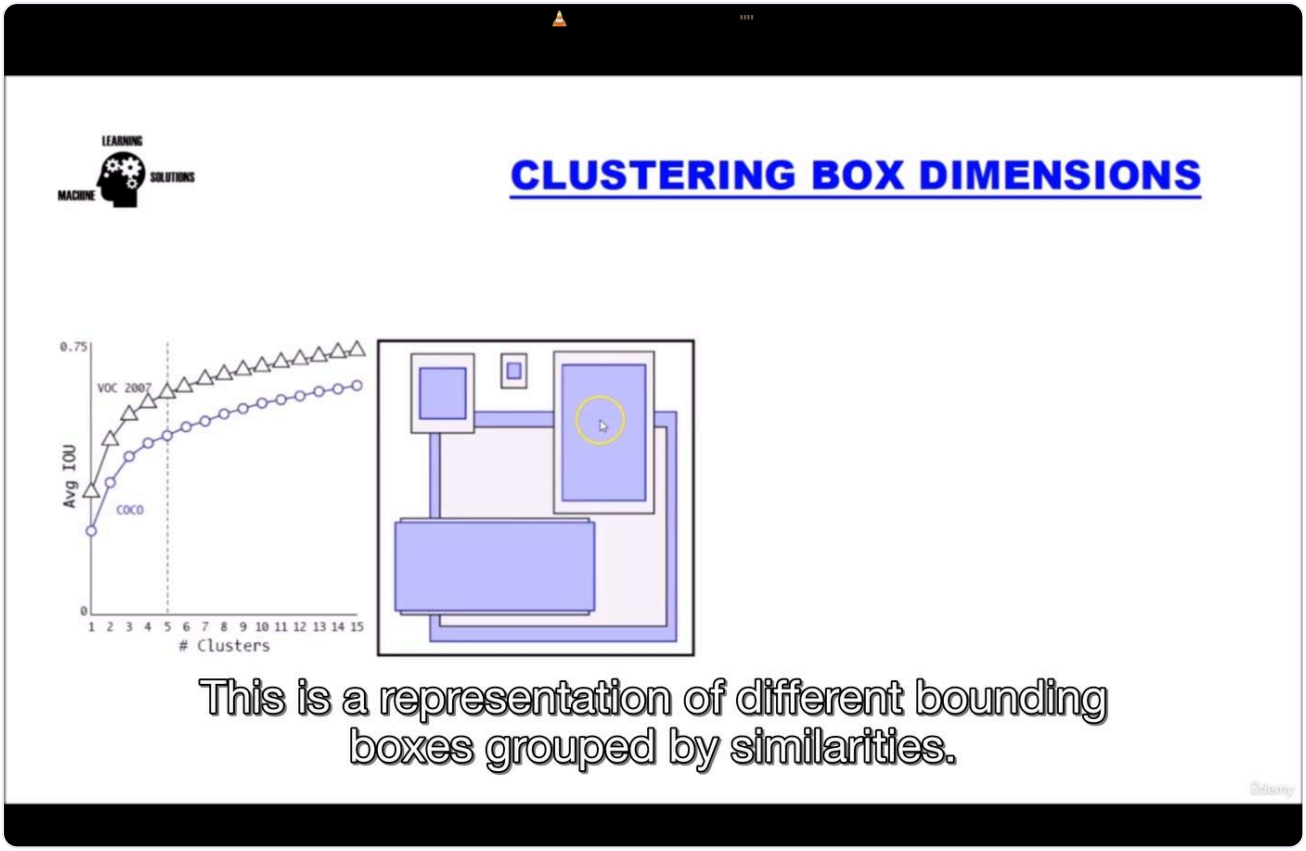


FIGURE 1.8

This is a representation of different banding boxes grouped by similarities. These shapes on black correspond to the Pascal VOC and these light blue shapes correspond to COCO dataset. And in order to calculate the best distances in the clusters the algorithm applies not the Euclidean distance rather a distance based on the IOU metric. This distance is calculated between the box and the centroid and is equal to 1 minus the IOU metric between the box and the centroid.

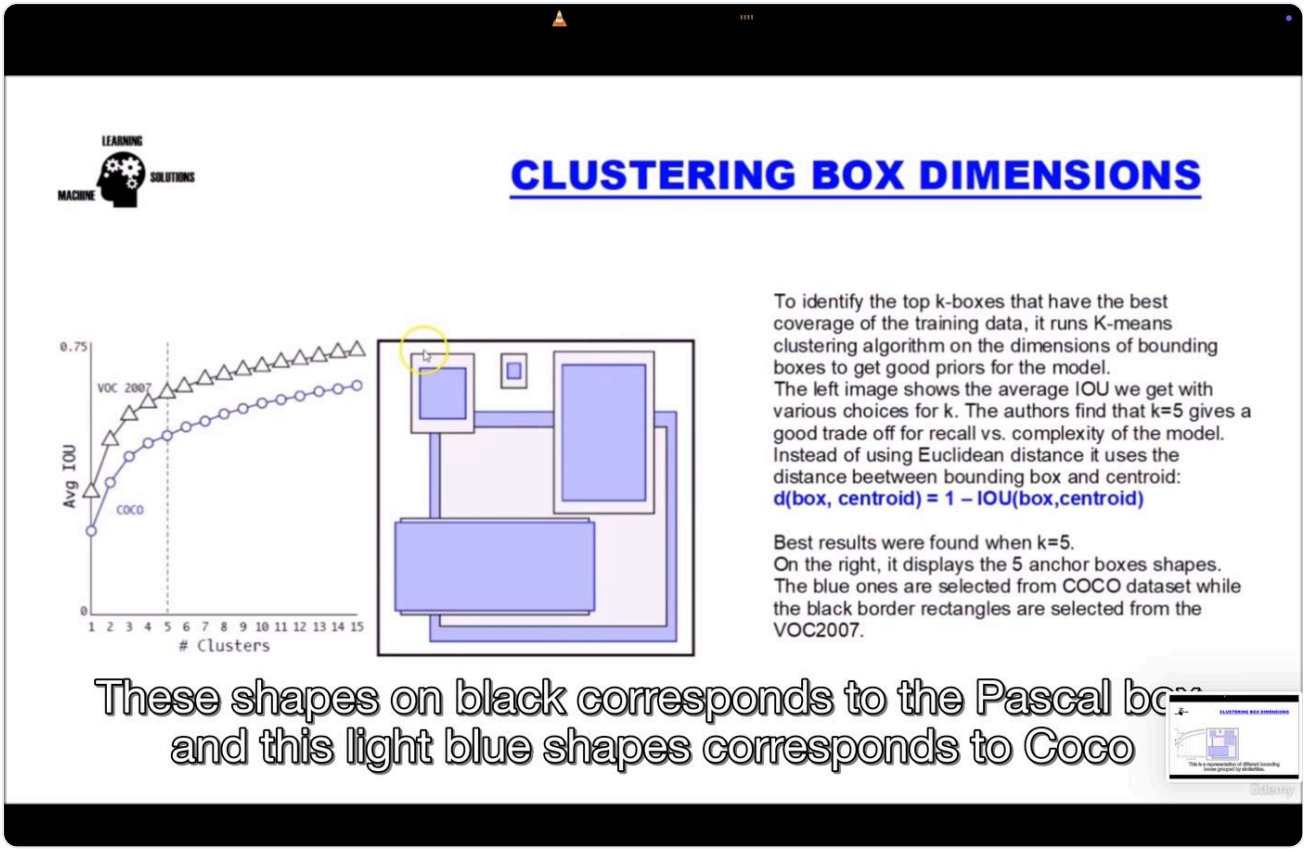


FIGURE 1.9 | To Identify the Top K-boxes That Have the Best

After running four different clusters the best results were found when k is equal to 5. This is a reason why this algorithm uses five anchor boxes as default per grid cell. On the other hand we have a new coordinate system. This approach is called direct location prediction and is represented with this image. The initial travel with YOLO version 1 is there is no constraints on location predictions. Then the initial prediction of a banding box could be very far away from the grand true box. Then the idea to direct location prediction is to constrain the location of the predicted boxes. We use the offset values for height and width with respect to the grand true box and these values are constrained with a sigma function in order to produce values between 0 to 1.

And we also constrain the confidence score with this sigma function because this is a probability with values in a range 0 to 1.

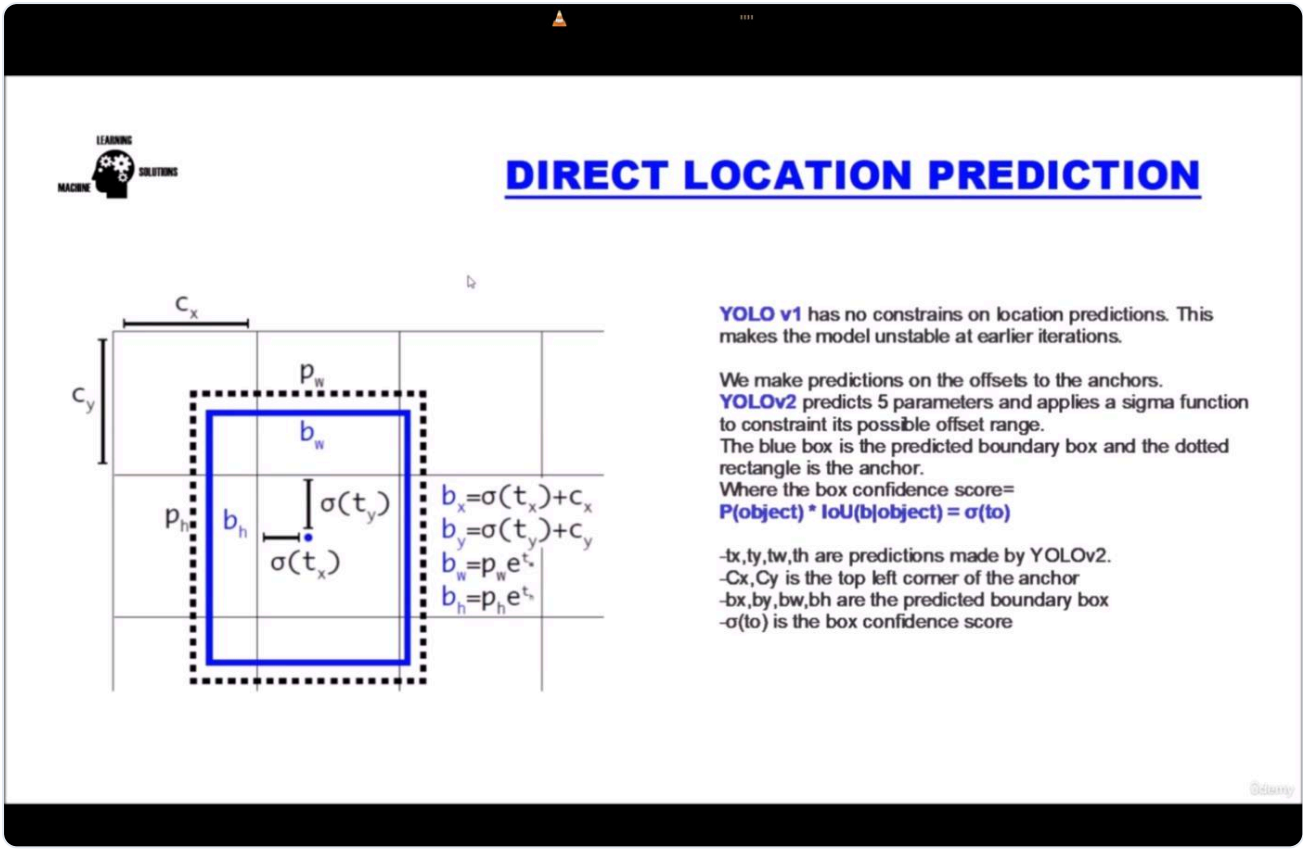


FIGURE 1.10 | YOLO v1 Has No Constrains on Location Predictions. This

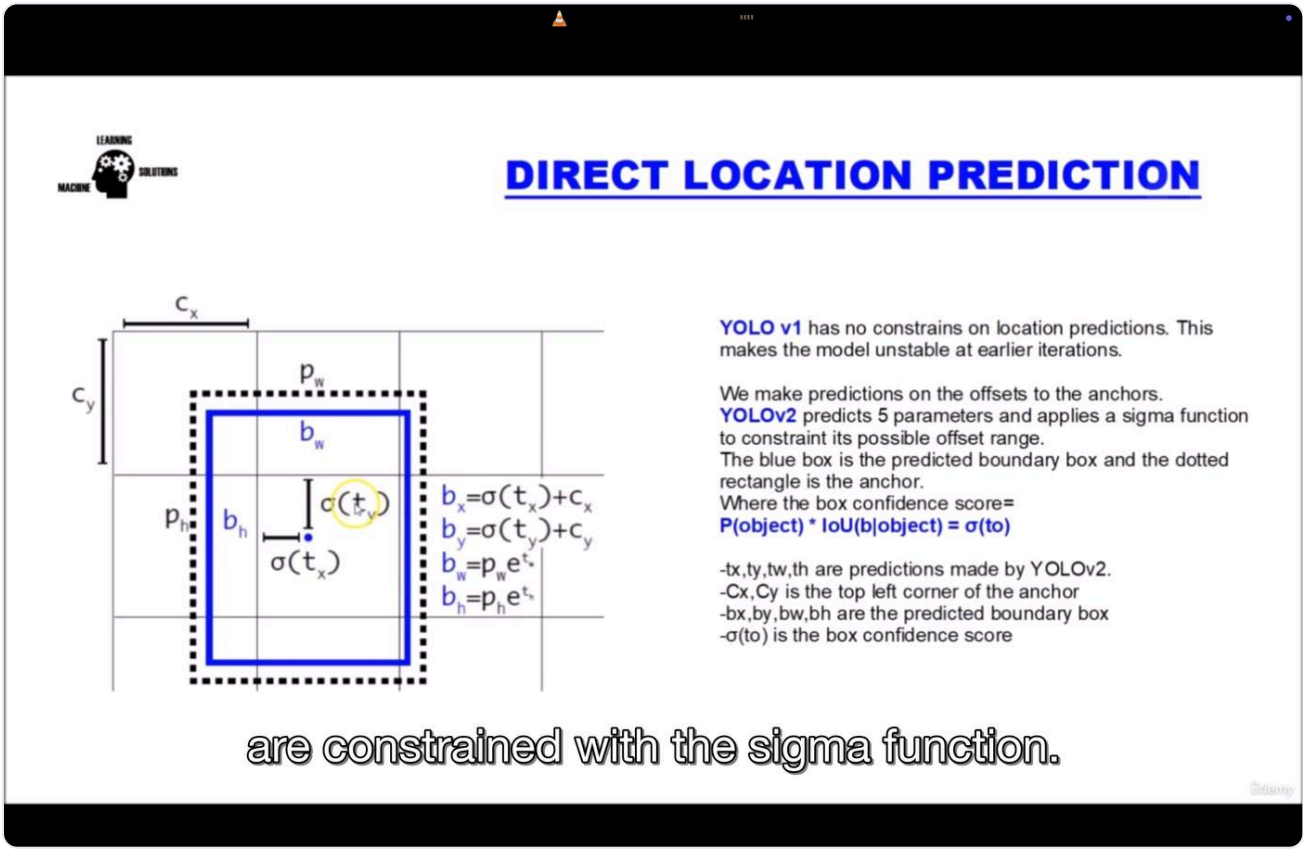


FIGURE 1.11 | YOLO v1 Has No Constrains on Location Predictions. This

And the calculation for the box confidence score is the same like in version 1. It's the probability to find an object multiplied for the IOU metric between the predicted banding box and the grand true box. In this new coordinate system the point of reference is not the center of the banding box, rather the top left corner of the anchor. Now let's see the concept of fine grained features. In YOLO version 1 we have the information of the output feature map to produce detections, but it suffers from detecting small objects because the final resolution is very poor and only can detect large objects. So in order to solve this issue we incorporate feature maps with higher resolution.

So Dalgo-Riden uses a feature map with a size 26 by 26 with higher resolution than the final feature maps. The idea is to add this information with higher resolution with information of the last layer with low resolution. So in order to perform this concatenation we need to change the shape of this feature map. So we convert this 26 by 26 by 512 feature map into a feature map with a size 13 by 13 with 2048 channels. This is because this higher resolution has a double size in height and width. Then we have four times in the number of channels to preserve the volume of the data.

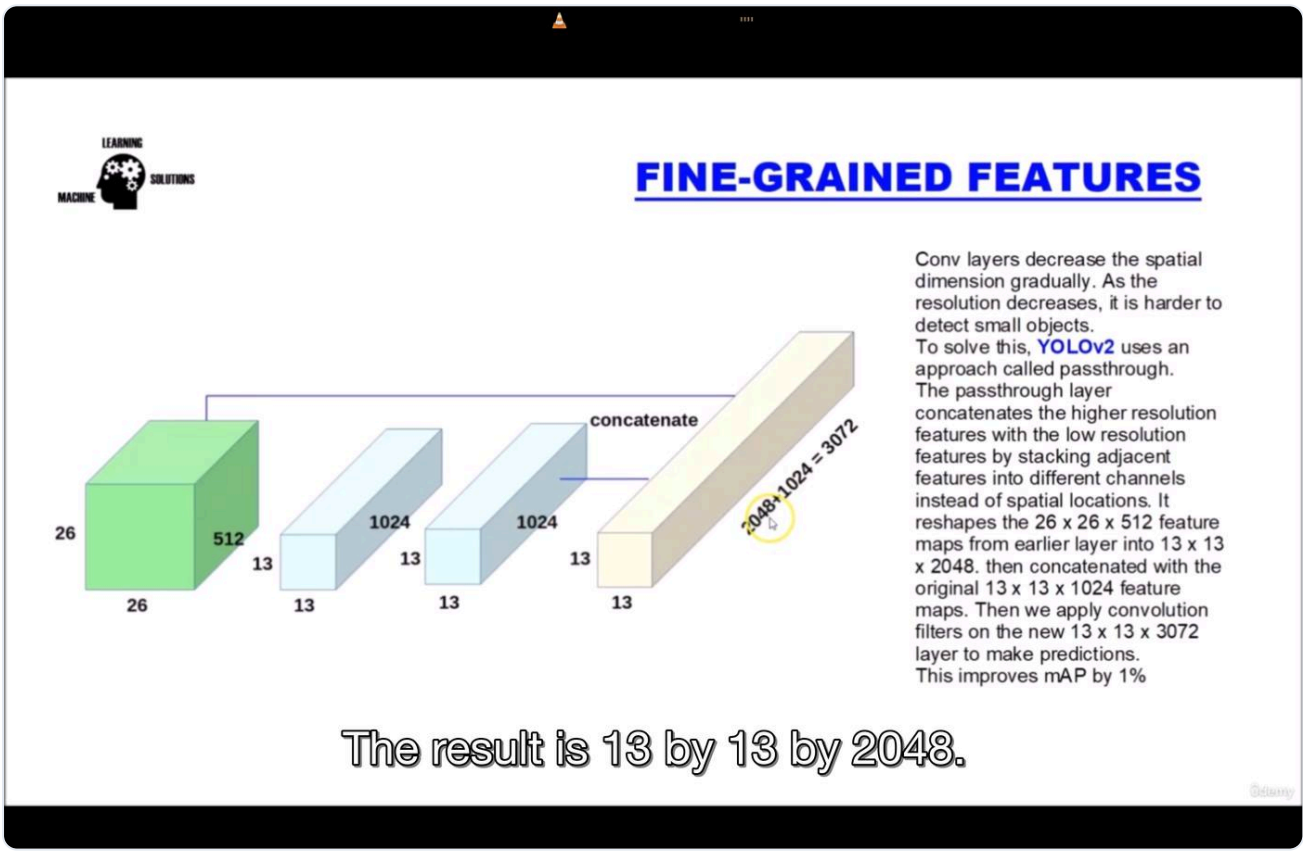
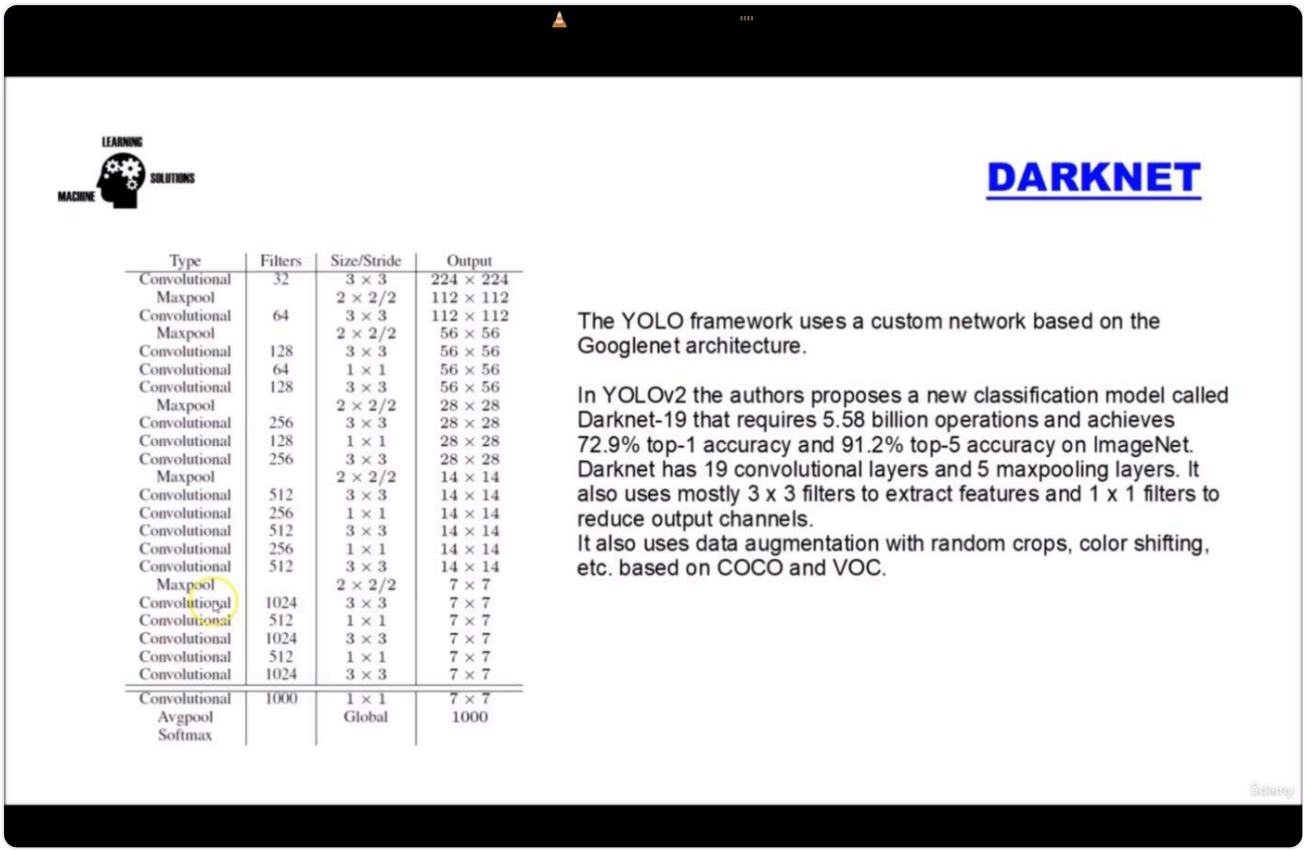


FIGURE 1.12 | Conv Layers Decrease the Spatial

The result is 13 by 13 by 2048. So we add the information of the last layer which is 13 by 13 the same spatial dimension and the number of channels here is 1024. Then if we have the same spatial dimension we can extract the information of the channels summing them in this third dimension producing in total 3072 channels. This kind of connection is also known as a residual connection. You can check it in the ResNet architecture. In YOLO version 2 this approach is called pass-through. So in conclusion the idea is to capture with this higher resolution small objects and with this low resolution large objects. This technique improves the mean average precision by 1%.

YOLO version 2 has two main parts. One part is the feature extractor or backbone and the other part is the layers for object detection. The backbone is new here which is called darknet.



Type	Filters	Size/Stride	Output
Convolutional	32	3 × 3	224 × 224
Maxpool		2 × 2/2	112 × 112
Convolutional	64	3 × 3	112 × 112
Maxpool		2 × 2/2	56 × 56
Convolutional	128	3 × 3	56 × 56
Convolutional	64	1 × 1	56 × 56
Convolutional	128	3 × 3	56 × 56
Maxpool		2 × 2/2	28 × 28
Convolutional	256	3 × 3	28 × 28
Convolutional	128	1 × 1	28 × 28
Convolutional	256	3 × 3	28 × 28
Maxpool		2 × 2/2	14 × 14
Convolutional	512	3 × 3	14 × 14
Convolutional	256	1 × 1	14 × 14
Convolutional	512	3 × 3	14 × 14
Convolutional	256	1 × 1	14 × 14
Convolutional	512	3 × 3	14 × 14
Maxpool		2 × 2/2	7 × 7
Convolutional	1024	3 × 3	7 × 7
Convolutional	512	1 × 1	7 × 7
Convolutional	1024	3 × 3	7 × 7
Convolutional	512	1 × 1	7 × 7
Convolutional	1024	3 × 3	7 × 7
Convolutional	1000	1 × 1	7 × 7
Avgpool		Global	1000
Softmax			

FIGURE 1.13

This new feature extractor or backbone produces better results than VGG16 and the original backbone in version 1. The architecture is here. This is darknet 19 because it uses 19 convolutional layers with 6 max pooling layers. The convolutional filters are 3 by 3 for feature extraction and 1 by 1 for changing the channel size. This architecture is used for training in classification. After that we remove the final layer and train the network for object detection. So let's see the components of the training. Here we have two training steps. We need to perform training for classification using darknet 19 trained on ImageNet which has 1000 classes. The initial resolution is 224 by 200 by 24 and then doubles the resolution for 10 epochs.

For the training time this algorithm uses data augmentation using random crops, rotation and so on. After the training on classification we need to train the detection layers.

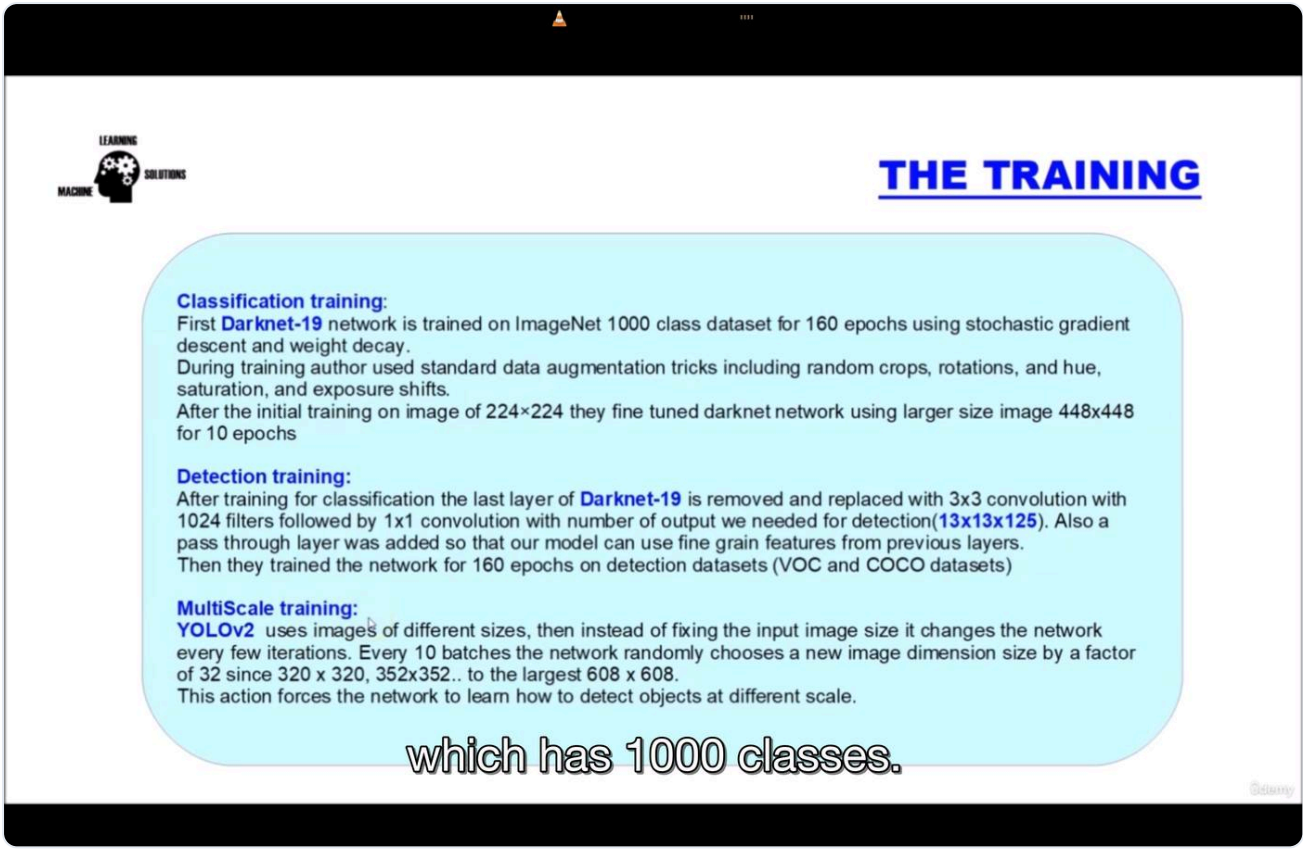


FIGURE 1.14 | Classification Training

The final layer of darknet 19 is removed and replaced with 3 by 3 convolutional layer with 1024 filters. And after that 1 by 1 convolutional filter changing the number of the channel dimension in order to produce this feature map 13 by 13 by 125. As we have seen this number represents the information of the banding boxes. Finally YOLO version 2 uses a multiscale training approach changing the input dimension every 10 batches using image sizes by a factor of 32.

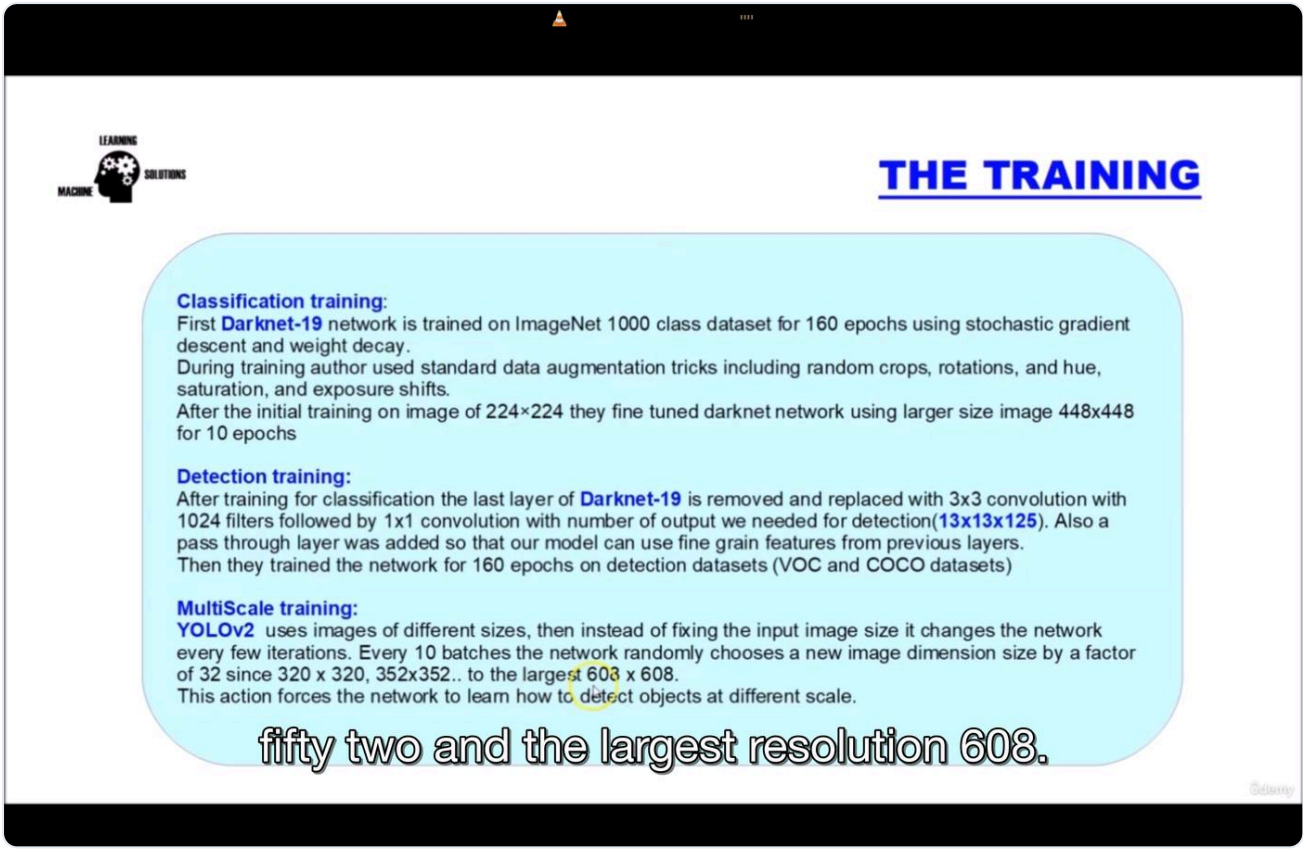


FIGURE 1.15 | Classification Training

For instance 320 by 352 and the largest resolution 608. With this technique we are forcing the network to detect objects at different scales. Finally let's see the accuracy and speed results in a comparison chart. YOLO version 2 has different competitors such as SSD and YOLO version 1. We have in horizontal the speed in frames per second and in vertical the mean average precision. For YOLO version 2 the base 1 outperforms in mean average precision to SSD 300 and the initial version of YOLO. Even more the results are better than faster R-CNN, SSD 512 and faster R-CNN plus ResNet. And in terms of speed there are several architectures with better speeds than the other competitors achieving good results in the mean average precision.

This evaluation is for Pascal VOC 2007 dataset. If you need more information about this YOLO version 2 algorithm you can check in detail the original paper. That's That's all for now. See you in the next class. Bye bye.

LECTURE 47

YOLO Model and Left Corner

https://pjreddie.com/darknet/yolo/ https://github.com/thtrieu/darkflow

Hello everyone, welcome back to the course. In this lesson I'm going to perform the implementation of YOLO for real-time object detection. I'm going to use the version 2 of YOLO. This is the webpage.

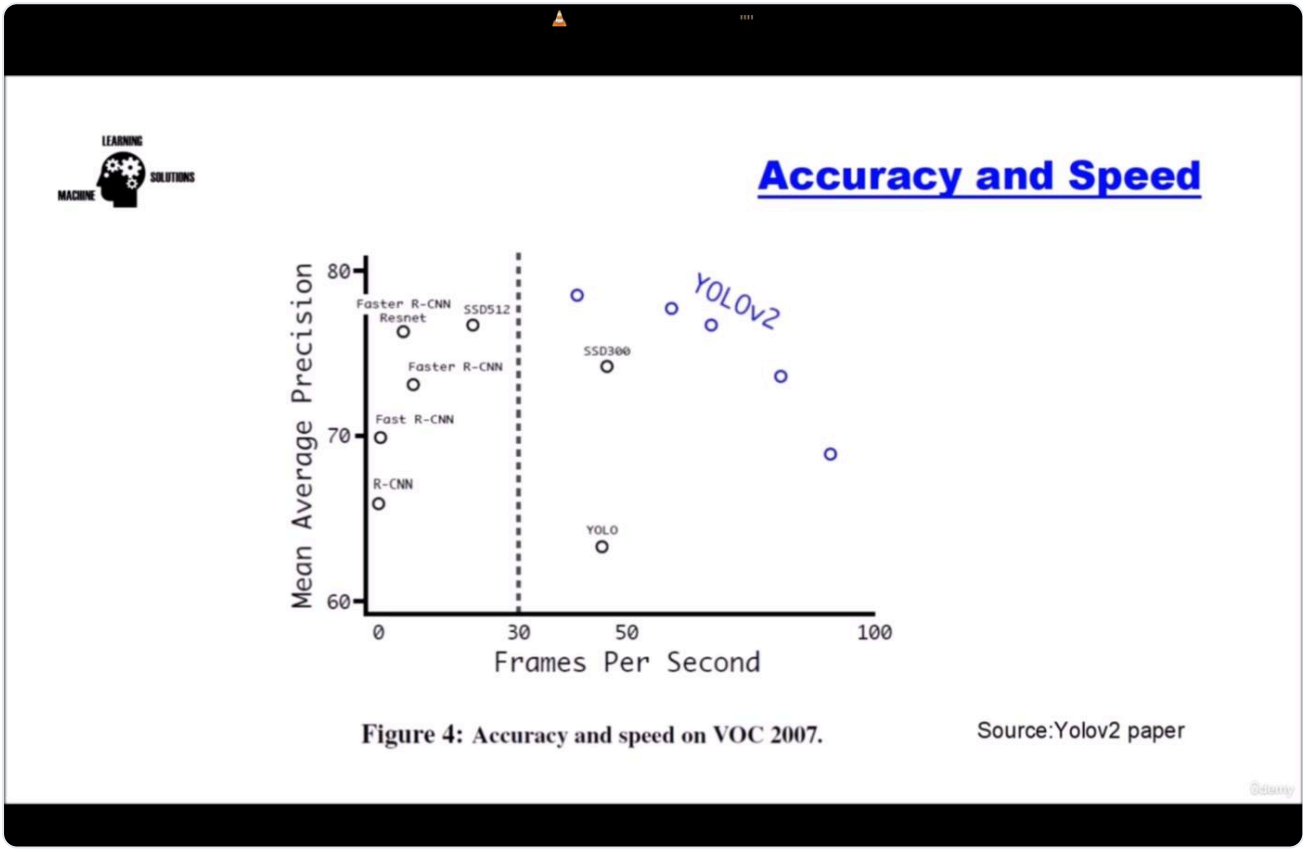


FIGURE 2.1

This algorithm performs the real-time object detection very fast, between 40 to 40 frames per second on VOC 2007. The MAP score is 78.6%. It's a high accuracy and 48.1% of accuracy in COCO dataset. Okay, here are the models. CFD is to download the model and weights is to download the pre-trained weights. You can find different models trained in this dataset VOC 2007 + 2012. There are other models trained in COCO dataset. This is the mean average precision score. All values are very good and the frames per second for each model. After that, you can find how it works, the explanation, what is new in version 2, detection using a pre-trained model, that is the goal in this lesson.

There is an example, a dog, a bicycle, and a car, very well identified. There is a threshold that is a parameter. If you set this threshold to zero, you will find all possible labels. Different kinds of labels are shown, but we need to set up this threshold in a specific value. This threshold is a confidence that the fault value is 0.25 or higher, tiny YOLO. It's very light and very fast, but it has less

accuracy than the normal YOLO model. For implementing real-time detection on a webcam, I'm going to perform this in the next lesson. If you want to access to the old YOLO site, that is the version 1, this is the link. Ok, I'm going to implement YOLO for object detection on images.

Ok, the first thing that I need to do is to download the YOLO model. I need to go to this webpage in GitHub. It's very easy, you need to select clone or download. If you have git, you can clone the address and paste it in a specific folder or you can download in a zip file. You have two options. This is the model, the complete model. After that, you will have something like this. This is a folder containing the YOLO model version 2 with the name dark-flow. I'm placing this model into this root. So, checking this model, you will find this folder "cfg" that contains all models. This is a default extension for different models in YOLO. To run YOLO, you will need not only the models, but the weights.

If you go here on this bin folder, you will find all weights. You will need to locate all weights into this folder, the bin folder. Now, you need to create the new file into this main folder dark-flow. For instance, this is the file of this lesson. Let's jump to Jupyter notebook. This is the file in Jupyter notebook of the detection on images with YOLO. Let's start in a new empty notebook. The first step that I need to do is to import the libraries. CB2 is OpenCV and this is new. Remember, this file is contained into the dark folder. So, I can import the YOLO model. I'm using darkflow.net.build. Import tfnet. This is the object that I need to access to the YOLO model.

This function is an API of the YOLO model. After that, I need my plotlib. I'm using this flag "svg" and tensorflow. I'm running this code in a GPU kernel with the version 1.10 in tensorflow. So, I need some instructions to deal with the GPU kernel. I need to create an object using tensorflow, the function config.proto and this flag "25, the graphic card". After that, I need to configure this flag in true, allowing the growth in the memory. I'm creating a new session using this parameter config equal to this previous variable. And into this session, I'm specifying the options that I need to perform YOLO. I'm specifying with these parameter options, there is a dictionary, the model, where is located the model, I'm specifying the folder "cfg" and this model.

After that, the root for the weights, that is, on this folder, the bin folder. This file contains the weights, the threshold in 0.1 and the GPU option 1.0. This number represents the use of 100% of the memory in the GPU card. You can set up this parameter in another value. Okay, I have the options, so I'm creating an instance of this function tf.net. In this instance, I'm specifying these options. So now we can run YOLO using the GPU card. It's very good. This is the structure of the YOLO model. As you can see, there are many convolutional layers, max pooling layers. I also have batch normalization, the leaky reload function for activation. The last layer has a linear activation function.

This is the output volume. GPU mode with 100% usage. This is the time in this loading process. Okay, we have loaded the model. Now we need to load the image. For instance, I have an image. I'm going to use OpenCV to load this image and convert it in an array. I'm using OpenCV, the function I am read. I'm specifying a root and the type of the image in color. After that, in OpenCV, I need to make a transformation of the format from RGB to BGR. This is the instruction to make the

transformation. This is a folder. Into this folder, I have different images. I'm going to load this image. Obviously this folder is into the main folder, dark flow. Now I'm going to check the shape of this image.

It's very easy. I need to use the function shape. Okay, I have three channels, 406 pixels in height, 1720 pixels in width. Now I'm going to perform the object detection, making predictions with YOLO. It's very easy. I need to use this variable, that is the previous instance, tfnet, and the function return_predict. I'm specifying this previous variable. The result of this expression is stored into this variable. Very good. Let's see this variable. Okay, it's a list containing dictionaries. The structure of each dictionary has these keys. The label is the class, in this case person. The confidence is the probability to find this class. The top left corner, the bottom right corner of the banding box.

There are the coordinates of X and Y. The next label is also a person, but I have a label handbag with this probability. There is another handbag. Person, person, person. There is a card label with this confidence. Remember I have set the threshold value in 0.1, so all values greater than 0.1 are shown. But if I set this threshold in 0.2, for instance, these values will not appear. Okay, this is the structure of this variable containing all possible objects detected in the picture. Now let's see a graphical representation of the object detection. So I'm using my plot lip and specifying the figure size. I'm browsing into this variable, extracting all results and creating new variables to extract the upper left corner, the bottom right corner from the banding boxes.

So I'm specifying the keys and the values for X and Y. In the case of label, I need to use the key label. So I have the coordinates and the label. After that, I can use OpenCV to draw a rectangle into these coordinates. So I'm using the image, the value of the upper left corner, the value of the bottom right corner, the color in a tuple in RGB format. So I'm specifying a green color and finally the thickness of this line. I'm drawing a rectangle. After that, I can write a text. So I'm specifying the same image, the information of the label that is this variable. I'm going to place this image in the top left corner. This is a type of the font, italic. This is the size of the font.

I'm specifying the color in white where all values are 255 and finally the thickness or line style. I'm drawing into this image a rectangle and I'm adding a text in the upper left corner of this rectangle. Finally, I'm using my plot lip to show this image. Okay, it's very good. All rectangles in green, the labels in white. This is a car. It is not a bird. There are many persons with a rectangle and the corresponding labels. So as you can see, there are many rectangles, but if I set the threshold in 0.2 or 0.3, many of the rectangles will disappear. Finally, I'm going to perform the same code, but applying on many images. So I'm going to perform object detection for all of these images into this folder.

Okay, I need always to browse into the folders. I need to specify the root. With this variable, I have the folder that I want to access. After that, I need to create a path. This is a path, os.pad.join and the name of this folder. After that, I need to browse into this path. I need to perform a cycle four. I'm extracting all images from this path. Now, I need to use OpenCV to extract the content of the images. This is my variable ing. I'm using the same function as before, but this time I'm specifying the path. The path contains the information of the main folder that I want to browse and the information of each image that is contained into this variable image.

After that, I need to set this parameter for retrieving the color images. The next step is to convert the formats in OpenCV from RGB to VGR. Next, I'm going to perform the predictions. This is the same as before. I'm using this function to predict the object detection on this image. Now to visualize the results, I need to create another cycle four to browse into this variable and extract the information of the labels and the position of the rectangles in the image. This is a cycle four and extracting the information from this variable into this variable with the name R. I'm retrieving into these three variables the position of the upper left corner, the bottom right corner and the label.

After that, I'm drawing a rectangle using the previous image and the position of the rectangle. I'm drawing the rectangle in a green color and specifying a thickness. After that, I need to write a test, in this case in the upper left corner, specifying the image, the label, the type of the font, the size of the font, and the color of the font. Finally, the thickness or line type. Finally, I'm showing the results. Okay, in the left I have a dog, in the right I have a cow. It is not good. In this picture, I have many persons, very well classified, many kites, very well classified. I have the image of a city, many persons are identified, there is a bike label or something like this.

I have also a motorbike. It is a laptop, a book, a dining table. It is not good, but there is a keyboard. It's okay. This is a cell phone and this is a mouse, not. It is a cup of coffee. It's not a mouse, but it's very good. There is another picture of the city, but it is the same picture as before. Okay, I have a dog. This is a cat, but there are another labels such as sofa and dining table. You can set up the threshold in a higher value to remove these banding boxes. The panda bear. Now, the next picture is this. I have a dog, a bicycle, a truck at the back. It's very good. There is a bird, it's okay. Zebra, a geography. There are horses, very well classified.

A person, a horse, a dog. It's very good. These are traffic lights. They are very well identified. Okay, the results are quite good. Okay, the next step is to implement YOLO to perform real-time object detection. So, in the next lesson I'm going to try this on videos and welcome. Until then, enjoy deep learning. See you in the next class. Bye bye.

LECTURE 48

Double Dash and Traffic Light

See section `Flowing the graph using flow` in https://github.com/thtrieu/darkflow.

Now, to detect objects in video, Run the command in terminal as:

```
uv sync

cd darkflow
../.venv/bin/python flow --model cfg/yolo.cfg --load bin/yolo.weights \
  --demo ../cars.mp4 --saveVideo --saveVideoPath ../videos_out/cars-yolov2.mp4 \
  --threshold 0.4 --gpu 0.8
```

Now, to detect objects using webcam, Run the command in 54.Deep-Learning-for-Computer-Vision-with-TensorFlow-2 folder and in terminal as:

```
.venv/bin/python yolov2_od_webcam.py
```

[MUSIC PLAYING] Hello, everyone. Welcome back to the course. In this lesson, I'm going to perform real-time object detection using videos and webcam. As you can see, I'm running YOLO in a video.



FIGURE 3.1

This is New York City.

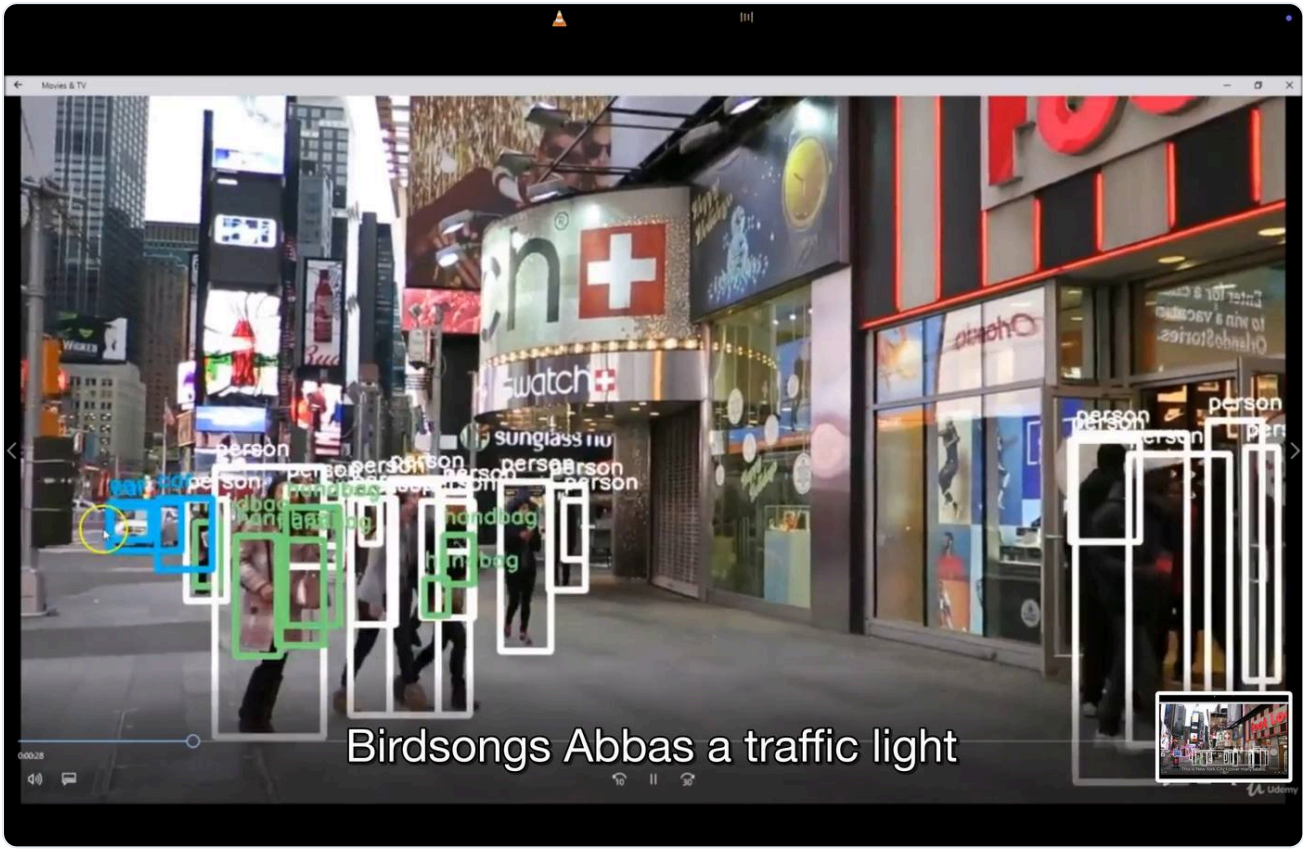


FIGURE 3.2

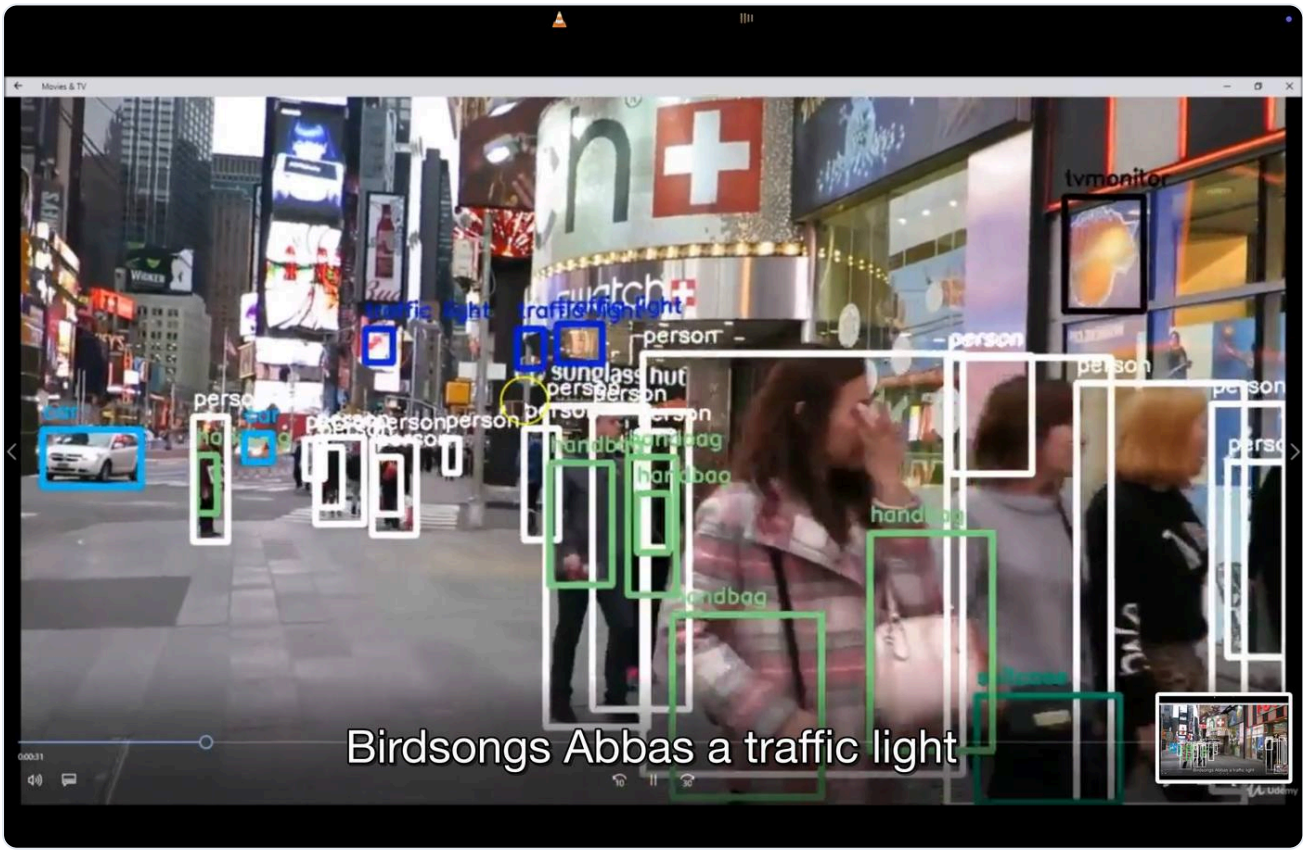


FIGURE 3.3

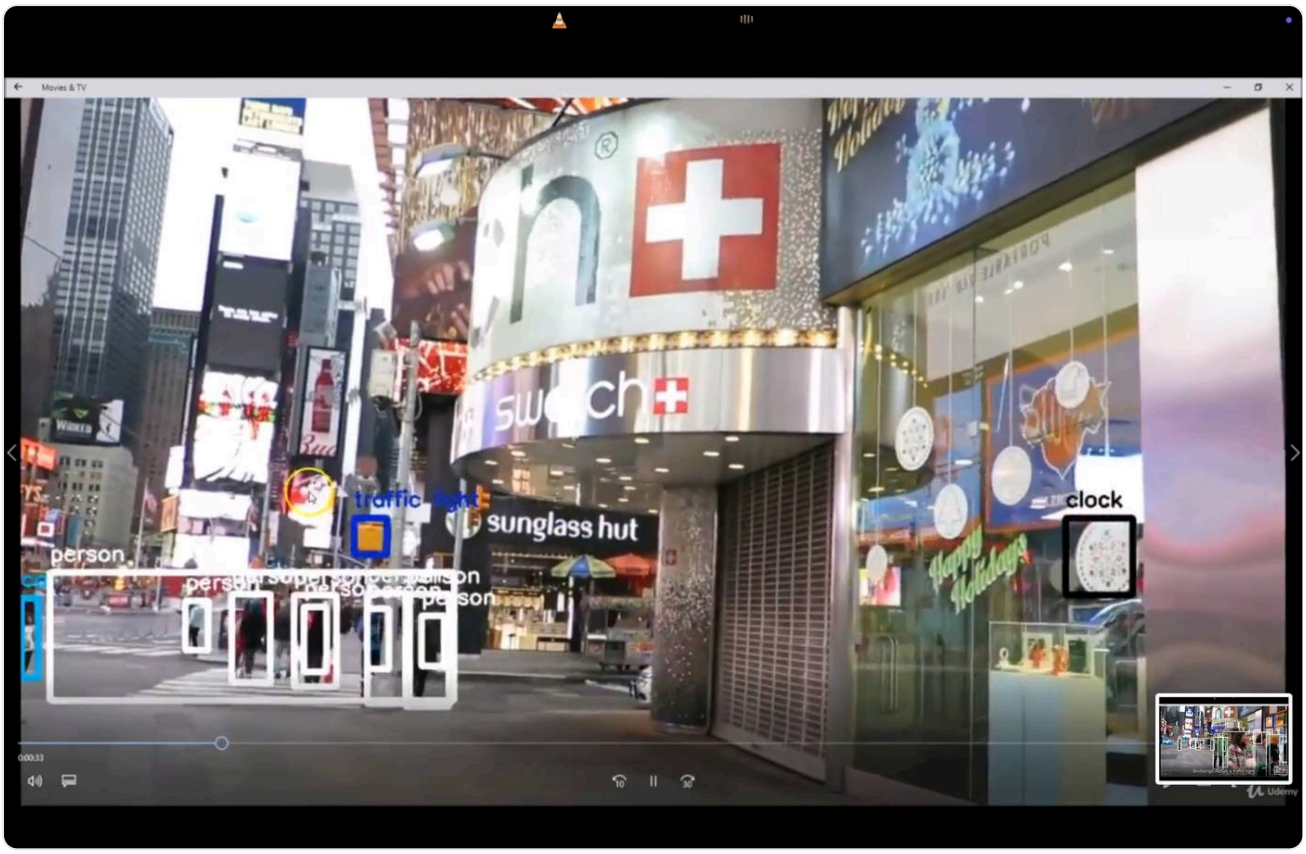


FIGURE 3.4

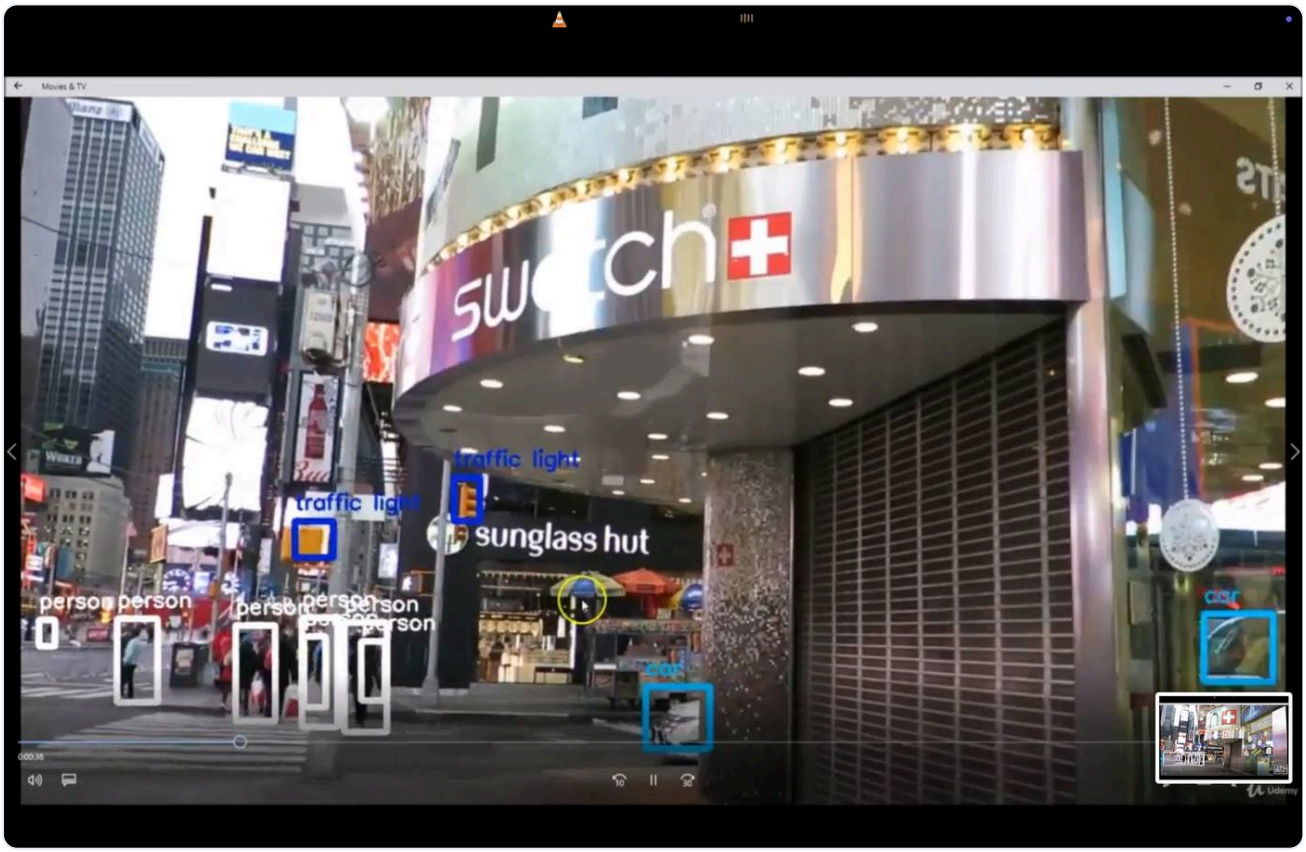


FIGURE 3.5



FIGURE 3.6

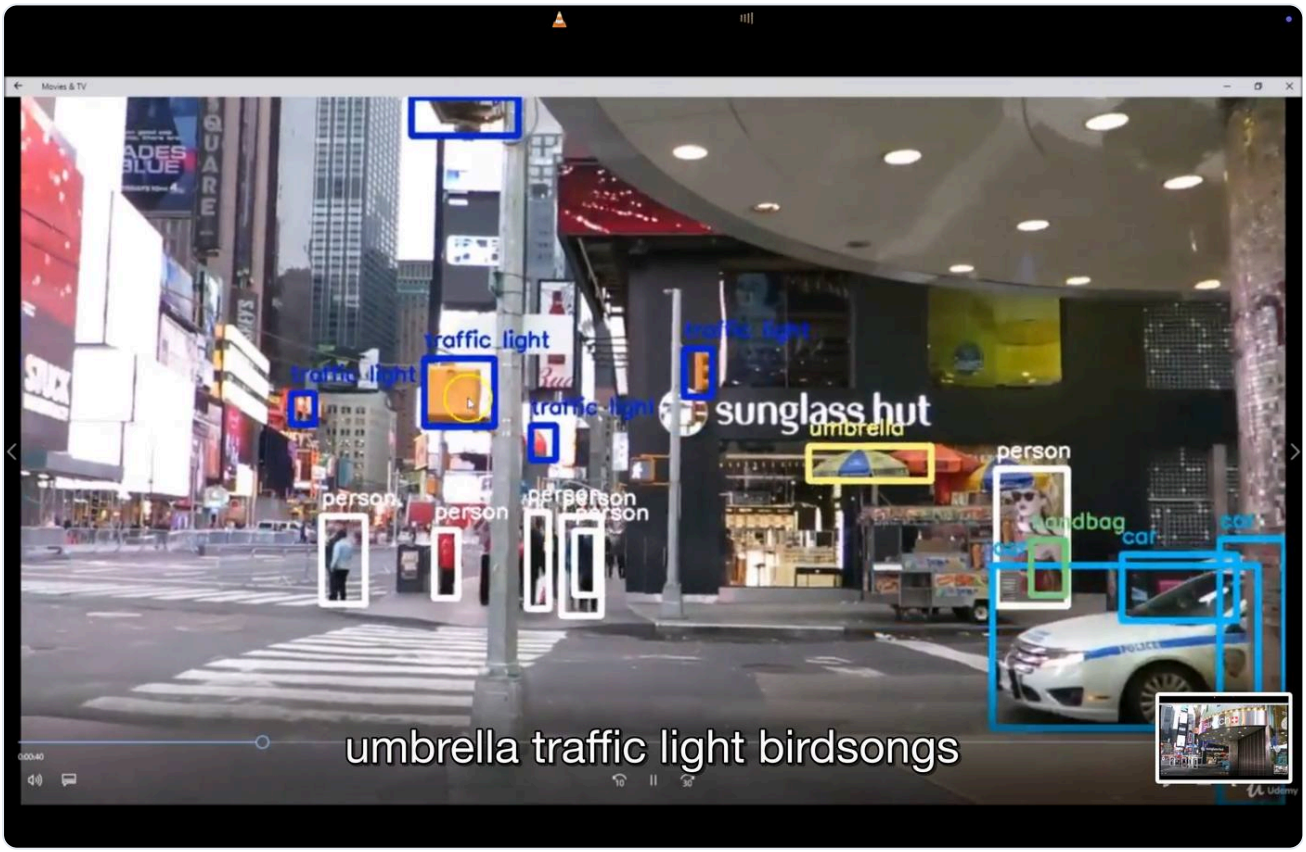


FIGURE 3.7

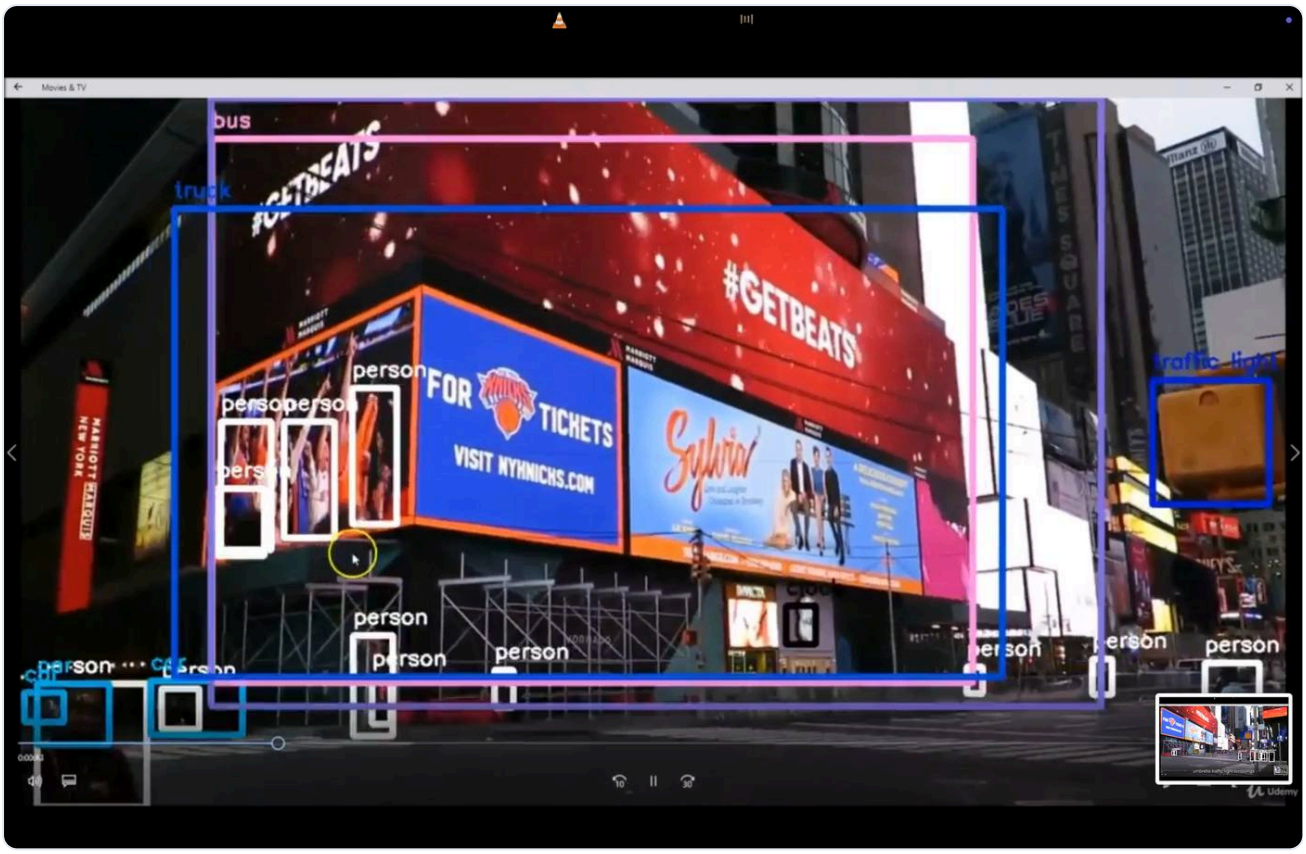


FIGURE 3.8

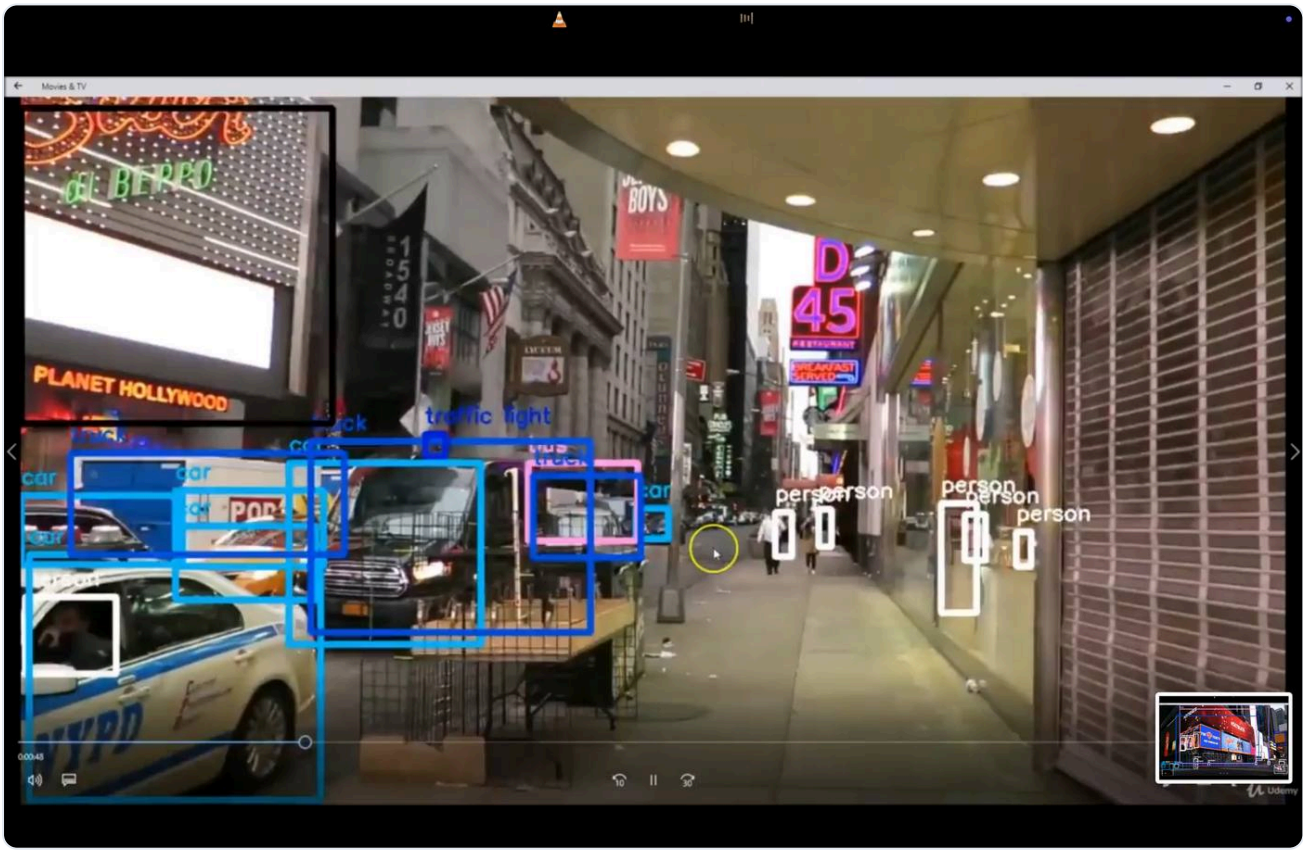


FIGURE 3.9

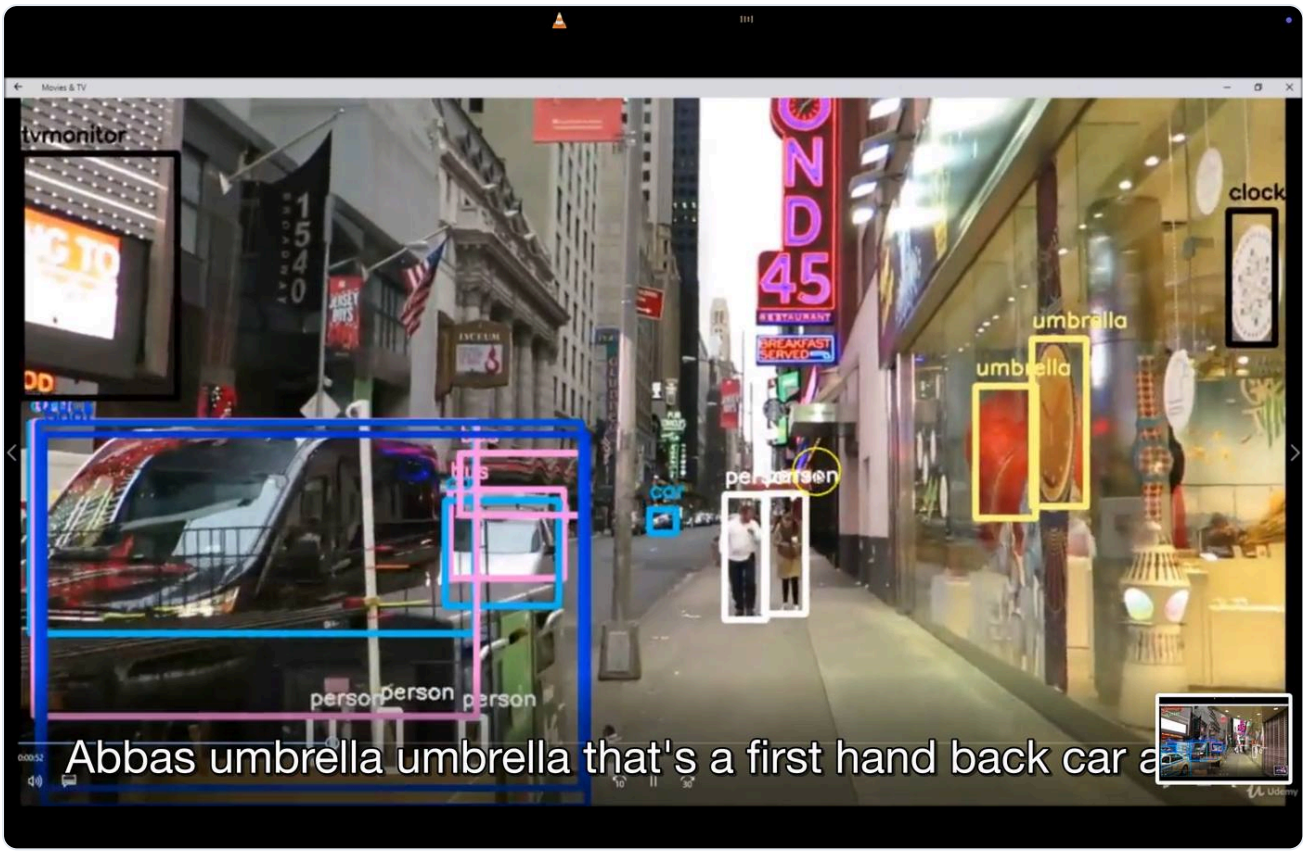


FIGURE 3.10

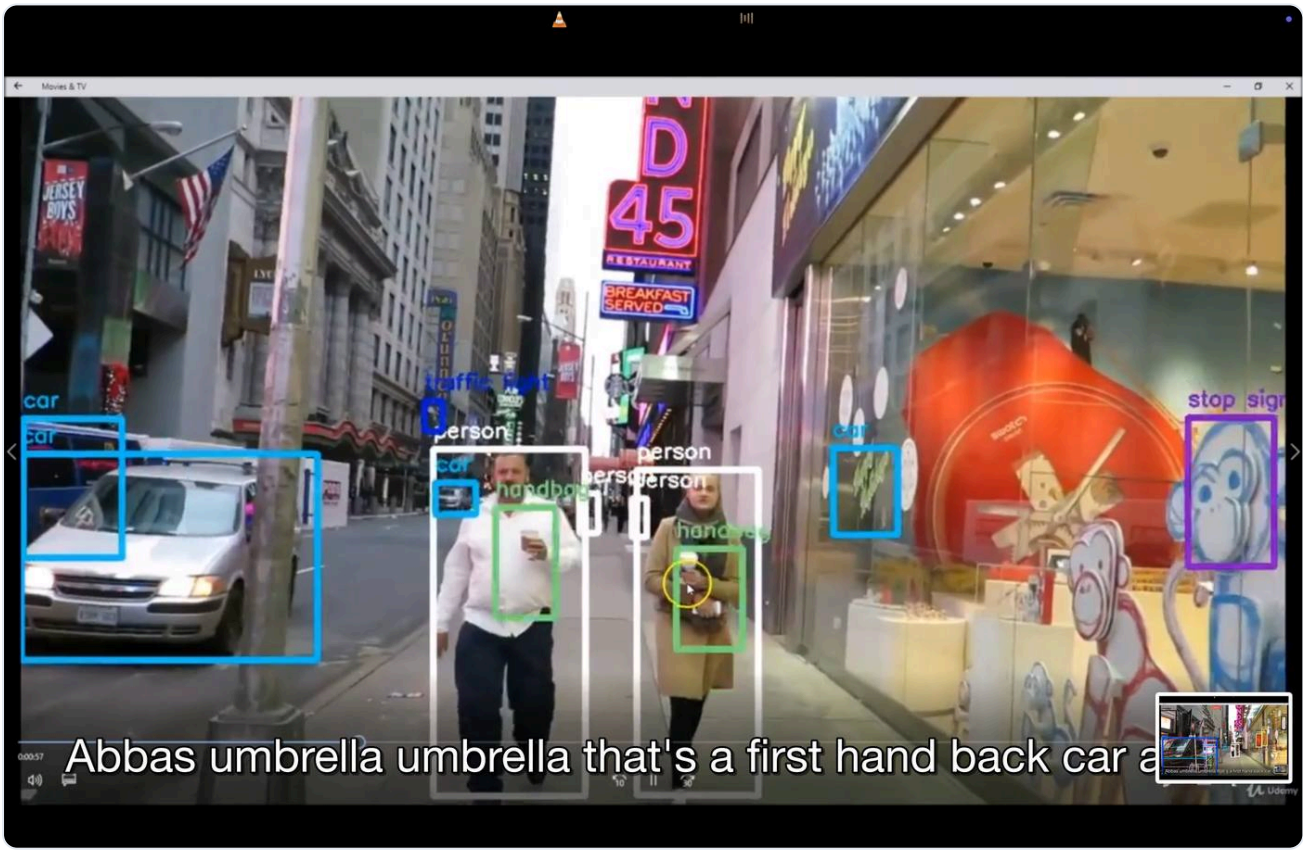


FIGURE 3.11

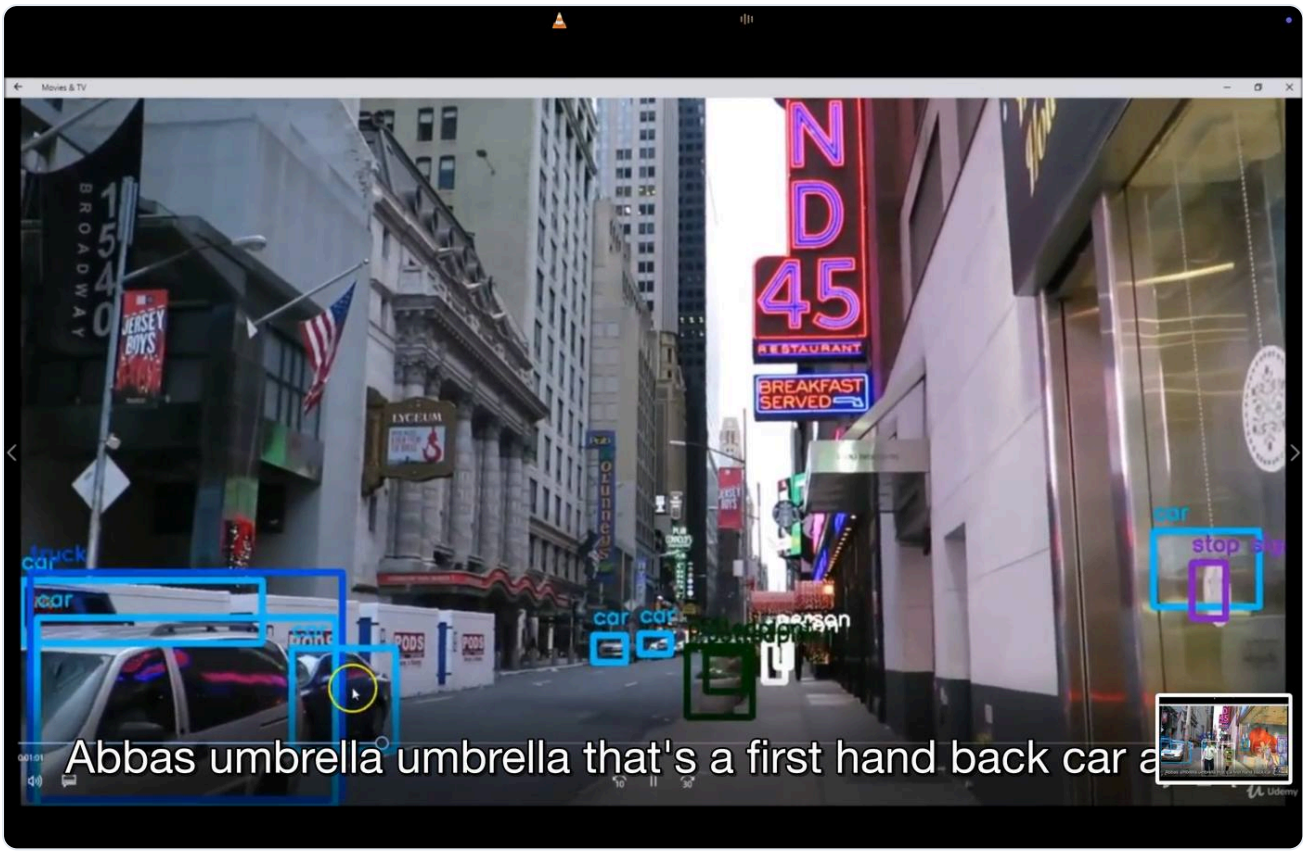


FIGURE 3.12

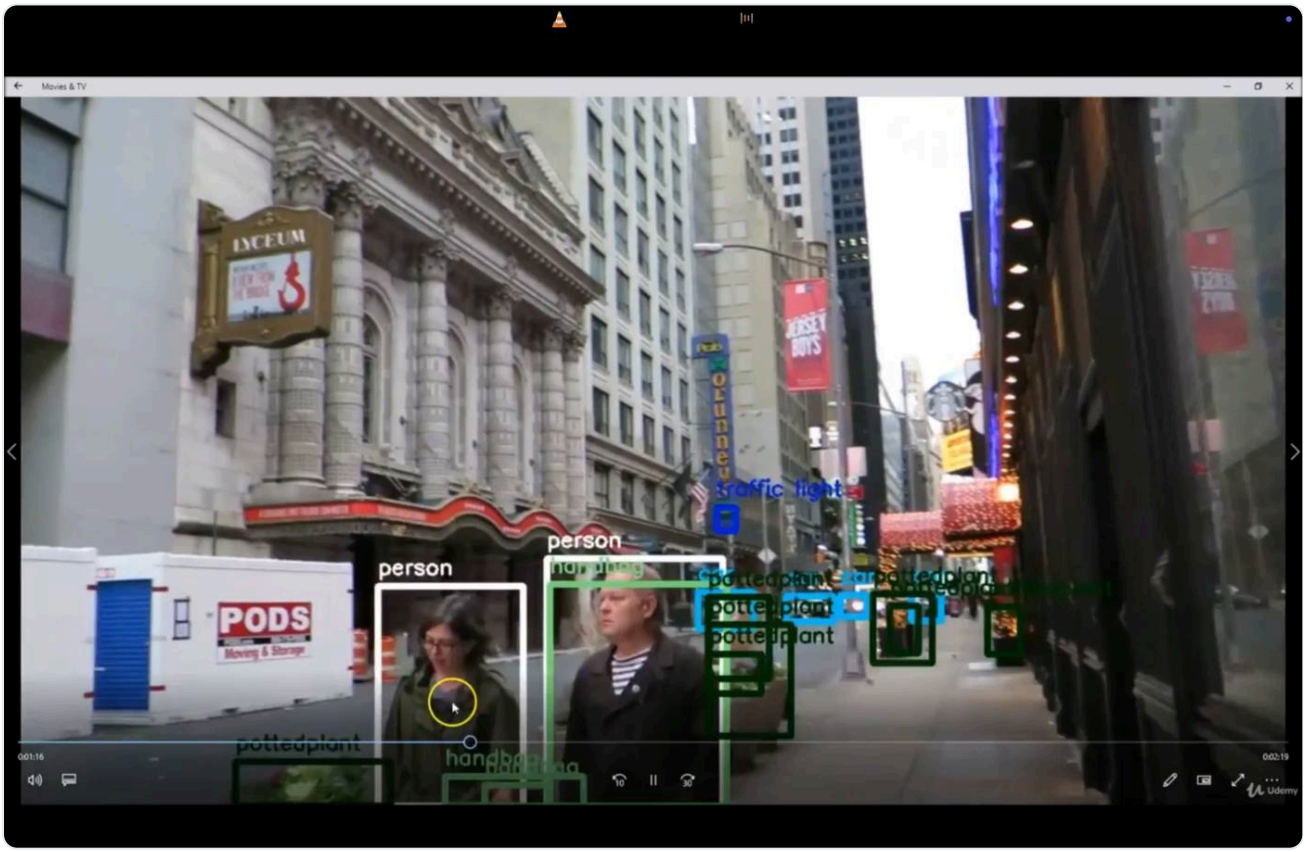


FIGURE 3.13

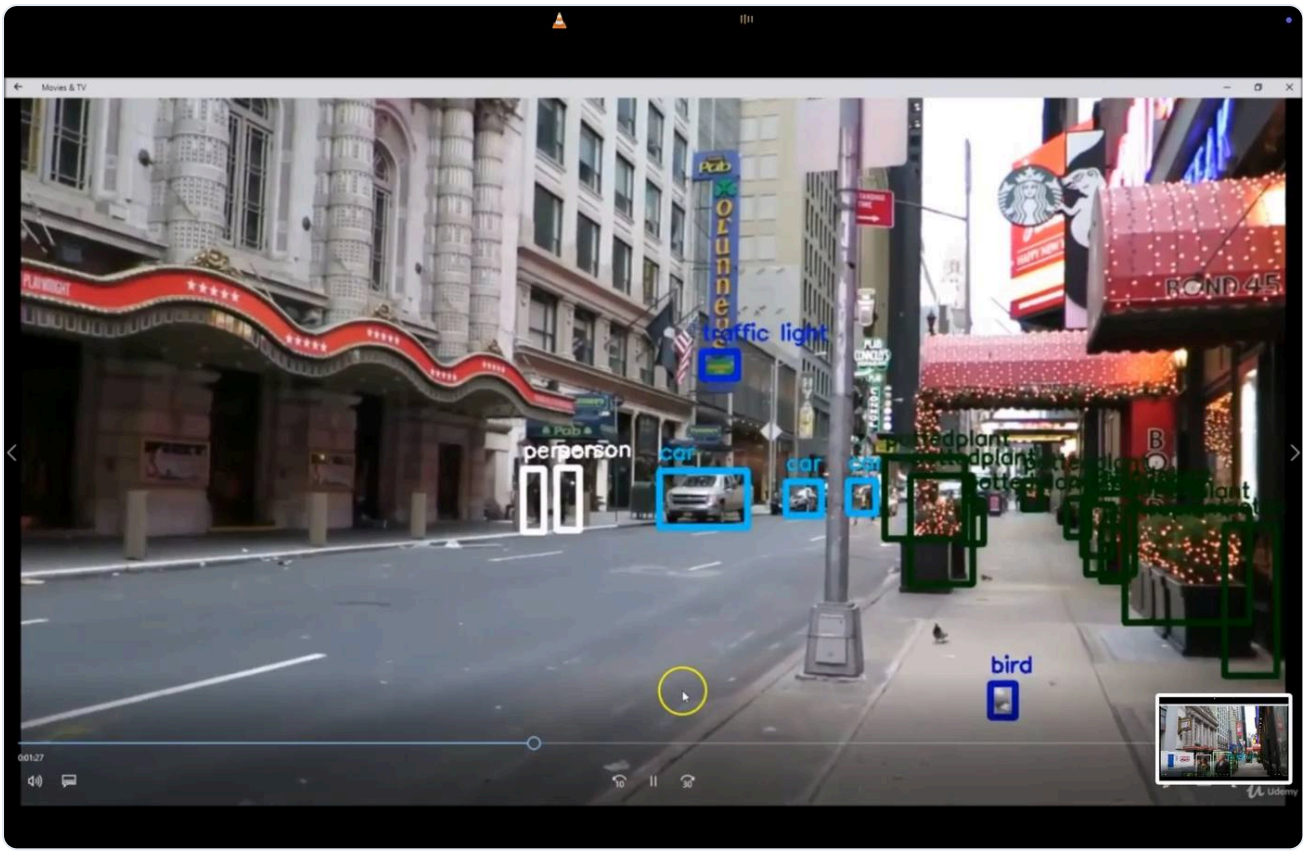


FIGURE 3.14



FIGURE 3.15



FIGURE 3.16

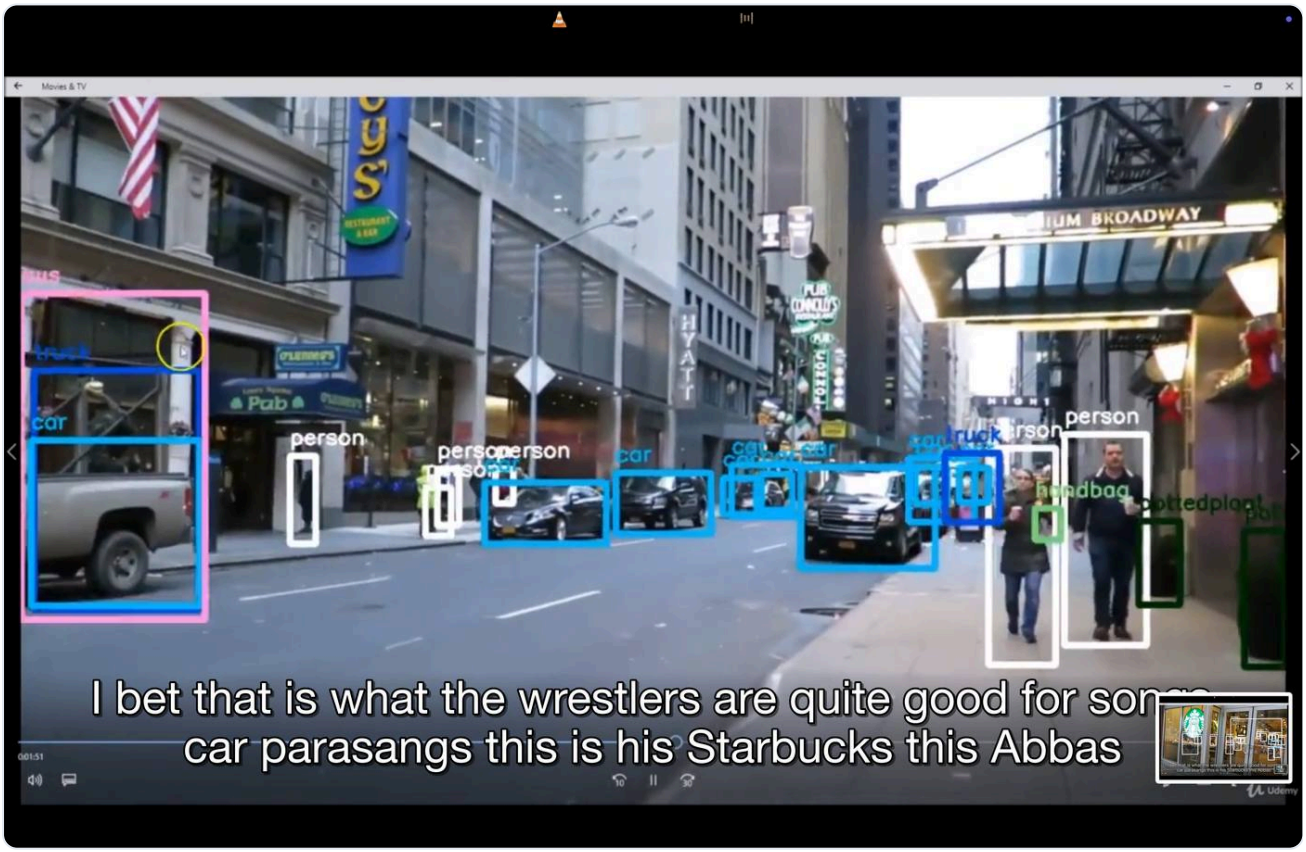


FIGURE 3.17

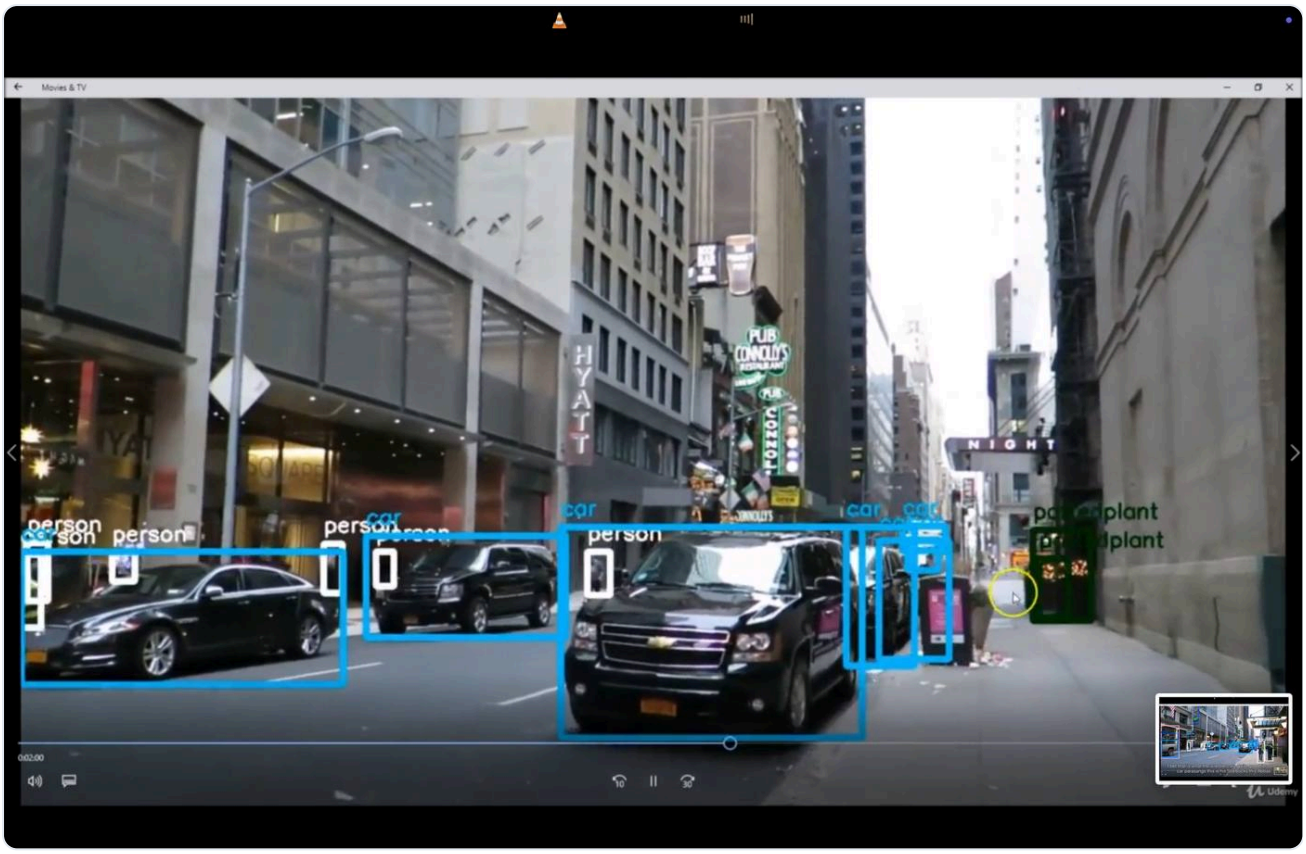


FIGURE 3.18

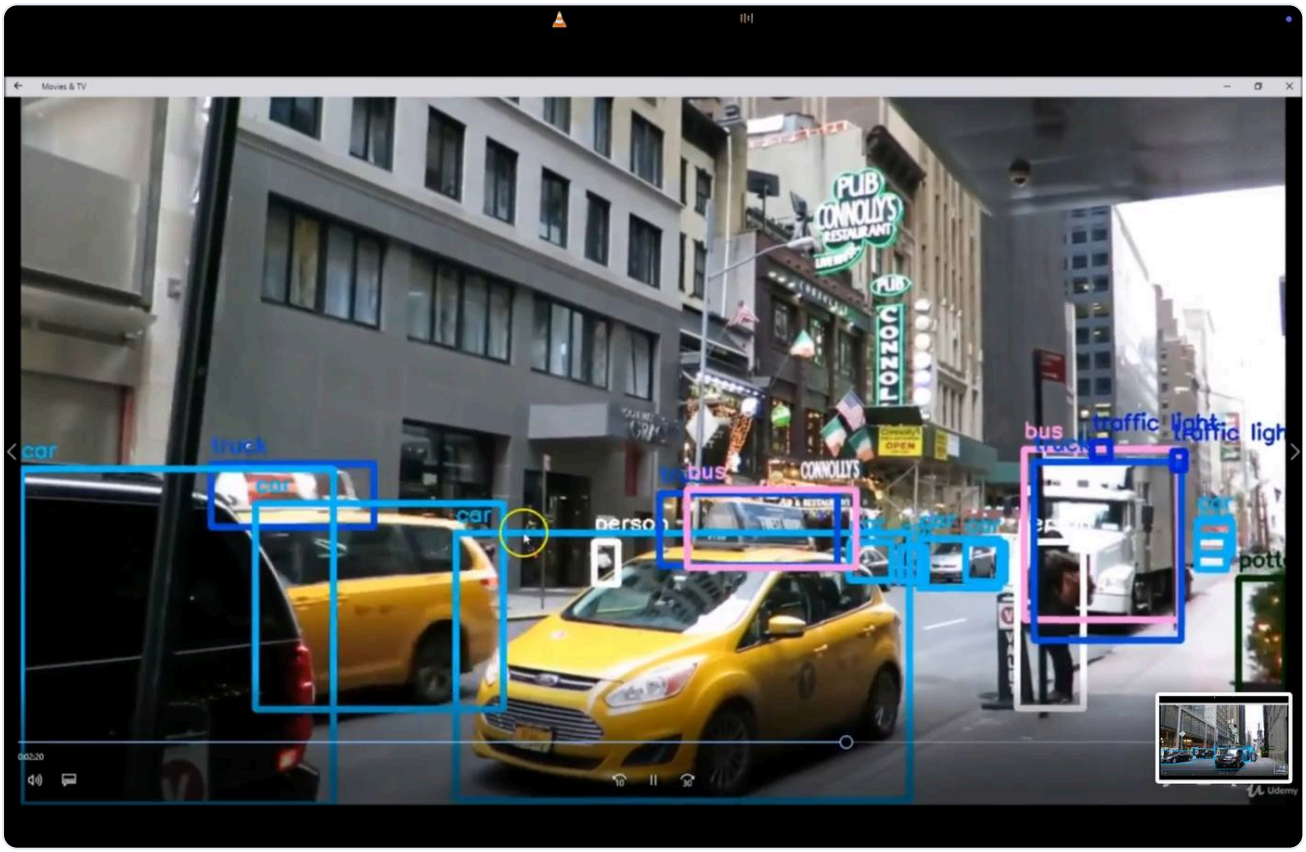


FIGURE 3.19

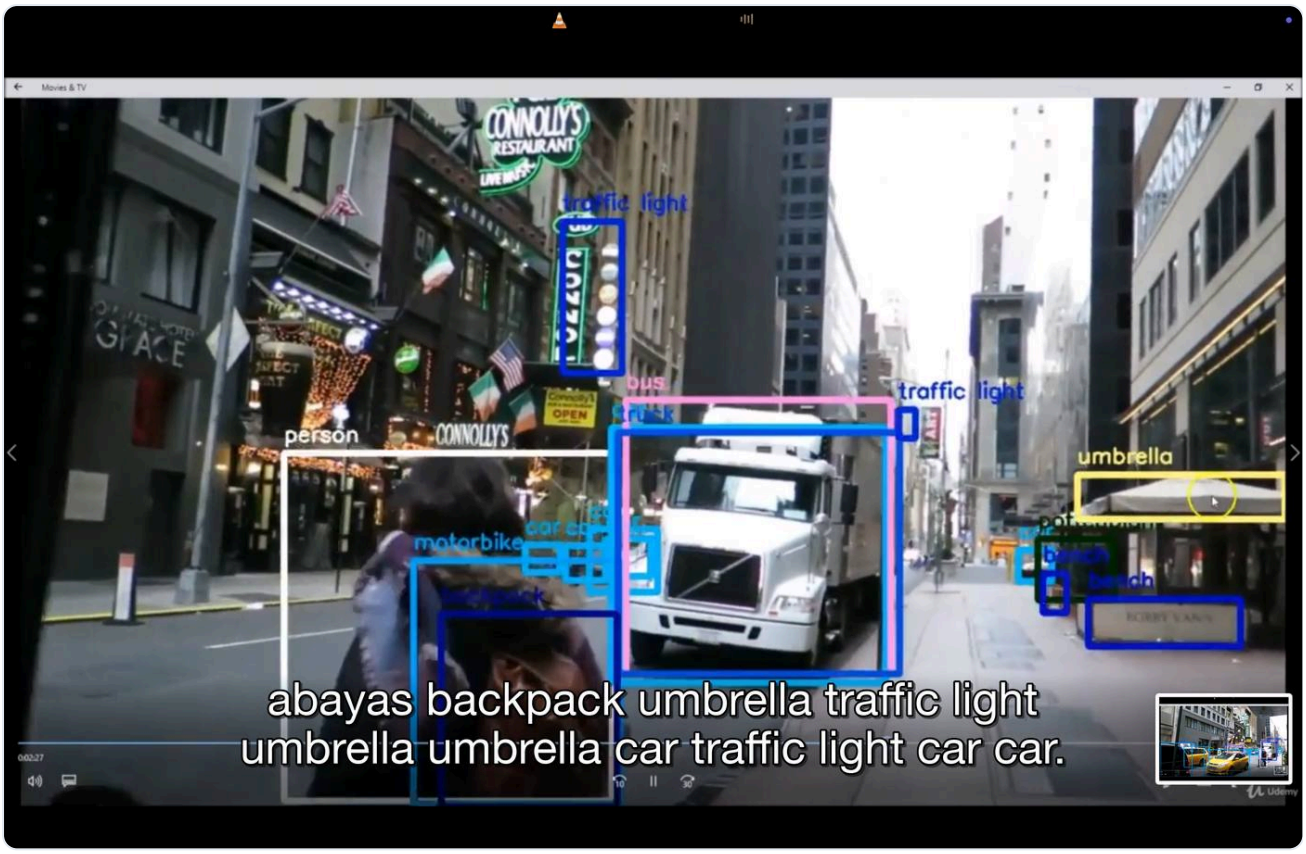


FIGURE 3.20



FIGURE 3.21

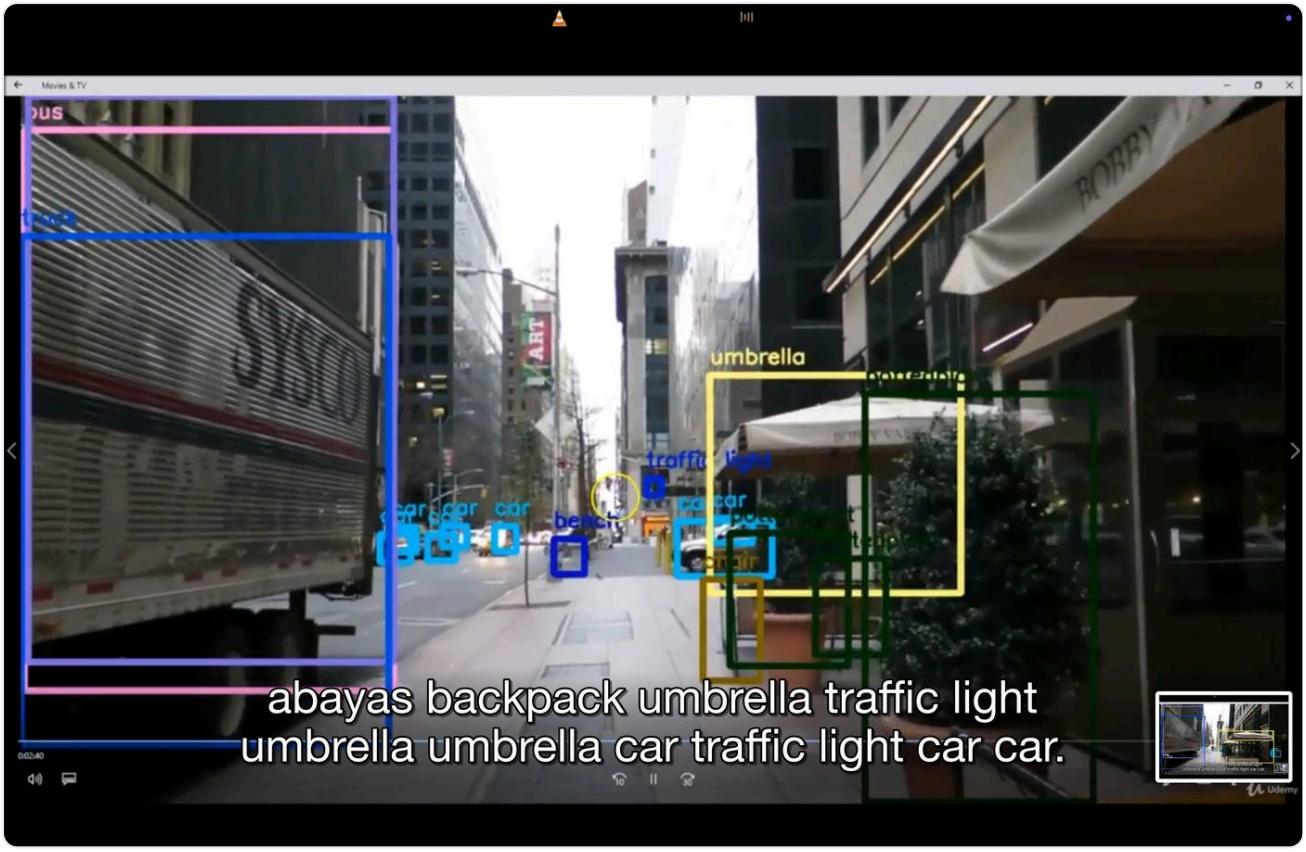


FIGURE 3.22

I have many labels, persons, a bus, a traffic light, umbrella, traffic light, persons, a bus, umbrella, person, person, handbag, a car, a car, different persons, cars, handbag, a bird, very good, the vessels are quite good, persons, car, persons, this is Starbucks, this is a bus, a plant, parking meter, a bus, backpack, umbrella, traffic light, umbrella, umbrella, a car, traffic light, car, car. So this is idea in this lesson.

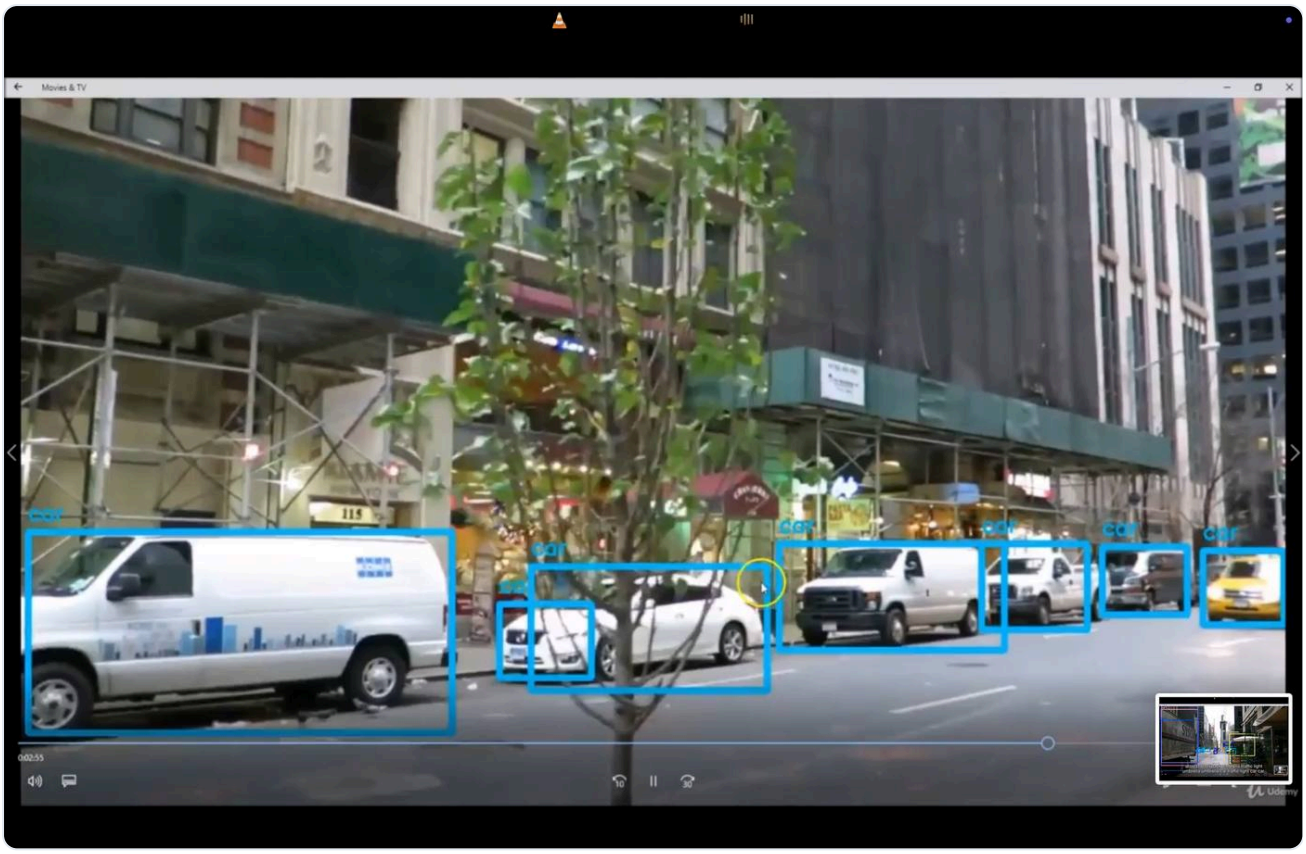


FIGURE 3.23

First, I'm going to start implementing YOLO for object detection in a video.

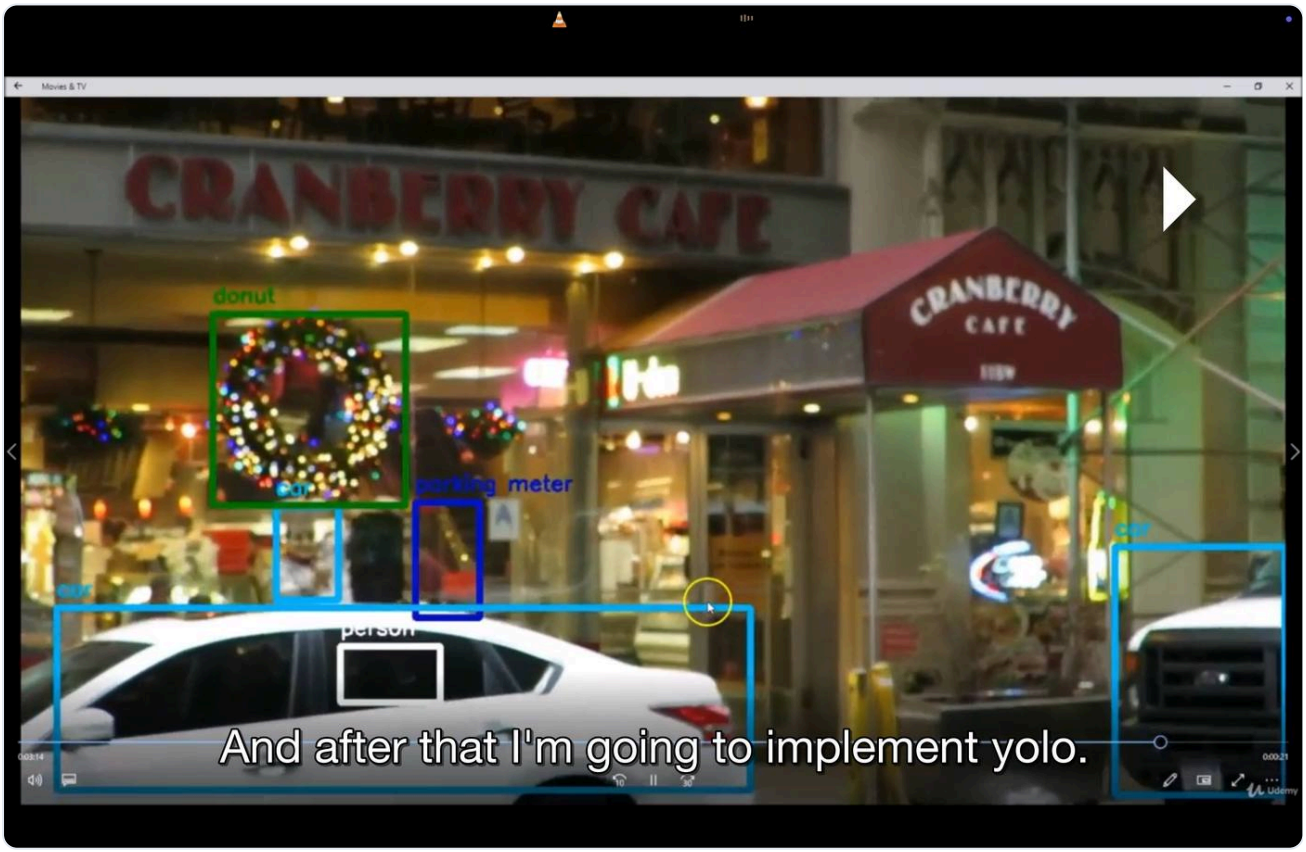


FIGURE 3.24

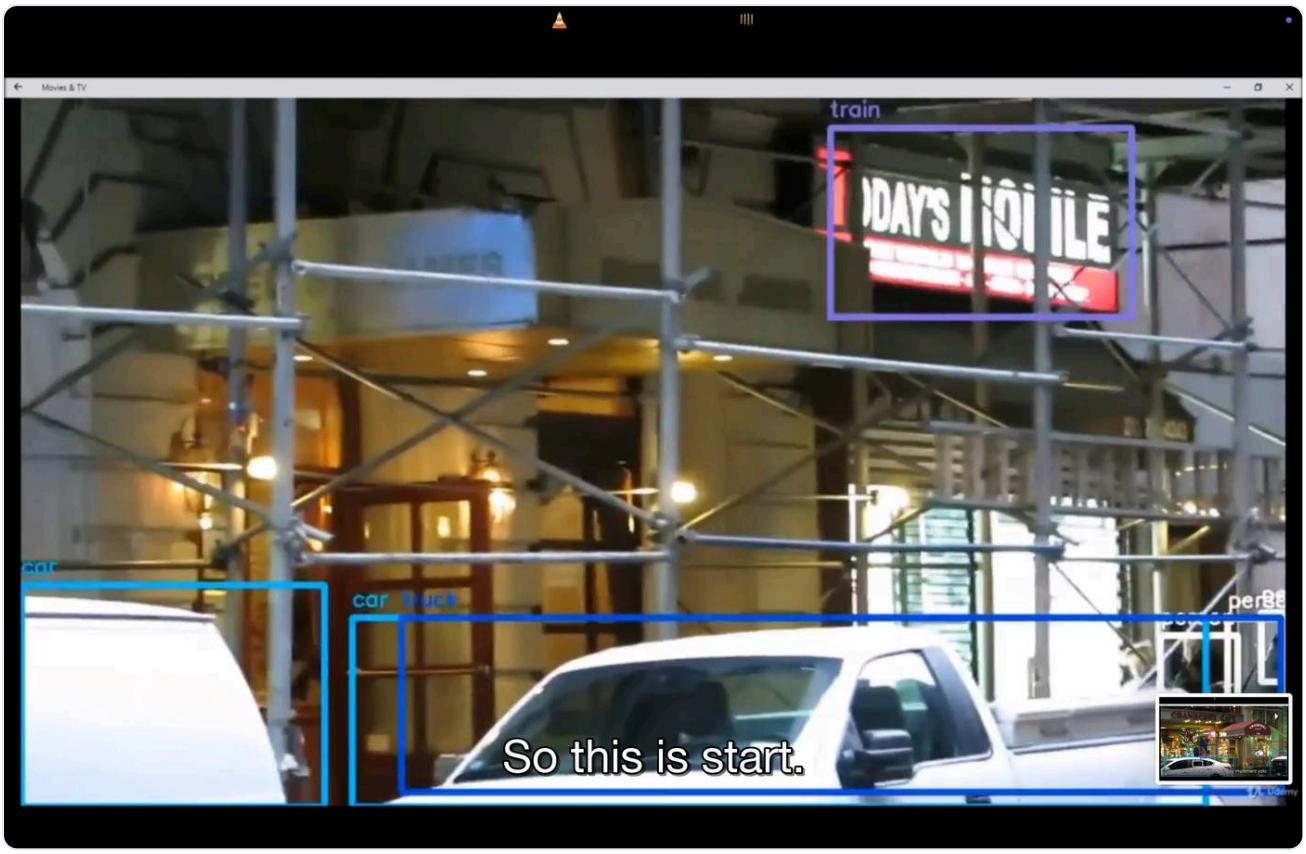


FIGURE 3.25

And after that, I'm going to implement YOLO in a webcam. So let's start. You can find the instructions here in the Darkflow repo. You can check the information for more explanation, dependencies, and many instructions. So let's start with the videos. I need to use this function, Flow. So I'm going to apply YOLO to a video. This is my folder, Darkflow. In this folder, I have this video, Miami. So I'm going to apply YOLO to this video. The first thing that I need to do is to open the Anaconda prompt. This is an instruction. I need to write Python, the function, Flow, to load the YOLO model. Double dash model, I'm specifying the model and the path where it's located.

Double dash load to load the weights. This is the path into the bin folder. Double dash demo, the name of the video file that I want to apply the object detection. Double dash tape video, I'm going to record the new video performing the object detection. Double dash GPU, I'm applying 80% of the total memory to avoid any issue. So I can run this line. The model is loading. I'm using 80% of the total memory. This is a GPU card, GTX 1070. I'm running with this speed. This speed is in frames per second, 16 frames per second, 17. OK, my new file is created. You need some time to process the entire file. And that is all. Now we can check it out, the new file. This is a new file, video.abi.

So let's see it. This is a file in Miami City. I have cars, persons, cars, a parking lot, persons, umbrella, traffic light, stop sign, umbrella, car, truck, umbrella. So as you can see, it's very good. You can try with different videos and different models of YOLO. You only need to take care about the match between the model and its corresponding weights. Each model has its own weights. Now I'm going to perform the object detection in a webcam. To do that, I'm going to create a new file in

Subline Text. Let's start. I have a new file. This file is created into the DartFlow folder. The first state that I need to do is to import the libraries. I'm importing CV2, that is, OpenCV, the same object from YOLO, TFNet, NumPy, Time, and TensorFlow.

After that, I need to set up the session in TensorFlow for the GPU card. These instructions are the same as the past lesson. I'm allowing the growth in the memory with this flag in true. I'm specifying my GPU card with this first statement. I'm creating a session using this config variable. After that, I need to create a dictionary with the options in YOLO. The first key is the model. I'm specifying the root and the name of the model that I want to load. The second key is load. I need to specify the root and the weights that I need to load, the threshold, in this case, 20%, and the GPU option in 100%. I'm going to use the total size of the memory of my GPU card.

Finally, I need to create a variable that is an instance of this function, TFNet, using the options variable. That is solved for configuring the session. Now I'm going to create a new variable to represent different colors for all classes. I'm creating this variable colors. That is a list that contains a tuple. I'm creating three random numbers for red, green, and blue. These three numbers represent a color. So in total, I'm creating 10 colors. Now I'm going to capture the image from the webcam. I need to use video capture, open CV, and here I need to specify the index for the webcam. Zero is the default index. In my case, I need to use 1, that is a second device.

The main device is 0, the second device is 1, and so on. I'm using this variable to set a size of the window that I'm going to show. This is a resolution in width and in height. Now I'm going to create an infinite loop to run the webcam. I'm performing the cycle while with this statement. It is an infinite loop. I'm creating a new variable that is an instance of the time function. The idea is to calculate the time because I'm going to show the frames per second. After that, I need to retrieve the image. It's simple. I need to use capture, that is this variable, the function read. This function will return two variables. The frame variable contains the image.

And this variable read represents if any frame is captured. Otherwise, it's known or false. So I can use it to check if I'm retrieving images. If this condition is true, I'm creating a new variable that contains the prediction of yellow model applied to the frame. Now I need to browse into this variable looking for all objects and assign each object to a specific color. So I'm creating a new cycle for using the function zip. I'm adding the information of the results and the colors. So I'm retrieving this information into these variables. Now I need to extract the information of the result variable. So I'm using result. I need to extract the coordinates of the upper left corner, the bottom right corner of the bounding box, the information of the label, and the confidence score.

Now I'm going to create a text to show the label and the confidence for each bounding box. So I'm using the function format, specifying the content of the label because I want to show the label and the value of the confidence. I'm normalizing this value by 100. Now I need to create the rectangle using OpenCV. cv2.rectangle. I'm specifying the frame, the upper left corner, the bottom right corner, the color that I want to use that is this variable that contains a random color, and the thickness of the line. After that, I can write the text in the upper left corner of the rectangle. So it's very simple, I need to use put text in OpenCV. I'm specifying the frame, the text that I want to write, the position, in this case the upper left corner, the type of the font.

You can set up different types. I'm going to use this, italic. This is the size of the font. The color 0 represents black for RGB, and the thickness. OK, I have the frame with a rectangle and a text. Now I need to visualize the whole frame. So I'm using InShow to visualize the frame. You can specify a name in the window, in this case frame. Now I'm going to calculate the frames per second. I'm using the function time and this variable that represents the time at the beginning. I'm formatting the expression in FlowAdd with one decimal. I'm going to print the result by console. OK, we are almost ready. The final step is to create some statement to break this infinite loop.

If you press the letter Q, you are breaking the cycle. Otherwise, the cycle is infinite. Finally, I need to release the resources, capture.release and cv2 destroy all windows. So you need to save this file, and that is all. This is a code to perform object detection using the webcam. Let's check it out. OK, you need to go to the dark flow folder where it's located this file. So after that, you only need to write Python and the name of the file. That is all. You only need to press Enter. I'm loading the model, the YOLO model, 100% usage. And this is the window. This is a bottle, a bottle of water. I have a cell phone. A mouse. This is a mouse. Very good. On the left side, you can see the frames per second, about 22, 23 frames per second.

It's quite good. I think it's better than TensorFlow Object Detection API. So it's very nice, isn't it? OK, to close this application, you only need to press the letter Q. Very good. We have used YOLO to perform real time object detection. It's quite good. OK, that is all for this lesson. In the next lesson, I'm going to use YOLO, but using the transfer learning approach to train our own data. The idea is to use YOLO to identify our own objects. So until then, review the concepts. See you in the next class. Bye bye.

LECTURE 49

YOLO: Real-Time Object Detection

Run the command in terminal as:

```
cd 54.Deep-Learning-for-Computer-Vision-with-TensorFlow-2

.venv/bin/python yolov2_custom_object.py prepare
.venv/bin/python yolov2_custom_object.py train --epochs 300 --gpu 0.8
.venv/bin/python yolov2_custom_object.py video --load -1 --threshold 0.3
.venv/bin/python yolov2_custom_object.py cam  --load -1 --threshold 0.3
```

[MUSIC PLAYING] Hello, guys. Welcome back to the course. This is the final lesson in YOLO. In this lesson, I'm going to perform the training of YOLO using custom objects. As you know, the training of YOLO was performed using different large data sets. But you can reuse the model for training any custom object that you want. So I'm going to apply the concept of transfer learning. YOLO has a knowledge because it was training to detect some objects. So I'm going to use this knowledge to train new data. What kind of data? Something similar like the previous classes that YOLO has been trained. So in theory, all kind of objects. So the first thing that you need to do is to collect the data.

For training, I recommend you at least 200 images.

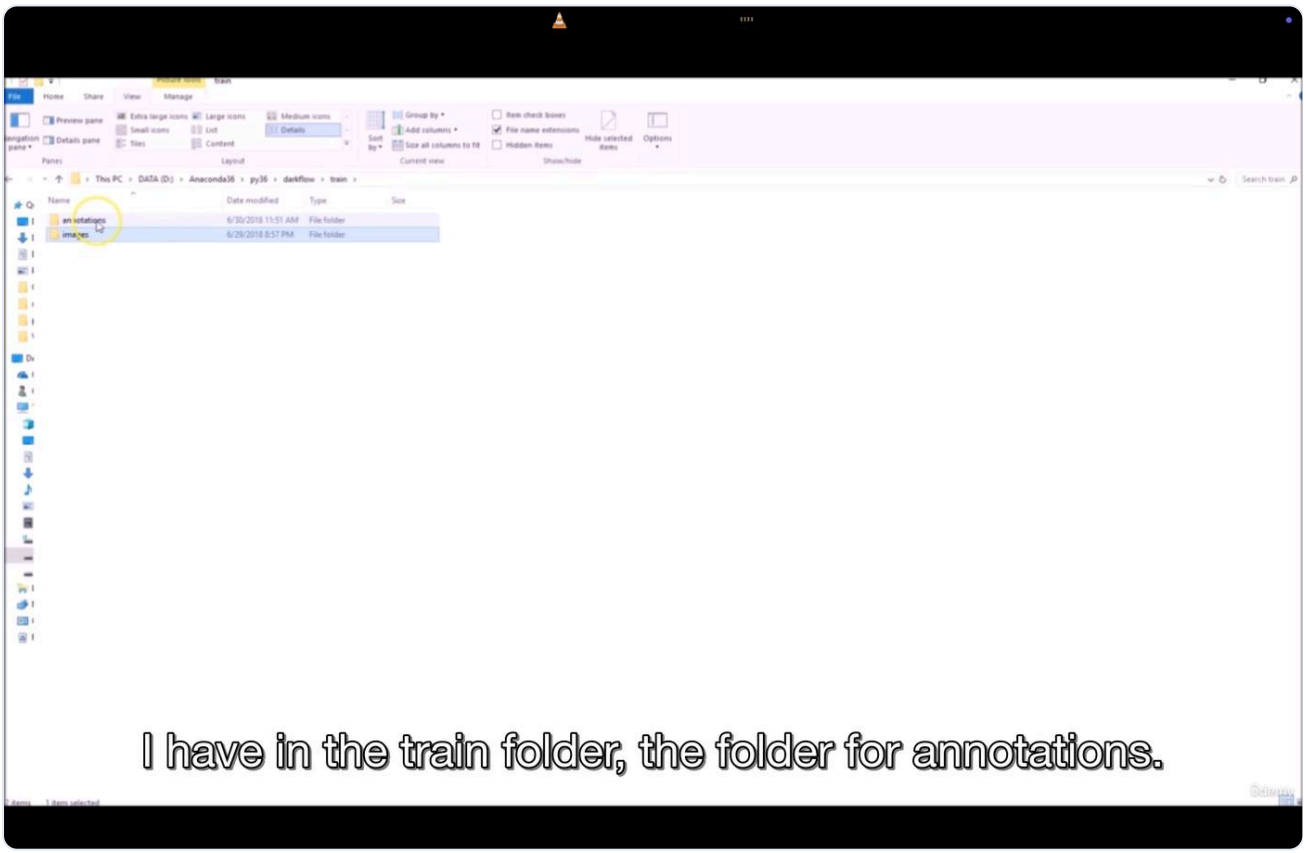


FIGURE 4.1

But if you can get 1,000 images, it's better. So let me show you my images. These are my images. I have a folder images in a train folder that is located in DartFlow. For instance, I want to train YOLO to recognize this R2D2 robot from Star Wars. First, I need to perform the scrapping of the web, collecting these images, or you can take pictures. I have used the code to rename all pictures with these names in a sequence. I have 200 pictures because I'm starting in zero. So I'm going to perform the object detection for only one class. You can do it for more classes if you want. But for each class, you need 200 images as a minimum. After that, you need to perform the annotations, selecting the regions with a bounding box where the object is located.

So you can use image label or another application. All annotations must be stored in a different folder. I have in the train folder the folder for annotations.

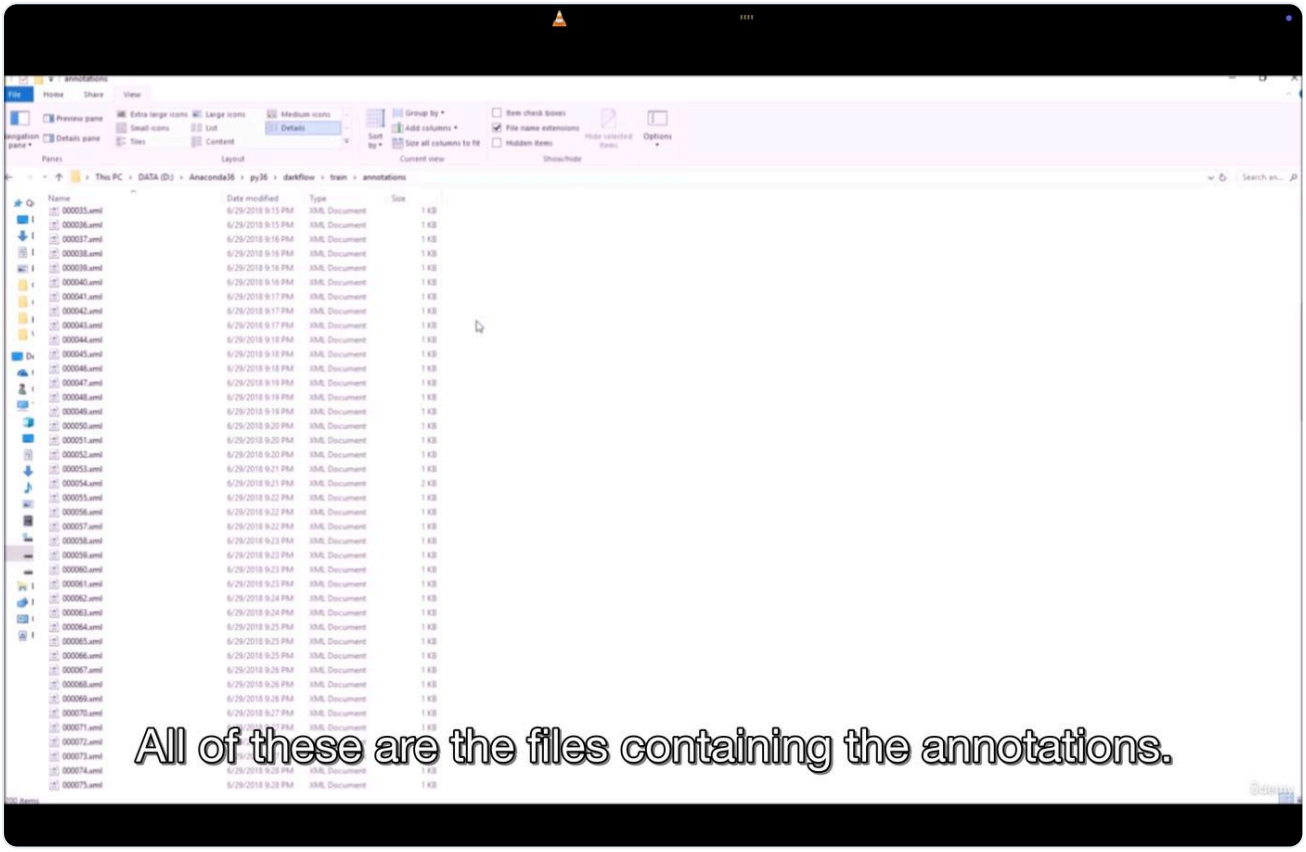


FIGURE 4.2

All of these are the files containing the annotations. I have 200 files, the same number as images.

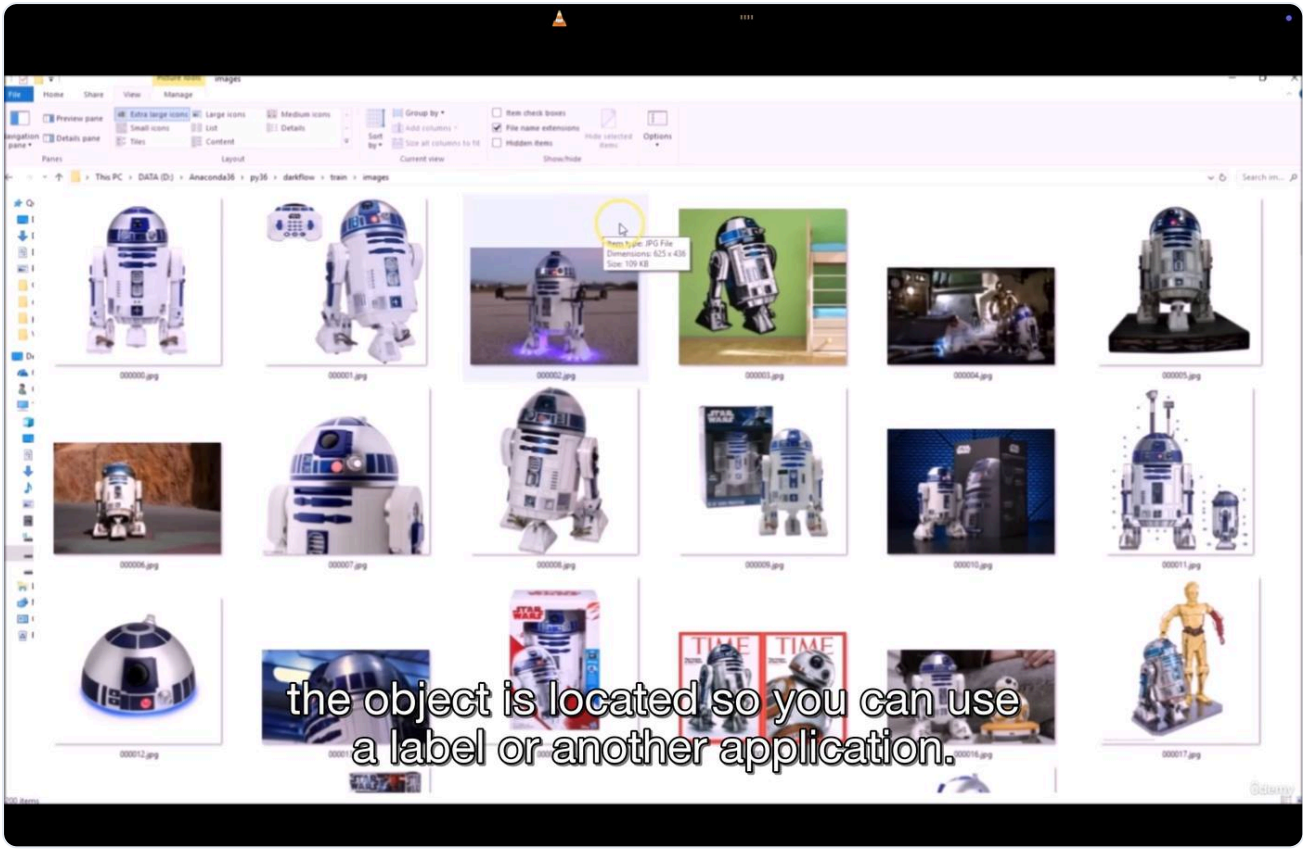


FIGURE 4.3

OK, I have the pictures and annotations. The next step is to download the model, the YOLO model, that I want to use.

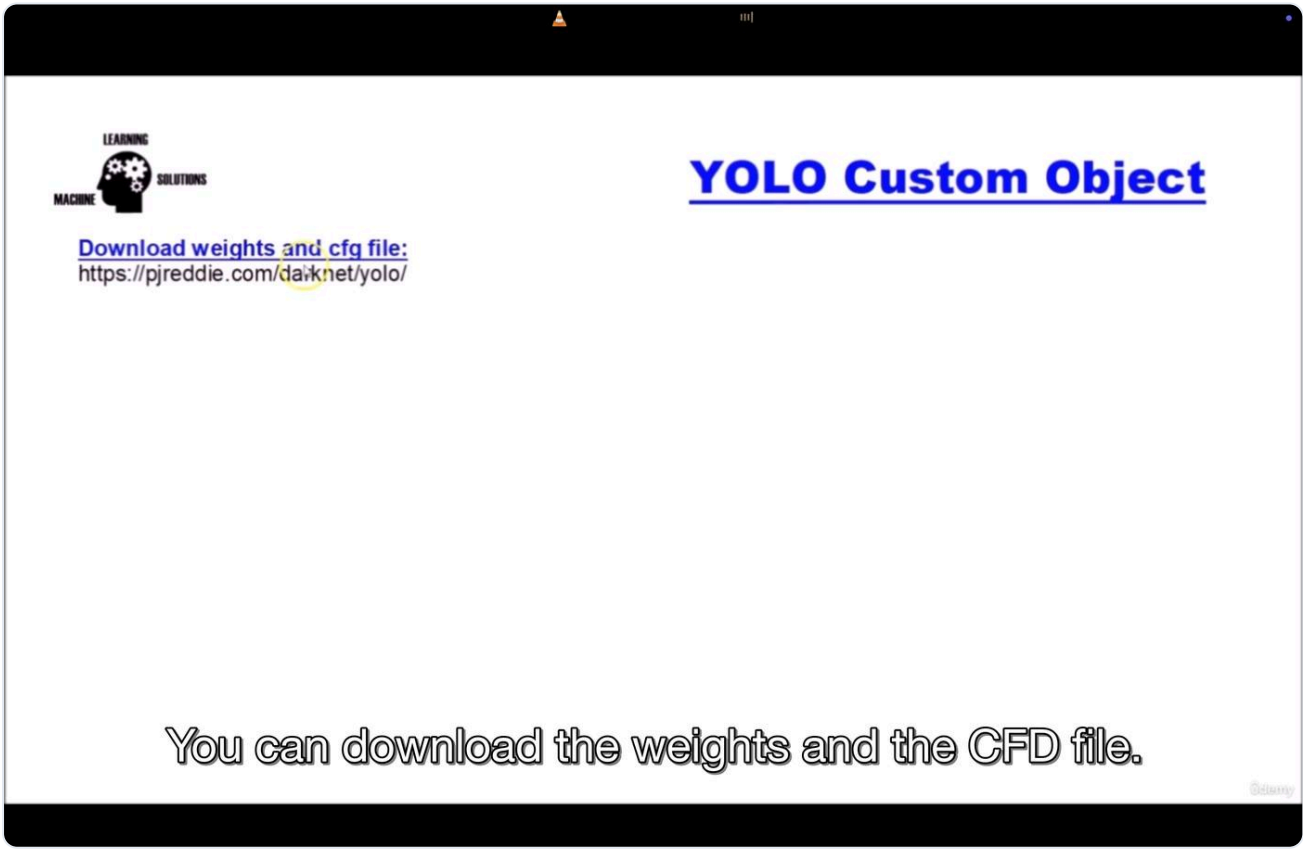


FIGURE 4.4 | Download Weights and Cfq File:

In this web page, you can download the weights and the CFG file. Let's check it out. This is the default web page of YOLO. The current version is 3.



FIGURE 4.5

You can find different models in this column, CFG. You will find the models. You can download from here the model and from here the weights. The YOLO version 3 runs in Linux. So if you are in Windows, you can try installing a virtual machine and Linux as operating system.

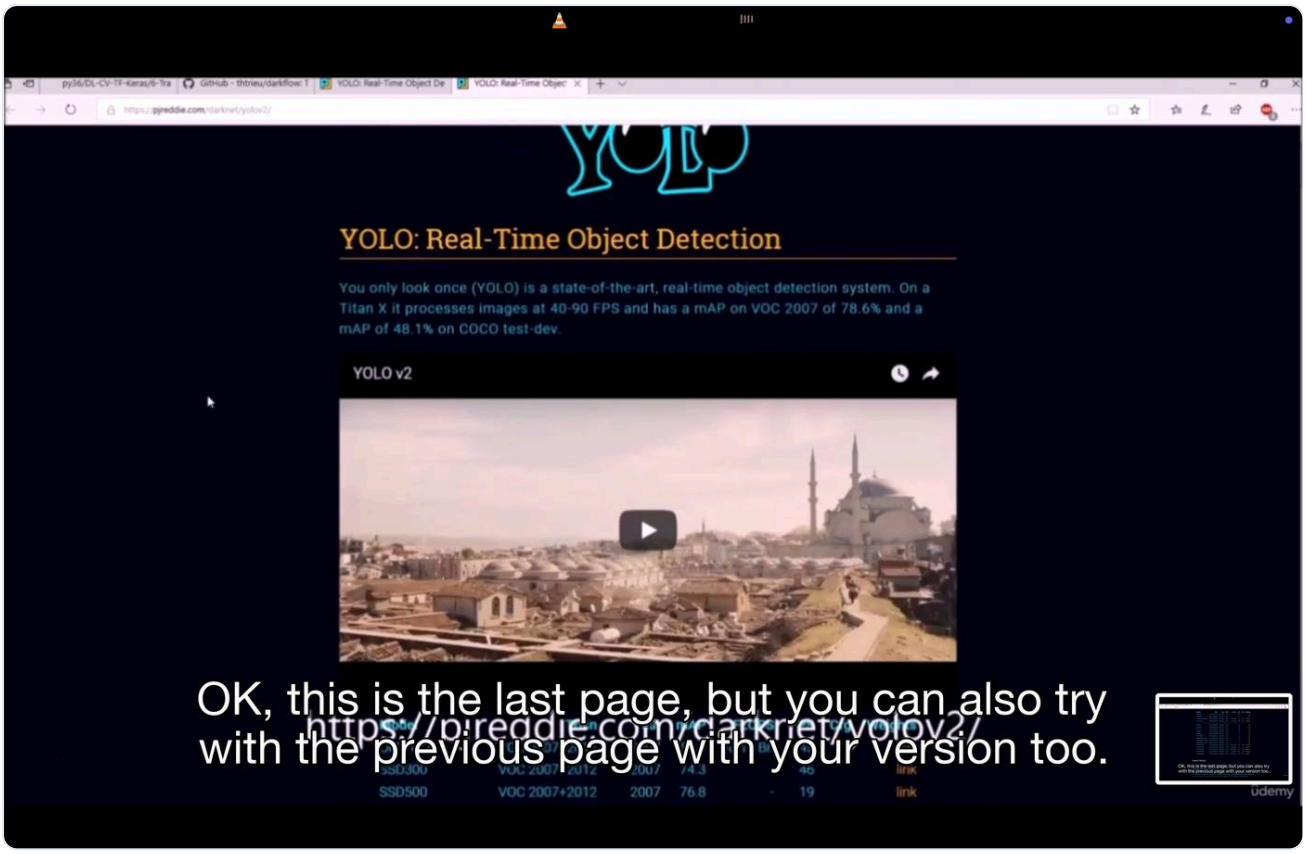


FIGURE 4.6 | YOLO: Real-Time Object Detection

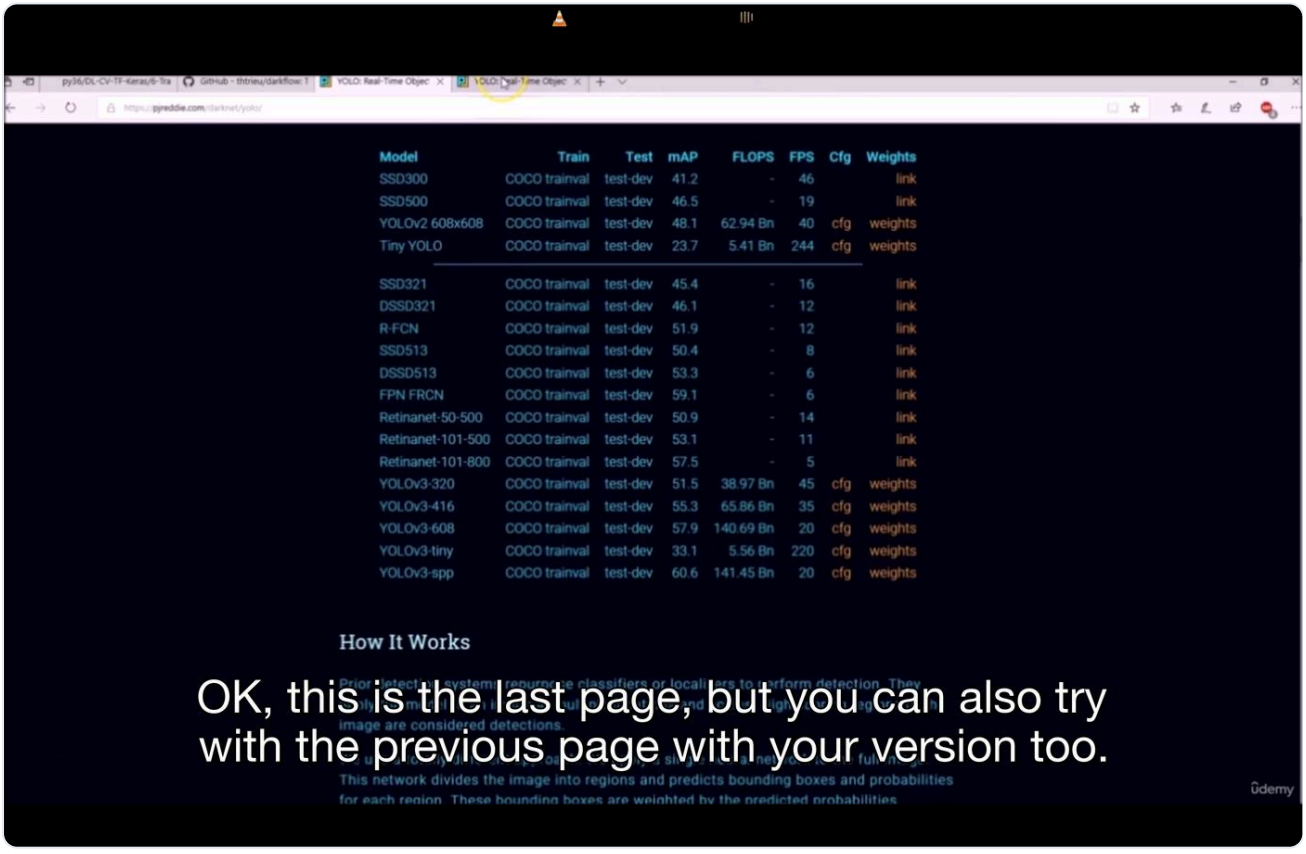


FIGURE 4.7

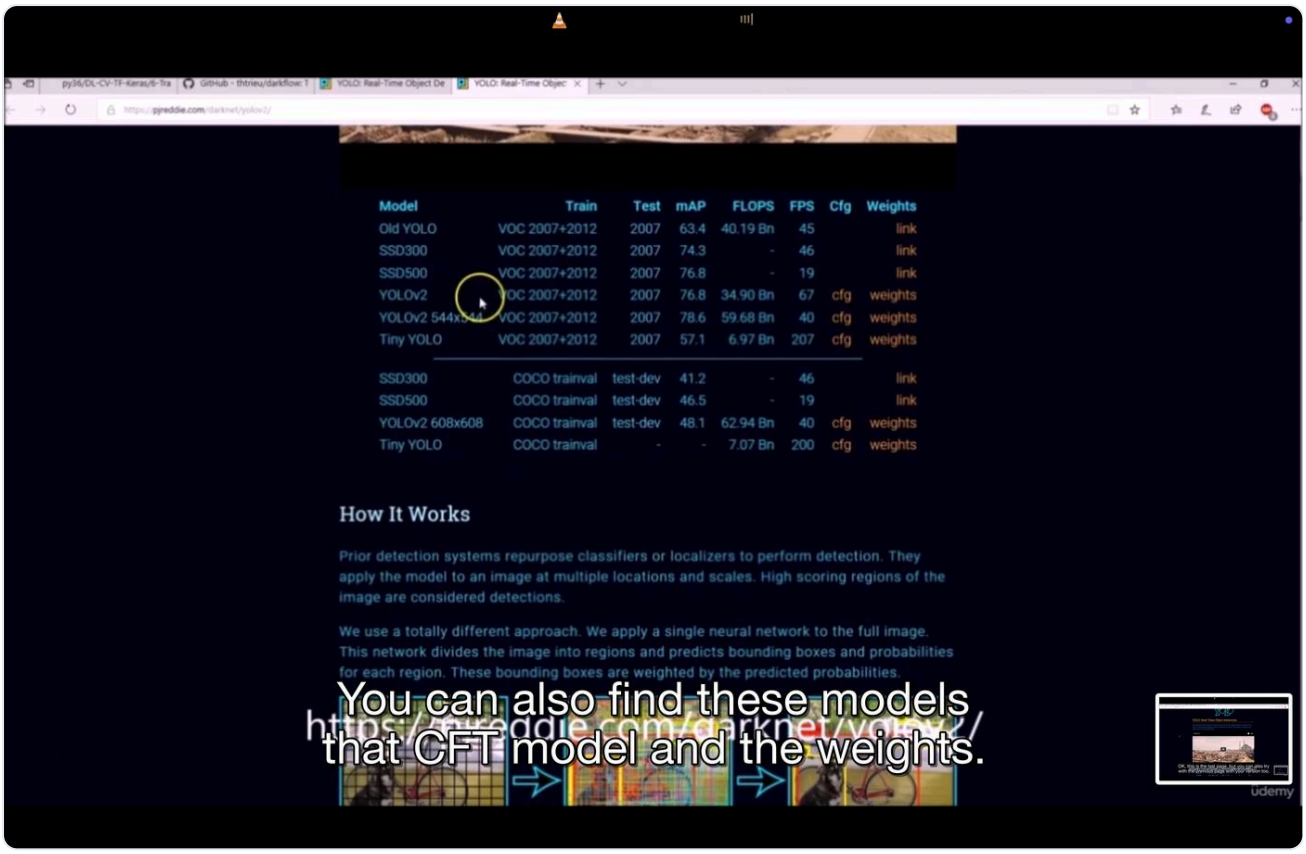


FIGURE 4.8 | Model Train Test mAP FLOPS FPS Cfg Weights

OK, this is the last web page, but you can also try with the previous page with YOLO version 2, you can also find these models, the CFG model and the weights. You have different options here. For this example, I'm going to use the YOLO version 2 model. OK, after downloading the model, you need to edit the model.

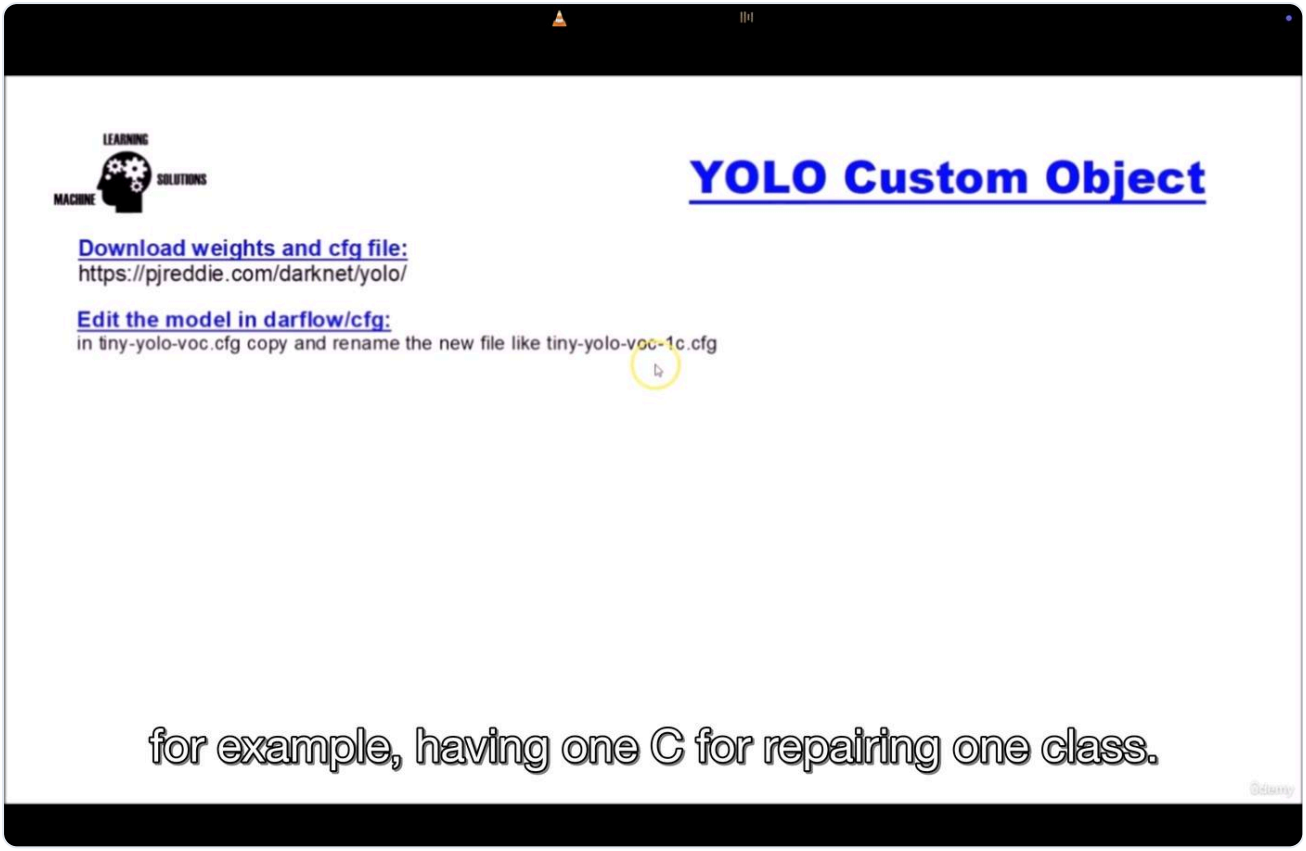


FIGURE 4.9 | Download Weights and Cfq File:

So if you are working with this model, the YOLO VOC, you need to rename this file with a new name. For example, adding 1C for referring one class. After that, you need to edit this file in a text editor. This is a new file. You need to change filters.

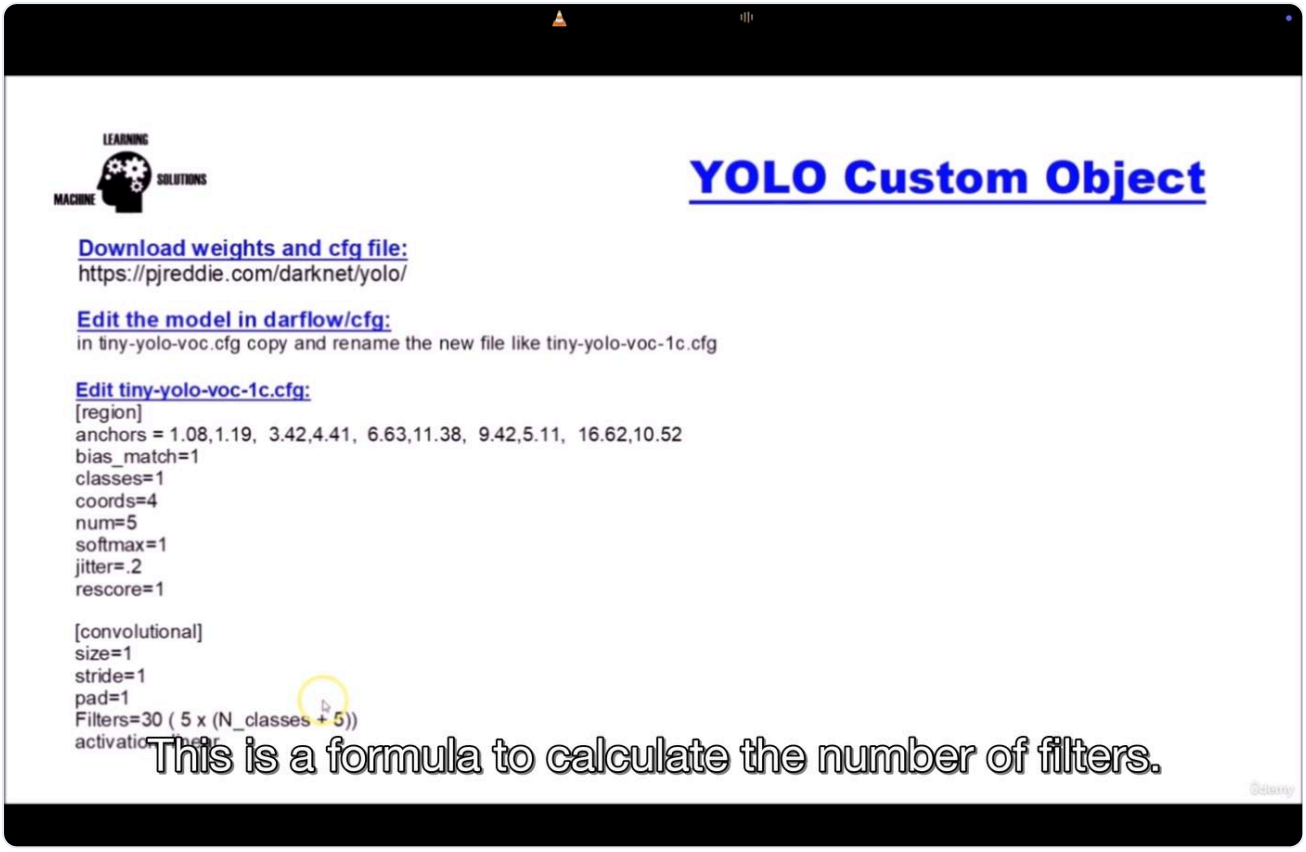


FIGURE 4.10 | Download Weights and Cfq File:

This is a formula to calculate the number of filters. You need to specify the number of classes. If you have one class, so 1 plus 5 is 6, 6 by 5 is 30.

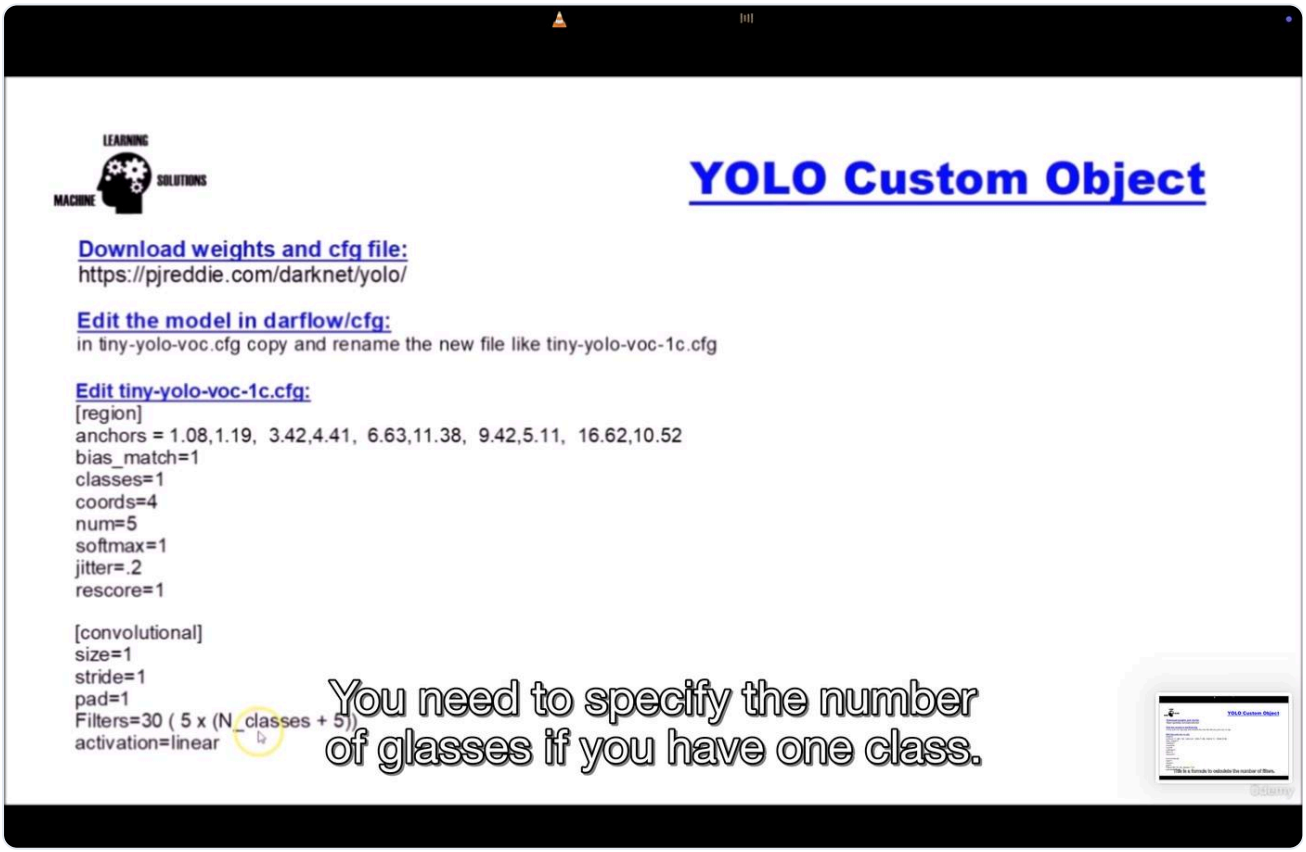


FIGURE 4.11 | Download Weights and Cfq File:

You need to replace the value with this new value. Let's check it out in the file. This is the file.

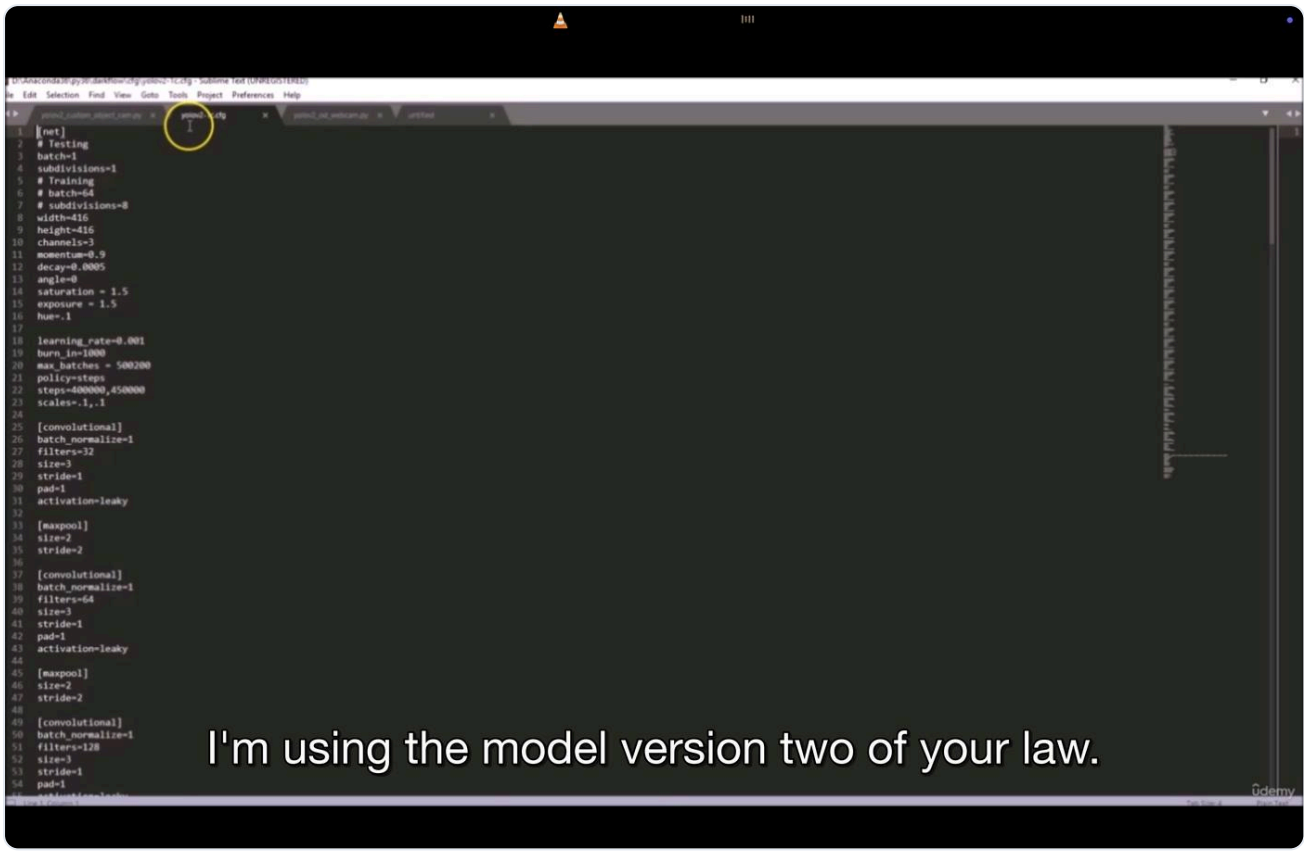


FIGURE 4.12

In my case, I'm using the model version 2 of YOLO. I'm renaming this file with one class.

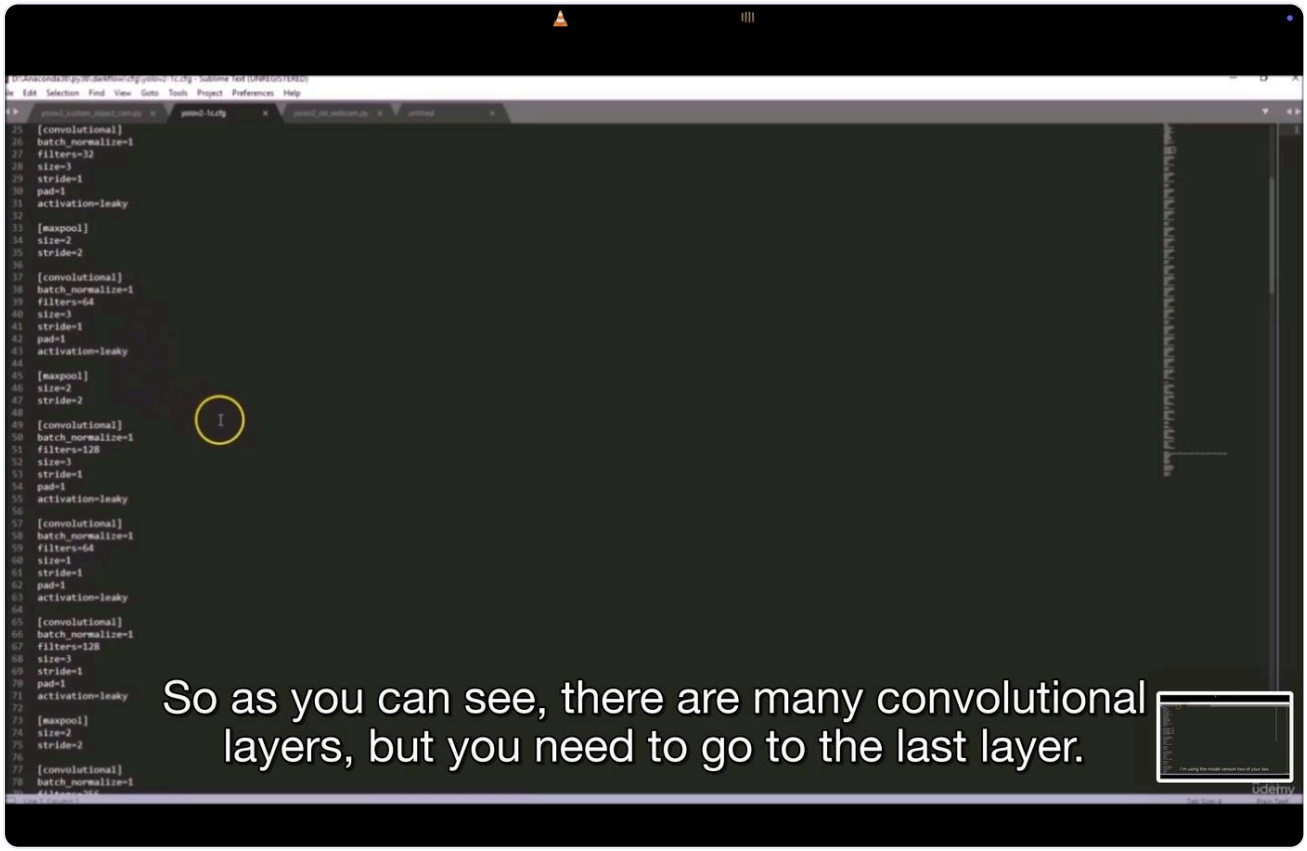


FIGURE 4.13

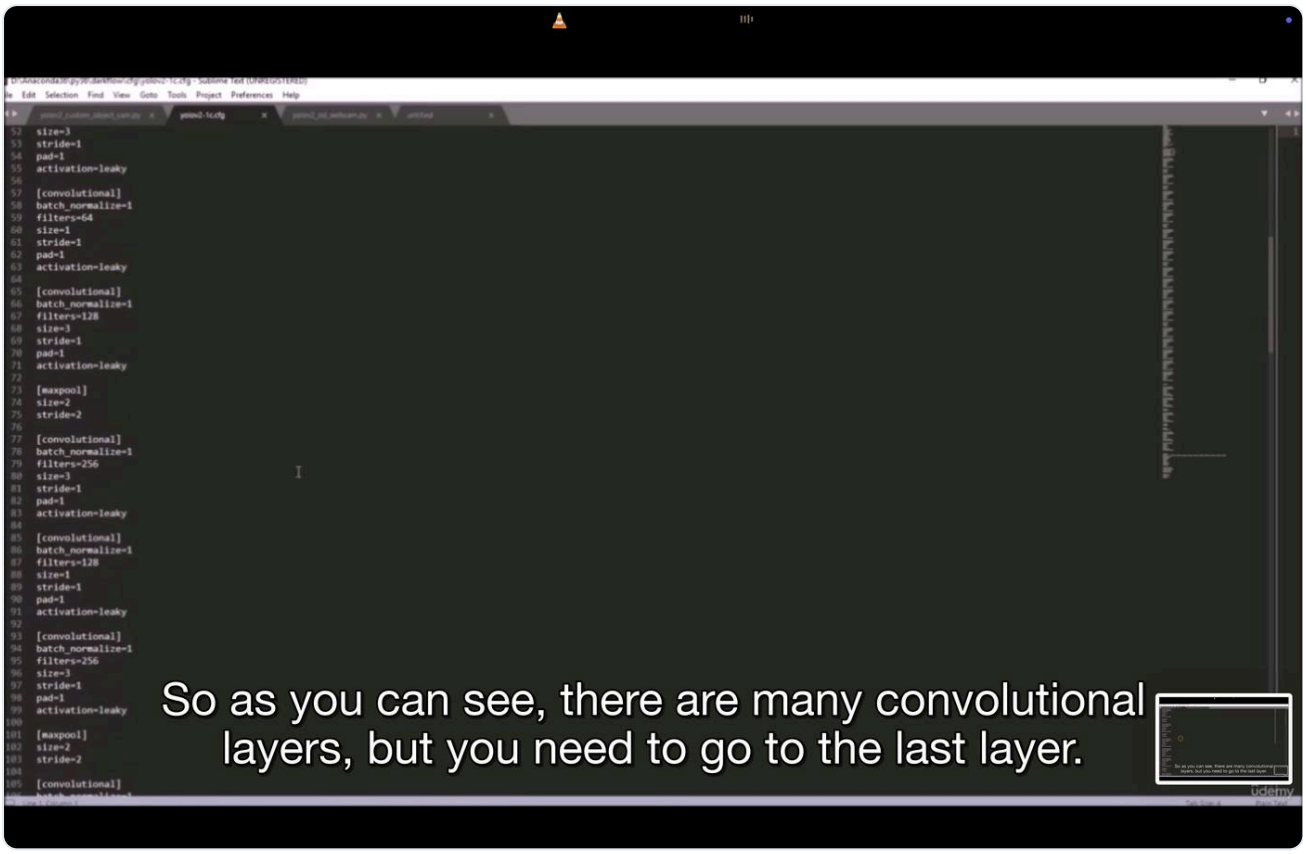


FIGURE 4.14

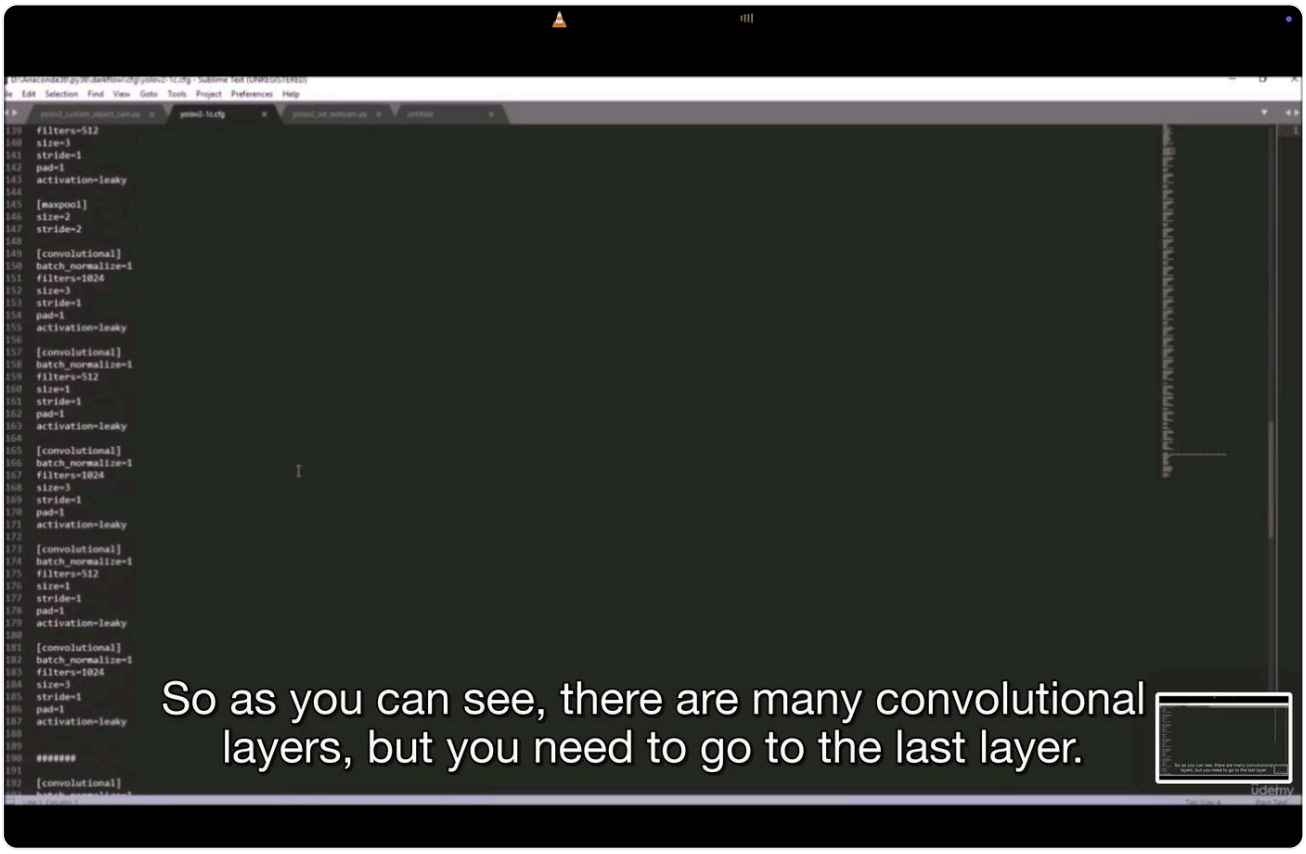


FIGURE 4.15

So as you can see, there are many convolutional layers, but you need to go to the last layer. OK, in this field region, you need to set the classes, the number of classes. In my case, it's 1. And in the last convolutional layer, you need to fix the number of filters.

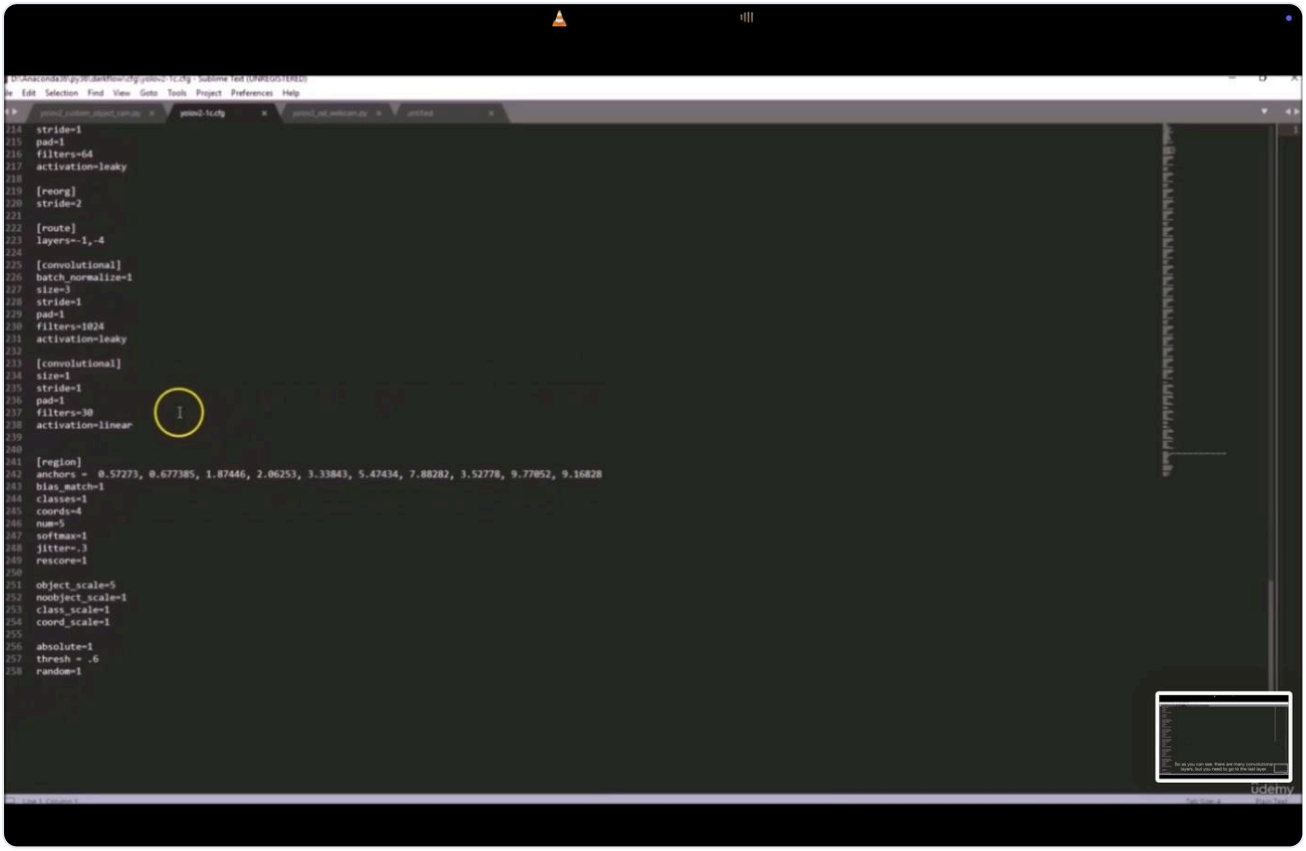


FIGURE 4.16

In this case, it's 30. These are only two changes. That is all. After applying the changes, you need to save your file. The next step is to create the labels for our own objects.

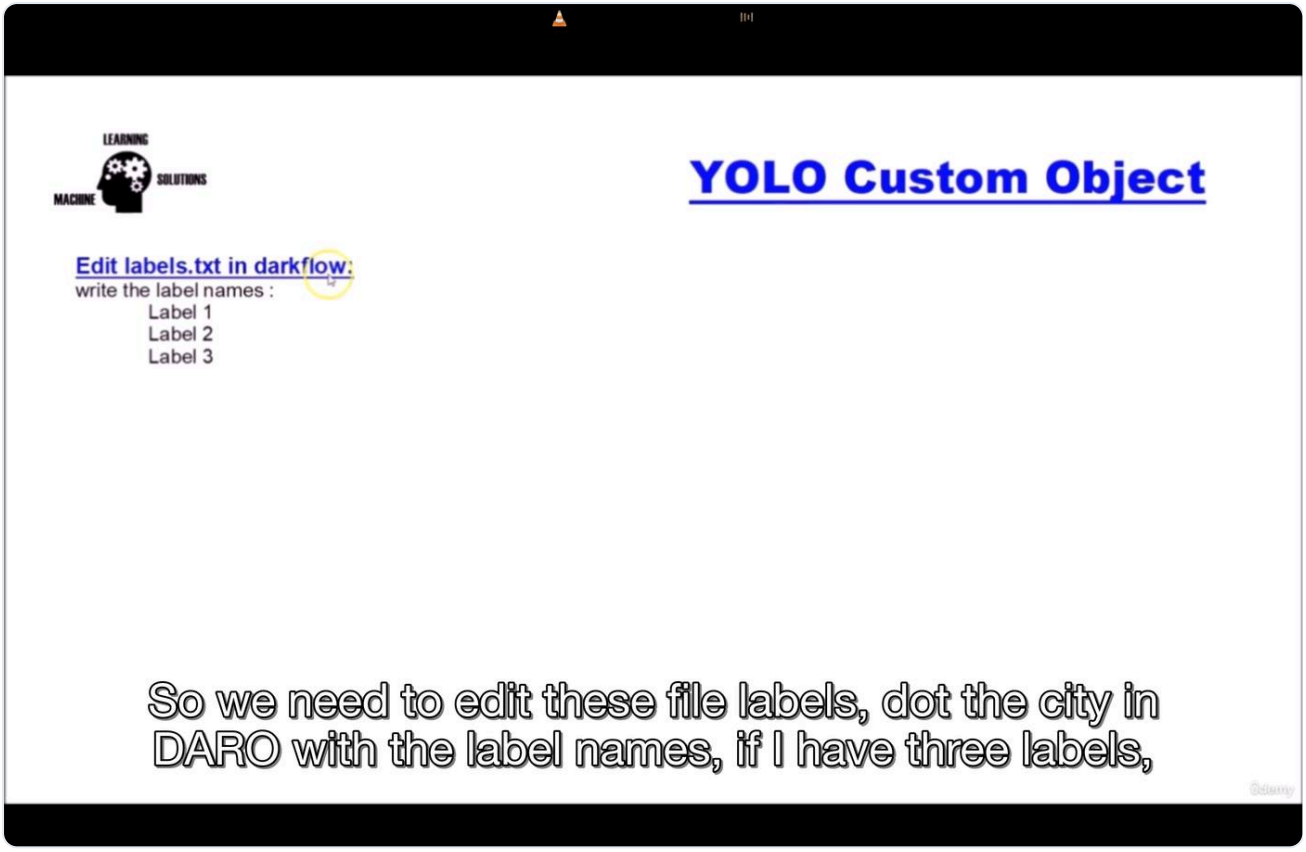


FIGURE 4.17 | Edit labels.txt in Darkflow:

So we need to edit this file labels.txt in DartFlow with the label names. If I have three labels, I need to write one label per line. So let's check it out.

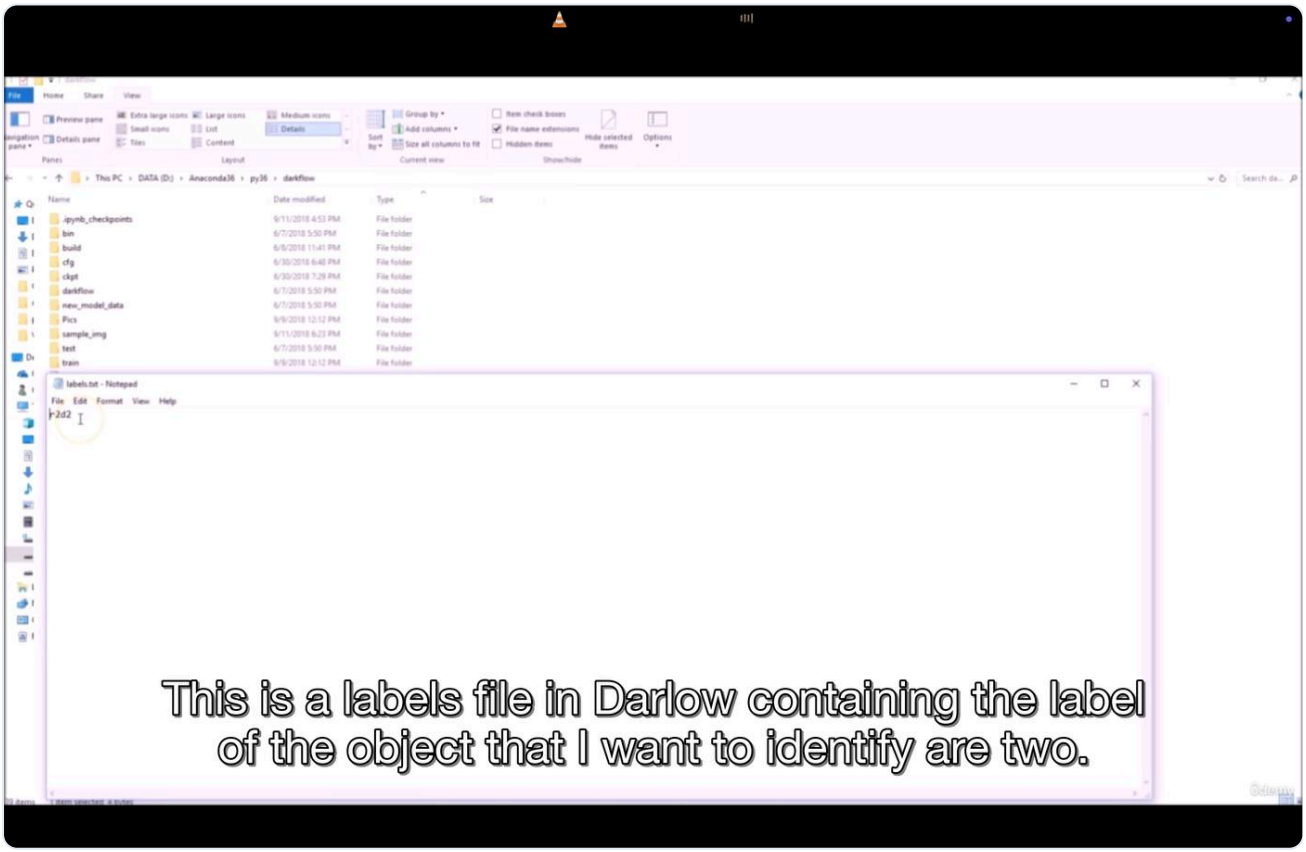


FIGURE 4.18

This is the labels file in DartFlow containing the label of the object that I want to identify, R2D2. That is all. It's very simple. You don't need to specify anything else. Save your file. OK, we have everything to train our model. I need to write an expression. Just open an Akonda prompt. You need to run this into the DartFlow folder.

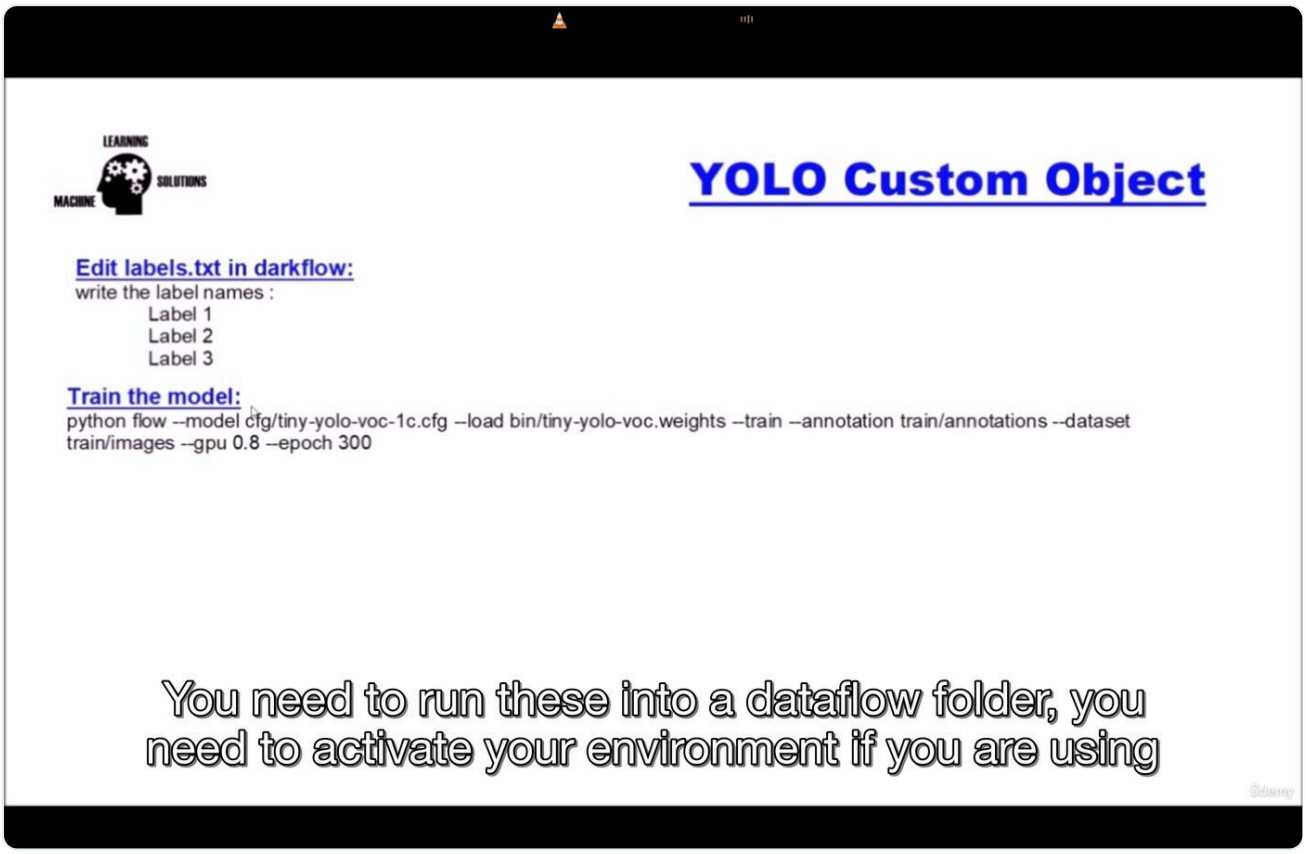


FIGURE 4.19 | Edit labels.txt in Darkflow:

You need to activate your environment. If you are using a GPU or CPU, you need to write this instruction. Python flow to load the YOLO model, double dash model, the route where it's located the model, and the name of the model. Double dash load the folder where it's located the weights. Double dash train, double dash annotation. I need to specify the folder where it's located all annotations. Double dash data set. So I need to specify the route where it's located the images. Double dash GPU, you can set up 80% to avoid any issue. And finally, double dash and the number of epochs that you want to perform. In my case, using one class, the YOLO version 2, 300 epochs, it takes about two hours or perhaps three hours to perform the whole training.

OK, you will see something like this. There are information about the loss for each step and the average loss and the points where the epochs are finished. You will see the average loss is continuously decreasing along the steps. At the final, you will find this screen. This is the last step, 3,600 steps. And this is the average loss. It's very tiny, so it's good. If you don't have any issue, you will find this message. Training finished, exit. So we have performed the training of our model. Finally, we need to load our new model to perform the object detection in a webcam. I can use my previous file for object detection in a webcam, but in this case, I'm going to load my new model.

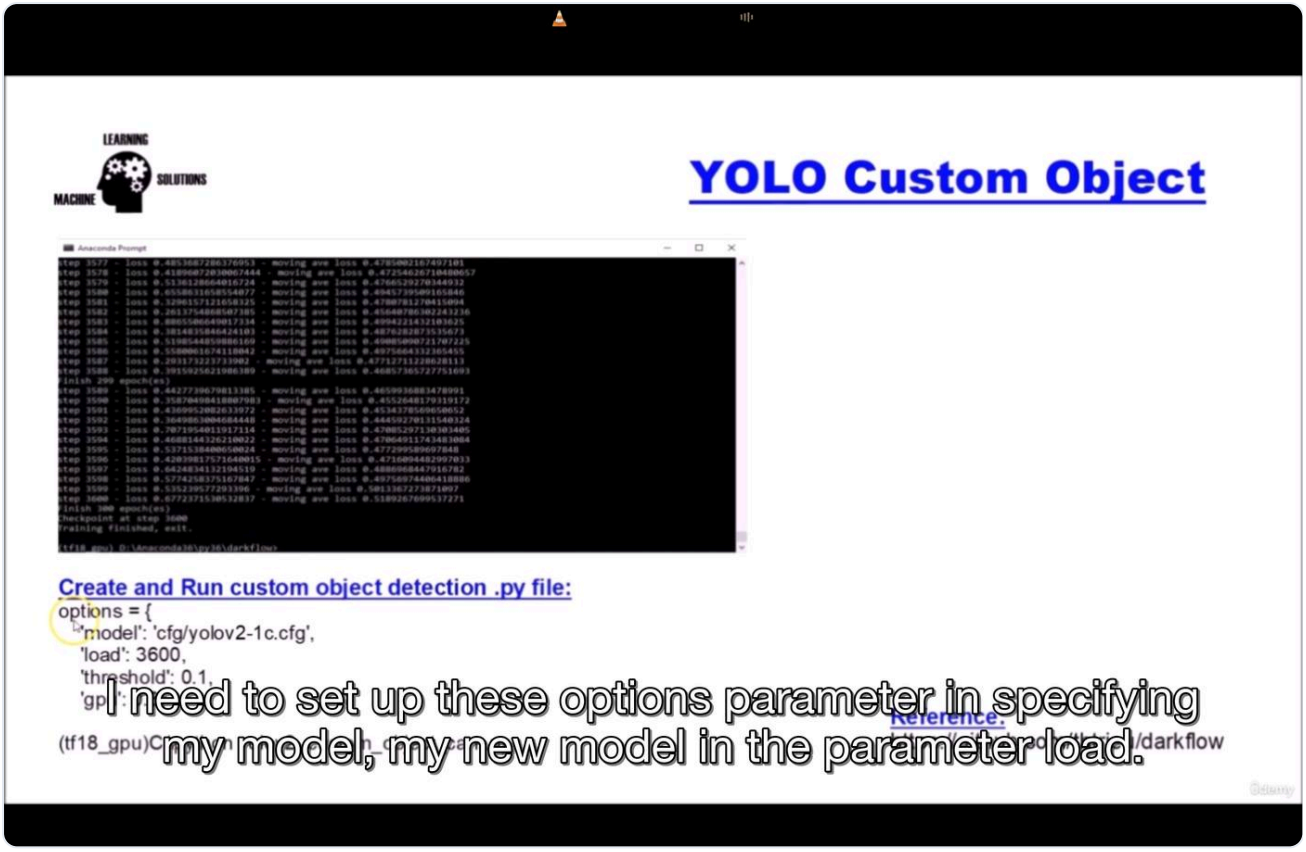


FIGURE 4.20 | Create and Run Custom Object Detection .Py File:

I need to set up these options parameter, specifying my model, my new model. In the parameter load, I need to consign the number of steps 3,600. That corresponds to this last number. This is the last step in the training. The threshold, I recommend you 0.1 because I don't have too many images in the training. And the GPU, it could be 1 or 0.8.

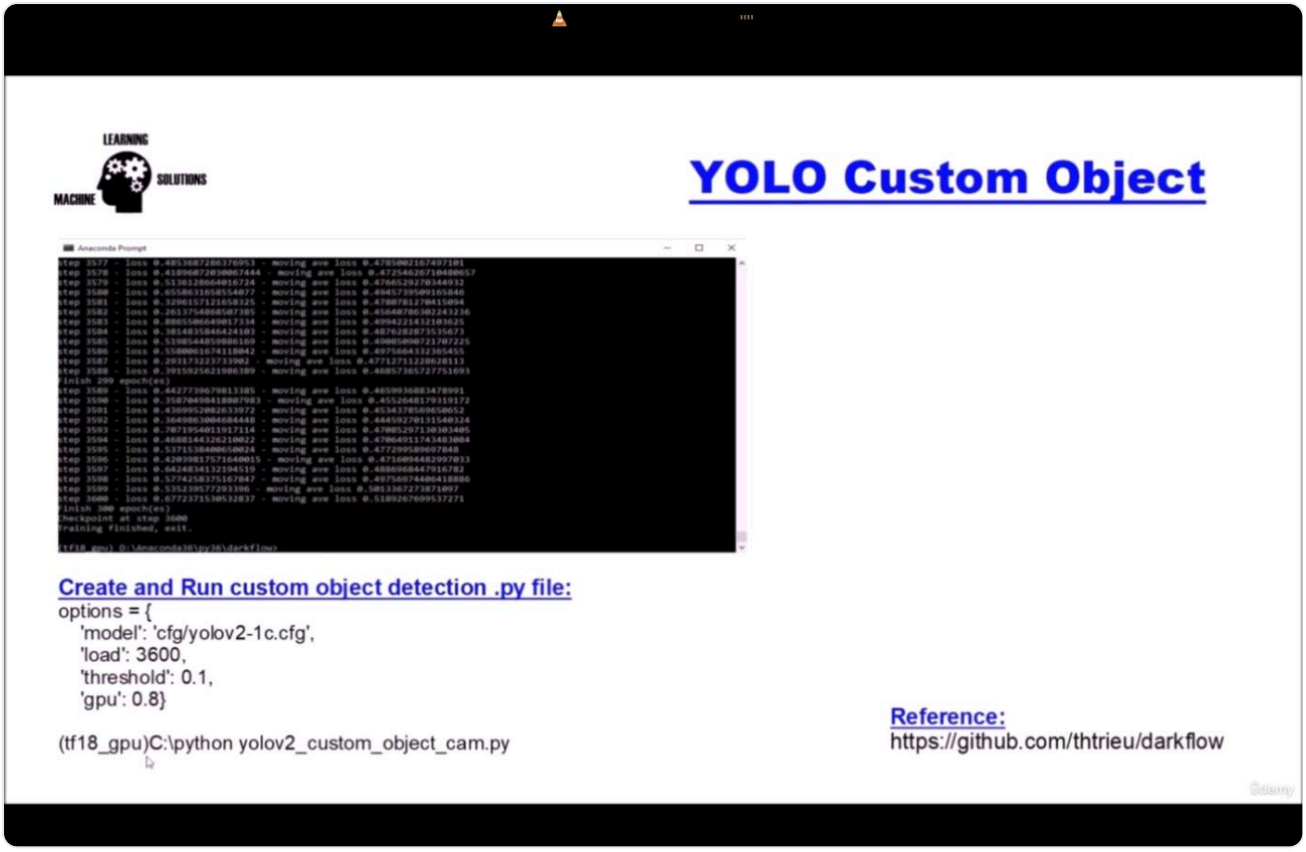


FIGURE 4.21 | Create and Run Custom Object Detection .Py File:

Finally, I need to activate my environment. I need to use Python and the name of this file to run our new YOLO model. So let's go to this file. OK, this is the file. I have made a copy of the YOLO webcam file. That is this.

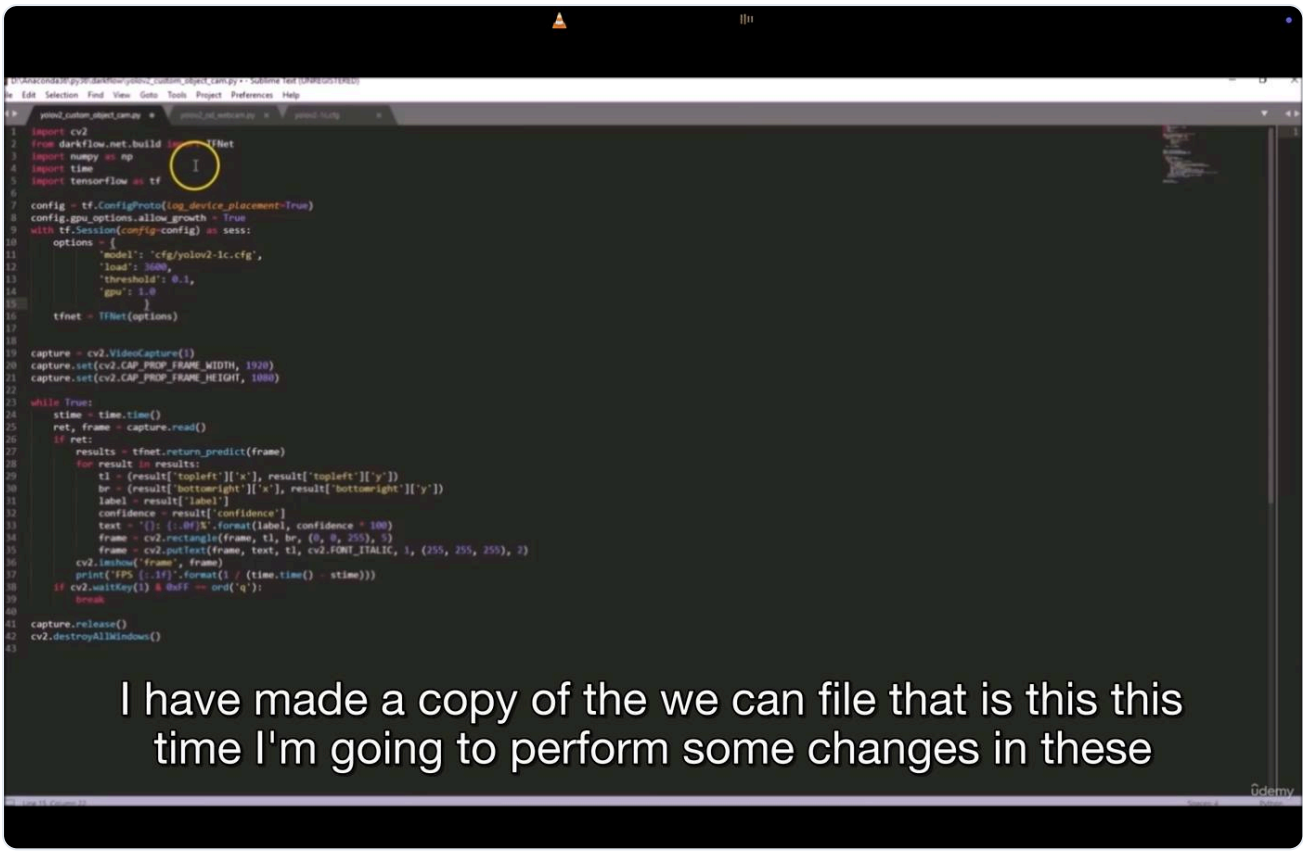


FIGURE 4.22

This time, I'm going to perform some changes. In these options, I need to specify the model.

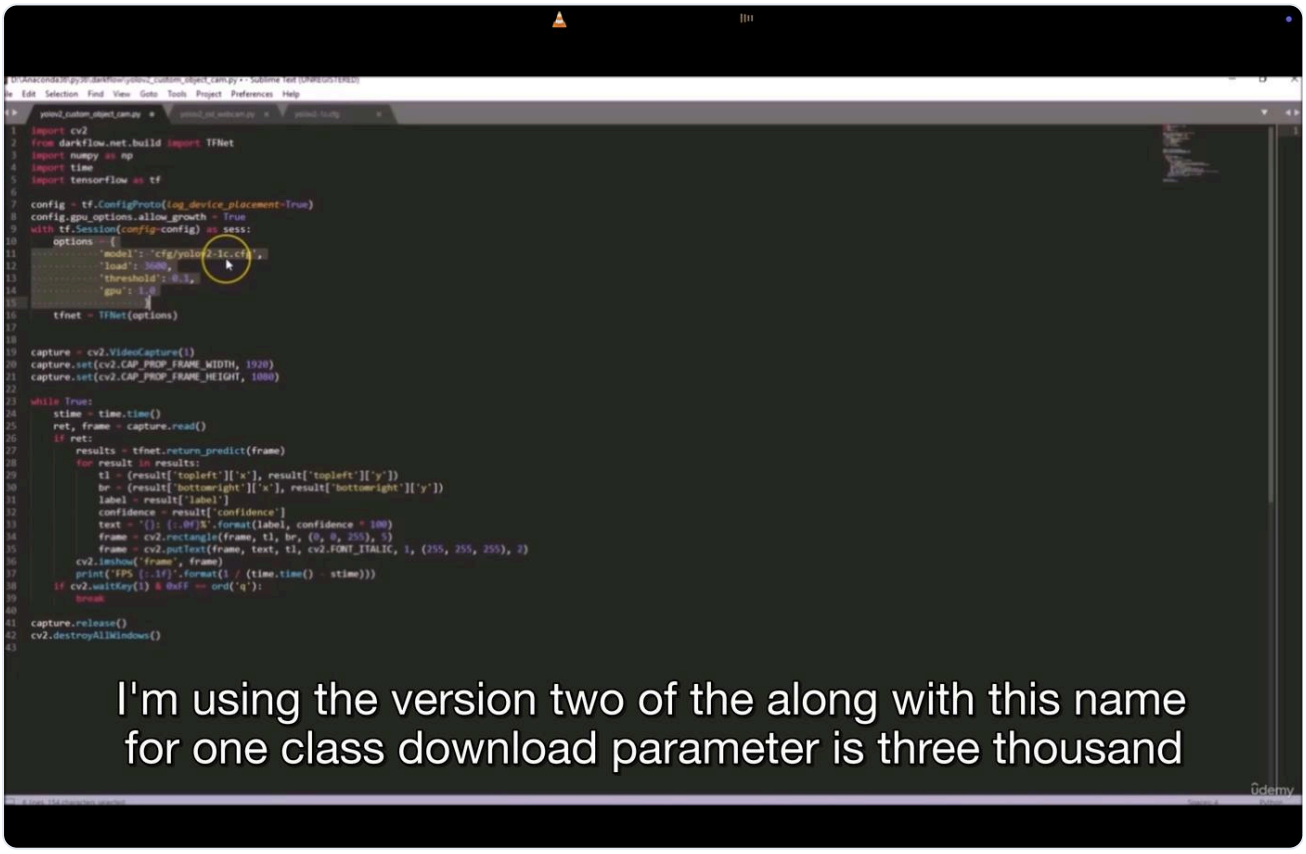


FIGURE 4.23

In this case, I'm using the version 2 of YOLO with this name for one class. The load parameter is 3,600. We don't need to specify a specific file for the weights. The threshold in 0.1 and the GPU option in 100%. I'm going to use the full memory size. The rest is the same, but now I'm going to set the window in a small size. You can use other values if you want. And because I have one class, I don't need to specify or create different colors for all classes. So I'm going to print the rectangle in a blue color. This is a blue color. The color of the text is white, all values in 255. The rest is the same. I'm calculating the frame rate. OK, that is all. You need to save your file.

And now, you need to open your Anaconda prompt. OK, you need to go to the DartFlow folder. I'm running in a GPU kernel with TensorFlow 1.10. So you need to write Python at the name of the file and press Enter. OK, I'm loading the YOLO model version 2. In the final layer, I have this output. This is the number of filters in the last layer, 30, that we have calculated. OK, this is the object, R2T2. I have the banding box with a score higher than 10%. I can rotate the toy. In this frame, I have near to 95% in the confidence. It's very, very good. The object is very well identified. I'm zooming it. I'm far to the object, but it's still recognized. Mission accomplished, guys.

You can try with more images, and the results will be better. On the left side, we have the frames per second. It's about 30 frames per second. It's flying. It's quite good. And that is all. You can apply this concept using YOLO to identify different kinds of objects. For example, pedestrians, cars, stop signs, traffic lights, or many objects in a line production. Applications are endless, so enjoy it. We are reaching the end of the course for any doubt or questions, you can write me on QA forums, or you can write me by email or in Twitter. I recommend you to learn the concepts before to practice the codes, OK? Bye-bye.

LECTURE 50

Don't Void the Original or YOLO

[MUSIC PLAYING] Hello, everyone. Welcome back to the course. I'm going to perform the training of YOLO using my ARM dataset back in Jupyter. Previously, I have explained to you how to use your ARM dataset for training YOLO. In that opportunity, I used Python files and the training process, writing commands in the console. Now, I'm going to present you the same procedure but using Jupyter. The idea is to see this procedure more clearly. We are going to need OpenCV for running this procedure. So let's make a recap of the installation. First, you need to go to your Anaconda folder. In my case, this is my Anaconda folder. Inside this folder, you need to download the OpenCV file.

You can also get OpenCV plus the contrib files. This is my version of OpenCV. The current version is 3.4.5. Remember to get the version in Python 3.6. After that, you need to open your Anaconda prompt, something like this. You need to activate your environment. In my case, I'm working in a GPU environment, TensorFlow version 1.11.

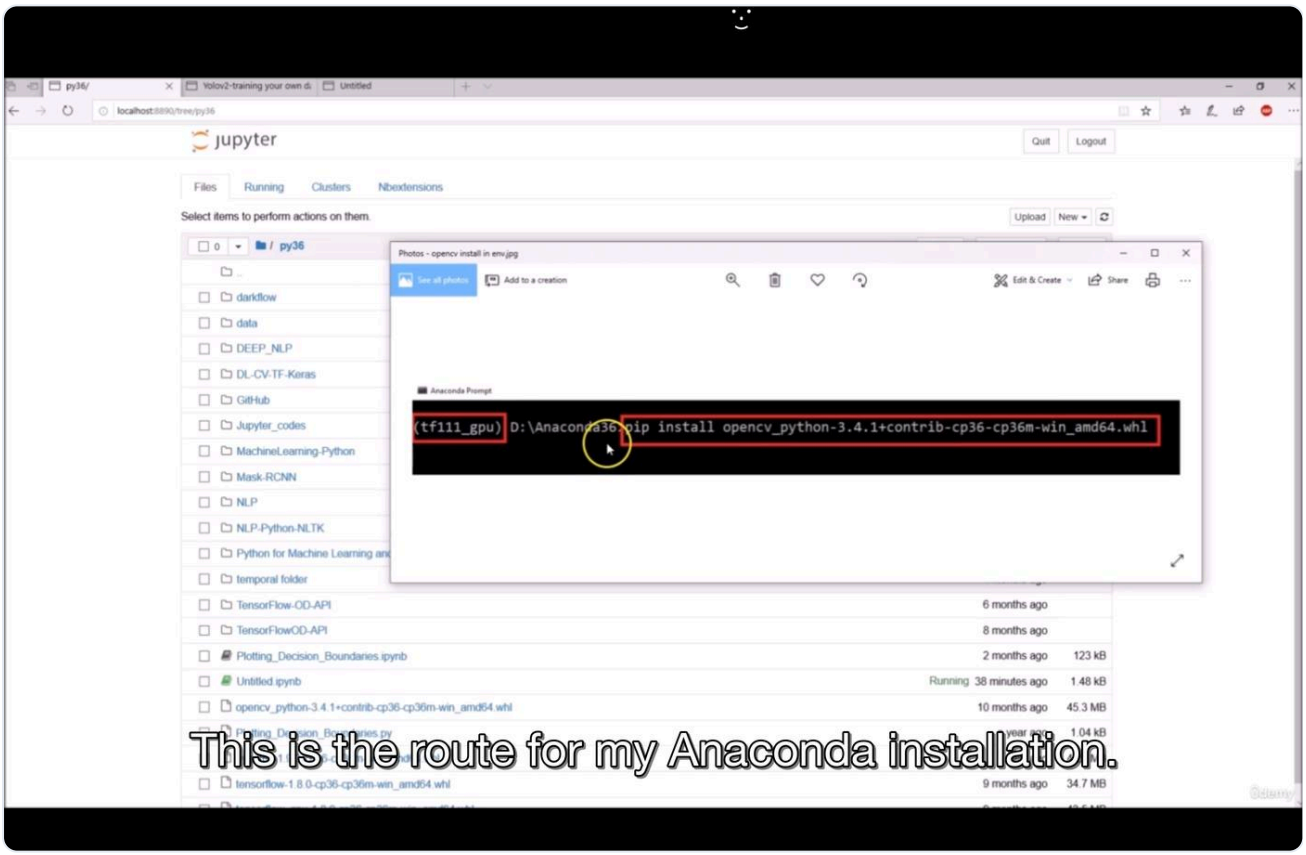


FIGURE 5.1 | This Is the.routte Tor My Anaconda Install 2

This is a route for my Anaconda installation. And into this folder, you need to write this expression-- pip install, and the name of the file. In this case, OpenCV Python, the version of Python. This is the extension for the installation with pip. W-H-O. So you only need to press Enter, and the installation will start. And that's all. It's very simple. OK, after the installation of OpenCV, you need to work with this file. This is a file for this class. You will find this class in a new chapter,

chapter 7, containing bonus classes. This is a new folder, bonus classes. Into this folder, you will find the file for this lesson. One important recommendation is to copy this file to the DartFlow project.

This is the folder, DartFlow. This folder is inside into the Anaconda folder. And here, this is the file with Jupyter extension. OK, now I can start. I'm going to create a new empty notebook. I need to import the libraries, map load lib, as usual, numpy. And this is important, OpenCV. You need CV2. One advantage to work with the DartFlow project is the capability to use an API using this function, tfnet. Now, with this function, tfnet, we have the complete model of DartFlow. We only need to work with this function. If everything is OK, you won't get any error. Now, I need to define some parameters for our model with this variable, option, which is a dictionary, and you're specifying different parameters.

Model, I'm using the version 2 of YOLO with one class. This is the file, YOLO version 2, one class. Remember to fix the number of classes. In my case, I only have one class. I'm going to perform object detection with only one object. If you have more objects, you need to specify the number of classes according to the number of objects that you want to identify.

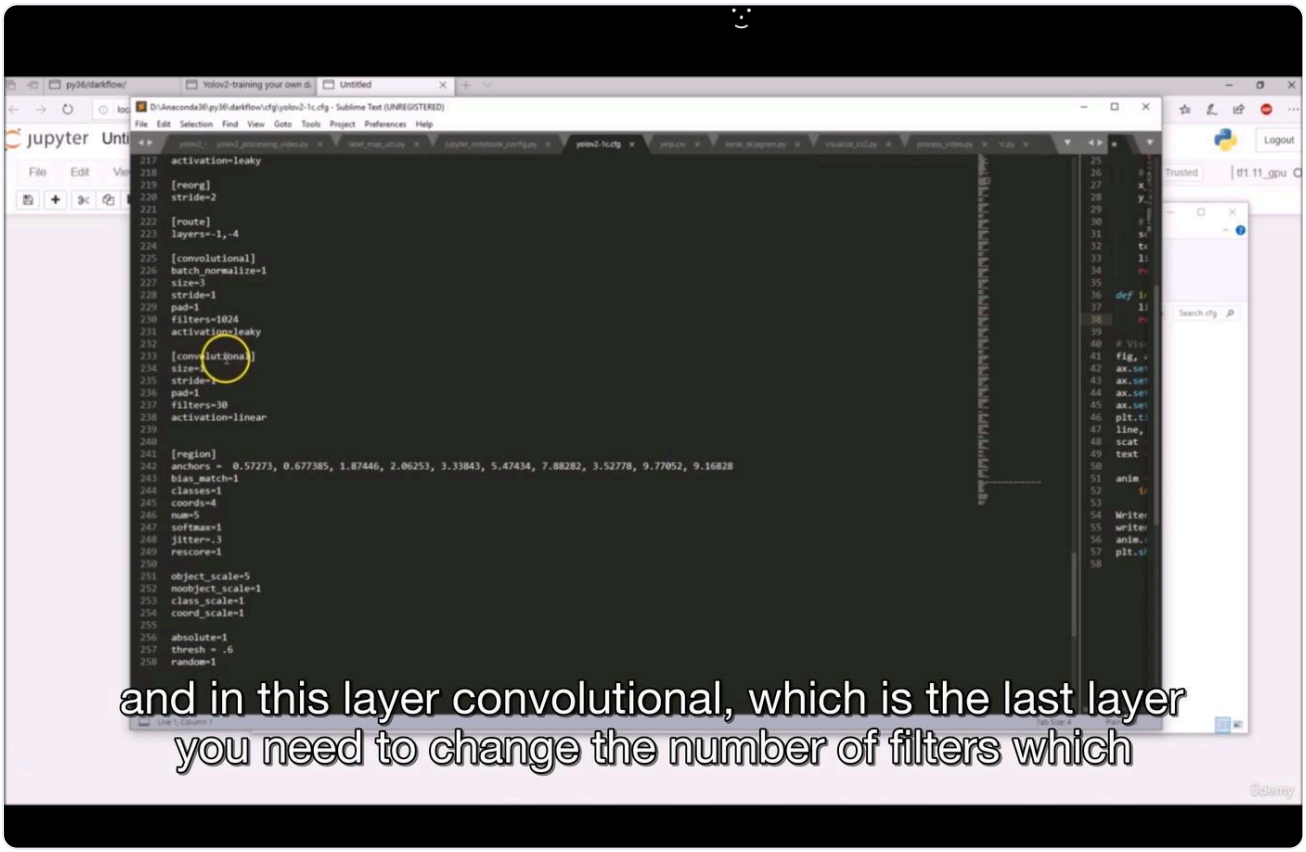


FIGURE 5.2

And in this layer, convolutional, which is the last layer, you need to change the number of filters, which are according to the number of classes. Remember this slide. Filters, 30, is a result to apply one class, because if I have one class, plus 5 is 6, 6 by 5 is 30.

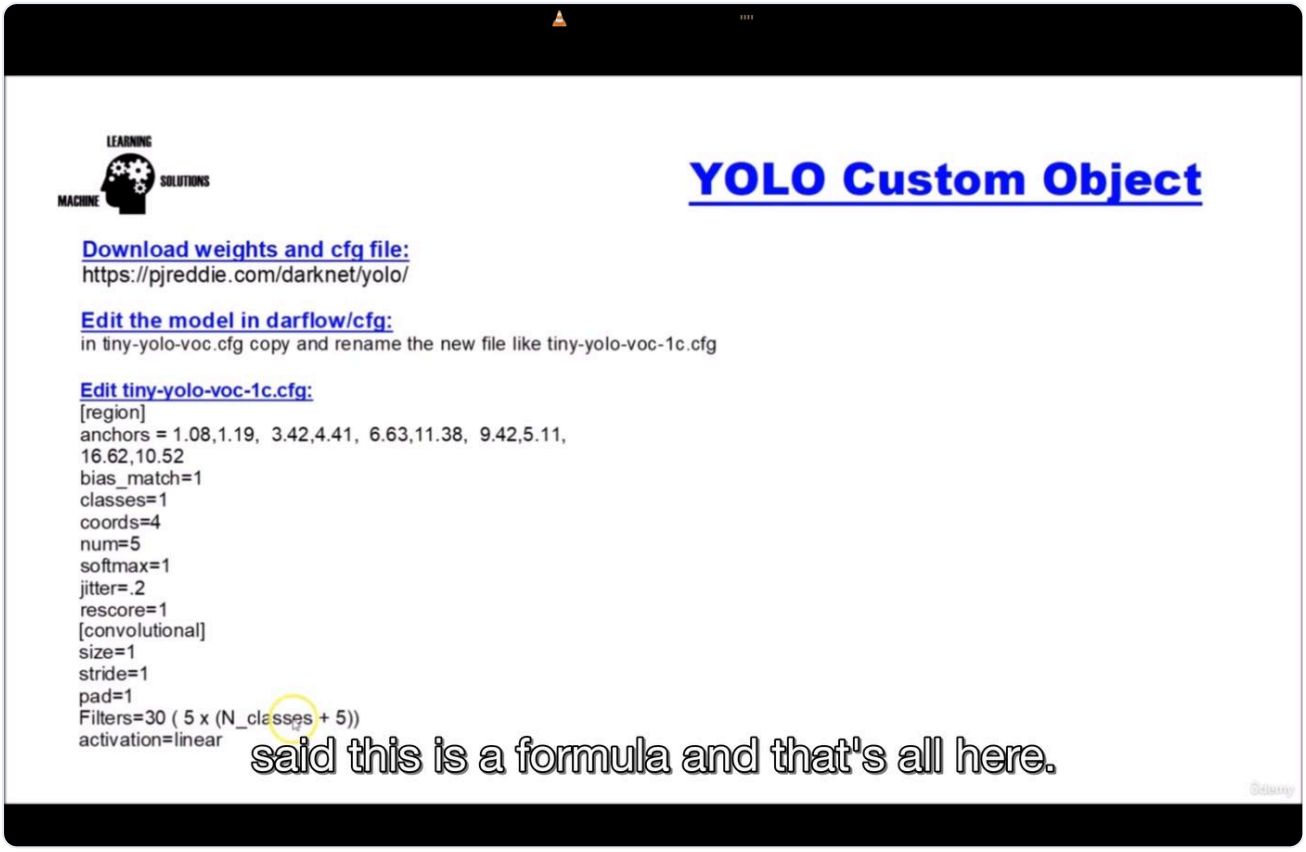


FIGURE 5.3 | Download Weights and Cfg File:

This is a formula.

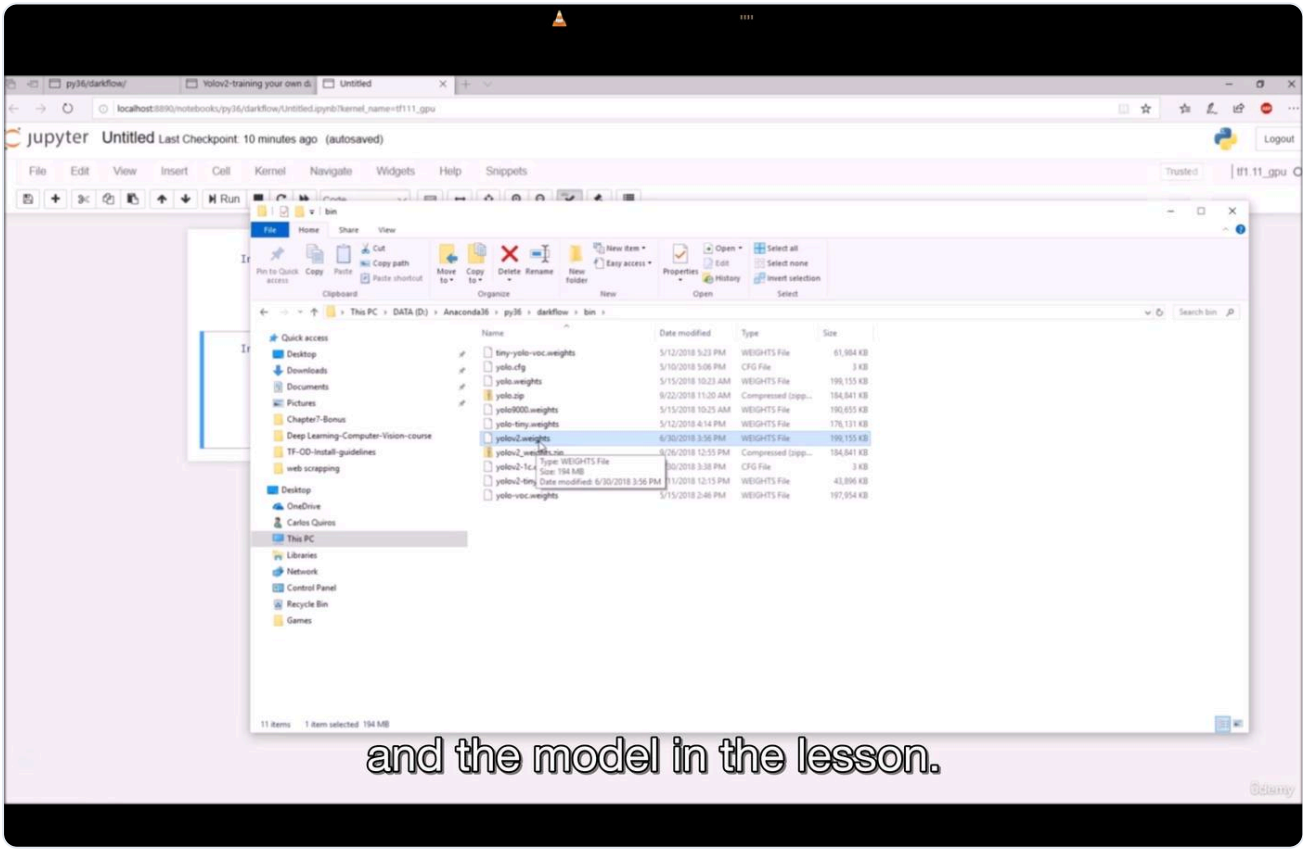


FIGURE 5.4

And that's all here.

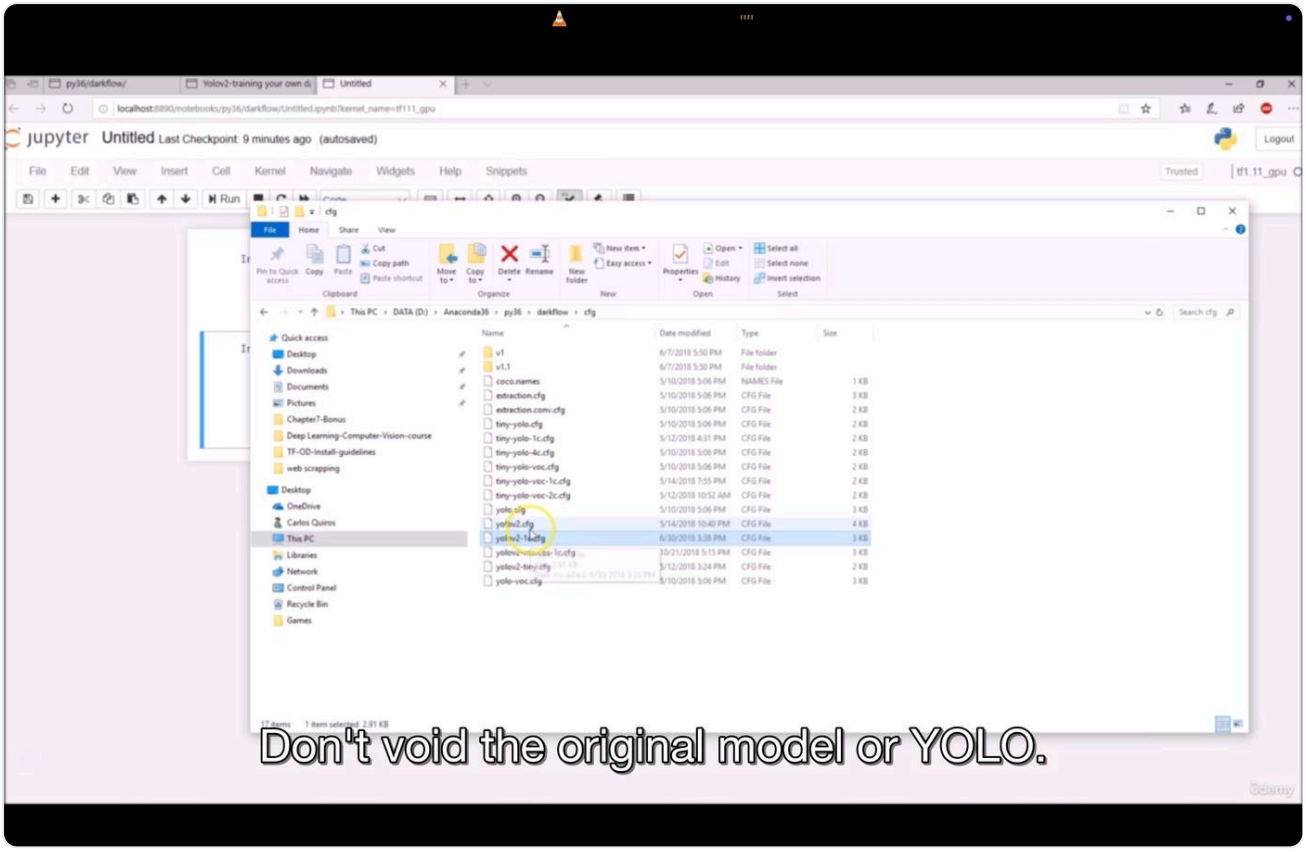


FIGURE 5.5 | Don't Void the Original or YOLO

Don't void the original model for YOLO. You only need to rename the original model with the name as you want. For this example, I'm going to use these weights. For YOLO version 2, I have uploaded these weights and the model in the lesson 51. You can check them. Now I have the model. I have the weights with this parameter load. Epoch is the number of epochs that I'm going to perform. GPU, 80%. This is a recommended value. Train through annotation. This is the route where your annotations are located. The same for the images. In my case, both of them are in the train folder. This is my train folder into the dark flow folder into an Akonda folder. I have the annotations folder and the images folder.

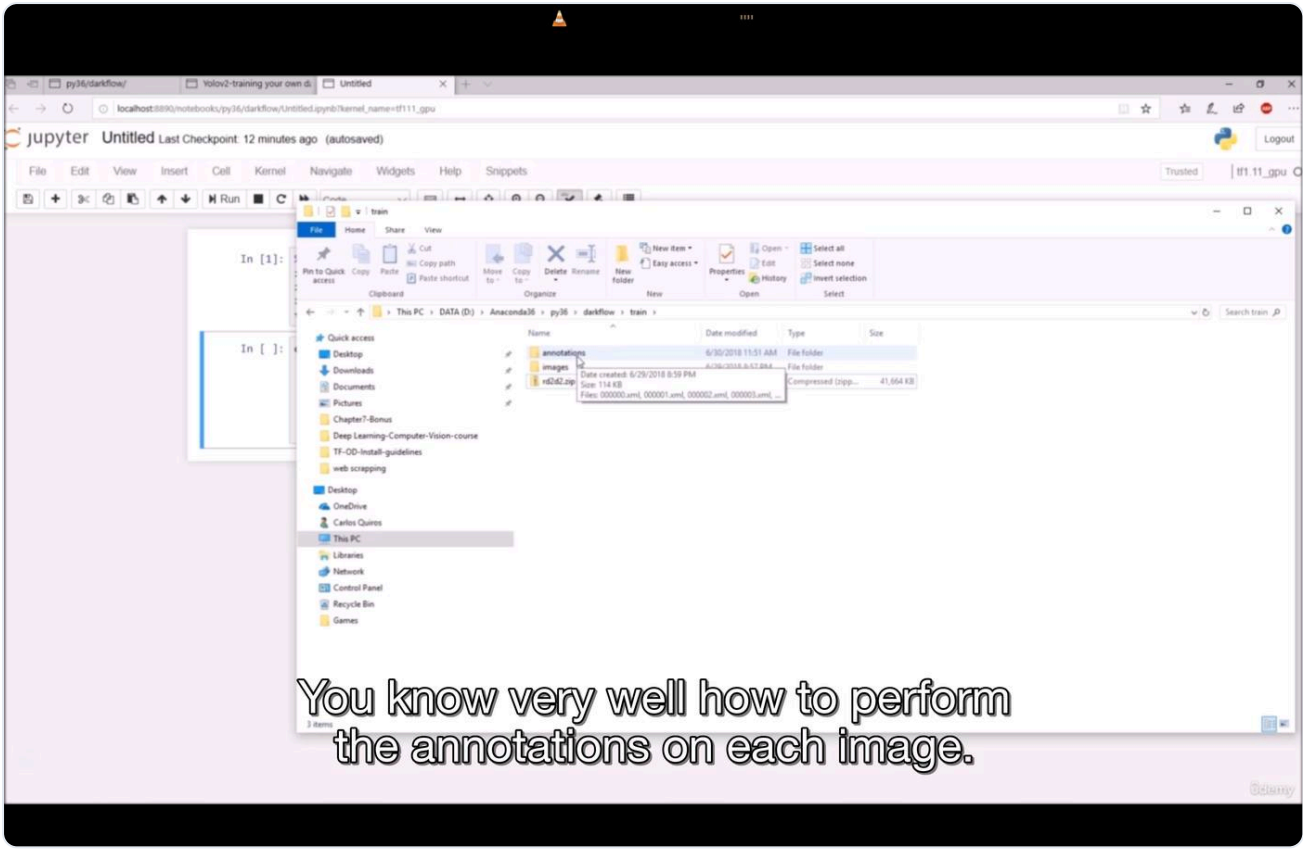


FIGURE 5.6

You know very well how to perform the annotations on each image.

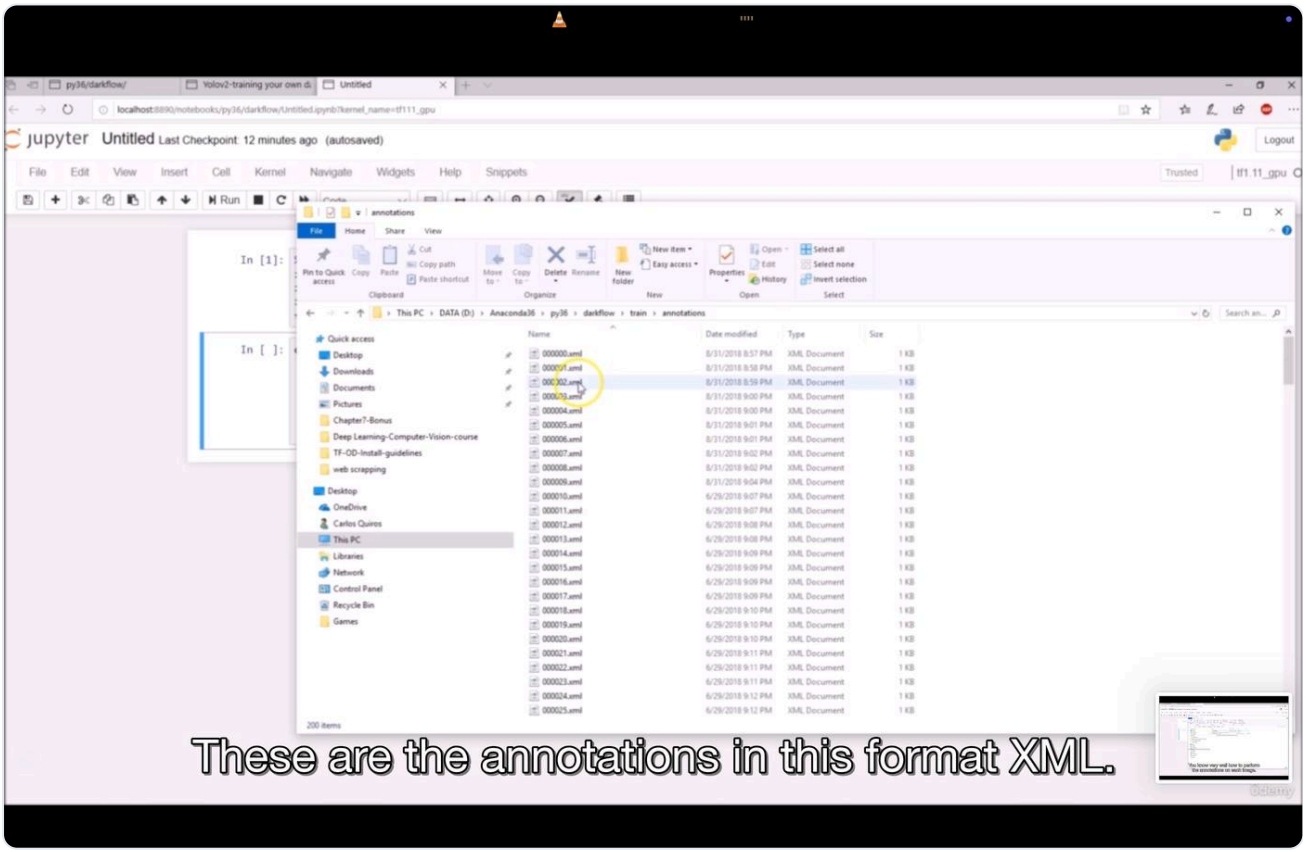


FIGURE 5.7

These are the annotations in this format XML about the images.

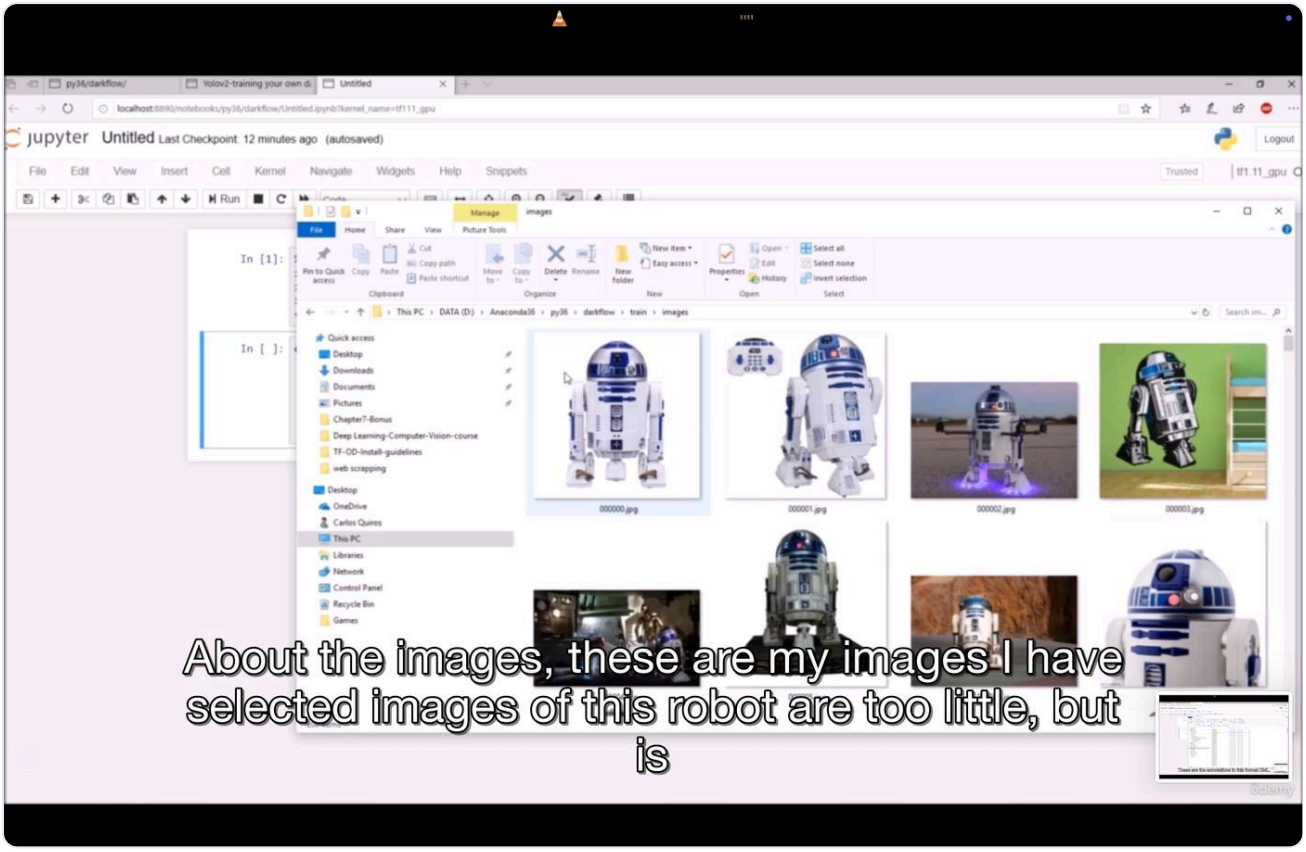


FIGURE 5.8

These are my images. I have selected images of this robot R2D2. But it's very important to note the format of the name. We need this kind of format with numbers. You need to normalize the names. I have previously told you how to do it. All pictures in JPG format, you can use PNG. It's not a problem. So we are ready with all requirements. We can launch our model with an instance of this function. I am specifying this previous variable with these parameters.

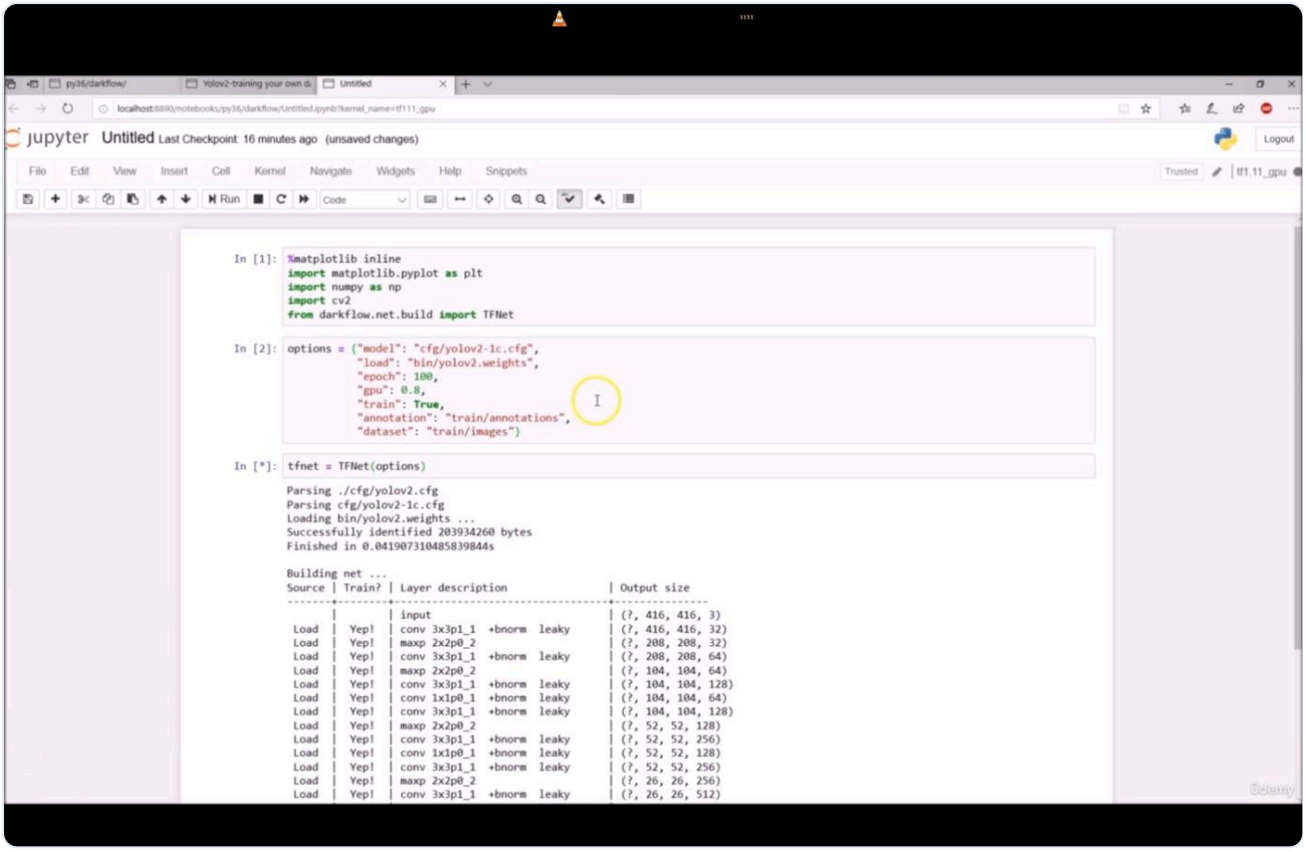


FIGURE 5.9

OK, I'm loading the model. This is a YOLO version 2 model. Very good. Now we are ready for the training. I'm going to measure the time with this expression. I only need to use the function train. Let's start the process. Very good. 200 instances. The number of epochs, 100. This is the number of objects in the images, 235 R2D2.

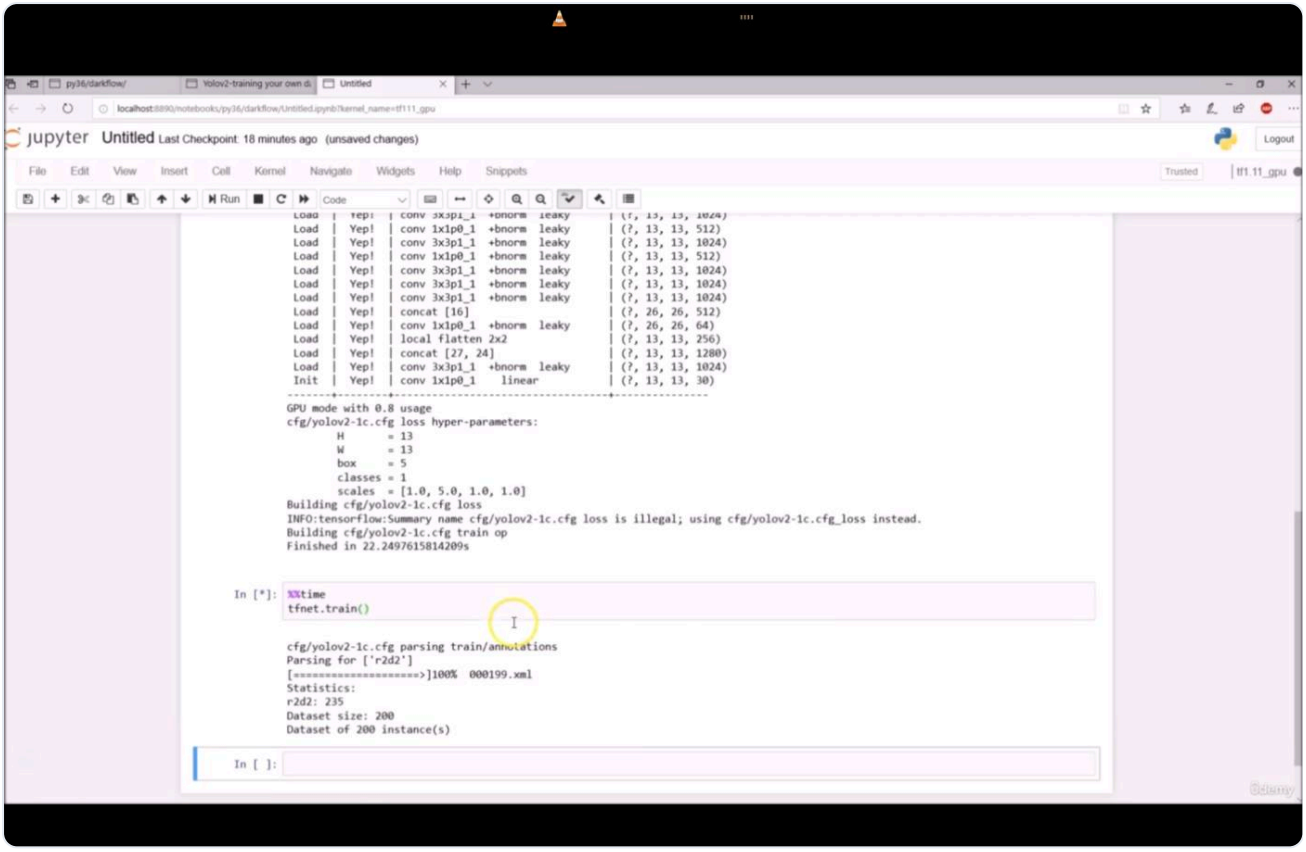


FIGURE 5.10

This is a label. Remember, I have a file containing this label. Very good. This is one epoch. It takes a while. So I'm going to pause the video. When you are training your model, you are creating checkpoints. This is a folder containing the checkpoints. There are some checkpoints. You will get more during the training process. This is a labels file in TXT format.

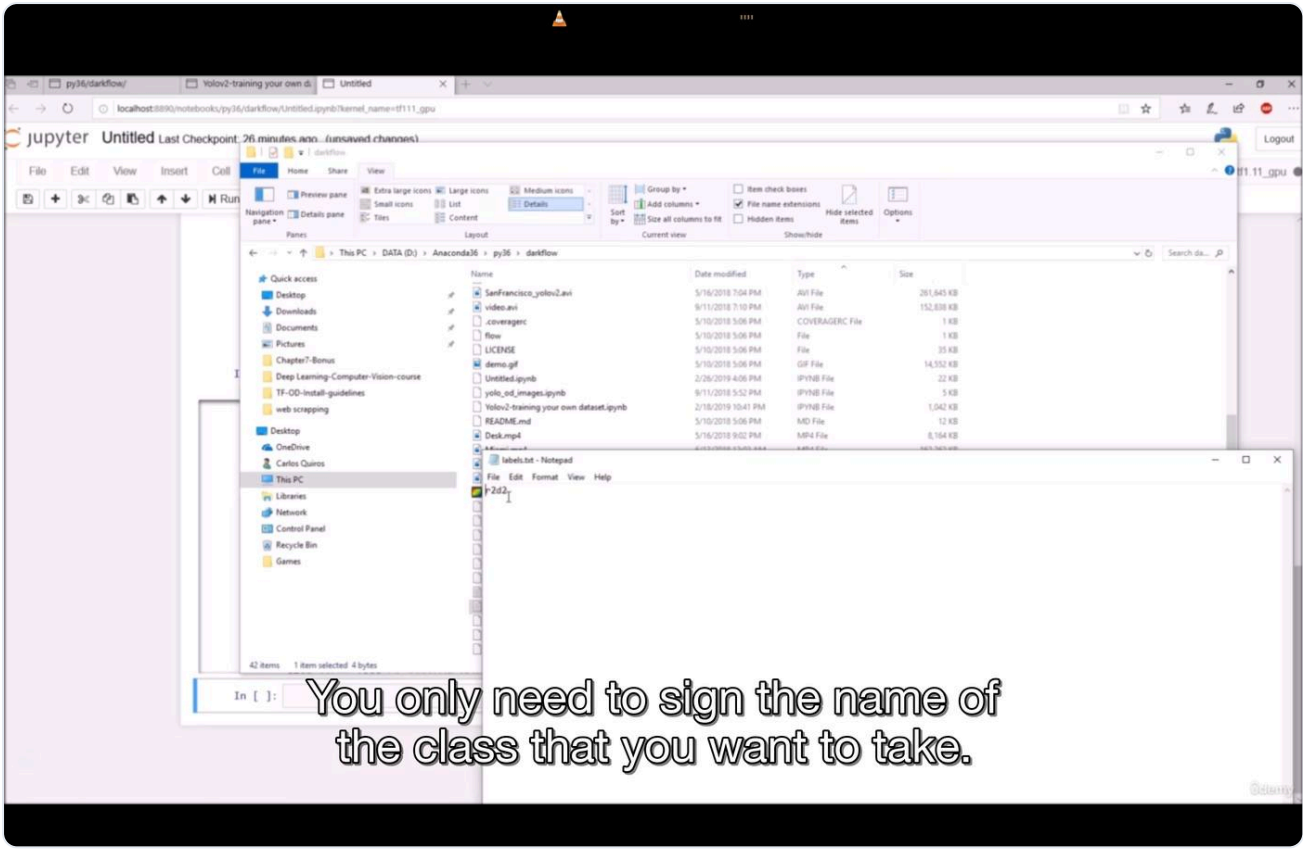


FIGURE 5.11

You only need to consign the name of the class that you want to detect. You can change with the name as you want. If you have more classes, you need to write one name per line. We have a start in a loss 104. Now we are in the epoch 32. We have a loss 9.9. We are continuously decreasing the loss. I'm going to pause the video. Everything is going smoothly. Now we have in the epoch number 50, the loss is about 2.7. It's a small loss. It's very good. OK, we are reaching the end. This is a checkpoint at step 1,125, epoch 94. In the epoch 96, the loss is 0.73. It's a small value. Epoch 99, the loss is about 0.66. It's also a smaller loss than before. Epoch 100, checkpoint at the step 1,200.

This is the last checkpoint. And this is the final loss, 0.68. The total training time, 47 minutes. OK, we have performed the training process. Now we have checkpoints. The last step is this number, 1,200. Now this is the folder containing the checkpoint files. This is the last number, the last step, 1,200.

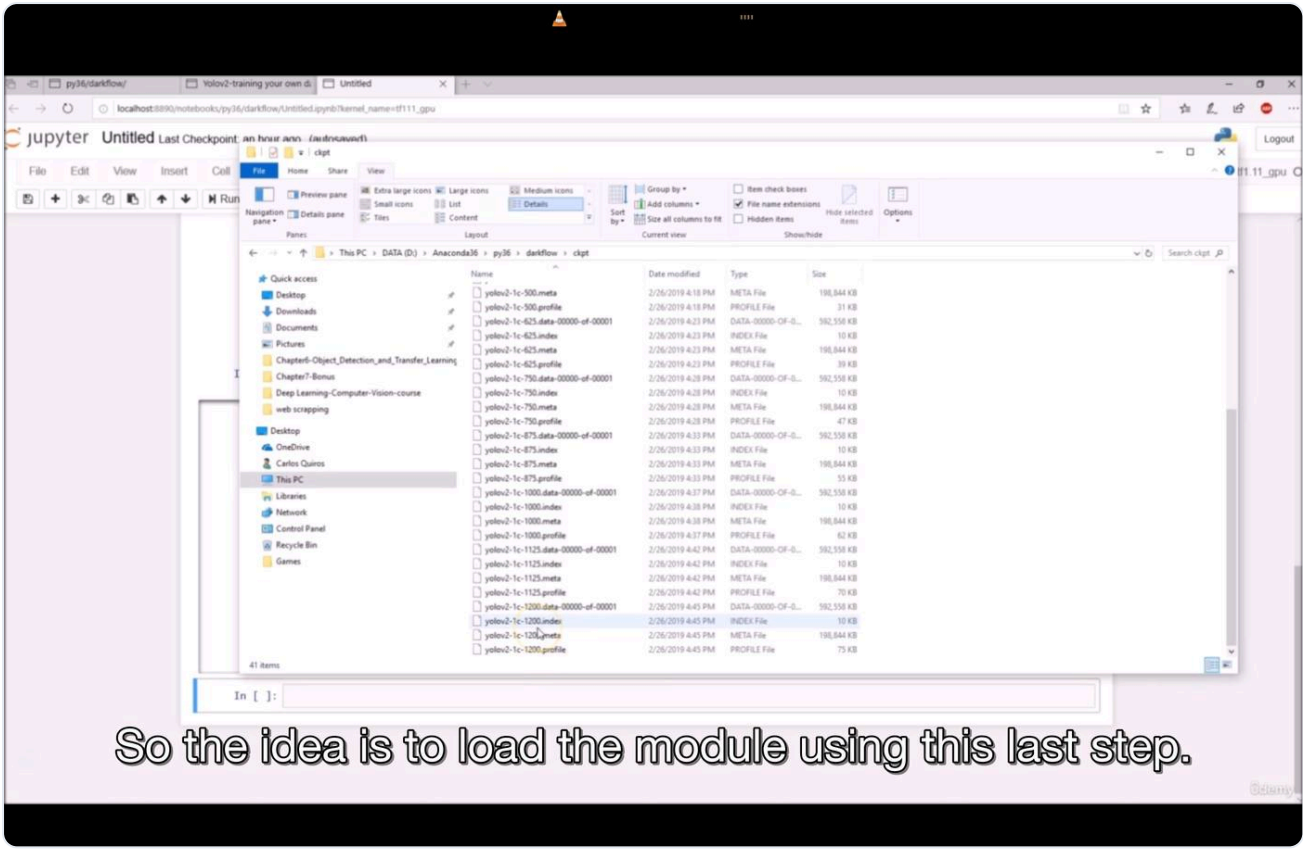


FIGURE 5.12

So the idea is to load the model using this last step. So you have two options. You can refer to this number, to this step, or you can use minus 1. With minus 1, I'm referring to the last step. Now I'm going to load the model. I need to define new options. The threshold is a limit, a limit in the confidence. In this case, 10%. Every object with a confidence higher than 10% will be classified with the label R2D2. Load, this is a point where the model will be loaded. The last point, the step, 1,200. GPU, 1.0. I'm going to use the total size of the memory in the GPU card.

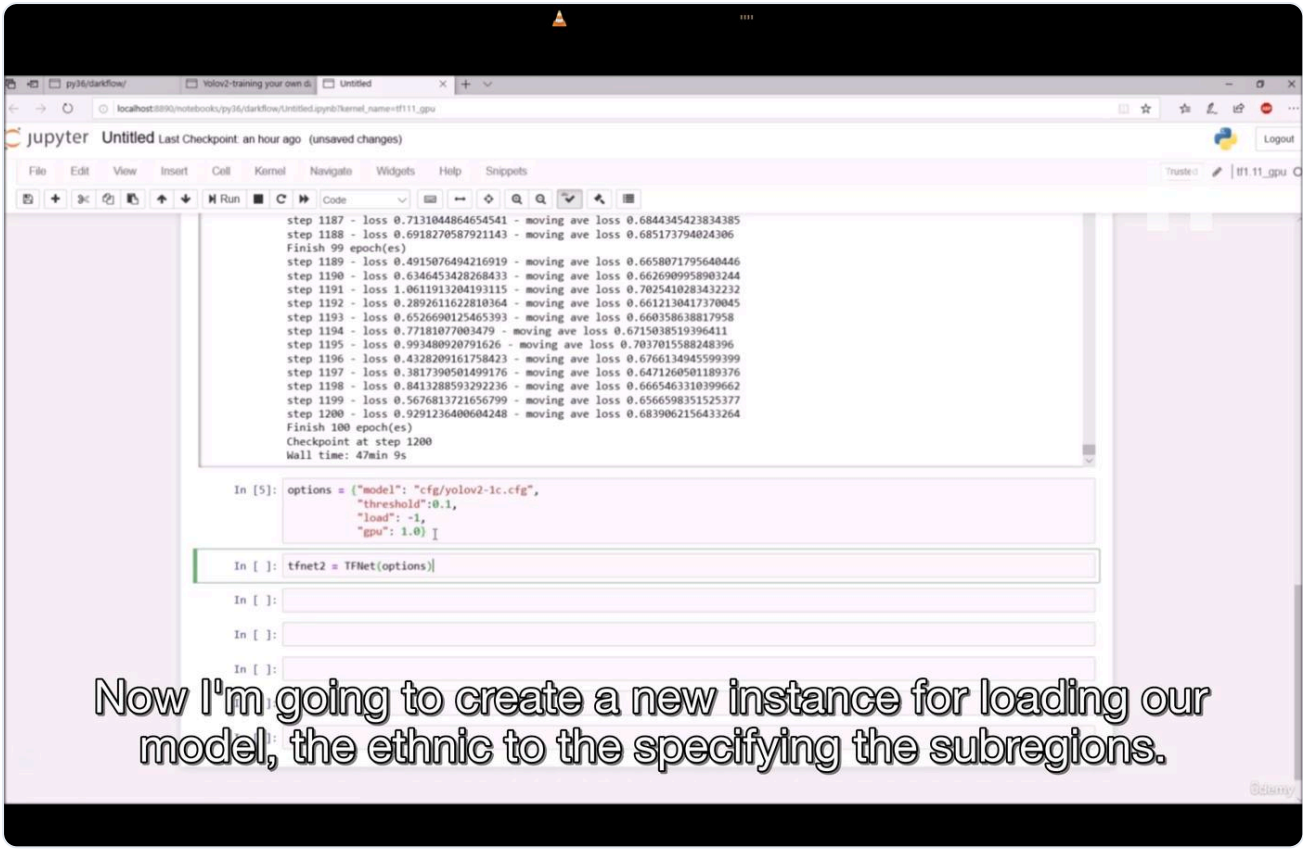


FIGURE 5.13

Now I'm going to create a new instance for loading our model. TFNet2, specifying these options. I'm loading the YOLO model.

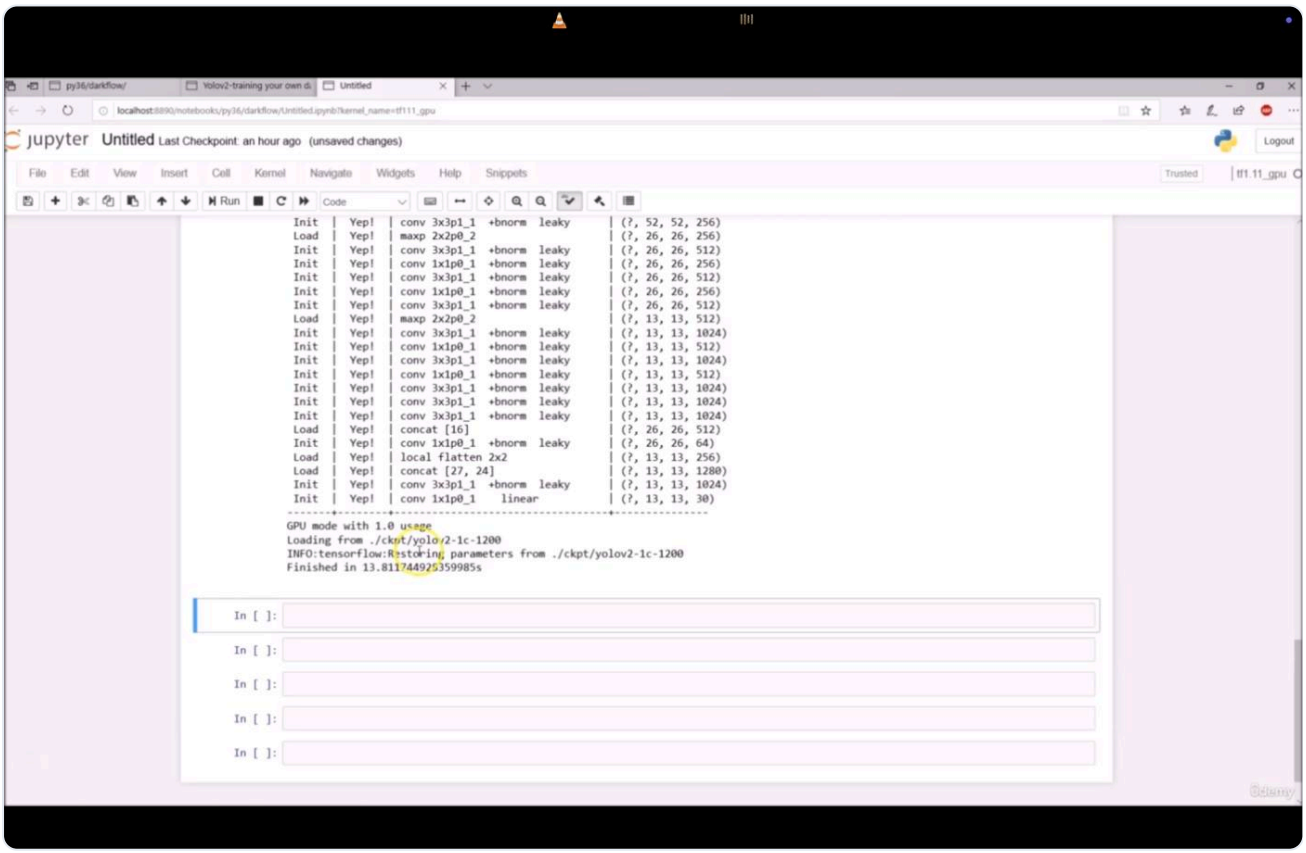


FIGURE 5.14

This time, the GPU is in 100 usage. This is a model that I'm loading with the last step. Now this function to load the last step of this model into this variable.

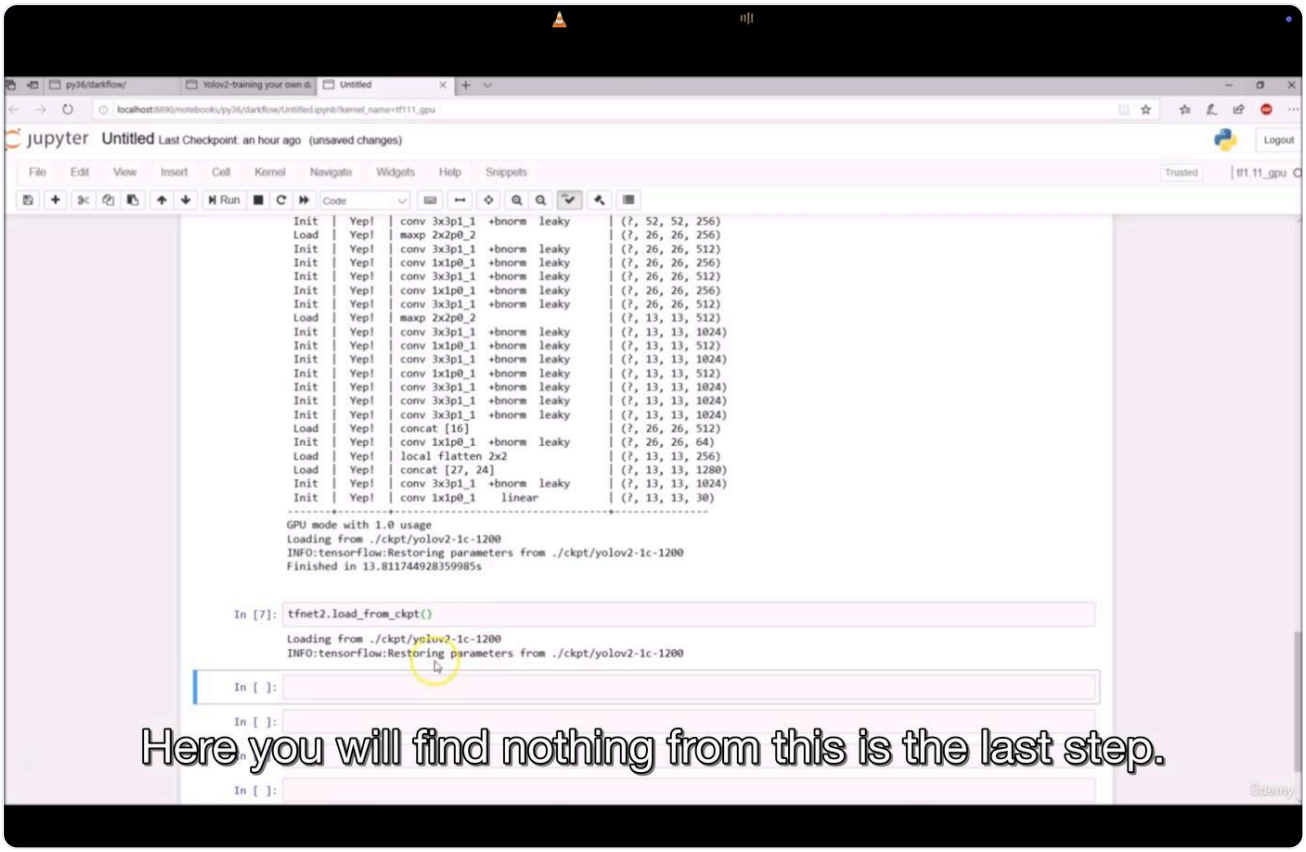


FIGURE 5.15

Here you will find loading from. This is the last step. Now we can test our model. First, I'm going to take an object into an image. So I'm using this image that is contained into the images folder. I'm using OpenCV to read this image. I'm performing the conversion of the formats from VDR to RGB. This is the new image. And after that, I'm passing this image to the model to perform the prediction. I'm using return and the line predict, which is a function for prediction into this model TFNet2. Finally, this expression will return an output.

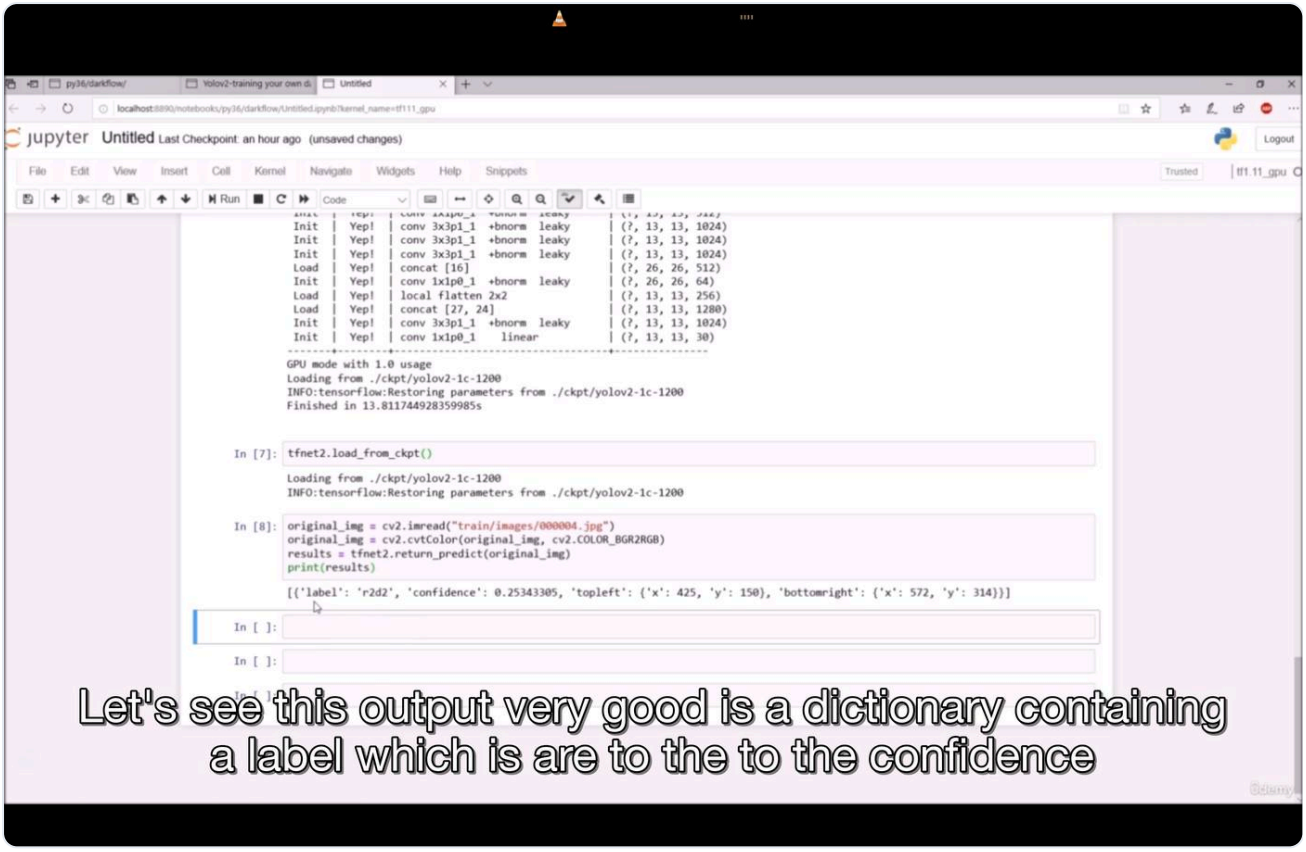


FIGURE 5.16 | Let's See This Is a Dlictionary Containing

Let's see this output. Very good. It's a dictionary containing a label, which is R2D2. The confidence factor or accuracy is 25%. It's greater than 0.1. It's OK.

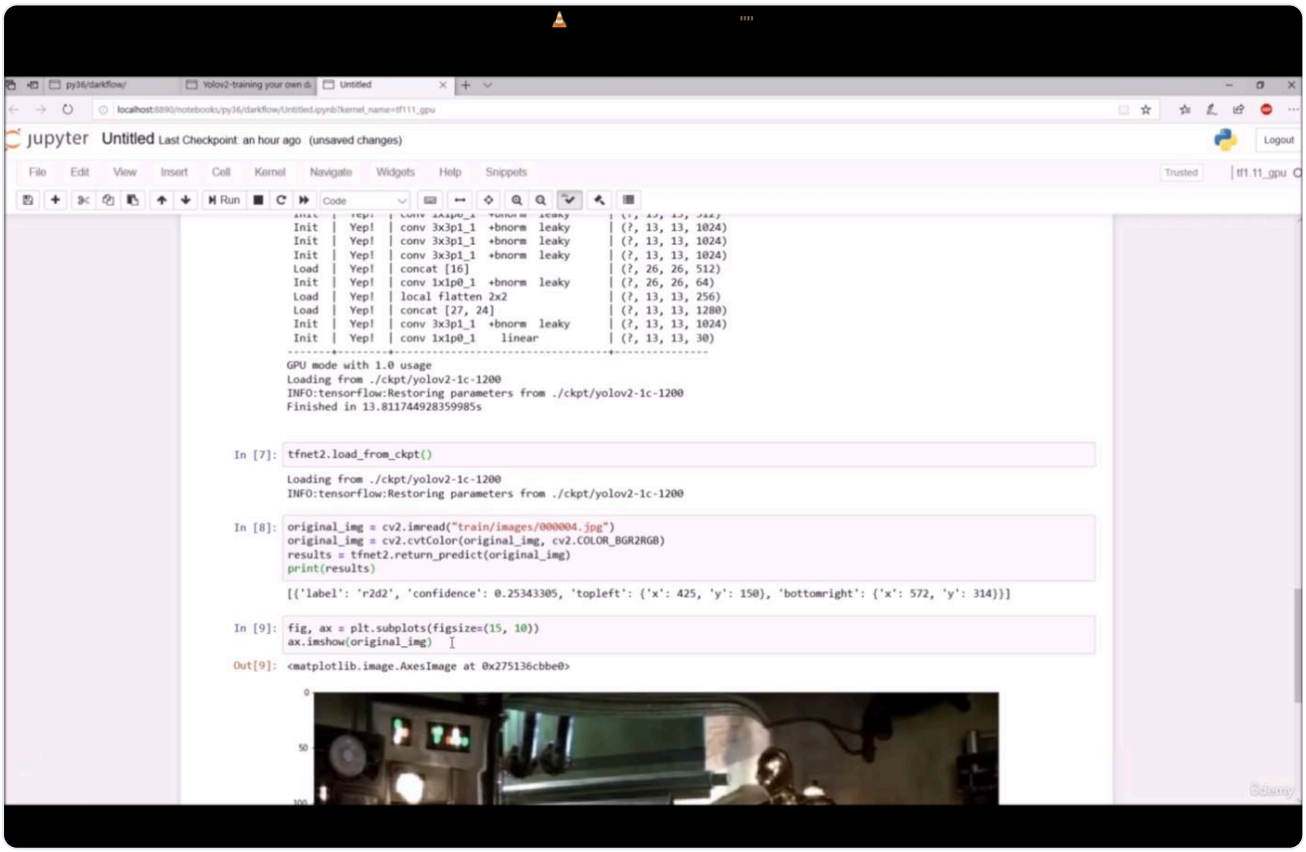


FIGURE 5.17

And these are the coordinates of the bounding box, the top left coordinates and bottom right coordinates. With these coordinates, we are identifying where it's located, this object, the robot R2D2. Let's see how it looks like the original image. I'm creating a graph with my plot leaf. I'm using the function inshow to visualize the image. OK, this is the original image.

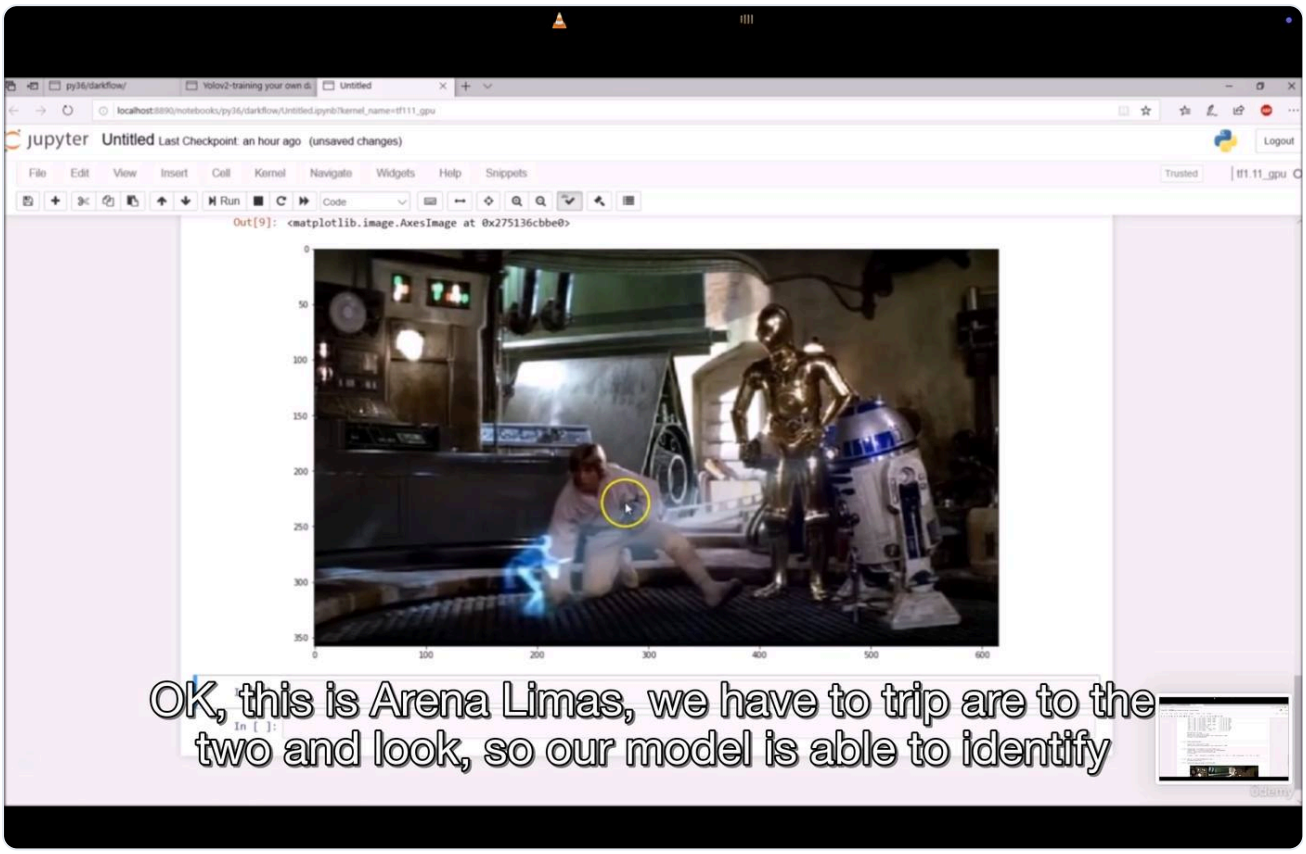


FIGURE 5.18

We have the TRIPIO R2D2 and look. So our model is able to identify where it's located R2D2 into this image. So the next step is to create a function for plotting the bounding box using the coordinates, these coordinates, and the label. OK, this is our function. It's very similar as before. I'm receiving an image. I'm receiving the predictions. Predictions is a dictionary. So I'm creating a copy of the original image, creating a new variable, new image. I'm browsing into this dictionary, extracting the coordinates to plot the bounding box. I'm also extracting the confidence factor, multiplying this value by 100, scaling the value in percentage. The label is the same.

I'm retrieving the content of the label and I'm creating a string adding the information of the confidence, showing the results with two decimals. So we have the data. Now I'm going to use OpenCV to draw a rectangle to write a text using the image and using the values of the coordinates. The procedure is the same as the previous lesson. Now we can use my plotlib to visualize the same image but with the prediction.

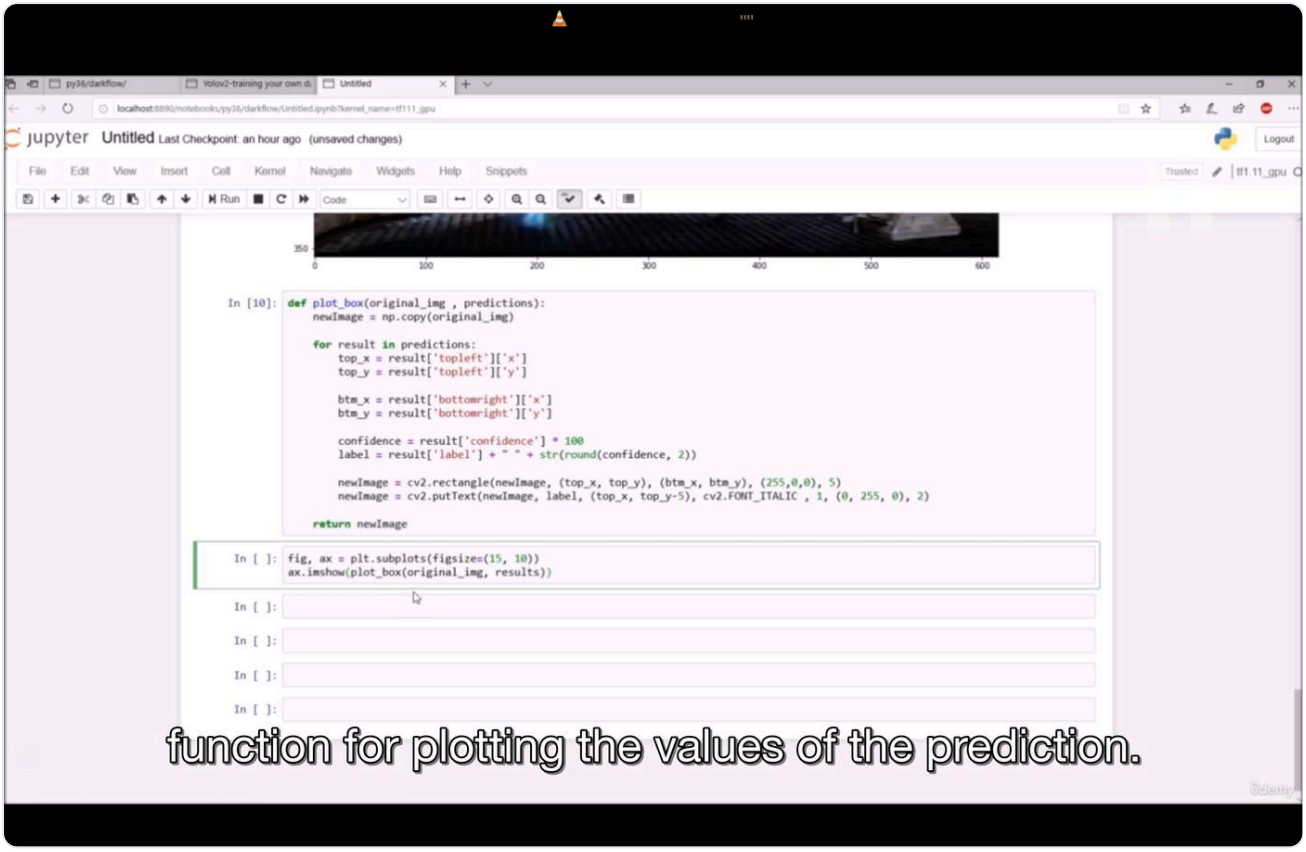


FIGURE 5.19

OK, I'm using this function for plotting the values of the prediction, the bounding box, the label, and the confidence. Let's see the results. It's very good. This is R2D2. This is a label, R2D2. This is a confidence factor.

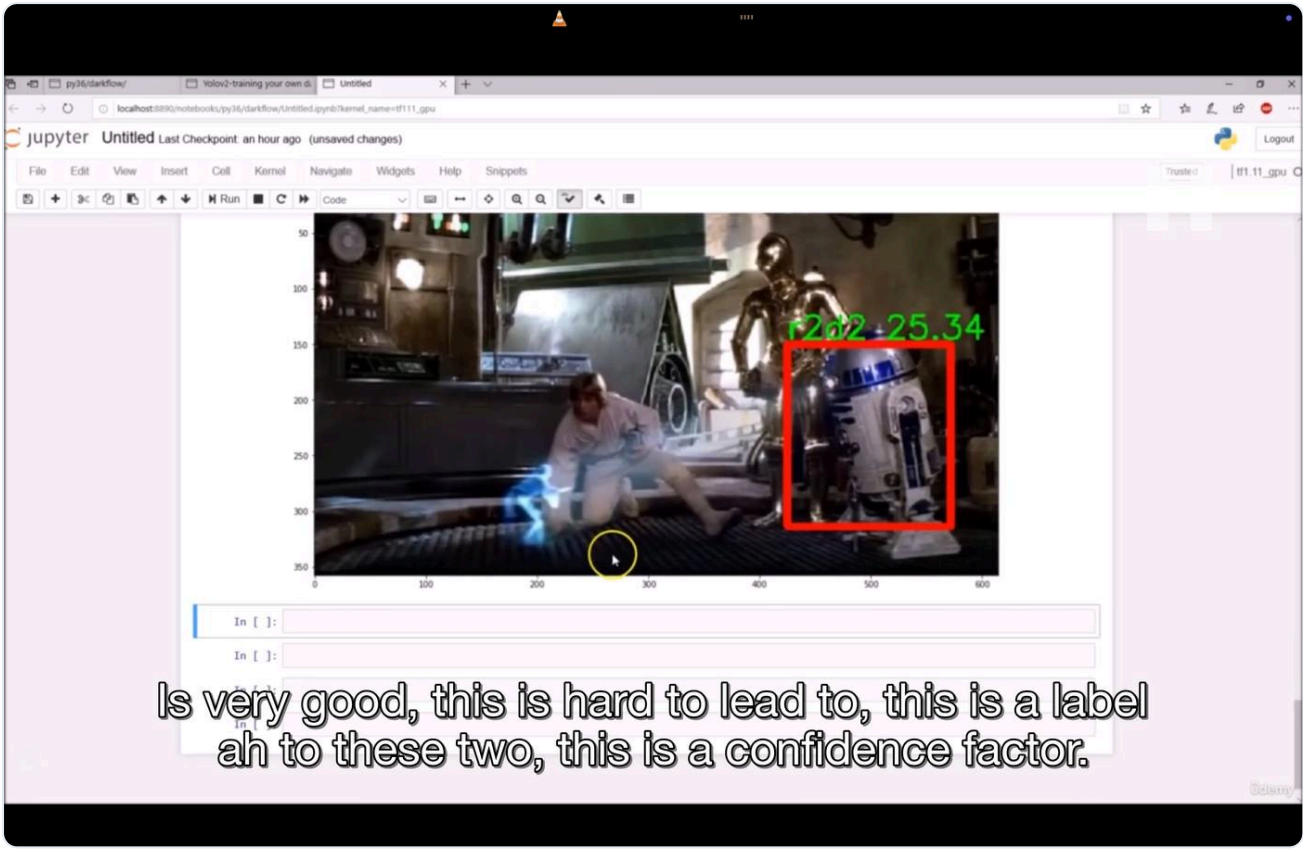


FIGURE 5.20

OK, to finish this lesson, I'm going to apply our model into a video. A YouTube video. To perform that, I need to import YouTube video from ipython.display. I'm going to visualize a video into this Jupyter notebook. This is a link of the video.

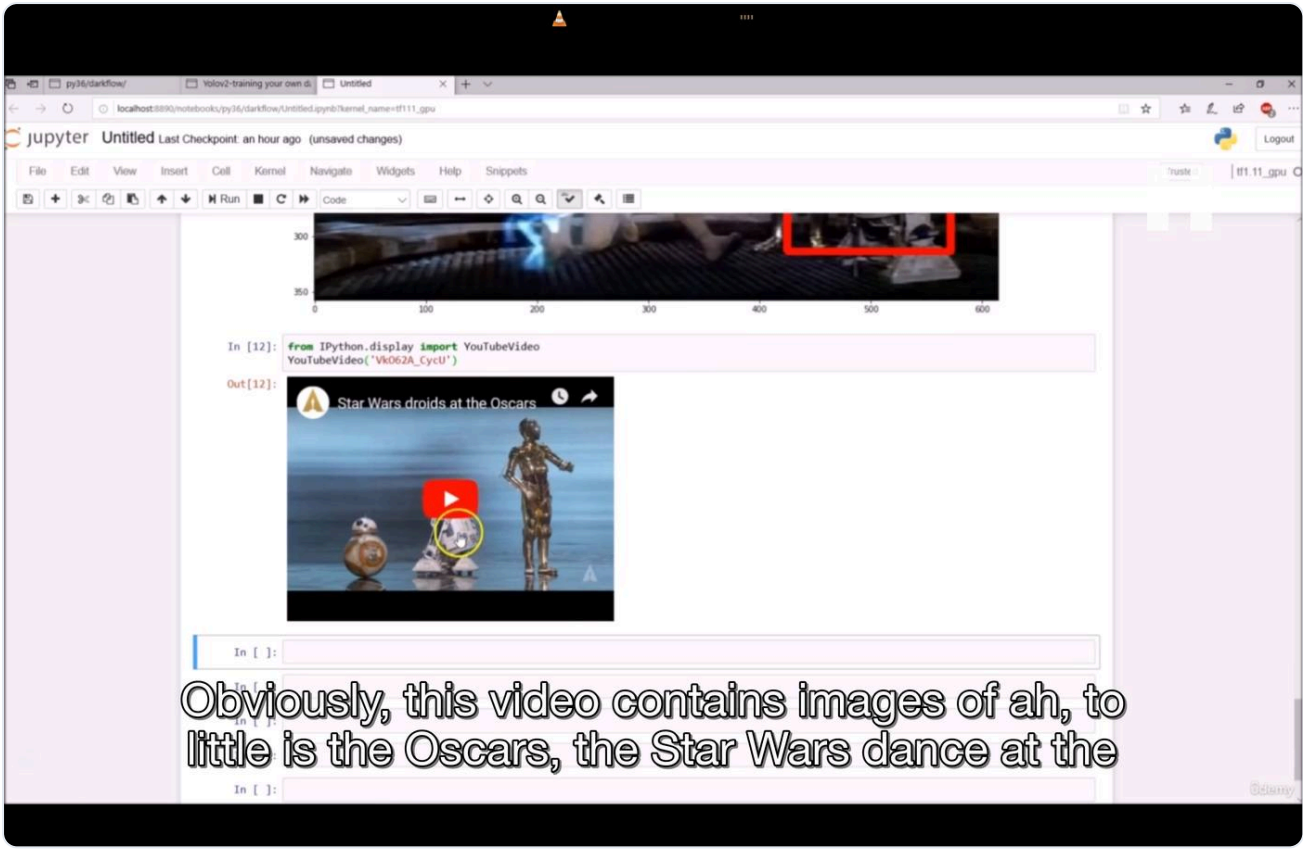


FIGURE 5.21

Obviously, this video contains images of R2D2. It's the Oscars, Star Wars droids are the Oscars. We have three different droids and the idea is to identify R2D2. So let's do it. OK, to perform that, I'm going to need a library. I need puffy. This is an instruction. You need to run these instructions in your Anaconda prompt. After that, you will be able to import puffy. Now I'm going to take a different approach. I'm going to create a video recording the images but with the predictions. I'm going to draw different bounding boxes with different confidence and the labels around R2D2. You can also perform live streaming if you want.

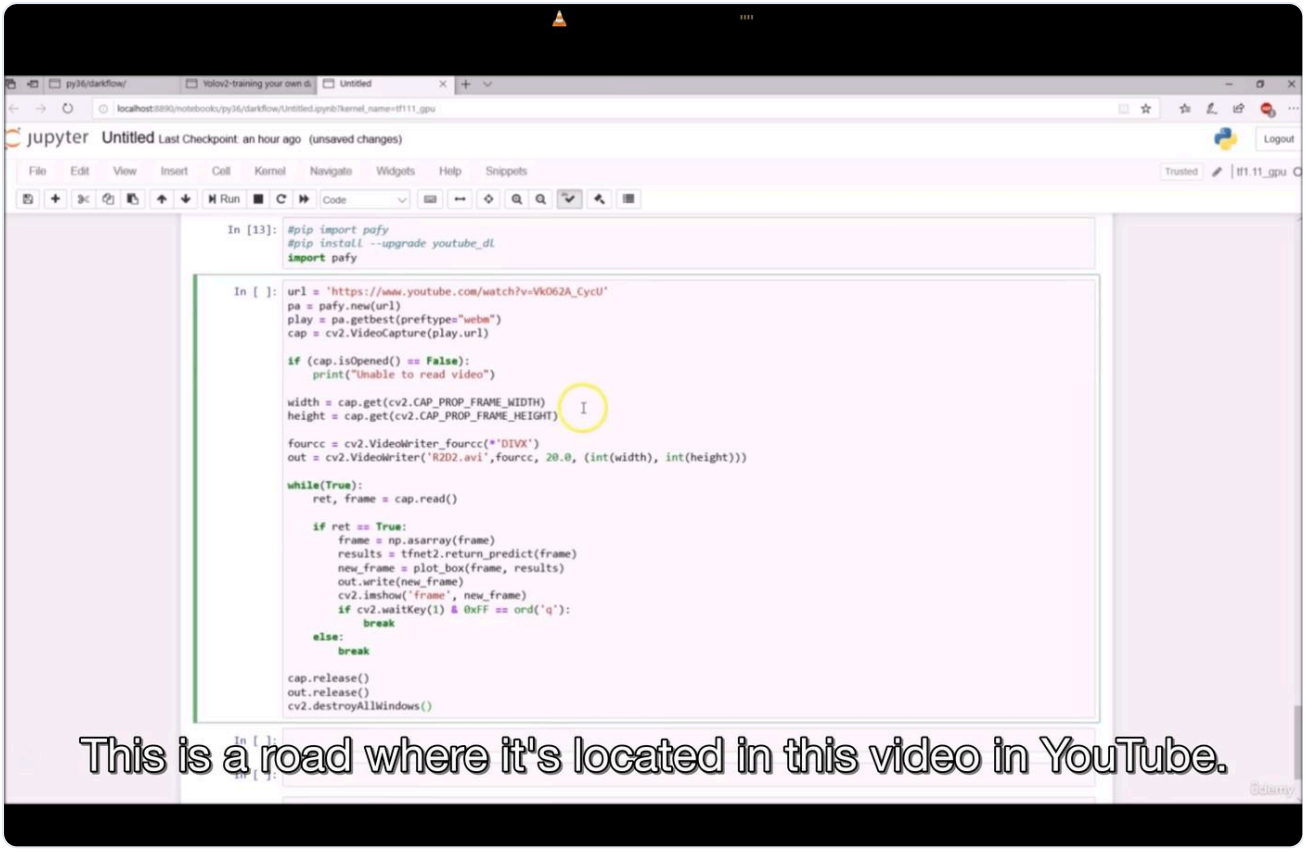


FIGURE 5.22 | This Is 4 It's Located in This Video in YouTube

This is a route where it's located in this video in YouTube. I'm using puffy and I'm retrieving the content of this video into this variable using this function. After that, I can use OpenCV to capture the content of this variable. Now in this variable cap, we have the video. If this variable contains a video, we can proceed with the code. On the contrary, if we don't have a video, we are going to print this message. I'm setting the size of the video with these variables in width and height. I'm creating a video. Because of that, I'm going to use VideoWriter to create a video. This is a format of the video. I'm specifying the video size, the frames per second, and the name of the video that I want to create.

After that, I'm going to pass the original video. I'm going to perform the prediction for each frame in the video. So I can use our function for plotting the bounding boxes. The idea is to draw the bounding boxes into a frame. After that, I'm going to pass the new frame to OpenCV to display the new frame. I can quit this process using the key q. Finally, I'm releasing the resources. OK, I'm reading the video. After that, I'm generating the new content with the bounding boxes, creating a new video. That's all. OK, this is the final result. This is the trip view. This is R2D2. As you can see, there is a bounding box into this robot. OK, they are very close because of that.

It's not simple to identify this robot, but now it's quite good. R2D2, the confidence factor is high. OK, here we go again. We have, again, R2D2.

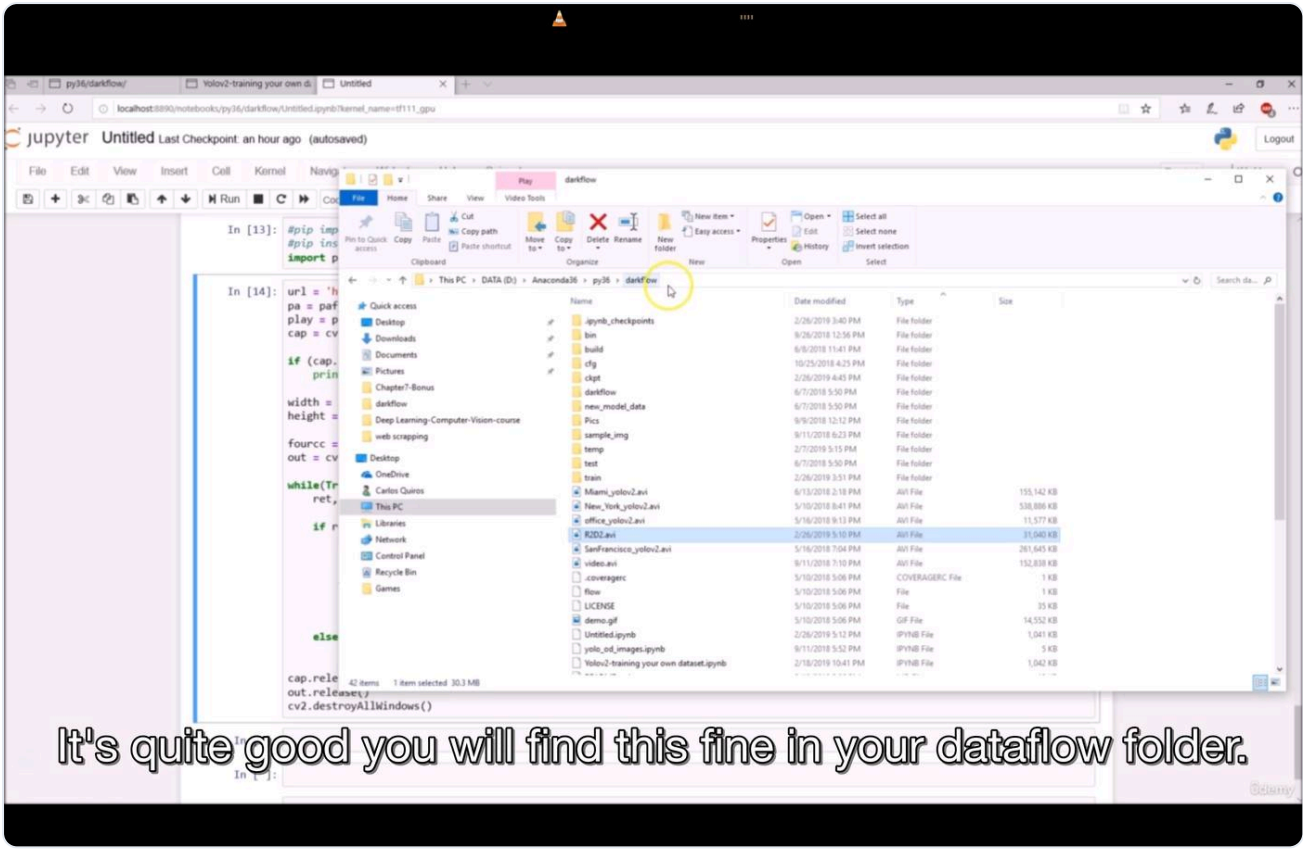


FIGURE 5.23

It's quite good. You will find this file in your DartFlow folder. Very good. That's all for now. If you have any question or doubt, you can contact me in QF forums, review this lesson, and practice a lot. Keep in touch. Bye bye.

LECTURE 51

Darknet Folder and Google Drive

[MUSIC] >> Hello everybody. Welcome to a new class. I'm going to present you an implementation on how to train your custom dataset using YOLO version 3. We have seen the implementation in YOLO version 2. At the final, we're going to compare these two results. YOLO version 3 represents an advantage of the previous model.

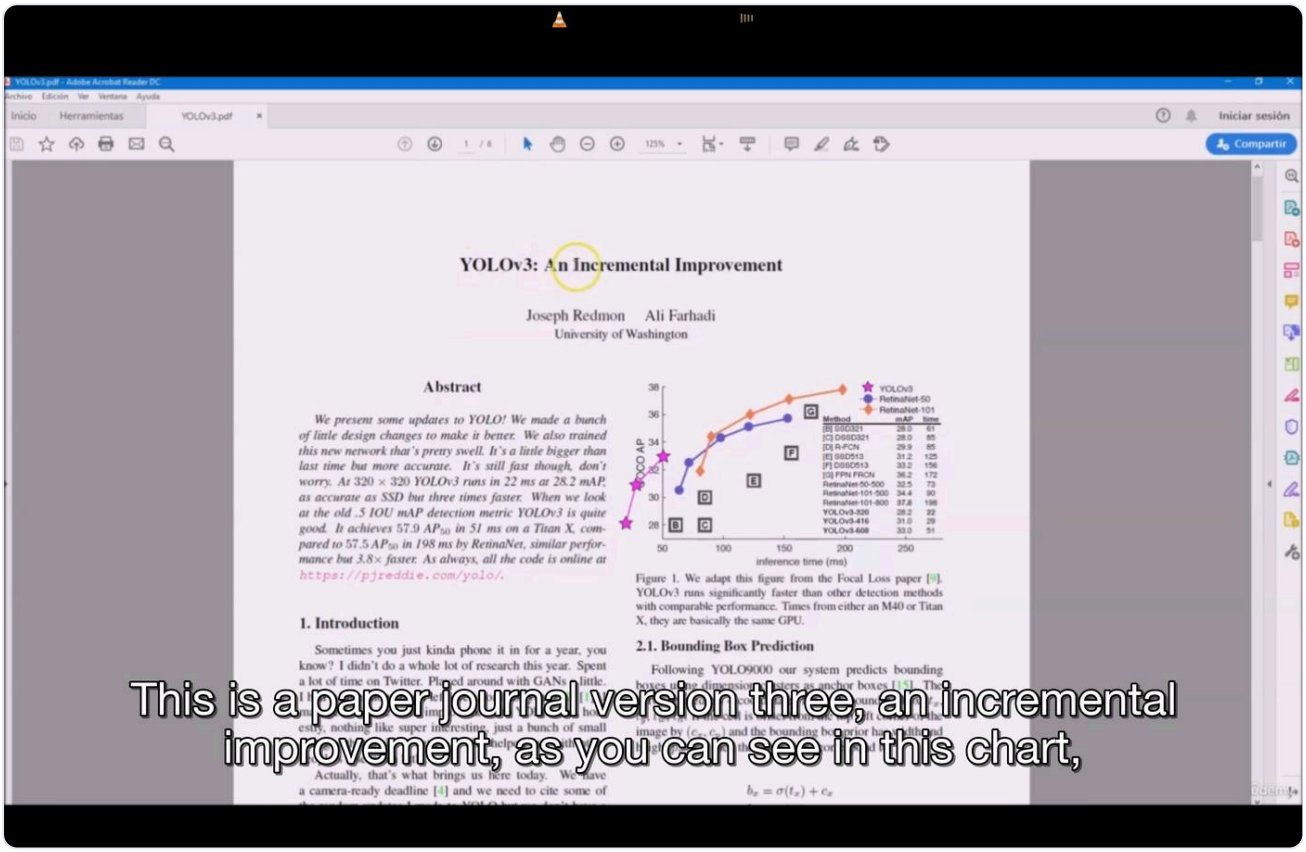


FIGURE 6.1

We have the same speed but we gain more accuracy. If you want to dive into the details of YOLO version 3 model, you can review the original paper I have attached in this lesson. This is the paper, YOLO version 3, an incremental improvement. As you can see in this chart, the pink starts represents the accuracy values on COCO dataset. If you read here, YOLO version 3 runs significantly faster than other detection methods. The results are comparable to other methods like RetinaNet-50 or RetinaNet-101, but the speed is greater than these methods.

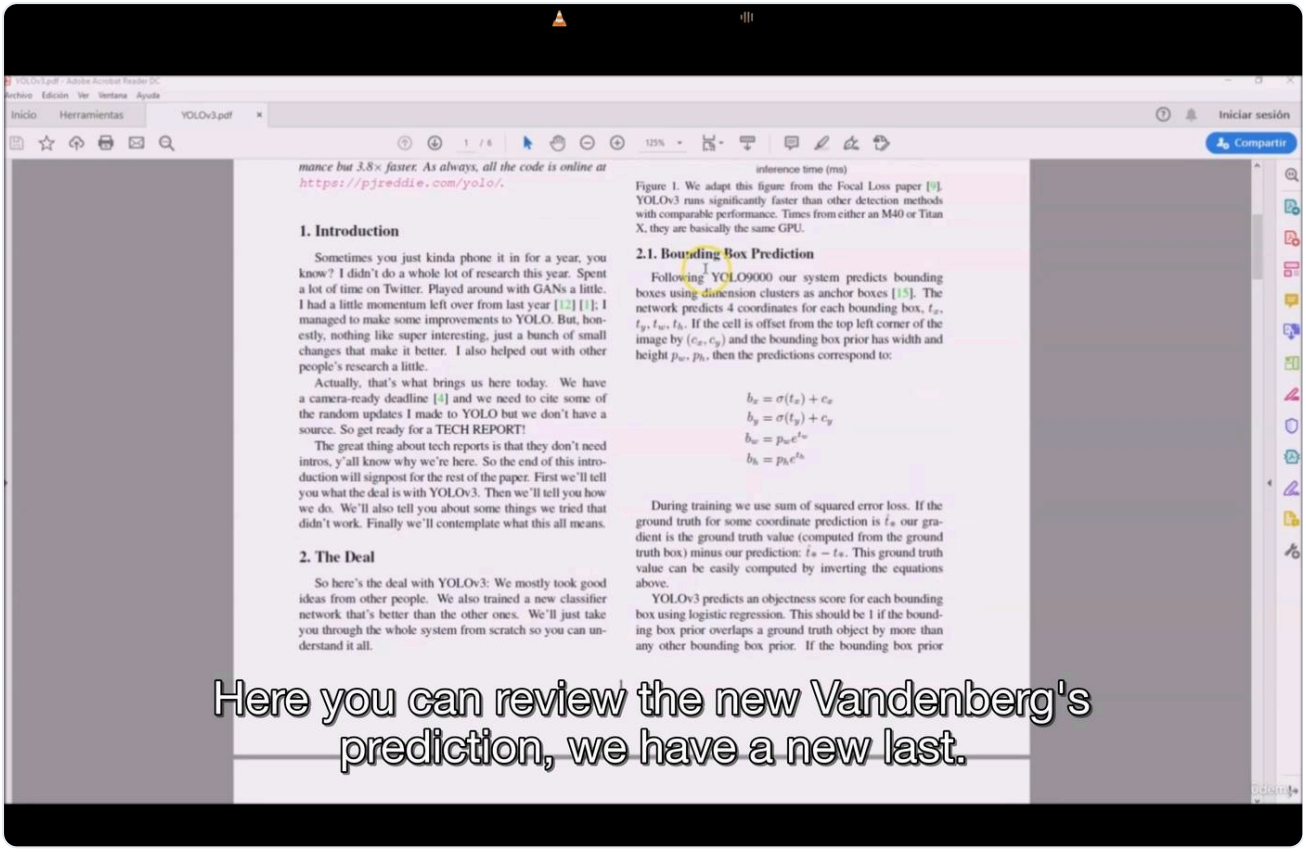


FIGURE 6.2

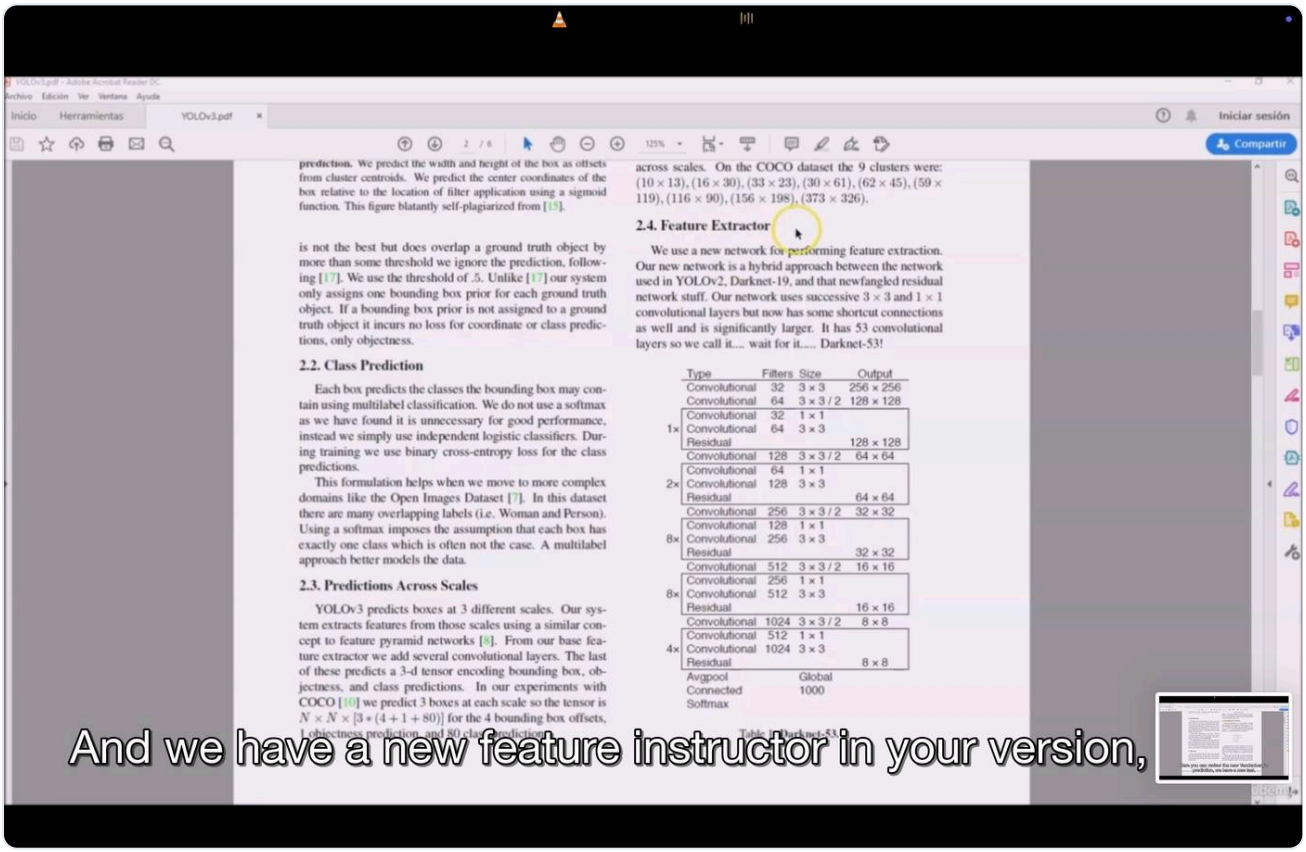


FIGURE 6.3

Here, you can review the new bounding box prediction. We have a new loss and we have a new feature extractor. In YOLO version 2, we had darknet with 19 convolutional layers, but now we have a new network with 53 convolutional layers. As you can see in this model, we don't have pooling layers. We only have convolutional layers and residual layers. The layers are connected in different stages. This model is also trained at different scales, and this is an advantage of the previous model because now the model learns objects in a large size, medium size, and tiny objects. Okay, you can review more in detail this paper, but now we're going to come back to a practical lesson.

We are here. For this lesson, I'm going to use a powerful tool that is Google Collaboratory, or Google Colab in short. With this service, we can access to a powerful GPU, that is Tesla K80 GPU with 12 gigabytes of RAM. The runtime is limited to 12 hours. After that, you need to reconnect the notebook. This service is totally free. This notebook is very similar to Jupyter notebook. You can run the same commands, but in Colab, you need to know about Linux. All commands about the operating systems are in Linux. If you want to know more about Colab, you can check these links. The first thing that I need to do is to check on this menu edit, here, no dot settings, if I'm running in GPU, because there is another options like none.

None represents CPU, but in this project, we need to use the GPU power. Runtime, Python 3, it's okay. Finally, if you want to use this tool, you need to have a Google account. In my case, I have a Google Drive account. The credentials are the same, like Gmail account. You can access to different products of Google with the same account. If you want to create a new Colab file, just enter in Google Drive, and press the right click of your mouse. Here is a new window, select more, and here you will have Google Colaboratory. And now we have a new file. Okay, let's come back to the original file. I don't pretend to explain you everything about Google Colab, so you need to have a previous knowledge about it, and you can review these two links, which are good references about the functionality of Google Colab.

Okay, let's start. On the left side, I have table of contents, which are the headers, introduction, Google Drive. We also have snippet codes, if you want to run some of them. And here we have files. This is the structure of the folders. One of the main properties of Google Colab is you can link this notebook with your Google Drive. Then you can retrieve data and use this data for your models. I'm going to run every cell explaining the purpose of this code, cell by cell. Okay, the current version of TensorFlow is 2.x, but for this model, we need to run TensorFlow in previous versions, version 1.x. To do that, we need to use this magic sign, percentage, and this command, TensorFlow version, specifying 1.x.

After that, we can import TensorFlow and check the version. We are running the first cell. This is the current version. We're going to use 1.15. Perfect. As you can see in this course, many projects run properly with previous versions of TensorFlow or other libraries. We don't need to use the latest versions of all programs. So my advice is always check the version of the programs that you are using. Okay, next we need to link the Google Drive account to this notebook. To do that, I need to import this function drive. I'm going to link my Google Drive account and all content in this account to this notebook. Using the function drive and the function mount, I need to specify the route.

This is a default route when we want to access to the content of Google Drive, /content/gdrive. Let's run this. And here, it's a link. Just press here. You need to select an account. I'm going to select this account. Press on this button, Allow, and a security code is generated. Just press here to copy it. Come back here. Enter your authorization code. Paste it and hit Enter. Perfect. Mount it at this address. After that, we can check this link. Here, we have the structure of the data. Then you can refresh here. And now we have this folder, gdrive. In gdrive, we have my drive. Now we are connected to Google Drive. You can also check the content of a specific folder using this command in Linux.

In this case, I need to use this magic sign. A exclamation sign. With this expression, we are going to retrieve the content of this darknet folder. Where is located this folder? In Colab, in my drive. My drive, Colab, darknet. This is the content of darknet. I'm going to run here. And the content is the same like this. Very good. Okay, when we are dealing in GPU, we need to use CUDA, and we need to check the version of CUDA we are using. I'm going to run here. We are using version 10. It's okay. With CUDA, we need to use another important library for GPU computing, which is QDNN. You need to download this QDNN version for CUDA 10. And here, you have the links.

After downloading the file, you can use it. I have placed this installer in QDNN folder. This is a file for Linux. Then you can copy this file to the virtual machine. We are running this Colab file in a virtual machine. Then we use these commands, and we can run this out. Perfect. After that, we need to include this QDNN.h file to our local directory in the virtual machine. Okay, we have done the CUDA sync. Now we can clone the model. The original model is here in GitHub using the exclamation sign, the program GIT, the command clone. We can copy the original YOLO version 3 model. And this is a reference for you, which is another implementation. You can also use it.

There are many implementations in GitHub about YOLO. In this example, I'm going to use the default implementation. After cloning this model, we need to go to the darknet folder. I use %cd, very similar like in Windows, and darknet to specify the route where I want to go. Okay, let's run this. Cloning into darknet. Now we are in darknet folder. We can check the content of darknet with the command ls and the exclamation sign at the beginning. And here we have the content of the model. Please note that this darknet folder is not this folder because this folder is on Google Drive. This is a folder that I have created, but the model, the original model, was created in a separated folder.

This is a model that we have cloned previously. This is the content of the model. You can check using the command, or you can check visually accessing to this directory. Perfect. Now, inside this darknet folder, I need to install this library, vim. I need to run here. Perfect. We have installed the vim library. Now we can get the weights. We are using the transfer learning approach, so we need to use weights on the base. After that, we can train our custom dataset. When we are training our custom dataset, we are not starting since scratch. We are using pre-trained weights. So, YOLO version 3 has these weights. We only need to use the command in Linus, wget, specifying the route where it's located these weights.

We are downloading the weights. Remember, we are working inside this darknet folder. Okay, it's done, and we have here the weights. Now, the next step is to edit some configurations of the makefile, because I'm going to use OpenCV and the options of the GPU computing. Where is this file? We can check it. Here is the makefile. Now we have edit this file. Okay, we need to come back to the previous route using cd double dot and slash. We can check the new location using ls. Now we are in the content directory. That is the route. In this directory, we have darknet, the model, gdrive to accessing the data from Google Drive, and sample data folder. Very good. In this location, we need to install these libraries.

G++ and GCC. Let's run this cell. Perfect, it's done. Now I need to enter again to darknet directory, and here inside this directory, we need to run the makefile, because we are ready for building this model. So inside darknet, we use the command make, and let's run this cell. Don't be concerned about some annotations, it's okay. When we are running a cell, this circle is continuously moving, and new folders are created inside darknet folder. When you are building a model, you use a makefile. Okay, we have completed the building process. Now we have the complete model inside this darknet folder. So we can copy this compiled version of the model in Google Drive.

We use this command cp for copy, and after that, I need to specify the route. I'm going to copy the whole content of darknet into this route in the bin folder in Google Drive. This is for backup. Every time when you want to run this code, you don't need to repeat the whole process. You can access to a compiled version of the model, where it's now located in Google Drive. Then you can copy the model from Google Drive to the virtual machine. Now we are using the same expression as before, specifying the source where it's located the model in Google Drive, and the destiny folder, in this case is darknet folder. This is a reference for you in the future runnings.

Okay, at this point, we are ready. We have built our model, we have the weights, we have done the configuration of OpenCV, we have implemented CUDA and QDNN, so everything is okay. Now I'm going to need some utility functions. First, I need to use files from Google Colab. This function allows us to import files from your local host or computer directly to this notebook. Now we have three functions. The first function for showing images. We are using OpenCV and UploadLib for representing the images in a graphical way. We also have an upload function for uploading files to this notebook, and a download function if you want to download the content of a file in your local host, specifying a path.

Perfect, we can start testing our model on images. For example, we can use a picture where it's contained in data folder. This is a model, darknet, this is data. In data, we have some pictures, for example, dog, eagle, giraffe. I'm going to use this image, person. So for running the model, we need to use the darknet file. In this case, this is darknet file. The function detect, specifying the route where it's located the model. All models in YOLO have CFG extension. If you see here, CFG, there are different models. And here we have YOLO version 3.CFG. This is a model that I'm going to use. Weights are located in the root of this folder in darknet. Here are the weights.

And after that, the path of the picture. This picture is on data folder, and the name of the picture is person. So we can run this line. The model is loading, GPU count one. We have a GPU. When you run this model, we have the predictions. These are the predictions. Dog 99%, person 100%, horse 100%. But where are these predictions? The predictions are in a file with the name predic-

tions. Here, predictions.jpg, an image containing the predictions. This file is in the root of darknet folder. So we can call our function, showing, specifying the name of the file. Let's run this. And perfect. We have three classes identified on this picture. We have done our first test in YOLO version 3.

Now we're going to make a break. In the next lesson, I'm going to continue the implementation on how to train our custom dataset using this YOLO version 3 model. Review the code, take note about the configuration, and I'll see you in the next class. Bye bye.

LECTURE 52

YOLO Version and File

[MUSIC PLAYING] Hello, everybody. I'm going to continue the past class. Previously, we have seen how to configure our system in Google Colab, including CUDA, QDNN, and OpenCV. We have built our model and tested it with this image. As you can see, the model has found three classes. Now, we are ready to construct our own model for object detection. So what requirements we need to train our custom data set? First, we need images. This is my folder containing all images in Google Drive. I'm going to import these images in the Colab notebook. Here, you will find the image and also the file containing the annotations. For every image, we need an annotation file in TXT format.

I'm going to use the same images and annotations like in YOLO version 2. The idea here is to compare the results between YOLO version 2 and YOLO version 3. Then you can use the same annotation tool as before. You don't need the same size in the images. As you can see, the sizes are different because the YOLO model will crop the images in a standard size. How many images do I need? About 50 images per class. In my case, I have two classes because I'm going to detect two kinds of robots, R2D2 and Z3PO. Here, it's an image containing these two classes, R2D2 and Z3PO. Obviously, you can perform this example for any other kind of objects. OK, once you have the images, the next step is to define your own model based on YOLO model.

This is my file containing the model, YOLO version 3-Robots.cfg. I also have this file in my Google Drive account. This is darknet in my Google Drive account, YOLO version 3-Robots.cfg. And this is the file. This is a standard size in the YOLO model for the images, 416 in width and height. You will need to go at the end. These are the convolutional layers. Remember, this is a new network with 53 convolutional layers. And here, on filters, you need to edit this value. How can I calculate this number? This is the number of classes plus 5 because we have 5 values for each bounding box, including the [INAUDIBLE] score, multiplied by 3. Here, I have two classes.

2 plus 5 is 7. 7 by 3 is 21. And finally here, you will need to define the number of classes, in my case, 2. So in this file, you only need to edit these two fields. Rename the file with the name as you want and save it in your Google Drive account. What else do I need? I need a train and test file containing the images for training and testing, but in TXT format. Let's check them. Here, in my Google Drive account, I have the train and test file. Train file containing all images, specifying the route, where are located, these images. And test file containing also the images for testing. OK, the next step, to define two files, a data file and a names file.

Let's check them. First, this is the file, a data file. This is the data file containing the number of classes, the name of the train file in TXT format, the same for test file. And here, we have the file containing the names of the classes we are going to use. This is the file. And finally, a folder containing the backup of our model. Basically, this file is a config file. And what is the content of this file? This is a names file containing the name of the classes, R2D2 and C-Tripio. One line for this class. That's all. OK, we are almost ready. After that, we are going to need weights. In this case, we need pre-trained weights from DartNet. And here are the weights.

You can find these weights in the YOLO repo. But also, I have attached it for your reference. OK, when we are performing the training, we need to save the weights. Because our model will learn for every step, the model saves the weights in a folder. What is this folder? The backup folder. The same as we have consigned in the data file. OK, at the beginning, we are going to use these weights, DartNet 53. But after about 200 epochs and more, the model will save the last weights in the training. So we are going to need these last weights, not the weights at the beginning. This information will be saved in the backup folder. OK, until here, everything is done.

We have all requirements for training our custom data set in YOLO version 3. But now, we have two choices to perform that. First is to load all the requirements directly in the virtual machine. And the second approach is to make a reference to your Google Drive account for retrieving the data from Google Drive. My recommendation is to have the first approach. Because it's more stable in the training to load all data in the virtual machine directly, instead to work with the links to Google Drive account. Here, you have all instructions to copy the content of Google Drive, the source files, the model, the images, and rest of the requirements to our virtual machine.

Here, in the first instruction, we are copying all images in the virtual machine. Here on the left, you have DartNet. On DartNet, you will have this folder, IMG, containing all the images. And here, you will find the images. It's more stable, this approach. OK. Here, we have the same approach. We are going to copy this file from Google Drive to our virtual machine. But in this folder, CFG, here you have CFG containing different models. Now, we are going to make the copy. And here, you will have the file. The same approach with the resource files, the training file, the text file, the data file, the names file, and also the initial weights. We are copying these files in the DartNet root folder.

OK. And now, we are in this step. In my case, I already have done a pre-training process. So we have these weights, the last weights in the training. I'm going to use these weights and make a copy in the backup folder. But where is located this file? In my case, it's in the backup folder into this folder, DartNet, in Google Drive. But here, I have different weights for 1,000 steps, 2,000 steps, and the last weights. I'm going to use these last weights. So here is some backup. And I'm going to copy these last weights in this backup folder in the virtual machine. Remember, in the first training, you will need to work with these weights, DartNet 53. You can't use my last weights because these weights are based on my own model.

This approach is to load all files in the virtual machine. But if you want to work with a link with your Google Drive, you can specify the roots, where it's located, these files, something like that. But now, I'm going to start the training. I need to run the detector file, specifying the train, the data file, the model file, and the weights. Let's start. First, I'm loading the model. And now, it's starting the training. These are the results, the loss, and the mean average precision starting in this step, 2,101. So my last weights are based in 2,100. As you can see, this is a small loss, only 0.11 in the loss. So try to run your model and achieves a same loss.

With this kind of loss, your model will be very precise. OK, I'm going to stop the training because I already have my pre-trained weights. Just press here to interrupt the execution of this cell. OK, after training, what is the next step? It's to use our pre-trained model in a video. So I'm going to load the video containing the predictions of the YOLO version 2 model. Here, this is the video.

And with this video, I'm going to run my new model comparing the predictions between YOLO version 2 and YOLO version 3. So I need to use the detector file, in this case, the function demo. Because in this case, I need to test the model, not to train. I need to specify the data file, the model file, where it's located, the weights.

In this case, I'm going to use these weights with 2,000 steps. I can also use the last weights as before. And this is the file where I'm going to implement my new model, R2D2.avi. And I need to specify the output file, where the predictions are performed. Robots underline YOLO version 3.mp4. I can use this format, mp4. So let's run this cell. First, I'm loading the model. OK, all results containing the predictions will be stored in this file. Where it's located, this file, in the root. This is the file, robots underline YOLO version 3.mp4. So you can download this file here. And now we can see the results. Here you have the file. The sticker bounding boxes are from YOLO version 2.

For example, it's a misclassification because a boy is classified as a robot in YOLO version 2. But here, this prediction, the thinner bounding box from YOLO version 3. As you can see, in many cases, it's more precise than YOLO version 2, which are in these blue bounding boxes. As you can see, YOLO version 3 is predicting two robots, C3PO and R2D2. And YOLO version 2, only R2D2. In conclusion, YOLO version 3 is more precise. Here we have the same results. And here, YOLO version 3 is detecting very well C3PO. Very good. Mission accomplished, guys. With this model, YOLO version 3 and Colab, you can use your own data set and perform this example. But what else you can do?

You can construct your own model step by step since scratch. Because here, I'm using a model from GitHub. So if you want to construct your own model, I recommend you to use the Keras functional approach or PyTorch. PyTorch is very Pythonic. You can work with classes and functions, constructing your layers step by step. But this is a subject for another course. OK, finally, if you have questions or doubts, you can contact me in QA forums. Thanks for supporting the course with high ratings. Enjoy the course. I'll see you in the next class. Bye-bye.

LECTURE 53

MS COCO Object Detection

Hello everybody, welcome to a new class. In this class I'm going to present you the latest state-of-the-art algorithm for object detection, which is YOLO version 4. This algorithm uses many of the latest discoveries in the computer vision field. As you know, any object detection algorithm has two important goals, the accuracy in the predictions and the speed. The speed is represented in frames per second. How many frames per second can we take for object detection?

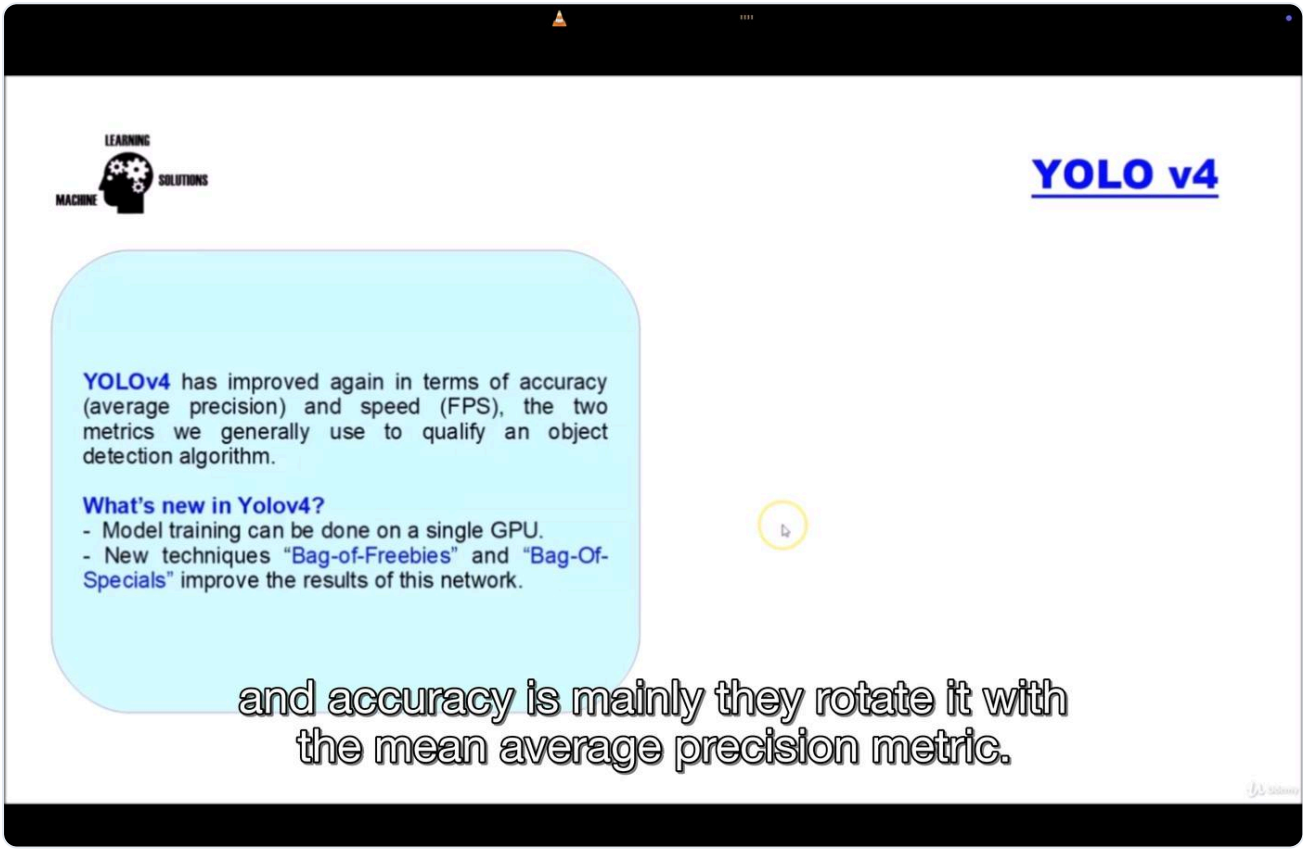


FIGURE 8.1

And the accuracy is mainly denoted with the mean average precision metric. The situation here is most of the object detection algorithms meet one of these two goals, but not both, so YOLO version 4 represents a great improvement in both goals. What is new in this version 4? We have a model that can be trained in a simple GPU and also incorporate many recent techniques in computer vision, which are called backup freebies and backup specials. The main purpose is to improve the results in the precision and also the speed of the model. This chart is on the paper if you want to dive deeper in the theory behind YOLO version 4, you can check the paper that I have attached in the resources for this lesson.

Previously we have seen YOLO version 3, which represents an improvement of YOLO version 2. Now, this is the latest model, YOLO version 4. It has been released on April 2020 and as you can see in this chart, on vertical we have the average precision and in horizontal the frames per second, which is the speed. In both metrics we have very good results.

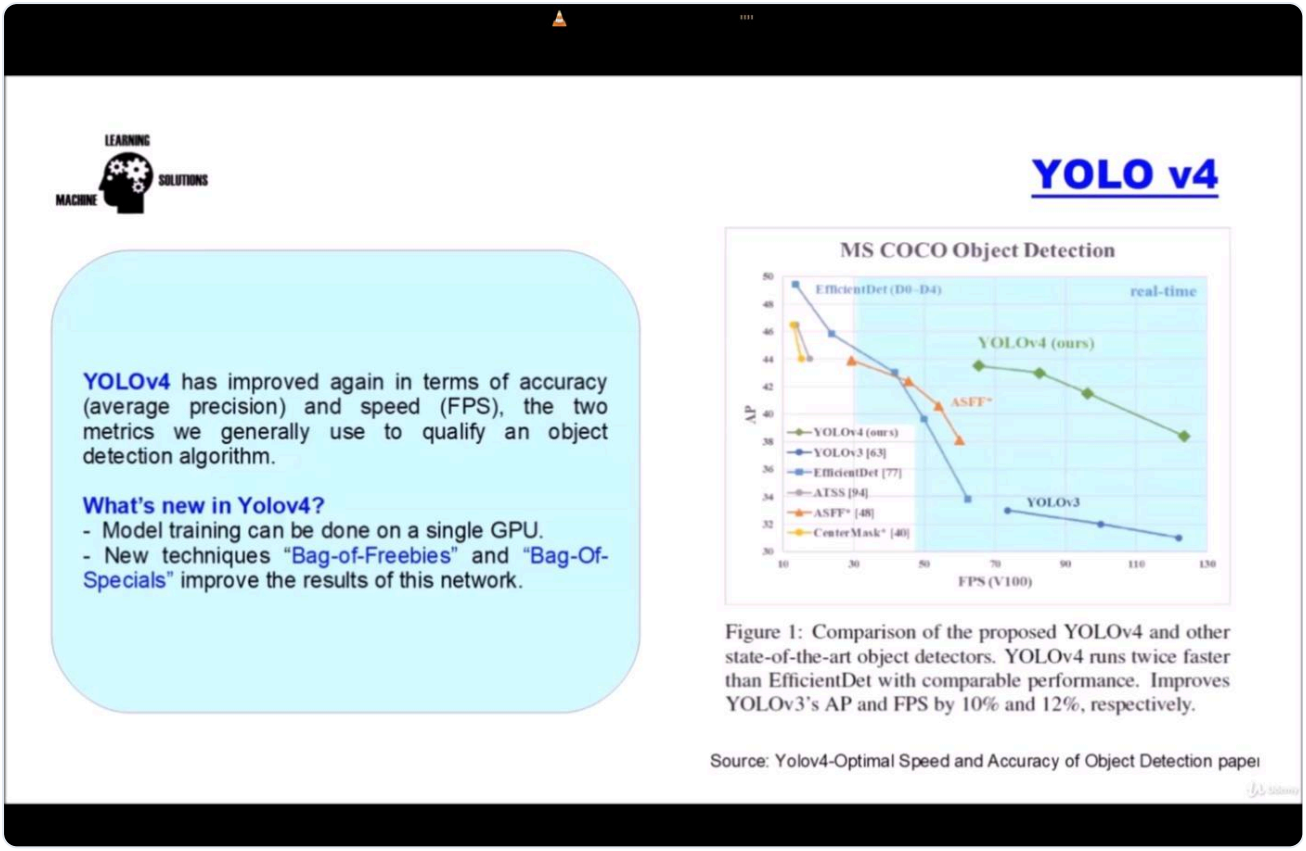


FIGURE 8.2 | MS COCO Object Detection

This algorithm represents an improvement of the previous version, YOLO version 3, because the average precision and the frames per second are improved in factors 10% and 12% respectively.

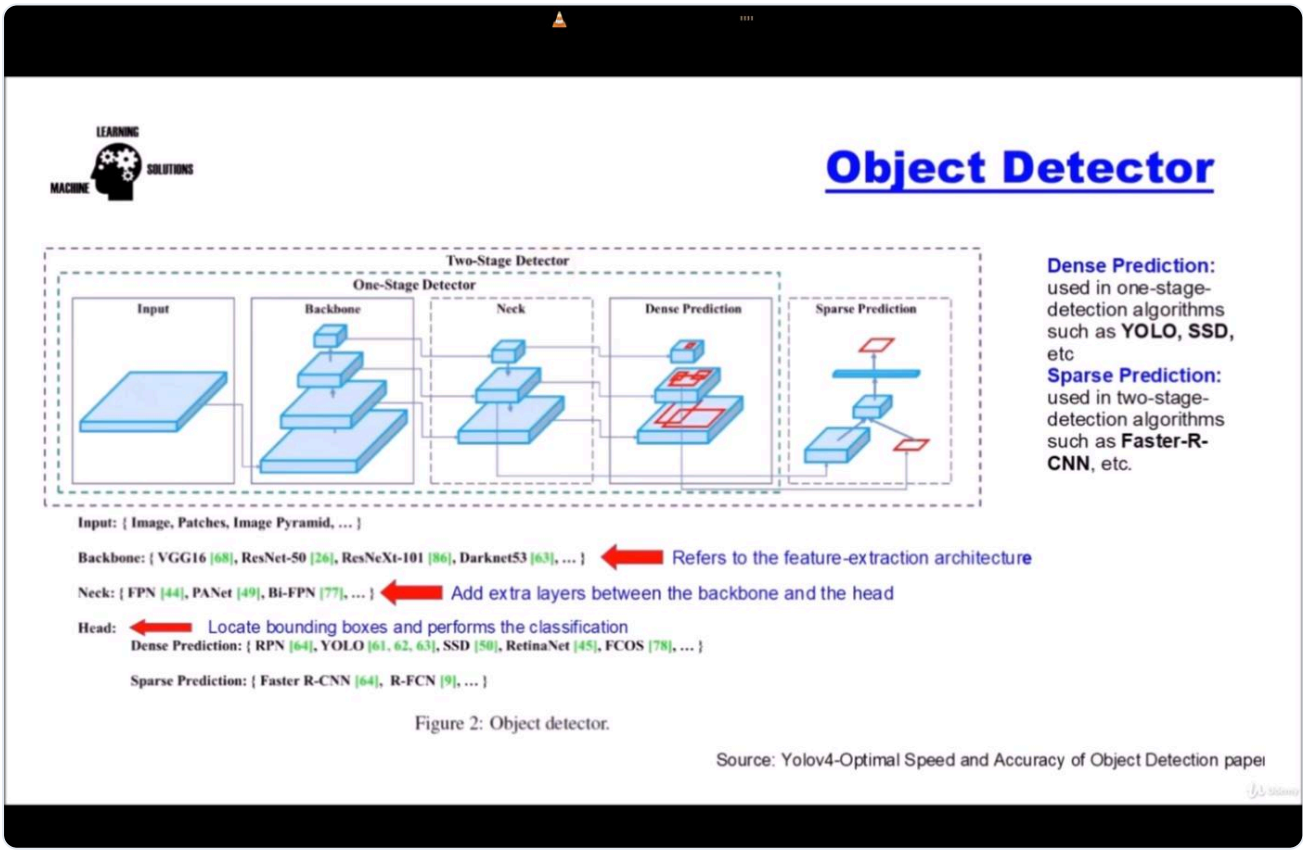


FIGURE 8.3

Ok, now I'm going to make a quick overview about the structure of the object detector, which is present in this model YOLO version 4. After that I'm going to review the main new techniques, this backup freebies and backup specials.

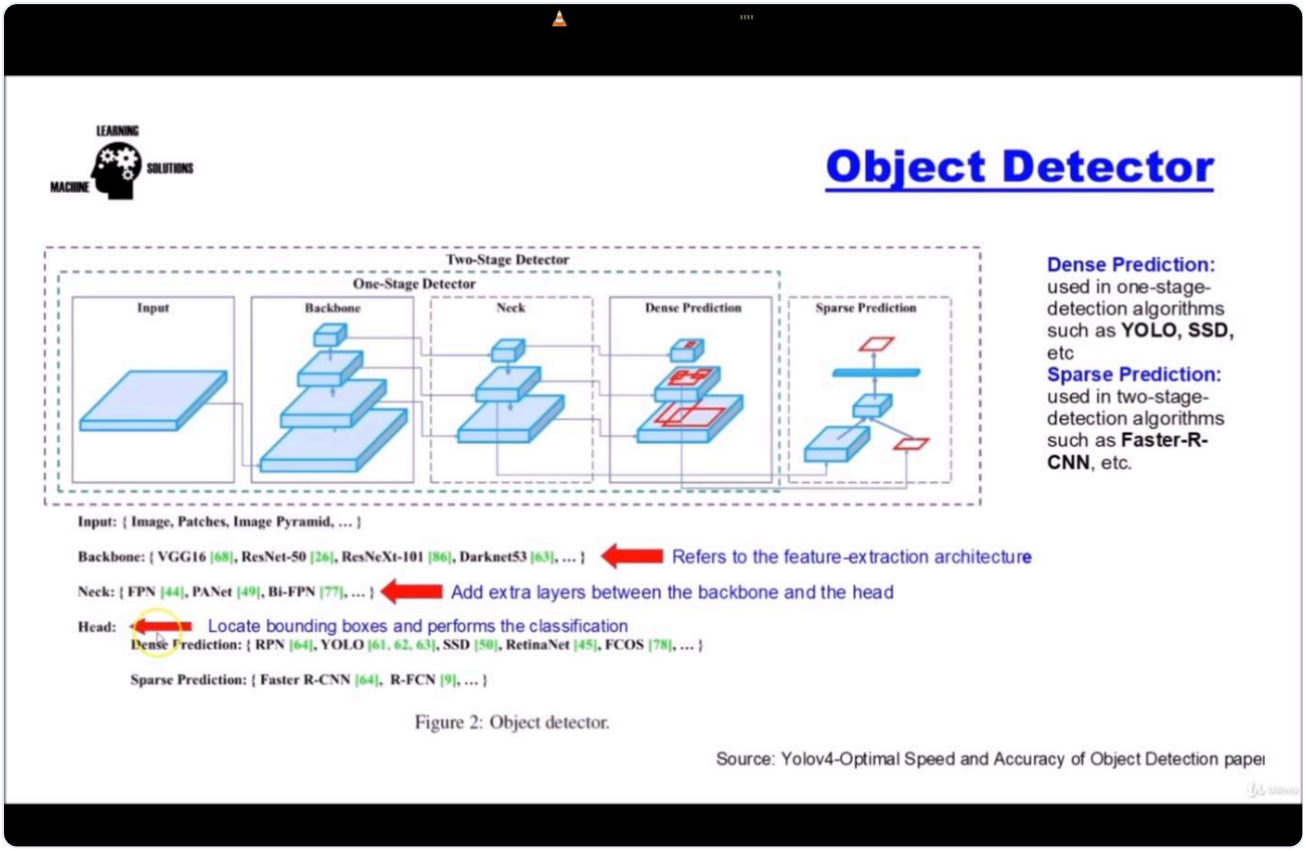


FIGURE 8.4

So, let's start with the object detector structure.

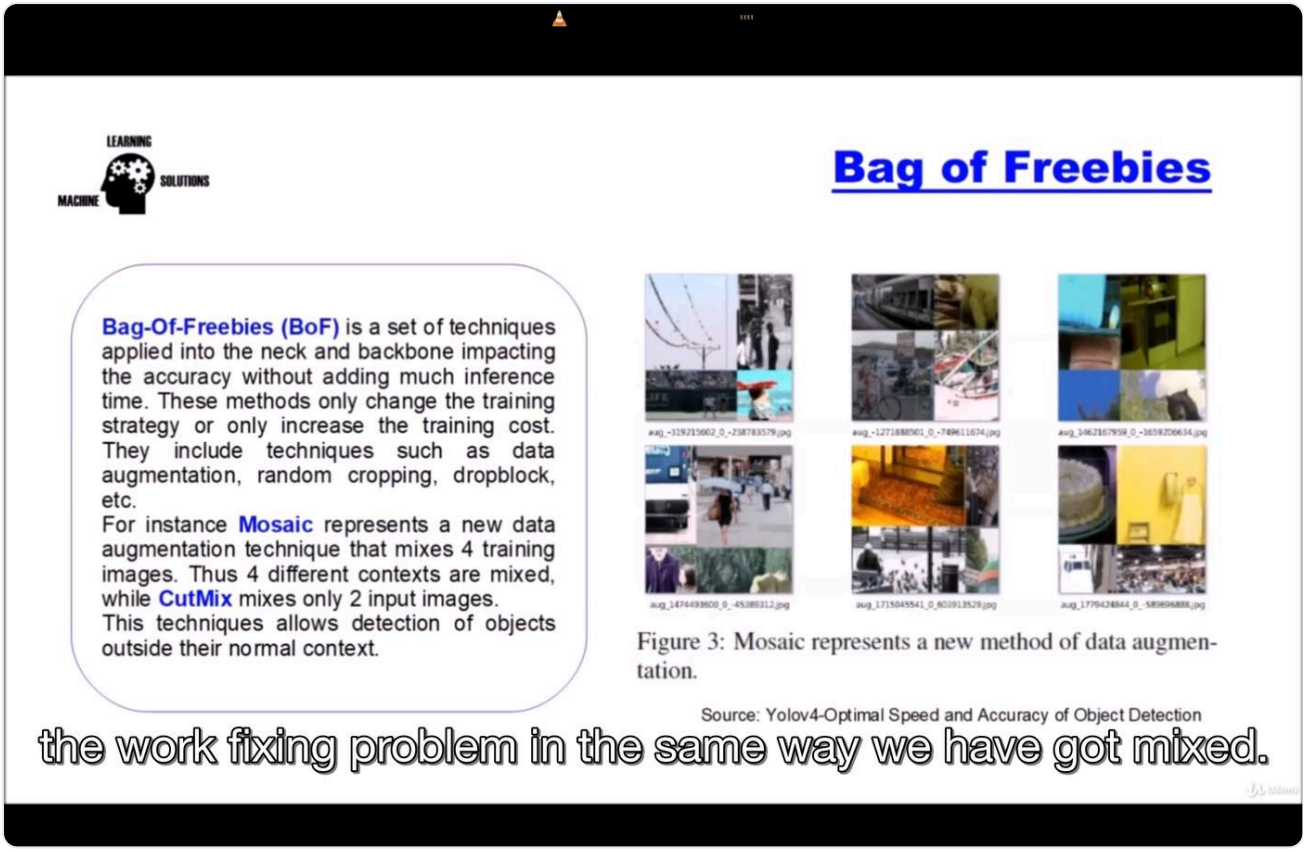


FIGURE 8.5

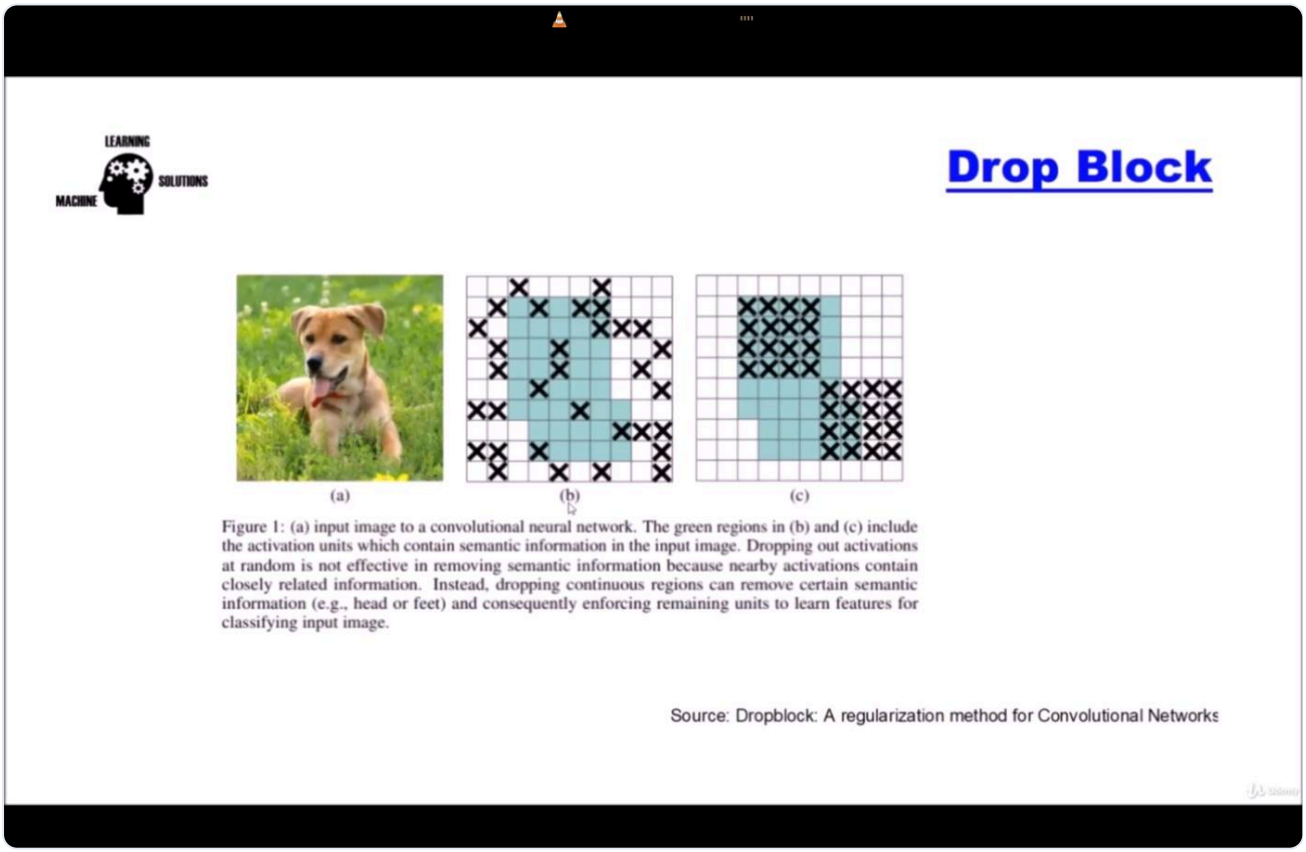


FIGURE 8.6

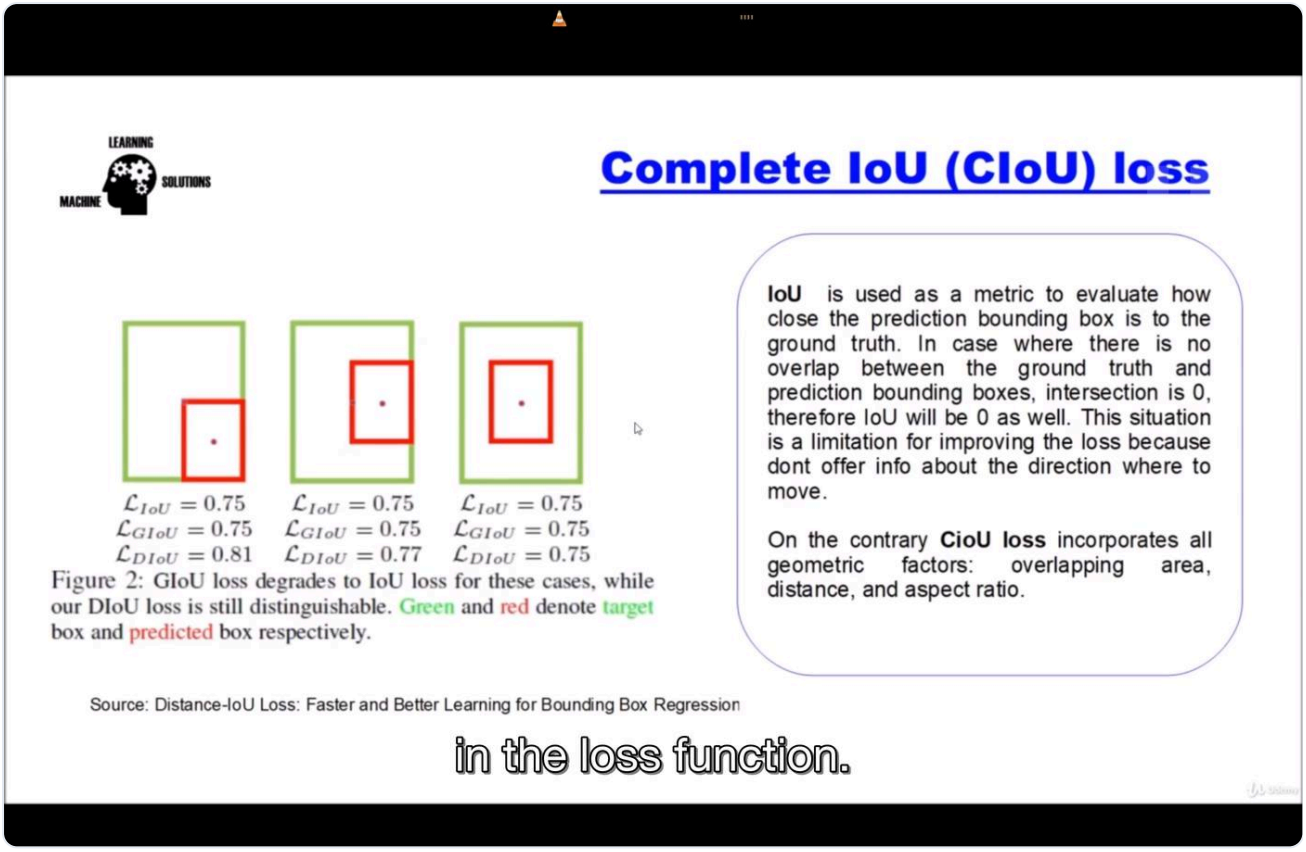


FIGURE 8.7

When I'm referring an object detector, I'm referring to the network, the network which is in charge for extracting features from the images to perform the final object detection. The structure is here, mainly there are two kinds, one stage detector and two stage detector. In one stage detector we extract the features passing the image only once. On the contrary, in two stage detector we need to use two networks for performing the detection, which is a case of faster RCNN. In this case we have the region proposal network for extracting the features, we propose different regions and for each region we apply the whole network for object detection, but YOLO has one stage detector.

In one stage detector we have these four main blocks, the input for taking the information from the image, the backbone which is in charge for feature extraction, this is a case of YOLO version 3 with the network darknet 53, ResNet, VGG16, the family of ResNet models, so this backbone is a convolutional neural network specially designed for extracting features from the image. After this module we have the neck module, this block adds more layers with the proposal to detect more fine grained features. Remember one of the problems in YOLO is about to detect different sizes in the objects, in special tiny objects. The idea here is with these extra layers and with the special architecture we can deal with different scales in the features, which is a case of feature pyramid network or FPM.

So in this module we can use different approaches, for the backbone, for the neck. Finally we have the final block, which is the head. In this block we are performing the location of the bounding boxes and the final classification. As you know in YOLO, including this last version, version 4, we use vectors containing the classification for the classes, the location of the bounding boxes and the size of the bounding boxes. So in conclusion, the modules backbone, neck and

head are the same for many one stage detectors. The difference is about the specific technique or network we are going to use on every module. Now I am going to continue presenting you a quick overview of the main techniques behind YOLO version 4.

I am going to start with back of freebies, which is a collection of different techniques. The idea with these techniques is to improve the accuracy. There are new techniques about the augmentation, random cropping, the block. For instance, mosaic is a new technique created by the authors of this version of YOLO. This is a representation of this technique. We create new training images consisting in four different images taken from the original training dataset and making a collage of these images. In this way, we are forcing the model to learn in different contexts at each time and helping us in the overfitting problem. In the same way we have catmix but taking only two images from the original dataset.

These techniques are applied in the training process. Another important technique is drop block, which is similar to dropout. Previously we have seen dropout but now drop block represents an improvement of this vanilla version. For example, we have this picture of the dog. The image in B represents the approach in the regular dropout algorithm taking randomly different pixels from the images and removing them. But the approach in drop block is not to use a certain region of the pixels. So in this way we are forcing the model to learn specific features without some part of the information. So in this way we can achieve more accuracy and also to prevent the overfitting problem.

Another important technique to achieve a better accuracy is about a new loss. This new loss is complete IOU. As we have seen in previous lessons, we have a metric. The main metric for object detection is IOU. But now we are going to apply IOU not only as a metric but as a loss, incorporating this metric in the loss. We have different situations. The green box is the target box, the red box is the predicted box. If this red box is far away from this green box and both don't have an intersection, we have an IOU score equal to zero. So in many cases we are going to add different zeros in the loss and this situation is a problem because we are not improving the learning, adding zeros in the loss.

So, in order to solve this problem, it has been created different metrics as a losses. For instance, we have the IOU, but takes in consideration how far away is the predicted box to the target box. This loss helps when there is no intersection between these bounding boxes. But in these three situations this loss is the same because the predicted box is contained into the target box and also in order to solve these kind of situations it has been created another metric as a loss which is called distance IOU. This metric takes in consideration the distance between the center points from the target and predicted boxes. So, in this way we can measure how precise is our prediction in relation to our target box.

So, as you can see, the metric is not the same in all cases. The best case is here because the centers are aligned, outputting the lowest value in the loss. But it is not enough because we are not measuring the aspect ratio and the overlapping area. Because of that, it has been created the complete IOU loss, which is also a metric, but it is used in the loss function. This loss incorporates different factors such as overlapping area, the distance between the center points and

aspect ratio. So, it's an improvement to the vanilla IOU loss. In addition, using this kind of loss we can avoid the use of regularization factors in the loss. Ok, that's all for now, I'm going to make a break.

In the next lesson I'm going to continue with the theory behind back-of-face shells. Review the theory and the paper. See you in the next class, bye bye!

LECTURE 54

Attention Module

Hello everybody, welcome to a new class. I'm going to continue with main techniques under the hood yellow version 4. In this lesson I'm going to explain you the techniques under the name bag of specials. These techniques have the purpose to increase the speed of the model and also to improve the accuracy. Some of them are used in the neck and in the backbone.

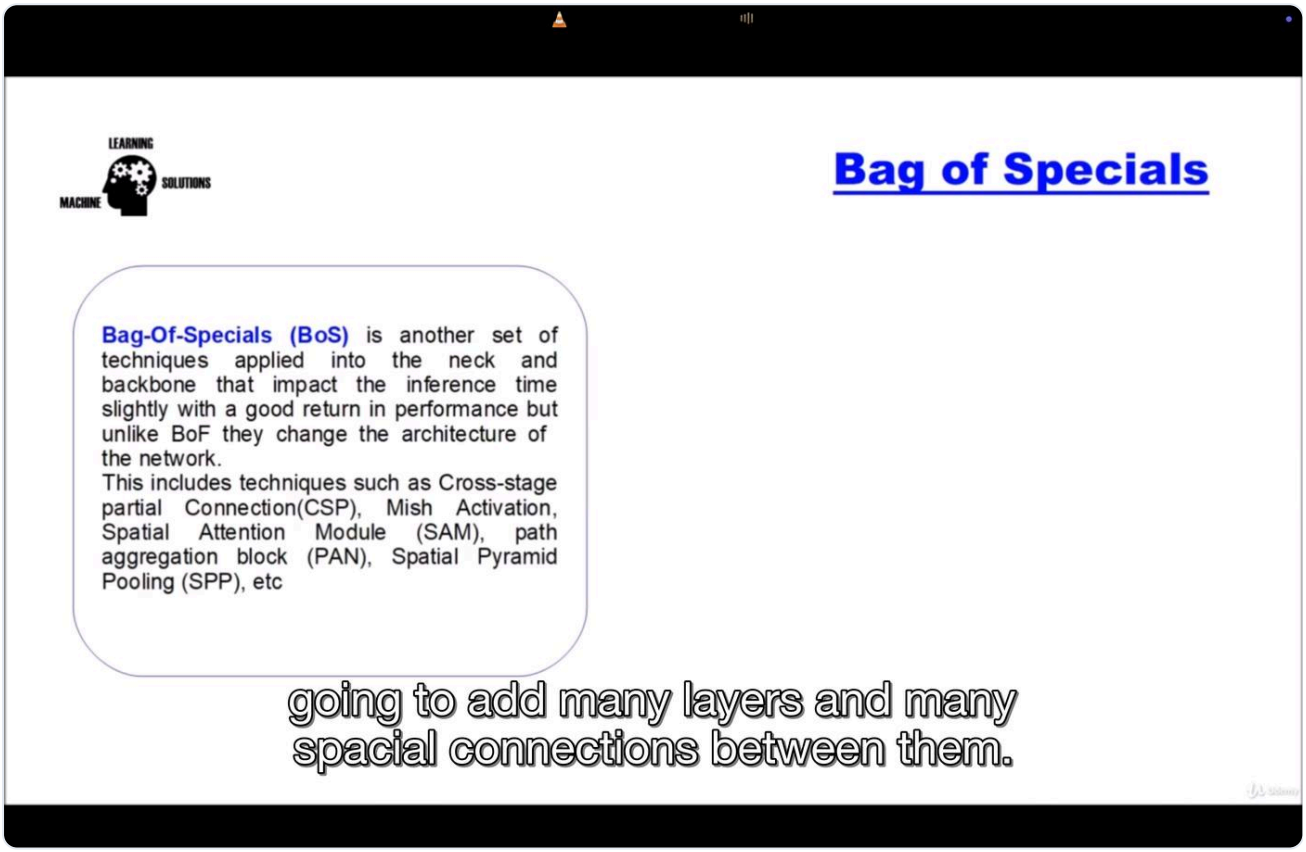


FIGURE 9.1

But unlike bag of freebies, these techniques change the architecture of the network because we are going to add many layers and many special connections between them. The initial point to understand these techniques is the building block of feature pyramid networks.

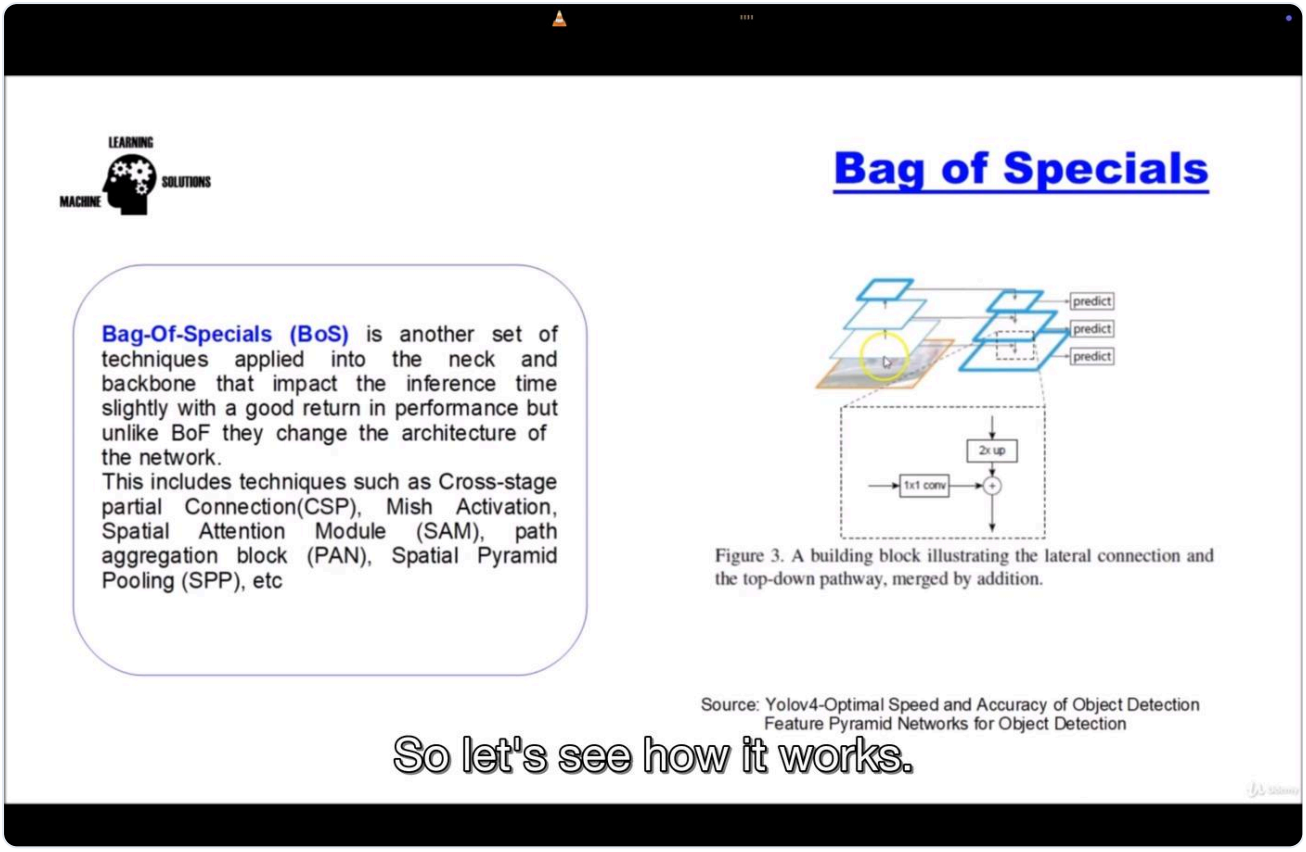


FIGURE 9.2

So let's see how it works. This architecture works in the neck, linking the backbone to the head. In the left side we have a standard convolutional network composed in different stages. Each layer here represents a stage and each stage has different convolutional layers and pooling layers. In this case we have different stages reducing the size of convolutional feature maps. In a structured button app the factor of reducing on each stage is the factor of two, applying pooling layers. So in the top we have strong semantic features but with low resolution because the size is tiny. And at the bottom we have the input which is the image. This shape is very similar to a pyramid.

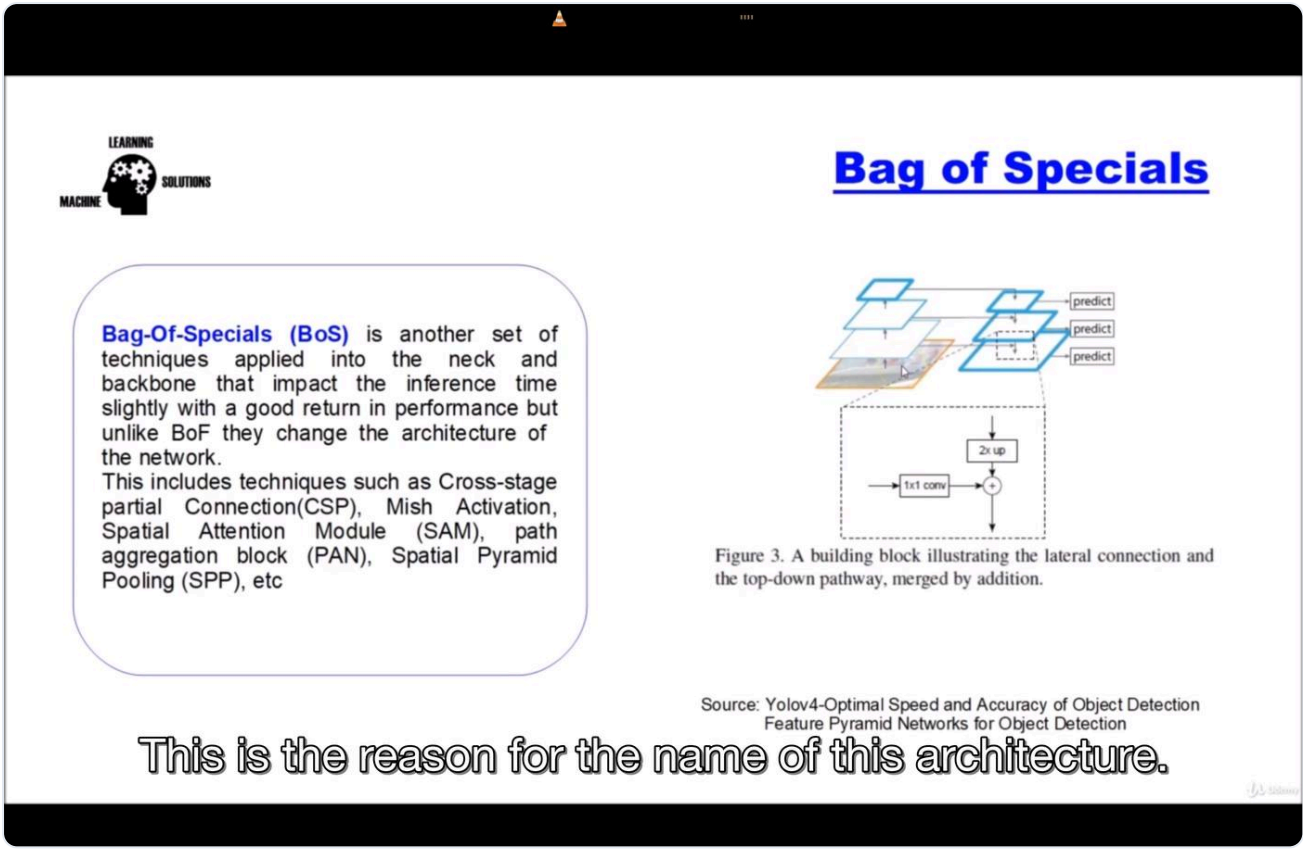


FIGURE 9.3

This is the reason for the name of this architecture. So this architecture offers us, for each stage, different scales. It is used in YOLO v3 but only this structure, the left side. But the complete building block is composed with another structure in parallel, linked by lateral connections. Very similar like in ResNet. Each lateral connection merges the feature maps from this button app structure and the feature maps from this structure which is a structure with a shape top-down but way, the inverse of this original structure, and producing different pyramid levels. But before merging these feature maps we need to scale these feature maps in a factor of two.

Here is the representation because on left side we have used pooling layers dividing the shape in a factor of two. So we need to match the shapes between feature maps before applying the merging operation, that is in this case the sum operation. And also in the lateral connection we have a convolutional filter with a size 1x1 extracting the feature map in 2D. If you want to dive deeper in the effect of convolutional filter with a size 1x1 I recommend you to review the lesson about ResNet and also Inception.

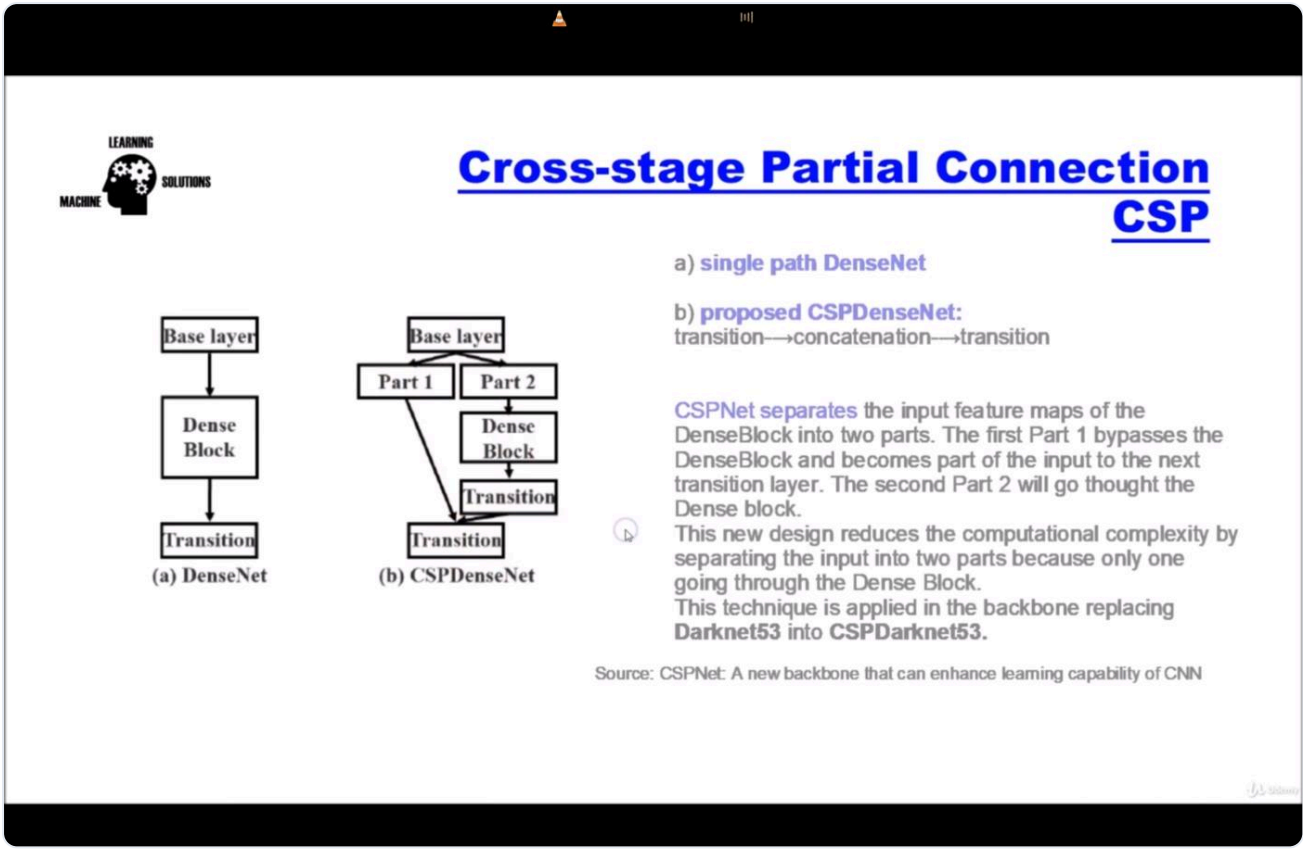


FIGURE 9.4

So the final prediction is applied in the head and we are going to have different predictions for different stages which represent different scales so we can detect different object sizes. The concept behind feature pyramid networks is very similar like ResNet but including the factor of the scales. If you want to dive deeper in the theory behind this building block you need to review this paper, feature pyramid networks for object detection. And that's it for this building block. Now I'm going to continue exploring these new techniques. I'm going to start with cross stage partial connection or CSP. What is this approach? Let's see this image. We have on A a regular approach, for example a DenseNet model.

In DenseNet we have different layers with different connections based on the idea of residual networks. So we have a base layer, different blocks and a transition. This approach cross stage partial connection splits the base layer in two parts.

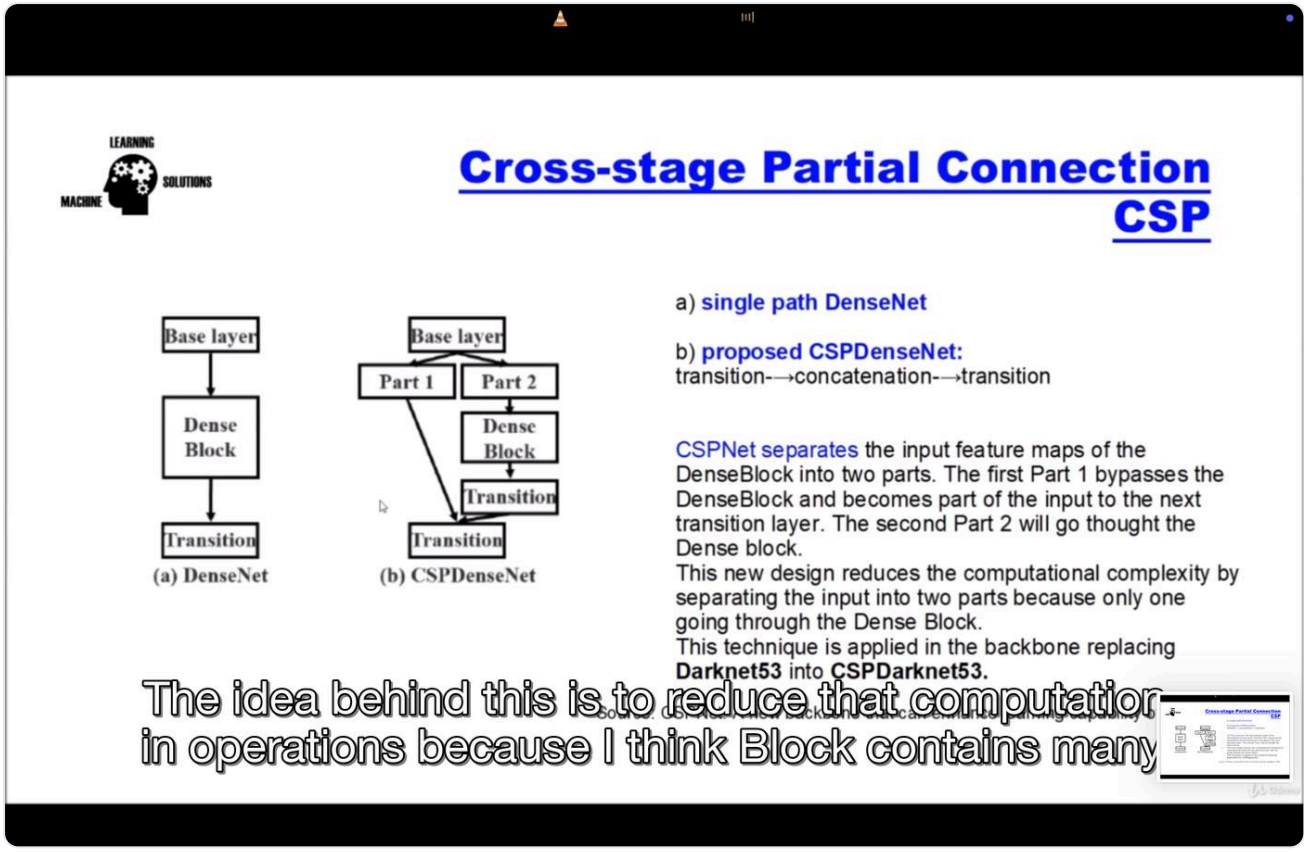


FIGURE 9.5

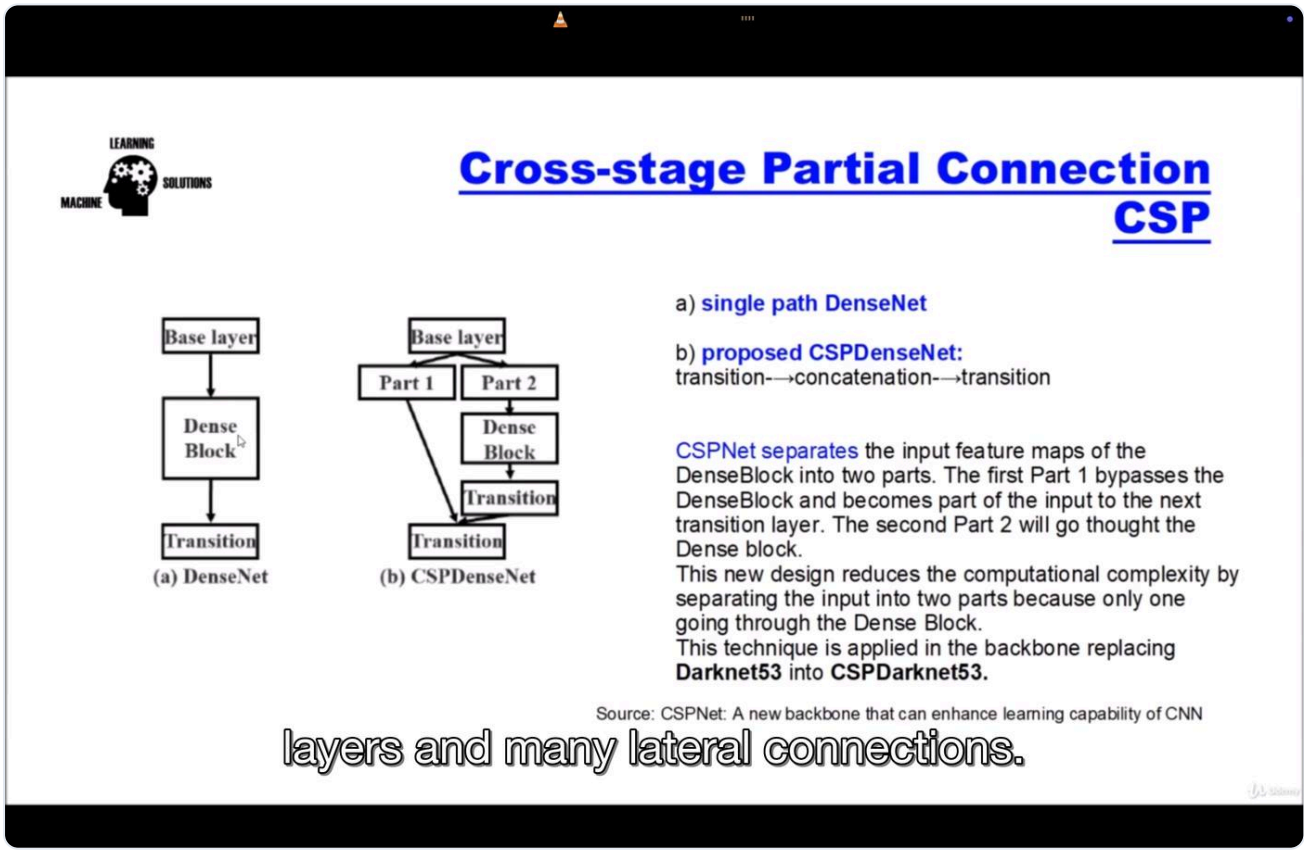


FIGURE 9.6

One part is preserved and the other part continues with the same procedure passing this part to the dense block to the transition block but finally the output of this splitted part is added, is concatenated to the part one in a transition operation. The idea behind this is to reduce the computation in operations because a dense block contains many layers and many lateral connections. So if we split the information of the base layer we are performing less operations because we are going to apply this dense block not to the whole base layer but only in a part. Because of that this technique has a goal to increase the speed in the model rather than the accuracy.

So if we have a feature extractor like darknet 53 which is present in YOLO version 3, now with this model we convert this regular network in CSP darknet 53 and this feature extractor could be present in the backbone of YOLO version 4. Now let's see another technique, this is pad aggregation network or PAN. In this illustration we have in the option A the regular pad aggregation network, the concept is the same like in ResNet, we perform addition linking the layers with connections but in YOLO version 4 the app force uses a different operation not an addition but concatenation.

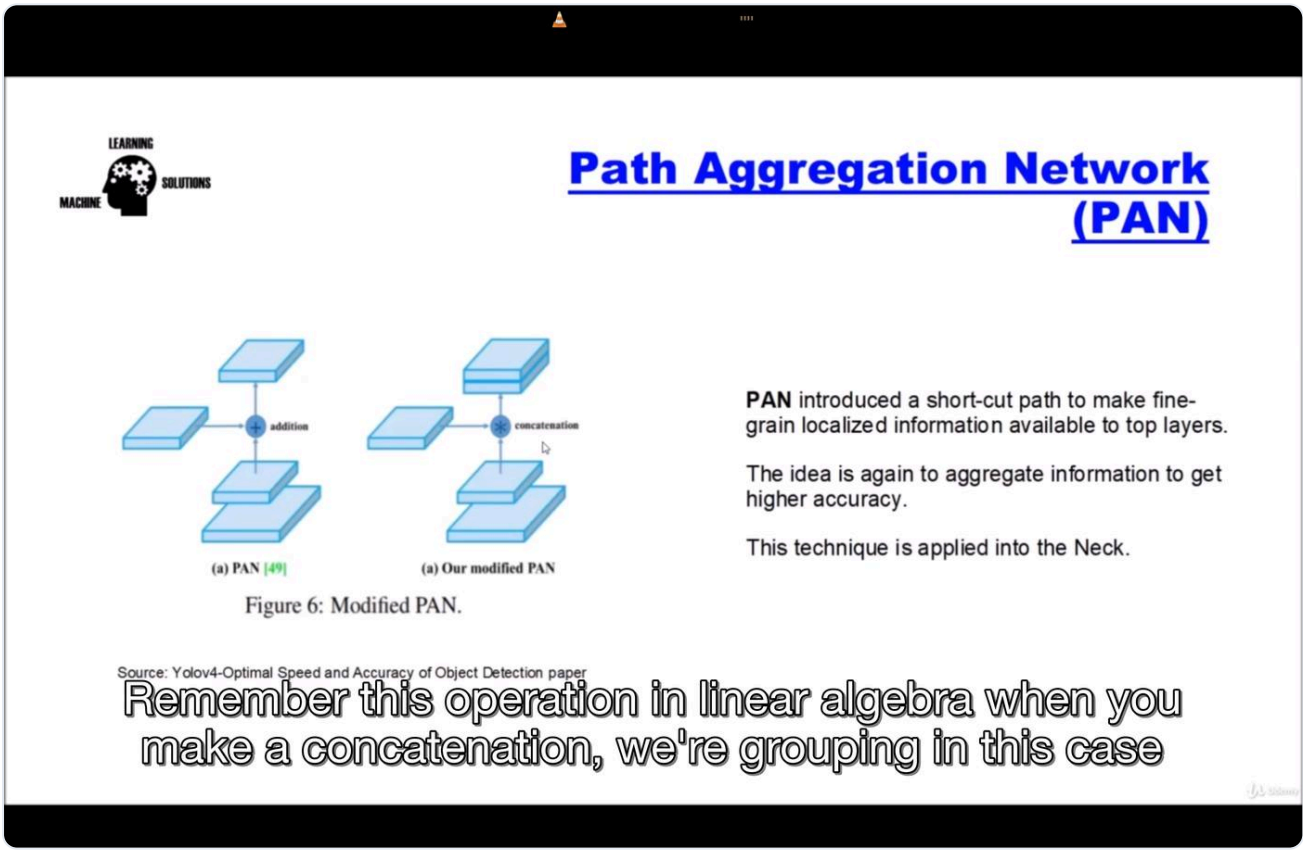


FIGURE 9.7

Remember this operation in linear algebra when you make a concatenation we are grouping in this case two feature maps. With this operation of aggregation we are adding more semantic information to the top layers.

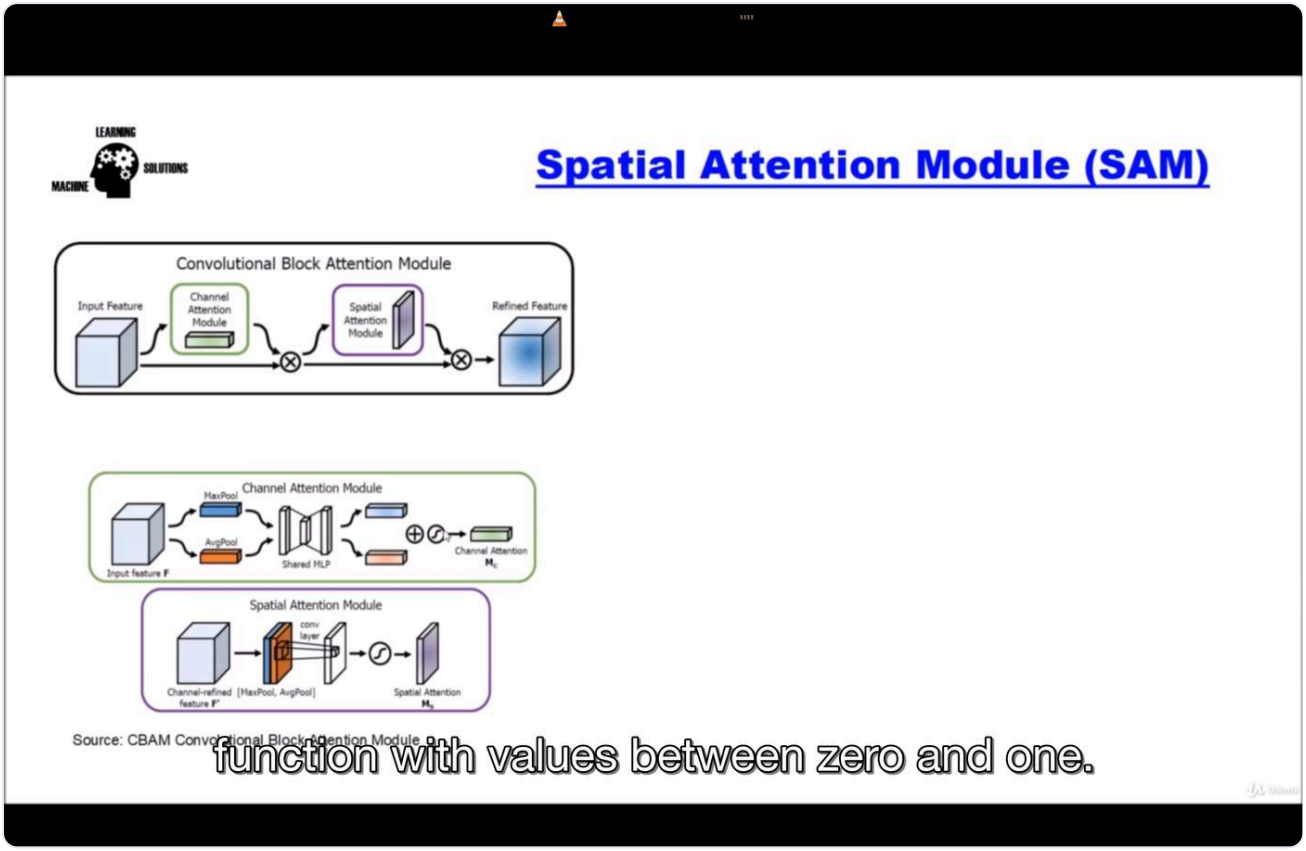


FIGURE 9.8

Now is the turn to the attention module.

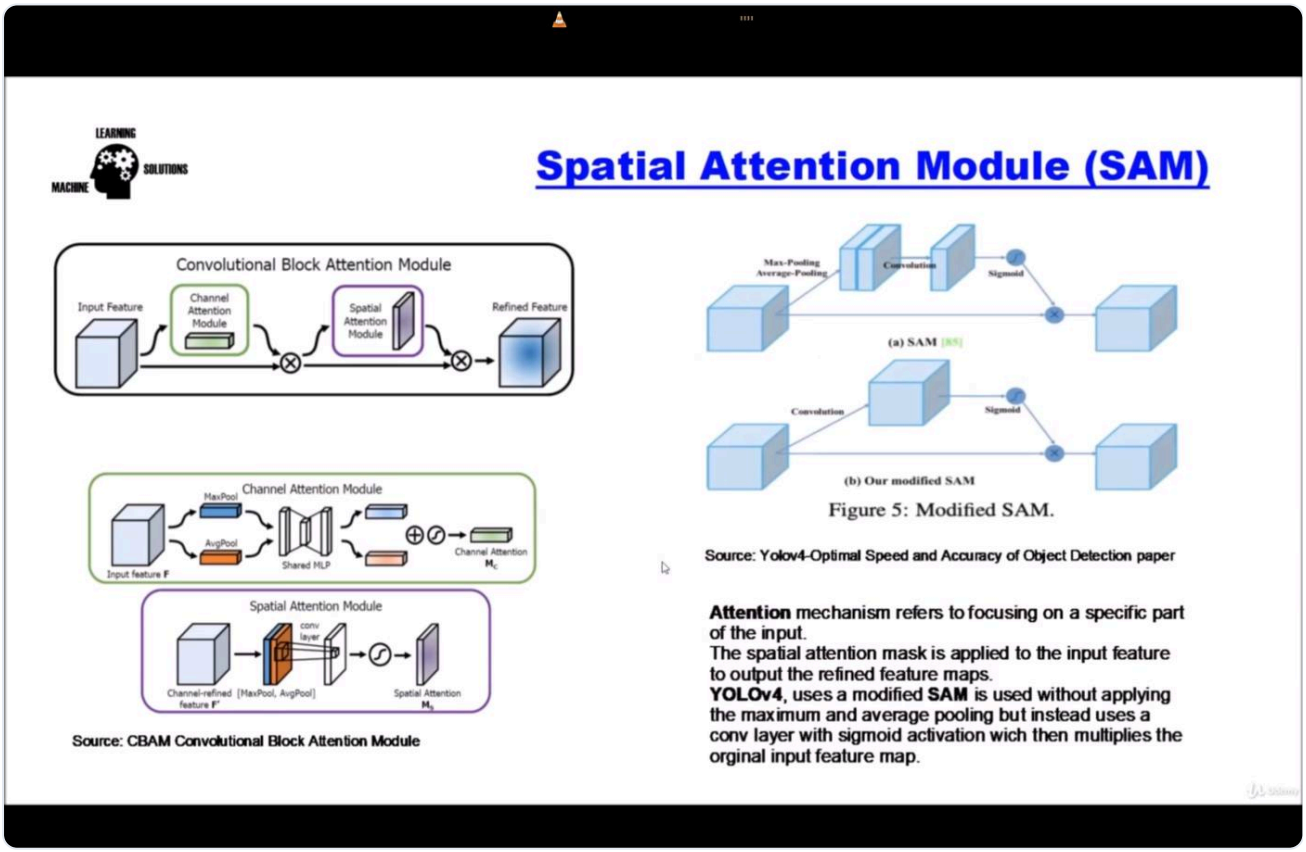


FIGURE 9.9

Attention mechanisms are present in different fields, for example in natural language processing the idea behind the attention modules is to focus on a specific part of the input. This illustration represents a convolutional block attention module, in computer vision the input is an image, in human vision if we want to pay attention in a specific object in a scene the human eye brings forward the object that we want to pay attention and blurs the remaining objects or the background. This effect is performed by the application of different filters so in the same way we need to use different filters to extract the information from the input on specific areas where is located the objects that we want to detect.

To perform that in this building block we need two sub modules, the channel attention module and the spatial attention module. This is in short SAM, the input feature is an input feature map which is a 3D tensor so first we pass this information to the channel attention module which is in charge to locate the relevant channel between all channels which are present in this input volume. After that we perform the element wise multiplication with the input and this output is fit into the spatial attention module. This spatial attention module is in charge to locate the specific areas in the feature map which are related to the objects that we want to detect.

After that we perform again an element wise multiplication to produce the final refined feature map which is a mask. In conclusion we split this 3D volume first analyzing the relevant channels and after that analyzing the specific areas in the spatial attention range related to the objects that we want to detect. We can see in detail these two sub modules. First in the channel attention module we split the input feature map which is a 3D tensor producing two different feature maps, one performing the max pooling operation and the other one performing the average pooling operation. These operations are performed in parallel, after that we pass by different convolutional layers, after that we perform the summing operation for these two feature maps and finally we pass this information to an activation function which is in this case the sigma function with values between zero and one.

The final result is an output volume which is a channel attention. Here we have the information what is the relevant channel between all channels present in the input feature volume. So this is for the channel attention module and the spatial attention module is in charge to identify the specific location of the object that we want to detect. So in this case the input of this spatial attention module is a refined feature map because we are going to apply this channel attention mask to the input feature volume which is f. This refined feature map is decomposed in two channels, one performing the max pooling operation and the other one performing the average pooling operation.

The output of these layers is feeding to one channel feature map. This feature map has in depth the dimension one. After that we apply the sigma function to produce the final spatial attention feature map. So in conclusion we need to generate different masks applying different convolutional filters and pooling layers for extracting the specific location which is related to the object that we want to detect. This spatial attention module in YOLO version 4 is modified with a new structure. At the top, in this option A, this is a regular sum module but in YOLO version 4 the affords have replaced this max pooling layer and average pooling layer for convolutional layers.

The rest of the module is the same. If you want to dive deeper in the attention modules for computer vision you can review this paper for CBAM which is the short name for convolutional block attention module. Now we are going to see another important tool for improving the accuracy which is the Mish activation function. This is Mish activation function which is a family related to ReLU function. There are different functions very similar in the shapes such as ReLU soft+ switch activation function and this Mish activation function. On the left side we have the representation of these functions, on the right side we have the derivative of Mish and switch function.

The idea to use this activation function is this negative part because in the positive range the results are the same but using this activation function we can deal in a better way with the negative values and mainly has a property to be non-monotonic function so when you have a monotonic function you always have a regular behavior increasing or decreasing the function but in order to accelerate the learning we need to represent eight different values positive and negative values so we need a non-monotonic nature in the function. The problem with ReLU is on the negative side we have a zero value but this curve helps us very well in the training process because it changes according to the values. This is the reason for this non-monotonic nature in this Mish activation function.

In addition if you see here the first derivative is very important when you are performing gradient descent. This red line has higher values than this switch function. We need higher values because we need to accelerate the training process. If this derivative is higher is better for the training. This is the formula is the value multiplied by tan function applied to another function which is soft plus.

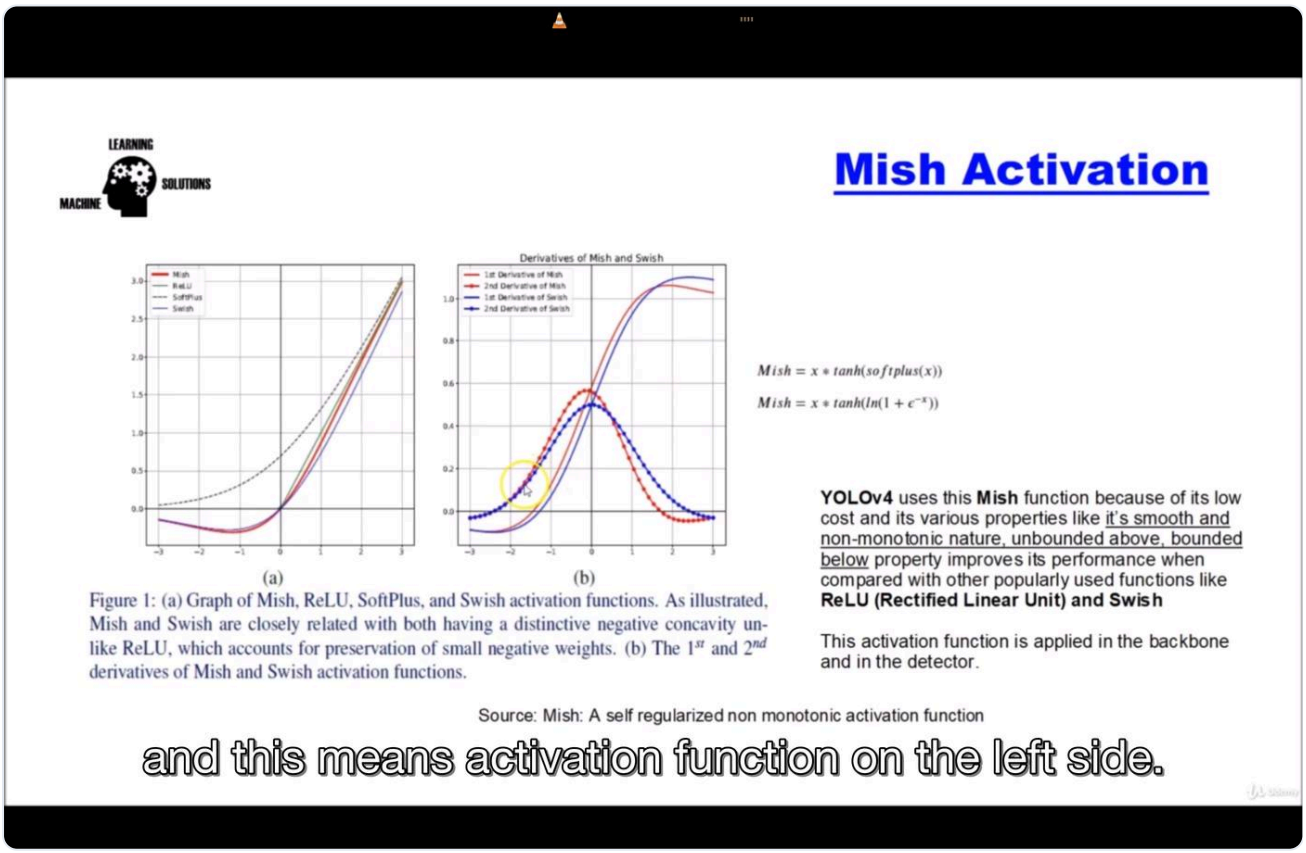


FIGURE 9.10

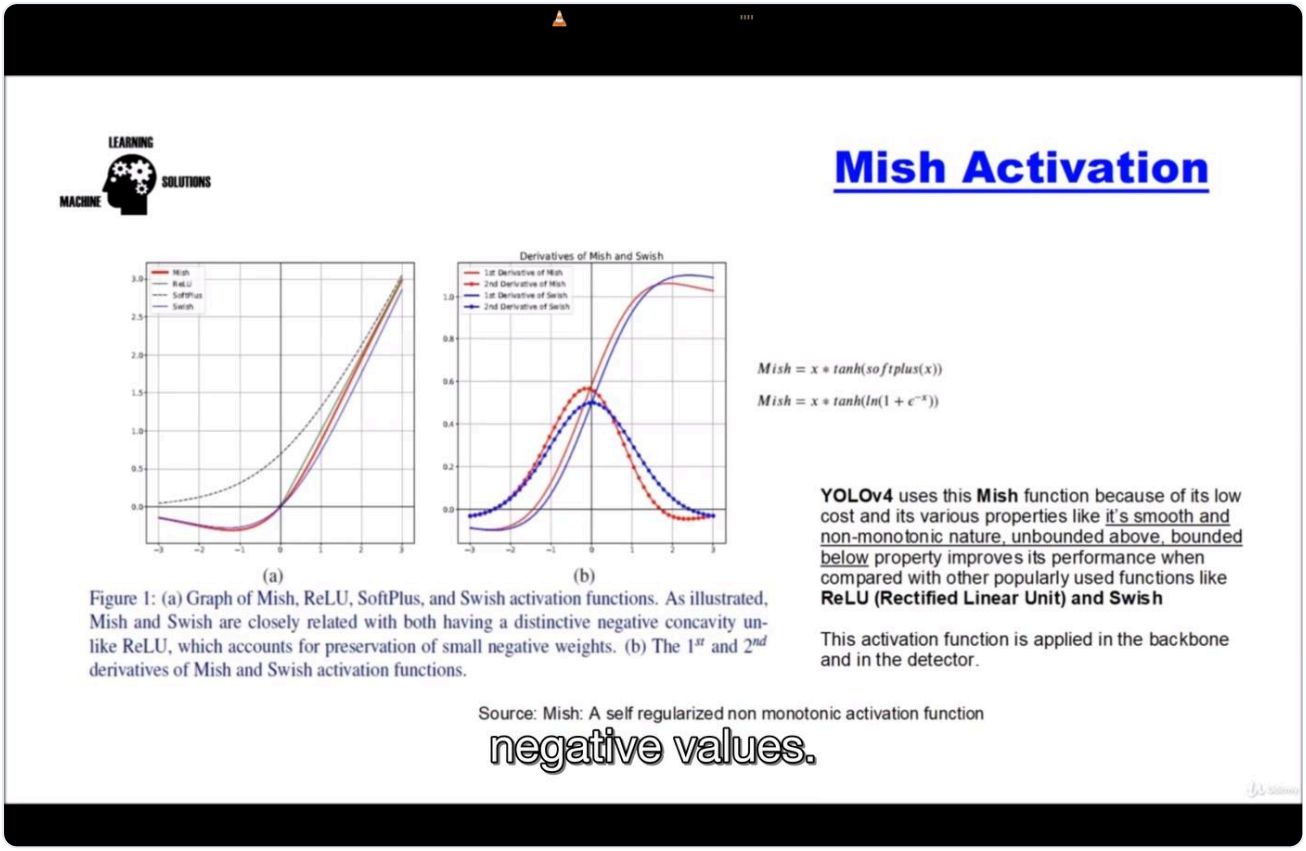


FIGURE 9.11

Soft plus is this the logarithm of this expression and this function is applied in the backbone and also in the detector. So that's all in a quick overview about the theory behind YOLO version 4. There are more techniques for backup freebies and backup fascias but you can review them in the YOLO paper. Now it's time to put in practice this model in the next lesson developing a code in Google Collaboratory. Until then review theory and the paper. See you in the next class, bye bye!

LECTURE 55

YOLO Version

Hello everybody, welcome to a new class. This class is dedicated to put in practice the theory I'm going to perform an example on how to train your own dataset using YOLO version 4. This implementation is on Collab, Google Collaboratory. The reason to use this tool is for supporting those students that don't have an available GPU card in their systems and also because it's very easy the implementation. We only need to pay attention to the configuration process. Depending on your location you can use these options for GPU using Google Collaboratory, Tesla K80 or Tesla T4. This last one has a higher amount in the RAM. The RAM time is limited to 12 hours.

You have here two links for more information about Google Collaboratory. Remember we are accessing to a virtual environment running under Linux, not Windows, so we need instructions in Linux. And here we have the repo for YOLO version 4. You can clone the repo and paste it in your Google Drive account, which is my case. In YOLO version 4 I have used a different methodology. Here I have created in my Google Drive account different folders. One folder with the name Collab, into this folder another folder with the name Darknet version 4. You can create the folders as you want, but into this folder I have the repo with the name Darknet. I have another folder for QDNN, the specific version that we need to use for this version of YOLO.

And also I have all iteration files that we need to implement this version of YOLO. One important thing to note is you need to construct your ARM dataset in the same way like in YOLO version 2 or YOLO version 3. The procedure is the same. You need the images and the annotations in TXT format. So in my case if you go here I have created a folder with the name ing and here I have all images and annotations. This is an annotation. You need to have all annotations in this format. TXT with the location of the bounding boxes in a scale between 0 and 1. The first number represents the class, the class of the object. You don't need a special format in the sizes for the images.

For example here on this image we have two objects. So the annotation contains two lines, zero for one class and one for the other. You only need one folder containing all images. In my case I have 320 images. You only need to specify with a path the images for the training and for testing. You can place your images in the location as you want but you need to consign the exact path to access to this location. Another important consideration in the setup is the configuration of your makefile. Here because I have the repo it's very easy to edit this file so you need to be sure about the specific parameters included into this file. You will need these specific instructions with these values because we are going to use a GPU card we need this flag GPU=1, QDNN=1 and these values.

You need to check if your makefile has these values. This is my makefile into the repo, open with text editor and here you will find the same flags as before. Very good. You need to be sure only for these initial instructions. You don't need to touch the rest of the code. Now you need to create a model. This model has an extension CFG. So I have the models here in this folder CFG. In my case

I have created a different file with the name YOLOv4Robots because I'm going to perform the same example like in YOLOv3 and YOLOv2. So here you need to edit many lines in this file. Let's see all changes. You need to make a copy of this custom model with the name as you want.

Second, in this new file you need to change these parameters. Max batches to classes is the number of classes. If you have three classes to detect you need to set 3 by 2000 equals 6000. This is a line, line 20. After that you need to change in the line 22 this range 0.8 by the max batches which is the previous variable and also this expression. In lines 8 and 9 it's recommended to set the network size in the width and height to this value for training. After training you can change this resolution to another values. After that in these lines you need to change the number of classes. 80 is because that the full implementation of YOLOv4Robots is for COCO dataset but in our case because we are performing a custom training you need to put the number of classes that you want to detect.

Finally, in these lines you need to change this number of filters for the result of this expression which is based in the number of classes. If you have three you need to replace this variable 3 plus 5 is 8 by 3 so the total 24. Very good. That's it in this model file. So you can use the editor, this text editor that comes with Google Colab to perform the changes. It's very simple. Okay, another configuration file is the data file. Here I have my data file so I need to consign the path where it located the train and test files containing all the information for training and testing. You need to specify the number of classes in names this file with the names of the classes that you want to detect and the folder for backup.

That's it. This path is related to the location where it's located the detector file that you are going to run. Okay, we also need a file containing the name of the labels. This is the file and it's the same file like in YOLO version 3. One name per line. In my case this is a name for class 0 and the name for class 1. Finally, you need to create two files specifying the information for training and for testing. These are the files in my case you can put in the location as you want but you need to consign the exact path to locate these files. For example, in this train file this is the path in my case. You can change or place this information in the location as you want and as you can see you only need to refer the location where it's located the images but for this implementation you won't need to upload all the images in the virtual machine.

I already have all the images in my Google Drive account and also you need to specify the location for the images to be used in the testing process. This is the same. Very good. This is all about pre-configuration. Now I'm going to come back to the Colab file. Very good. First, you need to go here on notebook settings and check the use of GPU for hardware accelerator. Second, you need to connect. Very good. This is the status of your virtual machine. Now we can check the information about the GPU which is available in this virtual machine. With this instruction we have the information. This is the version of CUDA 10.1 and this is the model Tesla T4. I have 15 gigabytes of RAM in this case.

Take note about this exclamation sign because we are running a command in Linux. After that we can link this Google Colab file to the Google Drive account. Okay, go to this link, select your account, allow. Very good. This is a link. You need to click here, copy this link and place this link here. Enter and that's it. Mount it at content/gdrive. If you go here on folders you will see this

folder gdrive. This is the access to your Google Drive account. Plotting here we have my drive. In my case I have Colab, darknet version 4 and this is the repo darknet. Very good. Let's check the CUDA version. This is the version 10.1. So we need QDNN for this specific version of CUDA.

The recommended version for this CUDA version in Linux is 7.6.5. So you need to download this version of QDNN and you have two options for making this download process. Very good. In my case I have this file which is the QDNN file located in this path. So extracting this file I need QDNN.h file. Only this file. I'm going to copy this file to the virtual machine using these instructions. Very good. Now I need to go to a specific location to perform different instructions. With this magic sign and cd command we are able to go to a specific path. This is a path. I need to go here into a repo and this is a path. Perfect. If you want to take a look of the content of this specific folder you can use ls.

This is the content and it's the same content as we have seen here darknet and all of these folders and files are the same as you can see here. Very good. Before using our model from the repo we need to build the model. To do that we need to use the instruction make and because of that it's very important that you can previously edit your make file. Very good. Let's run this all. Perfect. That's it in configuration. Now I have the same utility functions like in YOLO version 3 for showing the images in the predictions and also functions for uploading and downloading files. Very good. Now we are ready to use the YOLO version 4 model to detect objects on an image.

When you use by default the model of YOLO version 4 you are using a model which is pre-trained in cocoa data set and as you know this data set contains 80 classes. You need to use this instruction darknet detect. You need to specify the model weights. In this case YOLO version 4 weights the picture where you want to identify the classes and this parameter threshold is a threshold for a confidence factor. In this case 25% in the confidence any object with a lower confidence won't be detected. Let's run this all. Okay it's starting CUDA and QDNN GPU count 1. This is the version of OpenCV version 3. Okay the model is loaded and these are the confidence factors for the classes.

The model has identified three classes; dog, person and horse. This prediction is located here on this file predictions. If you want to visualize this file you need to use the utility function I have created before. As you can see these three classes are very well identified. Now the next step is to train our custom data set. In this case we need to change the instruction detect for detector. The first time you need to use this instruction using this pre-trained weights from YOLO yolo.conf.137. You need to specify your data file, your model file and after performing the initial training if you want to use pre-trained weights after so many epochs just replace these initial weights to your last weights.

After running this all you will have these values. The model will increase its accuracy after many epochs but it takes some time to have a good accuracy. I recommend you to run about 4000 or 5000 epochs. Let's see the results. For instance I have the situation for this step, 863 steps. On the left side the model will create the last weights for this specific step. This process is performed automatically and as you can see these are the losses for training and testing which are near to the

value of 2. You have the number of hours left to complete the process. This process is about 5 or 6 hours. Very good, let's see another value. This is for 1335 steps and here the model has created automatically two files.

Robots version 4 for 1000 steps and the last step for the weights. The loss for training and testing is reduced and near to 1. This is a result for 2000 steps. The loss is very tiny. Now I have another step. We have new files about 3000 steps, 4000 steps and the last weights. Finally these are the results for 4000 steps. These are the losses are very near to 0.5 which is a good result for object detection. I recommend you to achieve this value in the loss in order to have good accuracy in the predictions. Ok, after training you need to test your trained model in a video. In this case you need to use detector and the instruction demo. Specifying the same, the data file, the model file, the weights, in this case I'm using the last weights and the file, the video.

In my case this is a format, you can use MP4 if you want and finally the output file name, in my case with the extension MP4. So after running this cell you will find all the results in the detection, including the information about the frames per second, identifying all classes and the results in the confidence factor. So when you perform the testing process the model will create a new file which is this robots version 4 MP4 containing all predictions into the video. For accessing this file you need to use the utility function download specifying the file. So running this utility function you will get the file and see the results. A final note is about the configuration in the model.

At the beginning in this setup we have used a low resolution for the training but you can increase the resolution for detecting very small objects. This is in the case that you want to continue in the training using the last weights. Ok, finally let's see the results. I'm going to make the comparison between the results from YOLO version 2 and YOLO version 4. Ok, these are the results. Very well classified, C-3PO, R2D2, C-3P bounding boxes are about YOLO version 2, thinner bounding boxes YOLO version 4. Here it's a misclassification in YOLO version 2 but not in YOLO version 4. Very well classified, very good, YOLO version 4. But as you can see this Oscar statue is very similar to C-3PO so the model classifies this statue as C-3PO.

Both have the same gold finish. Very good, very good, no class to detect here. And here as you can see the confidence factor is about 100% in both cases but in YOLO version 2 it's only about 18 or 19%. In this case it's higher, 23, 40, 50 but YOLO version 4 has higher accuracies. Very good. This example is very useful because the shapes are very different between these two classes so you can try with different objects as you want. Ok, one final consideration is the version of TensorFlow that we are using so let's check it out. Very good, it's version 2.3, the latest version of TensorFlow. For this implementation we don't need to change the version of TensorFlow, you can use the latest version.

Ok, that's all in this application, I strongly recommend you to follow the instructions to avoid many issues and to save a lot of time. So, that's it, enjoy the course, I'll see you in the next class. Bye bye!

LECTURE 56

Traffic Lights

Hello everyone, this is the final class, I wanted to show you the results of object detection with YOLO version 4 applied to different videos. As you can see the results are excellent. You will find very precise detections for persons, traffic lights, cars, umbrellas, trucks, people walking, traffic lights, a mouse, keyboard, chair, laptop, and so on. But everything does not end here. What remains for you is to study and understand well the fundamentals exposed in the course and keep abreast of new algorithms that come out in a spiral of continuous improvement. Suggestions of new topics to be covered in the course, as well as doubts or ideas for improvement are welcome through the following means of contact.

You and I forums, email or by Twitter. Don't forget to support the course with a good rating to guarantee updates and support to the students. That's all, see you soon, best regards!